book 9

lua 5.5

a complete language manual

Raeliosol

version 5.0, compiled 10 2026

Contents

1. toolchain 7
   1.1. the project 7
   1.2. the interpreter surface 10
   1.3. loading chunks 11
2. lexical structure 13
   2.1. names and the compat caveat 13
   2.2. short strings and escapes 14
   2.3. long brackets 16
   2.4. numerals 16
3. values, metatables, metamethods 18
   3.1. numbers: one type, two subtypes 18
   3.2. strings and keys 19
   3.3. metatables 20
   3.4. the arithmetic and comparison events 20
   3.5. access and call 22
4. expressions and operators 25
   4.1. arithmetic 25
   4.2. bitwise 27
   4.3. comparison and coercion 27
   4.4. constructors and length 29
5. statements and scope 30
   5.1. attributes 30
   5.2. loops 31
   5.3. goto and returns 33
6. functions 36
   6.1. multiple results 36
   6.2. the 5.5 vararg model 37
   6.3. closures and tail calls 39
7. coroutines 42
   7.1. errors and the un-unwound stack 43
   7.2. close as cancellation 44
   7.3. coroutines as iterators 45
   7.4. yielding through C 46
8. environments, load, gc 48
   8.1. the collector 49
   8.2. weak tables and finalizers 50
9. inside the collector 54
   9.1. the count as an oscilloscope 54
   9.2. incremental anatomy 55
   9.3. the tricolor walk 57
   9.4. the pause and step multipliers 58
   9.5. generational anatomy 59
   9.6. the allocator hook, from lua 61
   9.7. weak tables and finalizers, in one breath 61
   9.8. the particle kernel, boxed 62
10. profiling: hooks, clocks, and p99 66
   10.1. clocks, on this machine 66
   10.2. the counting profiler 67

10.3. the census, honestly labeled ... 69
10.4. what a collector step costs ... 70
10.5. percentiles by hand ... 71
10.6. the jit side of the tree ... 73
10.7. the row kernel, ten million rows ... 74
10.8. across the six books ... 77
11. stdlib 1: basic, string patterns, utf8 ... 79
11.1. the string library ... 80
11.2. patterns ... 81
11.3. binary packing ... 83
11.4. utf8 ... 83
12. stdlib 2: table, math, io, os ... 85
12.1. math ... 86
12.2. io ... 88
12.3. os ... 90
13. stdlib 3: coroutine, debug, package ... 92
13.1. the debug library ... 92
13.2. package and require ... 94
14. the c api, hands on ... 97
14.1. luac ... 97
14.2. a first host ... 99
14.3. the stack ... 100
14.4. calling lua protected ... 101
14.5. c functions for lua ... 103
14.6. userdata in outline ... 104
14.7. references ... 105
14.8. the state model ... 107
14.9. chunk names ... 108
15. incompatibilities with 5.4 ... 109
15.1. the language list ... 109
15.2. the library list ... 110
15.3. beyond the list ... 110
15.4. the api list ... 112
16. idioms ... 113
16.1. errors ... 114
16.2. iteration and forwarding ... 115
16.3. state and caching ... 117
16.4. the sandbox shape ... 118
17. ffi: c without the api ... 120
17.1. the boundary ... 120
17.2. the statement lifecycle ... 121
17.3. who owns what ... 124
17.4. two runtimes, one tree ... 125
17.5. the editor lane gains a database ... 126
18. a c host: allegro 5 with lua embedded ... 128
18.1. why a c host owns time ... 128
18.2. the vendored toolchain ... 128
18.3. linking two mingw libraries ... 129
18.4. the event loop skeleton ... 130
18.5. events to commands ... 131
18.6. lua draws while c renders ... 133

18.7. headless by design 134
19. the http kernel 136
19.1. the seam and the handler shape 136
19.2. bytes into a request 137
19.3. the route table 137
19.4. the error envelope 138
19.5. json in and out 139
19.6. writing the response 140
19.7. the kernel routes and the root 141
19.8. what the kernel proves 141
20. the middleware chain 143
20.1. the chain fold 143
20.2. request id 143
20.3. the access log 144
20.4. the deadline owns the resume 145
20.5. recover 146
20.6. wiring order and what it proves 147
20.7. testing the chain 148
21. the user resource 150
21.1. the record and its etag 150
21.2. validation from first principles 150
21.3. merge patch semantics 151
21.4. keyset cursors 152
21.5. the store as three tables 153
21.6. projections and the admin list 154
21.7. golden vector replay 155
22. authentication 157
22.1. sha-256 from the spec 157
22.2. hmac and pbkdf2 157
22.3. login and the dummy verify 158
22.4. sessions in a fixed table 159
22.5. the cookie line 160
22.6. HS256 from scratch 161
22.7. rotation and reuse detection 162
22.8. golden vector replay 162
23. authorization and roles 164
23.1. two questions, two codes 164
23.2. the policy table 164
23.3. the object routes and the id 165
23.4. roles as data 166
23.5. deny by default and the audit 167
23.6. the matrix replay 168
24. the store 170
24.1. the ffi lane to the pinned engine 170
24.2. migrations as append-only records 171
24.3. the register transaction 171
24.4. guarded statements that keep an admin 172
24.5. reads, the keyset walk, the version guard 173
24.6. the analytics report as committed queries 174
25. concurrency control 176
25.1. the duel: two writers, one version 176

25.2. the conditional read answers 304 ... 177
25.3. the idem store and the replay contract ... 178
25.4. what the single vm buys ... 178
25.5. interleavings as scripted resume orders ... 179
26. caching ... 181
26.1. the lru in two tables ... 181
26.2. ttl with the injected clock ... 182
26.3. vary and the cache key ... 182
26.4. the guarded fill ... 183
26.5. path-scoped invalidation on write ... 184
26.6. what never caches ... 184
27. rate limiting ... 186
27.1. why limits exist ... 186
27.2. the token bucket from first principles ... 187
27.3. the keyed table and its sweep ... 187
27.4. the 429 contract ... 188
27.5. what to key on ... 189
27.6. limits under test ... 190
28. observability ... 192
28.1. the access line ... 192
28.2. structured beats interpolated ... 193
28.3. redaction as a closed set ... 193
28.4. traceparent from scratch ... 194
28.5. the fixed-table metrics registry ... 195
28.6. the exposition, byte for byte ... 196
28.7. one request fully observed ... 197
29. load and p99 ... 199
29.1. what load means here ... 199
29.2. nearest rank and the bucket walk ... 200
29.3. the harness over the real wire ... 201
29.4. the report and its two sources ... 201
29.5. committed captures ... 202
29.6. the lua leak, found by counting ... 203
30. shipping ... 205
30.1. what shipping means here ... 205
30.2. the oracle learns the report seams ... 206
30.3. the reports routes and vector 16 ... 206
30.4. the secret source ... 207
30.5. drain and readiness ... 208
30.6. the image ... 209
30.7. compose one command ... 210
30.8. the docker lane as a gate ... 211
31. the test suite ... 213
31.1. what the suite holds ... 213
31.2. the runner ... 214
31.3. the t lib ... 214
31.4. the two test shapes ... 215
31.5. the harness swap ... 216
31.6. the four fresh states ... 217
31.7. replay invariants over seeded streams ... 218
31.8. busted and luaunit as dated meta ... 219

32. capstone: auto chess, human versus bot 221
32.1. the game 221
32.2. the command queue, the automation seam 221
32.3. ecs in lua 223
32.4. the bot's brain, a state machine 224
32.5. deterministic combat 225
32.6. the c host, both interop directions 227
32.7. the headless suite 228
32.8. profiled, and why 229
32.9. the service part after the capstone 229
33. appendices: coverage and sources 230
33.1. keyword coverage 230
33.2. stdlib coverage 231
33.3. pinned sources 232

1. toolchain

Lua 5.5.0 shipped on 22 december 2025 and the current 5.5.1, a bug fix release, on 3 august 2026. This book pins 5.5.1, built from the official source tarball into `tools/lua55/build/bin/`, so every claim in every chapter runs against the same interpreter binary that `make verify-lua` invokes. The first test in the first sample suite pins `_VERSION` to the family name, and the `-v` banner with the exact point release is a later test in the same suite.

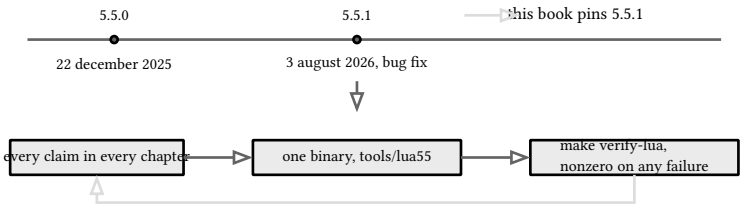


figure 1: the 5.5 release line, and the verify loop that pins every claim to one binary

Lua the language is 8 types, 23 keywords in 5.5 (`global` included, 22 before it), and one associative array constructor. The release highlights for 5.5 over 5.4 are four: global variable declarations with the new `global` statement, named vararg tables, a more compact internal array representation, and major collections inside the generational collector done incrementally. Each of those gets a full chapter treatment later; the toolchain chapter establishes how this book proves anything at all.

1.1. the project

Samples live in `lua/samples/`, one module per chapter, each returning a list of named tests. There is no npm, no sdk, no test framework: the harness is 43 lines of lua, registry through exit code.

```
-- sample suite runner: requires each chapter module, runs its tests,
-- reports per chapter, exits nonzero on any failure
local chapters = {
  "ch01_toolchain",
  "ch02_lexical",
  "ch03_values",
  "ch04_expressions",
  "ch05_statements",
  "ch06_functions",
  "ch07_coroutines",
  "ch08_environments",
  "ch09_collector",
  "ch10_profiling",
  "ch11_stdlib1",
  "ch12_stdlib2",
  "ch13_stdlib3",
  "ch14_capi",
  "ch15_incompat",
  "ch16_idioms",
  "ch18_allegro",
}

package.path = "./?.lua;" .. package.path

local failed = 0
local passed = 0
for _, name in ipairs(chapters) do
  local mod = require(name)
  local cpass = 0
  for _, t in ipairs(mod) do
    local ok, err = pcall(t.fn)
    if ok then
      cpass = cpass + 1
    else
      failed = failed + 1
      io.stderr:write(name .. " :: " .. t.name .. " FAILED: " .. tostring(err) .. "\n")
    end
  end
  passed = passed + cpass
  print(string.format("%-22s %d/%d", name, cpass, #mod))
end
print(string.format("total %d passed, %d failed", passed, failed))
if failed > 0 then os.exit(1) end
```

listing 1: the whole test runner (source: lua/samples/run.lua, lines 1-43)

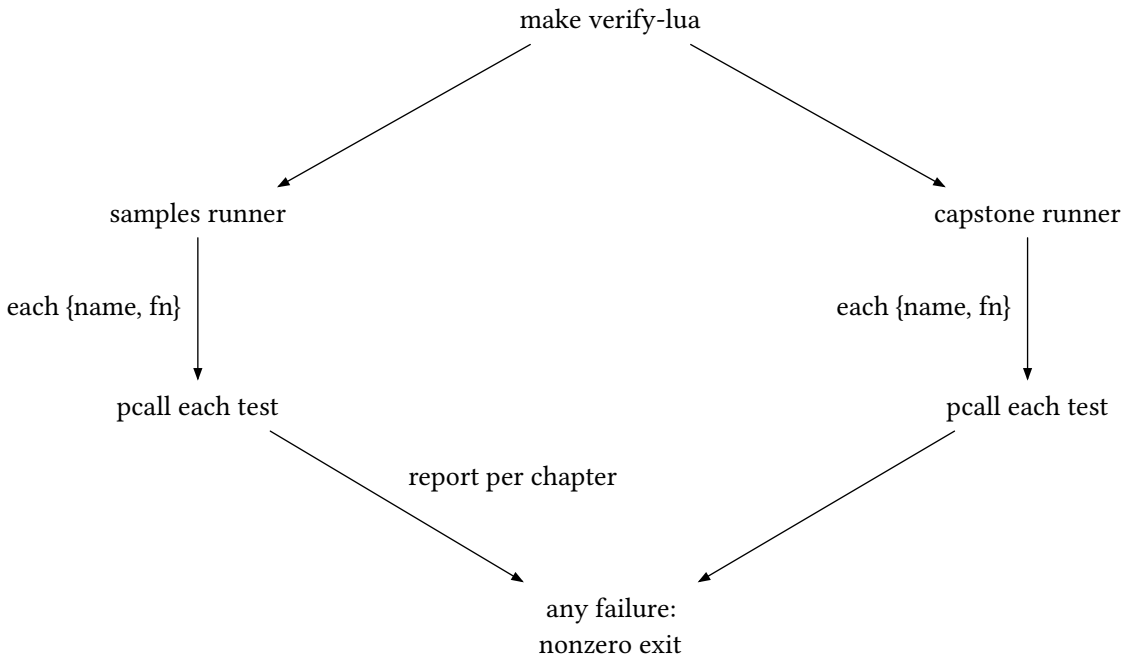


figure 2: the runner pipeline, chapter modules through pcall to the exit code

`make verify-lua` runs this file with the pinned interpreter from the samples directory and again from the capstone directory, and make fails if either exits nonzero. Chapters register themselves in the runner's list as they are written, the same grow-the-map discipline the other books use.

Assertions come from a shared helper module, deep comparing tables and reporting at the failing line:

```lua
local function deep(a, b)
  if a == b then return true end
  if type(a) ~= "table" or type(b) ~= "table" then return false end
  local seen = 0
  for k, v in pairs(a) do
    if not deep(v, b[k]) then return false end
    seen = seen + 1
  end
  for _ in pairs(b) do seen = seen - 1 end
  return seen == 0
end

-- compares with lua equality first, then structurally for tables
function M.eq(actual, expected, msg)
  if not deep(actual, expected) then
    error((msg or "values differ") .. ": got " .. show(actual) .. ", want " .. show(expected), 2)
  end
end

function M.ok(cond, msg)
  if not cond then error(msg or "assertion failed", 2) end
end

-- requires f to raise, with want as a substring of the message
function M.throws(f, want)
  local ok, err = pcall(f)
  if ok then error("expected an error, call returned " .. show(err), 2) end
  if want and not tostring(err):find(want, 1, true) then
    error("error message " .. show(tostring(err)) .. " lacks " .. show(want), 2)
  end
  return err
```

listing 2: deep equality, level 2 error reporting, and the error-capturing throws helper (source: lua/samples/lib.lua, lines 17-47)

1.2. the interpreter surface

The standalone `lua` program is the host this book uses: it reads a chunk from a file argument, from `-e` strings, or from stdin in the read eval print loop. The suite's first probes pin identity and the two number subtypes, since almost every later chapter leans on the integer versus float split:

```lua
  {name = "numbers split into integer and float subtypes", fn = function()
    T.eq(math.type(1), "integer")
    T.eq(math.type(1.0), "float")
    T.eq(math.type(0x10), "integer")
    T.eq(math.type(1e0), "float")
  end},

  {name = "integers are 64 bit and wrap around", fn = function()
    T.eq(math.maxinteger, 0x7fffffffffffffff)
    T.eq(math.maxinteger + 1, math.mininteger)
    T.eq(math.mininteger - 1, math.maxinteger)
```

listing 3: subtype split and 64 bit wraparound (source: lua/samples/ch01_toolchain.lua, lines 8-19)

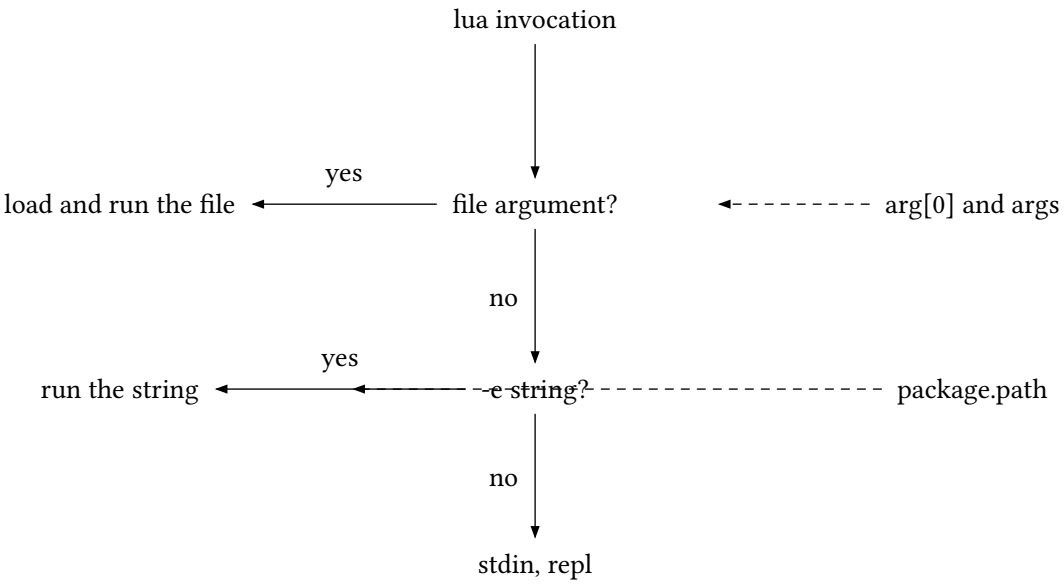


figure 3: how the standalone interpreter picks its chunk, with arg and package.path around it

`arg[0]` names the running script, and the `arg` table carries command line arguments around it, which is how the runner could tell you which file failed. `package.path` decides what `require` finds; the runner prepends `./?.lua` so chapter modules and `lib` resolve from the samples directory regardless of where lua was invoked.

NOTE: one binary, two shells

`lua.exe -v` prints the banner and exits. The suite shells out to it through `io.popen` with a path relative to the samples directory, which doubles as the book's proof that `io.popen` works on a plain windows build. `luac.exe` beside it precompiles and disassembles, and book 9, ch 14 (lua 5.5, the c api, hands on) uses it heavily.

1.3. loading chunks

`load` is the boundary between text and function. It takes source or bytecode, a chunk name for error messages, a mode that pins which form is acceptable, and since 5.2 an explicit environment table that free names resolve against:

```
{name = "load takes an explicit environment", fn = function()
  local f = load("return x + 1", "chunk", "t", {x = 41})
  T.eq(f(), 42)
end},

{name = "load mode t rejects bytecode, mode b accepts it", fn = function()
  local dump = string.dump(function() return 1 end)
  T.eq(load(dump, "bytes", "t"), nil)
  T.ok(load(dump, "bytes", "b"), "binary load should succeed")
end},

{name = "string.dump round trips through load", fn = function()
  local src = function(x) return x * 2 end
  local copy = load(string.dump(src))
  T.eq(copy(21), 42)
```

listing 4: environment argument, mode gate, and a bytecode round trip (source: lua/samples/ch01_toolchain.lua, lines 25-40)

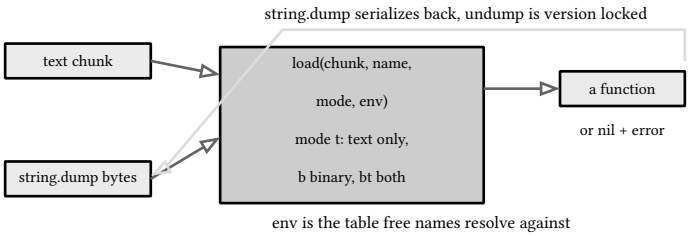


figure 4: load as the text to function boundary, the mode gate and the dump round trip

`string.dump` serializes a function back to binary and `load` accepts it with mode `"b"`, rejects it with mode `"t"`, and accepts either with the default mode `"bt"`. Undumping is version locked: a 5.4 dump will not load in 5.5 and the manual says so, which is why the book never treats bytecode as a distribution format.

sources: lua.org manual 5.5 sections 1, 6.2 (assert, load), lua.org/news.html release dates, lua.org/versions.html 5.5 features, accessed 2026-09-08. Interpreter behavior verified live with lua 5.5.1 built from the official tarball, 10 tests green through `make verify-lua`.

2. lexical structure

Lua is free form: whitespace and comments separate tokens and nothing else. Names are case sensitive, `And` and `AND` are two different valid names while `and` is grammar. In 5.5 `global` joins the reserved list, the first new reserved word since 5.2's `goto` in 2011, bringing the count to 23.

2.1. names and the compat caveat

The probe suite pins the reserved list by brute force, trying each word as a local name:

```
local reserved = {
  "and", "break", "do", "else", "elseif", "end", "false", "for",
  "function", "global", "goto", "if", "in", "local", "nil", "not",
  "or", "repeat", "return", "then", "true", "until", "while",
}

local function loads(src)
  return load(src) ~= nil
end
```

listing 5: the 23 reserved words and the loads helper (source: lua/samples/ch02_lexical.lua, lines 4-12)

```
  {name = "reserved words are not names, global is the compat exception", fn = function()
    local okcount = 0
    for _, w in ipairs(reserved) do
      if loads("local " .. w .. " = 1") then okcount = okcount + 1 end
    end
    T.eq(okcount, 1)
    T.ok(loads("local global = 1"), "stock 5.5.1 keeps global usable as a name")
    T.eq(load("local end = 1"), nil)
  end},

  {name = "global still acts as a keyword in statement position", fn = function()
    T.ok(loads("global x; x = 1"), "declaration form must parse")
    T.eq(load("global x; x = 1; y = 2"), nil)
    T.ok(select(2, load("global x; x = 1; y = 2")):find("variable 'y' not declared", 1, true))
    T.ok(loads("global = 5"), "assignment form parses under compat")
  end},
```

listing 6: exactly one reserved word parses as a name (source: lua/samples/ch02_lexical.lua, lines 22-37)

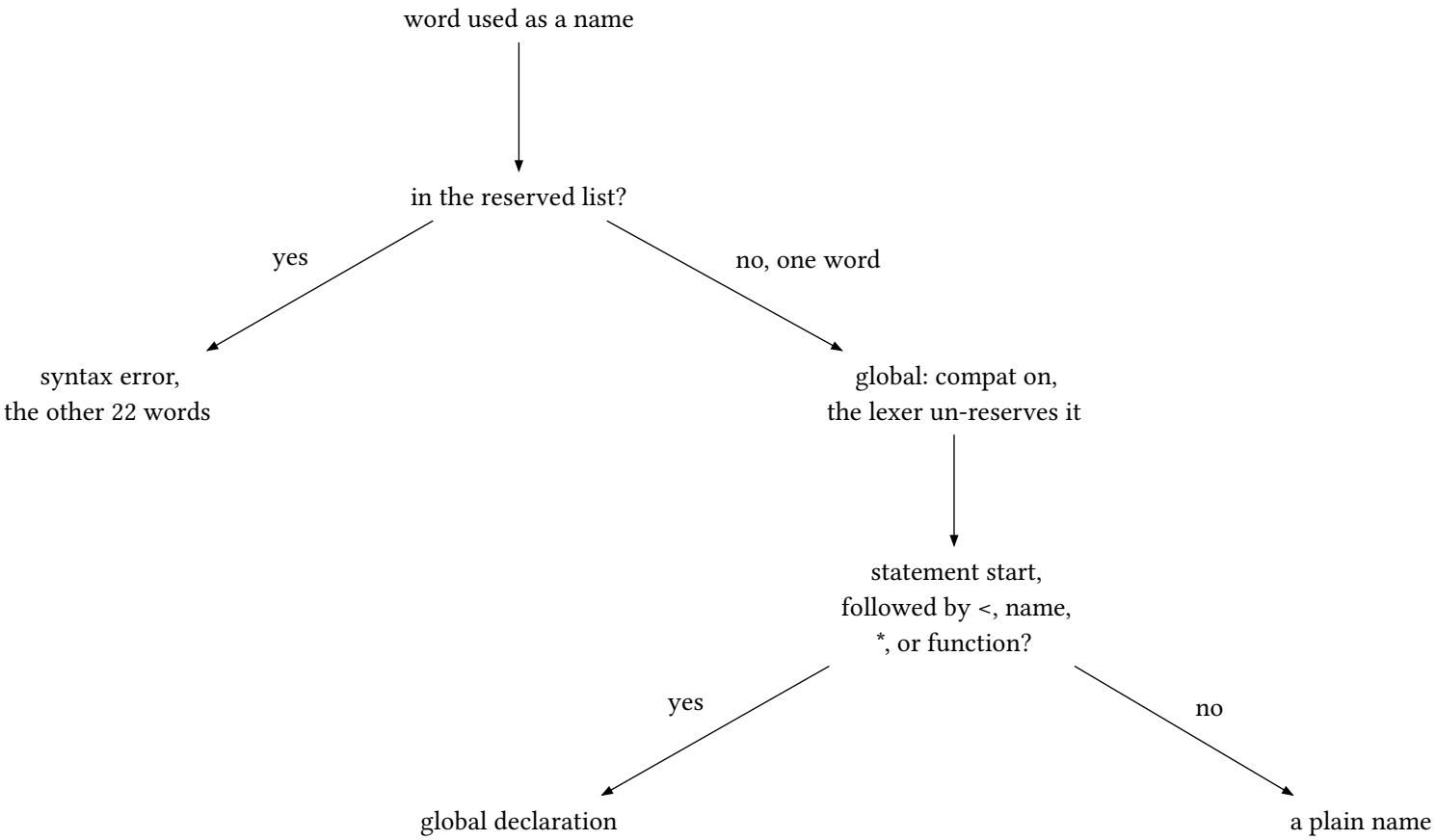


figure 5: why exactly one reserved word parses as a name, the compat un-reserving and the declaration lookahead

Exactly one word survives: `global`. This is the book's first honest finding about 5.5.1 as distributed. The manual reserves `global` outright, but `luaconf.h` ships `LUA_COMPAT_GLOBAL` defined to 1, and under it the lexer marks the word not reserved. The parser then recognizes a global declaration contextually: in statement position, the name `global` followed by a lookahead of `<`, a name, `*`, or `function` parses as the declaration statement. So `local global = 1` is legal, `global = 5` is a plain assignment to a global literally named `global`, and `global x; x = 1; y = 2` still fails to compile because `y` is undeclared. Code that needs the manual's strict reservation compiles with `LUA_COMPAT_GLOBAL` unset; this book tests the stock tarball and says so in every affected chapter.

The identifier alphabet has the same shape of caveat. The manual describes names as Latin letters, Arabic-Indic digits, and underscores, but the stock build's character table marks nothing above byte 127 alphabetic unless `LUA_UCID` is defined at compile time:

```
{name = "stock identifiers are ascii only despite the manual's alphabet", fn = function()
  local _x1 = 5
  T.eq(_x1, 5)
  T.eq(loads("local 1x = 1"), false)
  local f, e = load("local a\u{0661} = 7")
  T.eq(f, nil)
  T.ok(e:find("unexpected symbol"), e)
end},
```

listing 7: an utf 8 continuation byte is an unexpected symbol in the stock build (source: lua/samples/ch02_lexical.lua, lines 44-51)

2.2. short strings and escapes

A short string is single or double quoted, one line, with C-like escapes: the controls `\a \b \f \n \r \t \v`, quotes, backslash, plus `\z` to skip whitespace, a backslash line break for a newline, `\xXX` for a byte, up to three decimal

digits for a byte, and \u{XXX} for a code point in lua's original, non unicode restricted utf 8. The manual compresses the whole system into five spellings of one string:

```
  {name = "the manual's five spellings of one string", fn = function()
    local variants = {
      'alo\n123"',
      "alo\n123\"",
      '\97lo\10\04923"',
      [[alo
123"]],
      [==[alo
123"]==],
    }
    for i = 2, #variants do
      T.eq(variants[i], variants[1], "variant " .. i)
    end
  end},
```

listing 8: two escapes, two long brackets, one byte coded string (source: lua/samples/ch02_lexical.lua, lines 53-66)

The decimal escape has the classic trap the suite pins: \0658 is the byte 65 followed by the digit 8, while \658 tries to read one escape of 658 and fails to load:

```
  {name = "z skips whitespace, backslash newline is a newline", fn = function()
    T.eq("a\z
      b", "ab")
    T.eq("a\
b", "a\nb")
  end},

  {name = "hex and decimal byte escapes", fn = function()
    T.eq("\x41", "A")
    T.eq("\65", "A")
    T.eq("\0658", "A8")
    T.eq(loads('return "\\658"'), false)
    local f, e = load('return "\\256"')
    T.eq(f, nil)
    T.ok(e:find("too large"), e)
  end},

  {name = "u escapes emit raw utf 8, beyond unicode included", fn = function()
    T.eq("\u{48}", "H")
    T.eq(#"\u{263A}", 3)
    T.eq(#"\u{7FFFFFFF}", 6)
  end},
```

listing 9: z, backslash newline, byte escapes, and u with a beyond unicode code point (source: lua/samples/ch02_lexical.lua, lines 68-89)

spelling	produces
\a \b \f \n \r \t \v	the seven control characters
\" \' \\	the quote and the backslash themselves
\z	skips all following whitespace
\ + a real newline	a newline inside the literal
\xXX	one byte from two hex digits
\ddd	up to three decimals for a byte \0658 is byte 65 then 8, \658 fails
\u{XXX}	a code point up to 2^31^-1 in lua's original utf 8

figure 6: the escape grammar of a short string as one taxonomy

\u{7FFFFFFF} encodes to 6 bytes because lua implements the original utf 8 proposal, not the unicode capped one, and it accepts code points up to 2^31^-1.

2.3. long brackets

A long bracket of level n is [repeated = n times then [; the string runs to the matching close and ignores brackets of every other level. No escapes are interpreted. The first newline after the opening bracket is dropped, and any end of line sequence inside, including carriage return alone, normalizes to one newline:

```
  {name = "long brackets nest by level and drop the first newline", fn = function()
    T.eq([[
abc]], "abc")
    T.eq([==[a]=]b]==], "a]=]b")
    T.eq([[\n]], "\\n")
  end},

  {name = "end of line sequences become one newline in long strings", fn = function()
    local leading = load("return [[" .. string.char(13, 10) .. "abc]]")
    T.eq(leading(), "abc")
    local inner = load("return [[a" .. string.char(13, 10) .. "b]]")
    T.eq(inner(), "a\nb")
  end},
```

listing 10: level nesting, literal backslashes, dropped leading newline, crlf folding (source: lua/samples/ch02_lexical.lua, lines 97-109)

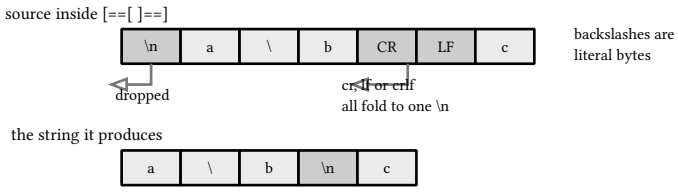


figure 7: long bracket scanning, the dropped first newline and the folded line ends

Comments reuse the machinery: -- to end of line, or -- plus a long bracket for a block comment, which is how you comment out code that itself contains]] by going up a level.

2.4. numerals

A numeral with a radix point or exponent is a float, in decimal or hexadecimal, hexadecimal exponents marked p. Without either, it is an integer if it fits, with two overflow rules that differ by base: hexadecimal integers wrap around, decimal integers that overflow become floats:

```
{name = "integer numerals", fn = function()
  T.eq(math.type(3), "integer")
  T.eq(math.type(345), "integer")
  T.eq(0xff, 255)
  T.eq(0xBEBADA, 0xBEBADA)
  T.eq(0xFFFFFFFFFFFFFFFF + 1, 0)
  T.eq(math.type(0xFFFFFFFFFFFFFFFF), "integer")
end},

{name = "float numerals including the manual's 0x1.fp10", fn = function()
  T.eq(math.type(3.0), "float")
  T.eq(math.type(34e1), "float")
  T.eq(314.16e-2, 3.1416)
  T.eq(0.31416E1, 3.1416)
  T.near(0x0.1E, 30 / 256)
  T.near(0xA23p-4, 2595 / 16)
  T.near(0x1.fp10, 1984)
  T.eq(math.type(9999999999999999999), "float")
end},
```

listing 11: the subtype rules and both overflow behaviors (source: lua/samples/ch02_lexical.lua, lines 111-129)

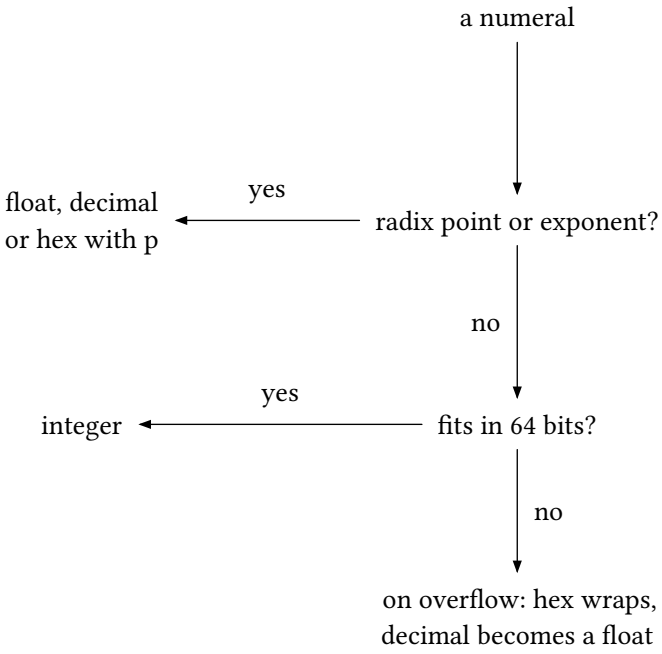


figure 8: numeral classification, what forces a float and the two overflow rules

The manual's own example `0x1.fp10` is 31 sixteenths times 1024, which is exactly 1984, and the suite holds it to that.

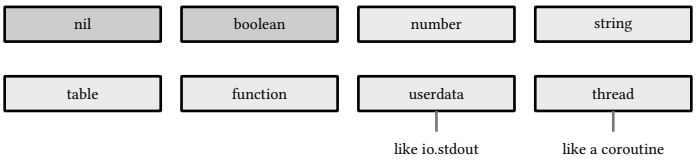
sources: lua.org manual 5.5 section 3.1, lua.org/versions.html 5.5 entry, accessed 2026-09-08. Lexer and parser behavior verified against the stock 5.5.1 tarball source, llex.c, lparser.c, lctype.c, lctype.h, luaconf.h, 15 tests green through `make verify-lua`.

3. values, metatables, metamethods

Lua is dynamically typed: variables hold anything, values carry the type. Eight types exist and the stock interpreter exposes all eight without leaving lua code, because `io.stdout` is userdata and a coroutine is a thread:

```
{name = "all eight types are observable from stock lua", fn = function()
  T.eq(type(nil), "nil")
  T.eq(type(true), "boolean")
  T.eq(type(1), "number")
  T.eq(type(""), "string")
  T.eq(type(print), "function")
  T.eq(type(io.stdout), "userdata")
  T.eq(type(coroutine.create(function() end)), "thread")
  T.eq(type({}), "table")
end},
```

listing 12: the eight types, each observed through a live value (source: lua/samples/ch03_values.lua, lines 5-14)



shaded: nil and false are the only falsy values

nil in a table is an absent key,

false is present with the value false

figure 9: the eight types, each reachable from plain lua, and the two falsy ones

Only `nil` and `false` are falsy. Zero, the empty string, and NaN all count as true. The distinction between nil and false is structural, not just truthiness: nil in a table means the key is absent, false means present with value false, which is why `next` sees one and not the other.

3.1. numbers: one type, two subtypes

`type` says number for both, `math.type` splits them. `1 == 1.0` is true and they make the same table key, but `tostring` keeps the subtype visible, `1` versus `1.0`, and a float with no integer representation cannot cross into `%d`:

```
{name = "integers and floats are one type, two subtypes", fn = function()
  T.eq(type(1), type(1.0))
  T.eq(math.type(1), "integer")
  T.eq(math.type(1.0), "float")
  T.eq(1 == 1.0, true)
  T.eq(tostring(1), "1")
  T.eq(tostring(1.0), "1.0")
  T.eq(string.format("%d", 3.0), "3")
  T.throws(function() return ("%d"):format(3.5) end, "has no integer representation")
end},
```

listing 13: equal but distinguishable, and the format boundary (source: lua/samples/ch03_values.lua, lines 32-41)



shaded: the subtype tag

math.type splits on it, type reports number for both

equal values, one table key, tostring keeps the subtype, %d wants an integer

figure 10: one runtime type, two tagged representations behind every number

3.2. strings and keys

Strings are immutable byte sequences, 8 bit clean, and encoding agnostic: an embedded zero is just a byte, and two strings differing only in that byte are different keys. Table keys accept anything except nil and NaN, and float keys holding exact integer values are normalized to integers so `t[2.0]` and `t[2]` are one entry:

```
{name = "float keys with exact integers normalize to integers", fn = function()
  local t = {}
  t[2.0] = "float form"
  T.eq(t[2], "float form")
  local count = 0
  for _ in pairs(t) do count = count + 1 end
  T.eq(count, 1)
  t[1 / 0] = "inf is a valid key"
  T.eq(t[1 / 0], "inf is a valid key")
  T.throws(function() local u = {}; u[0 / 0] = 1 end, "table index is NaN")
  T.throws(function() local u = {}; u[nil] = 1 end, "table index is nil")
end},
```

listing 14: key normalization, infinity as a key, nil and NaN rejected (source: lua/samples/ch03_values.lua, lines 53-64)

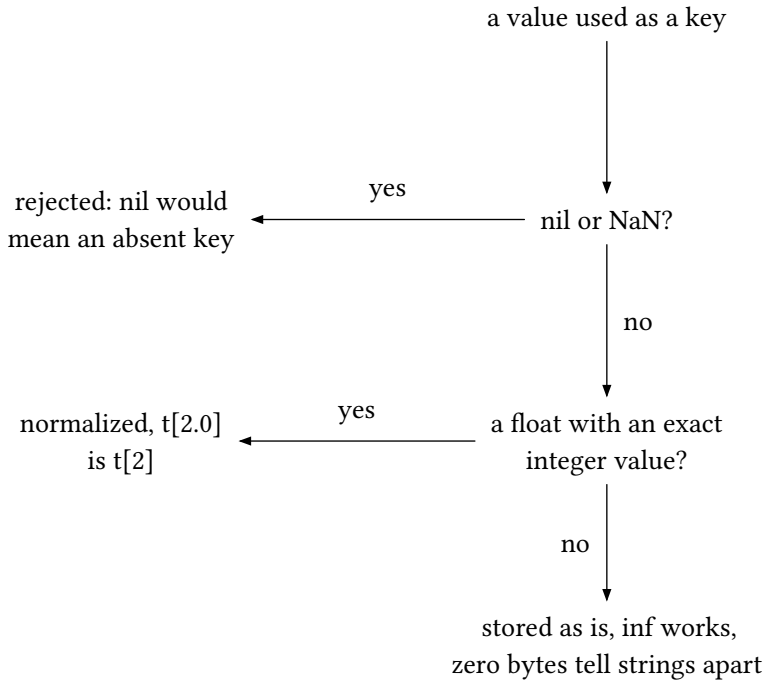


figure 11: what a table accepts as a key, the two rejections and the float normalization

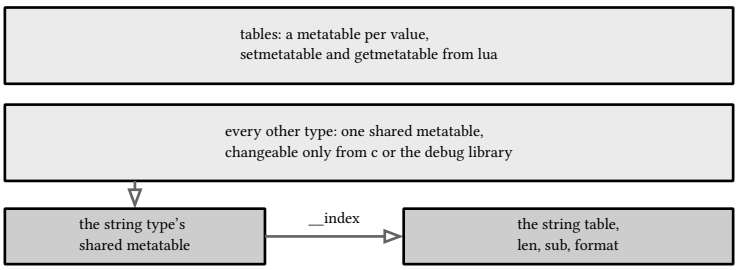
Tables are references: two names on one table alias, and fresh tables are never equal. Functions compare by identity, so the same C function equals itself and two closures from the same factory do not.

3.3. metatables

Any value may carry a metatable. Tables get theirs from `setmetatable`, other types share one per type and only C or the debug library can change those, which is why the string library can give every string an `__index` pointing at the `string` table and `("abc"):len()` works, the colon passing the string as `self`, the sugar chapter 6 defines. `getmetatable` returns the `__metatable` field instead of the real table when present, which is the standard protection idiom.

NOTE: address format is the platform's

`tostring` of a table is its type name, a colon, and the `%p` of the pointer. On this windows build that prints without an `0x` prefix, `table: 00000187...`. The suite matches only the prefix `table: ` because the format belongs to the C runtime, not to lua.



so ("abc"):len() is an index miss walked to the string table

__metatable returns a stand-in, __name names values in messages

figure 12: where metatables live, per value for tables, one shared per type for the rest

`__name` feeds the same machinery: a table with `__name = "Widget"` prints as `Widget: ...` and error messages say "on a Widget value" instead of "on a table value":

```
{name = "tostring uses the name field of the metatable", fn = function()
  local w = setmetatable({}, {__name = "Widget"})
  T.ok(tostring(w):match("^Widget: "), tostring(w))
  T.ok(tostring({}):match("^table: "), "plain tables say table")
  local err = select(2, pcall(function() return w + 1 end))
  T.ok(tostring(err):find("on a Widget value"), err)
  local cerr = select(2, pcall(function() return w() end))
  T.ok(tostring(cerr):find("call a Widget value"), cerr)
end},
```

listing 15: name in tostring and in error messages (source: lua/samples/ch03_values.lua, lines 84-93)

3.4. the arithmetic and comparison events

Arithmetic metamethods search the first operand first, then the second, and receive the operands in original order. Unary operators pass a dummy second operand equal to the first, an internal simplification the manual reserves the right to remove. Bitwise metamethods fire only when a float operand cannot coerce to integer, so `3.0 & 1` is plain machine arithmetic while `1.5 | 2` is an error without a metamethod:

```lua
{name = "add metamethod receives operands in order", fn = function()
  local log = {}
  local mt = {__add = function(a, b)
    log[1], log[2] = a, b
    return "sum"
  end}
  local t = setmetatable({}, mt)
  T.eq(t + 1, "sum")
  T.eq(log, {t, 1})
  log = {}
  T.eq(1 + t, "sum")
  T.eq(log, {1, t})
end},
```

listing 16: operand order pinned by capture (source: lua/samples/ch03_values.lua, lines 109-121)

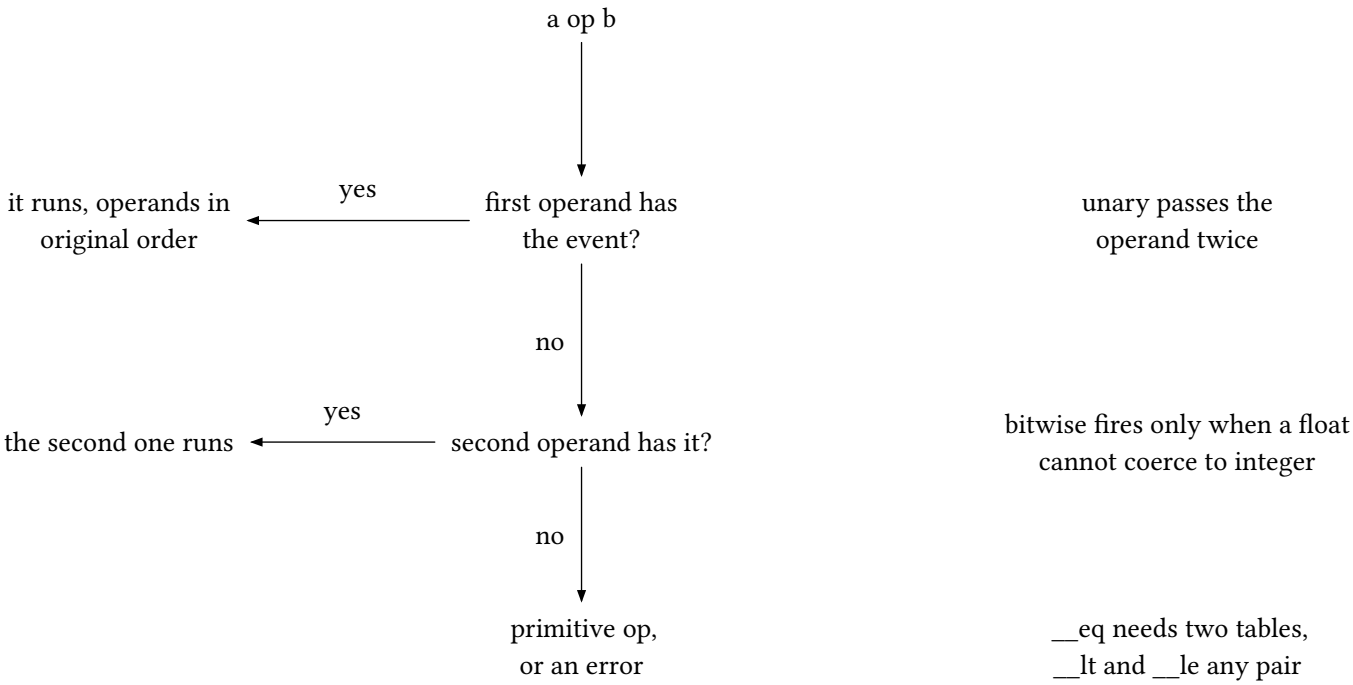


figure 13: metamethod lookup for a binary operation, the first operand's event first

```lua
{name = "bitwise metamethods fire only for non coercible floats", fn = function()
  T.eq(3.0 & 1, 1)
  T.throws(function() return 1.5 | 2 end, "number has no integer representation")
  local mt = {__bor = function() return "bored" end}
  T.eq(setmetatable({}, mt) | 2, "bored")
end},
```

listing 17: the bitwise coercion boundary (source: lua/samples/ch03_values.lua, lines 134-140)

`__eq` is the picky one: it runs only between two tables, or two full userdata, that are not primitively equal. `__lt` and `__le` are wider, any mixed pair qualifies, and both convert the metamethod result to boolean, so a numeric `0` return means true:

```lua
{name = "eq fires only between two tables not primitively equal", fn = function()
  local mt = {__eq = function(a, b) return a.v == b.v end}
  local a = setmetatable({v = 1}, mt)
  local b = setmetatable({v = 1}, mt)
  T.eq(a == b, true)
  T.eq(a == a, true)
  T.eq(a == 1, false)
  T.eq(a == {}, false)
end},

{name = "order metamethods convert to boolean", fn = function()
  local mt = {}
  mt.__lt = function(a, b) return true end
  mt.__le = function(a, b) return 0 end
  local t = setmetatable({}, mt)
  T.eq(t < {}, true)
  T.eq(t ≤ {}, true)
  T.eq(t < 1, true)
  T.throws(function() return {} < {} end, "attempt to compare two table values")
end},
```

listing 18: eq's both tables rule versus order's mixed pairs (source: lua/samples/ch03_values.lua, lines 151-171)

3.5. access and call

__index and __newindex fire only when the access is not a table or the key is absent. The metavalue can be a function, a table, or any value with its own metavalue, and the follow up access is regular, so metatables chain. Assignments routed through __newindex never perform the primitive store, rawset exists for that:

```lua
{name = "index as function, table, and chained", fn = function()
  local calls = {}
  local fn = setmetatable({}, {__index = function(t, k)
    calls[#calls + 1] = k
    return "fn:" .. k
  end})
  T.eq(fn.z, "fn:z")
  T.eq(calls, {"z"})
  local base = {y = "from base"}
  local viaTable = setmetatable({}, {__index = base})
  T.eq(viaTable.y, "from base")
  T.eq(rawget(viaTable, "y"), nil)
  local grand = {z = "from grand"}
  local chained = setmetatable({}, {__index = setmetatable({}, {__index = grand})})
  T.eq(chained.z, "from grand")
end},
```

listing 19: function, table, and chained index metavalue (source: lua/samples/ch03_values.lua, lines 172-188)

```lua
  {name = "newindex routes assignment and rawset bypasses", fn = function()
    local store = {}
    local t = setmetatable({}, {__newindex = function(_, k, v)
      store[k] = v
    end})
    t.a = 1
    T.eq(rawget(t, "a"), nil)
    T.eq(store.a, 1)
    local backing = {}
    local u = setmetatable({}, {__newindex = backing})
    u.b = 2
    T.eq(backing.b, 2)
    rawset(t, "direct", 3)
    T.eq(rawget(t, "direct"), 3)
  end},
```

listing 20: newindex as function and as backing table (source: lua/samples/ch03_values.lua, lines 189-204)

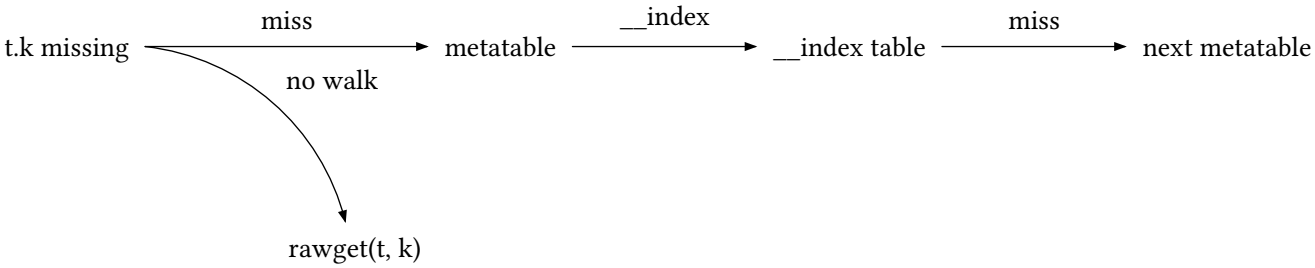


figure 14: a miss walking the __index chain, rawget exits the walk

`__call` is the only multimethod-return metamethod: the callee is prepended to the argument list and every result passes through. And a metavalue does not need to be a function, any callable works, which lets a callable table serve as `__add`:

```lua
  {name = "call prepends the callee and returns everything", fn = function()
    local t = setmetatable({}, {__call = function(self, ...)
      return self, select("#", ...), ...
    end})
    local a, n, x, y = t("one", "two")
    T.eq(a, t)
    T.eq(n, 2)
    T.eq(x, "one")
    T.eq(y, "two")
  end},
```

listing 21: call passes everything through (source: lua/samples/ch03_values.lua, lines 205-215)

```lua
  {name = "metamethods may be any callable value", fn = function()
    local callable = setmetatable({apply = function(_, a, b) return a.v + b end}, {
      __call = function(self, a, b) return self.apply(self, a, b) end,
    })
    local mt = {__add = callable}
    local t = setmetatable({v = 40}, mt)
    T.eq(t + 2, 42)
  end},
}
```

listing 22: a callable table as the add metamethod (source: lua/samples/ch03_values.lua, lines 216-224)

The remaining reserved keys, `__gc`, `__close`, and `__mode`, belong to their own machinery and get chapters later: finalization with the collector, to-be-closed variables, and weak tables.

sources: lua.org manual 5.5 sections 2.1, 2.4, accessed 2026-09-08. Metamethod behavior verified live with lua 5.5.1, 21 tests green through `make verify-lua`.

4. expressions and operators

Operator precedence, lowest to highest: `or`, `and`, comparisons, bor, bxor, band, shifts, concatenation, addition and subtraction, multiplication family, unary operators, exponentiation. Two rules in that list surprise people every time: unary minus binds looser than `^`, so `-2 ^ 2` is `-4`, and `^` is right associative, so `2 ^ 3 ^ 2` is 512. `and` and `or` short circuit and return an operand, not a boolean:

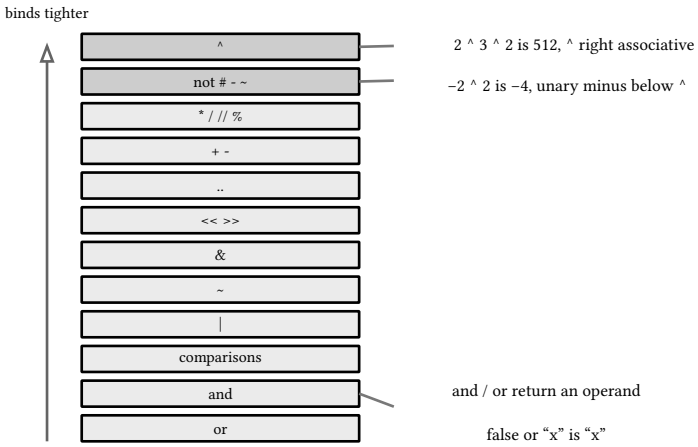


figure 15: the precedence ladder from loosest to tightest, with the two traps

```
{name = "precedence: or under and, power right, unary under power", fn = function()
  T.eq(2 or 3 and 4, 2)
  T.eq(1 + 2 * 3, 7)
  T.eq(-2 ^ 2, -4)
  T.eq(2 ^ -2, 0.25)
  T.eq(2 ^ 3 ^ 2, 512)
end},

{name = "and or return operands, short circuit", fn = function()
  T.eq(false or "x", "x")
  T.eq(nil and 1, nil)
  T.eq(1 and 2, 2)
  local probed = false
  local function side() probed = true return 1 end
  T.eq(false and side(), false)
  T.eq(true or side(), true)
  T.eq(probed, false)
end},
```

listing 23: precedence traps and operand-returning conditionals (source: lua/samples/ch04_expressions.lua, lines 5-22)

`not nil` is the one true boolean producer at that level; `not 1 == 2` parses `(not 1) == 2`, which is false, because unary `not` also binds below comparisons.

4.1. arithmetic

The arithmetic rules divide by operator, not by symmetry. `/` is always float and yields infinities and NaN quietly. `//` and `%` floor toward minus infinity and follow the divisor’s sign, and on integers they reject zero outright while on floats they extend to infinity:

```lua
{name = "floor division rounds toward minus infinity", fn = function()
  T.eq(7 // 2, 3)
  T.eq(-7 // 2, -4)
  T.eq(7 // -2, -4)
  T.eq(-7 // -2, 3)
  T.eq(math.type(7 // 2), "integer")
  T.eq(7.0 // 2, 3.0)
end},

{name = "modulo follows the divisor's sign", fn = function()
  T.eq(7 % 3, 1)
  T.eq(-7 % 3, 2)
  T.eq(7 % -3, -2)
  T.eq(-7 % -3, -1)
  for _, pair in ipairs({{13, 5}, {-13, 5}, {13, -5}, {-13, -5}}) do
    local a, b = pair[1], pair[2]
    T.eq(a % b, a - ((a // b) * b))
  end
end},

{name = "true division is always float, infinity and nan", fn = function()
  T.eq(6 / 3, 2.0)
  T.eq(math.type(6 / 3), "float")
  T.eq(1 / 0, math.huge)
  T.eq(-1 / 0, -math.huge)
  T.eq(0 / 0 == 0 / 0, false)
  T.eq(1.0 // 0, math.huge)
  T.throws(function() return 1 // 0 end, "attempt to divide by zero")
  T.throws(function() return 1 % 0 end, "attempt to perform 'n%0'")
end},
```

listing 24: floor division, modulo sign, division's float world (source: lua/samples/ch04_expressions.lua, lines 30-59)

operator	rule
1. • *	integer wrap, no promotion: maxinteger * 2 is −2
/	always float, infinities and nan arrive quietly
// %	floor toward minus infinity, sign follows the divisor
// 0, % 0	integers reject zero, floats reach inf or nan
^	always float, 2 ^ 10 is 1024.0

figure 16: the arithmetic rules, operator by operator

`^` is always float, so `2 ^ 10` is `1024.0` and a negative base with a fractional exponent is NaN. Integer arithmetic wraps in two's complement, `math.maxinteger * 2` is `-2`, with no promotion to float, ever.

PITFALL: integer zero division has two spellings

Integer `// 0` says `attempt to divide by zero`; integer `% 0` says `attempt to perform 'n%0'`. Both spellings are unchanged since 5.3, and the suite pins both.

4.2. bitwise

Six operators over 64 bit integers only, no float mixing unless the float has an exact integer value. Shifts reverse direction for negative counts, `1 << -1` is `1 >> 1`, and shifting by 64 or more saturates to zero rather than masking the count:

```
{name = "integer arithmetic wraps, no float promotion", fn = function()
  T.eq(math.maxinteger * 2, -2)
  T.eq(math.mininteger * -1, math.mininteger)
end},

{name = "bitwise operators are 64 bit with reversing shifts", fn = function()
  T.eq(0xFF & 0x0F, 15)
  T.eq(~0, -1)
  T.eq(1 << 63, math.mininteger)
  T.eq(1 << 64, 0)
  T.eq(1 << -1, 1 >> 1)
  T.eq(5 >> 1, 2)
  T.eq(0xF0 | 0x0F ~ 0xFF, 0xF0 | (0x0F ~ 0xFF))
end},
```

listing 25: the 64 bit unsigned view and reversing shifts (source: lua/samples/ch04_expressions.lua, lines 67-80)

operator	rule
& | ~	over the 64 bit pattern, integers only
<< >>	negative counts reverse direction, 1 << −1 is 1 >> 1
by 64 or more	saturates to zero, the count is not masked
floats	join only with an exact integer value, 1.5 | 2 errors



minus one is all ones in the unsigned view

figure 17: the six bitwise operators over the 64 bit unsigned view

4.3. comparison and coercion

Mixed integer float comparisons are exact, not approximate: `2 ^ 53 == 9007199254740993` is false, `2 ^ 53 == 9007199254740992` is true, and `math.maxinteger + 0.0` rounds to a float that no longer equals the integer it came from. Strings compare in byte order, and a number never compares against a string, `1 == "1"` is false and `1 < "a"` is an error.

Numbers coerce into concatenation with the same format `tostring` uses, integral floats keeping their `.0`. Strings coerce into arithmetic when they read as numbers, hex and exponent forms included, keeping the subtype the numeral implies. Bitwise operators refuse strings entirely. 5.5's error messages for failed string arithmetic name the operation and both operand types:

```
{name = "strings compare in byte order, never against numbers", fn = function()
  T.eq("abc" < "abd", true)
  T.eq("\255" > "z", true)
  T.throws(function() return 1 < "a" end, "attempt to compare number with string")
  T.eq(1 == "1", false)
end},

{name = "concatenation formats floats like tostring", fn = function()
  T.eq("n=" .. 42, "n=42")
  T.eq("" .. 1.5, "1.5")
  T.eq("" .. 1.0, "1.0")
  T.eq("" .. 1e15, "1e+15")
  T.eq("" .. 1 / 3, "0.33333333333333331")
  T.throws(function() return {} .. "x" end, "attempt to concatenate")
end},

{name = "arithmetic coerces numeric strings, bitwise does not", fn = function()
  T.eq("10" + 5, 15)
  T.eq(math.type("10" + 5), "integer")
  T.eq("0x10" + 0, 16)
  T.eq("1e2" + 0, 100.0)
  T.eq("10" * "2", 20)
  T.throws(function() return "a" + 1 end, "attempt to add a 'string' with a 'number'")
  T.throws(function() return -"a" end, "attempt to unm a 'string' with a 'string'")
  T.throws(function() return "1" & 1 end, "bitwise operation on a string value")
end},
```

listing 26: concat formats, string arithmetic, and the new message shape (source: lua/samples/ch04_expressions.lua, lines 89-114)

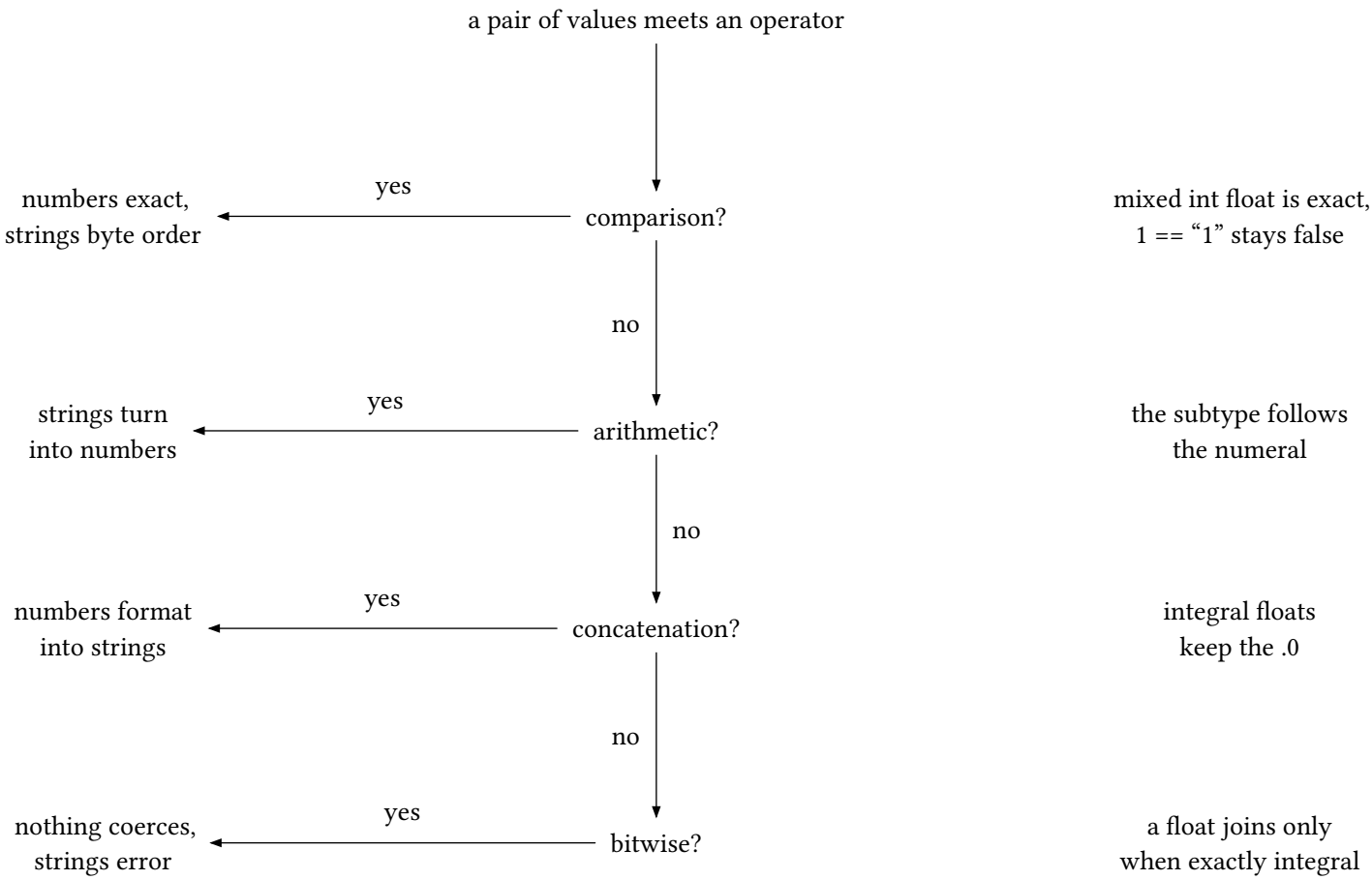


figure 18: which operators coerce which operand types, and where the boundary is

4.4. constructors and length

A constructor takes positional items, `name = value` sugar for string keys, `[exp] = value` for computed keys, and nested constructors as items, with commas or semicolons freely as separators. The assignment order inside a constructor is unspecified, and the observed behavior is that keyed fields assign before positional ones, so a positional item wins a key collision in either textual order:

```
  {name = "table constructors take every key form", fn = function()
    local k = "dyn"
    local t = {1, 2, 3; x = 10, [k] = "k", [1 + 1] = "two", {nested = true}}
    T.eq(t[1], 1)
    T.eq(t[2], 2)
    T.eq(t[3], 3)
    T.eq(t.x, 10)
    T.eq(t.dyn, "k")
    T.eq(t[4].nested, true)
    local u = {[2] = "two", 1, 2}
    T.eq(u[2], 2)
    local trailing = {1, 2, 3,}
    T.eq(#trailing, 3)
  end},
```

listing 27: every key form, and positional winning the collision (source: lua/samples/ch04_expressions.lua, lines 116-129)

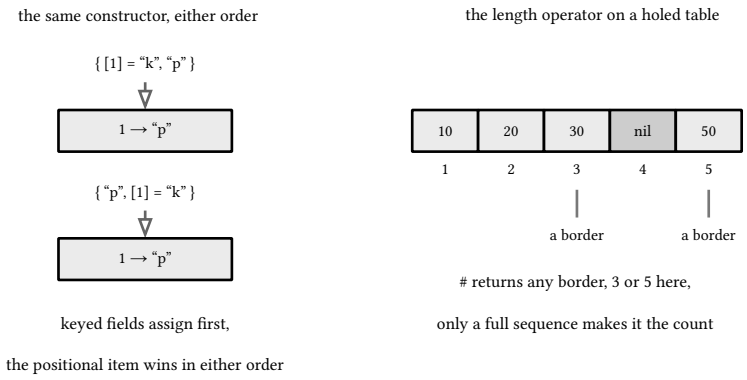


figure 19: constructor collisions and the length border

`#` is byte count on strings. On tables it is a border, any positive index where the next index is nil, which on a contiguous sequence is the count but on a holed table is only guaranteed to be one of the borders:

```
  {name = "length is bytes on strings, a border on tables", fn = function()
    T.eq(#"a\0b", 3)
    T.eq(#{10, 20, 30}, 3)
    local holed = {10, 20, 30}
    holed[5] = 50
    T.ok(#holed == 3 or #holed == 5, "# is a border, got " .. #holed)
  end},
}
```

listing 28: length as border, not count (source: lua/samples/ch04_expressions.lua, lines 131-138)

sources: lua.org manual 5.5 sections 3.4.1 through 3.4.7, accessed 2026-09-08. Error message wording verified live with lua 5.5.1 against the 5.4 spellings, 15 tests green through `make verify-lua`.

5. statements and scope

Assignment is a list operation: lua evaluates every value on the right before storing anything on the left, which makes the classic swap a one liner, and adjusts lists to the target's length exactly the way calls adjust arguments, padding with nil and dropping extras:

```
{name = "multiple assignment evaluates values first", fn = function()
  local a, b = 1, 2
  a, b = b, a
  T.eq(a, 2)
  T.eq(b, 1)
  local t = {n = 0}
  local function bump() t.n = t.n + 1 return t.n end
  local x, y = bump(), bump()
  T.eq(x, 1)
  T.eq(y, 2)
end},

{name = "assignment lists adjust like parameter lists", fn = function()
  local a, b, c = 1, 2
  T.eq(c, nil)
  a, b = 10, 20, 30
  T.eq(b, 20)
  local function two() return "p", "q" end
  local u, v = two()
  T.eq(u, "p")
  T.eq(v, "q")
  local w, x, y = two()
  T.eq(y, nil)
```

listing 29: swap, call-shaped adjustment, multiple returns spread and truncate (source: lua/samples/ch05_statements.lua, lines 5-27)

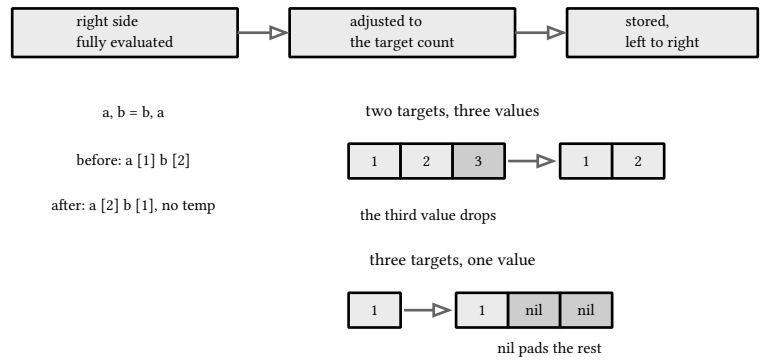


figure 20: assignment as a list operation, evaluate, adjust, then store

A `local` declaration's scope begins after the statement itself, so `local x = x + 1` reads the outer x and defines a new one. Blocks are `do end`, function bodies, and control structure bodies, and the chunk is a block.

5.1. attributes

`local` takes `<const>` and `<close>`. `<const>` is a compile time lock: the assignment is a syntax error, the same mechanism 5.5 now applies to for loop control variables and global declarations. `<close>` registers a to-be-closed variable for the block's exit: reverse declaration order, skipped quietly for nil and false, and an error at scope exit for anything without a callable `__close`:

```
  {name = "const attribute is a compile time lock", fn = function()
    T.eq(load("do local C <const> = 1; C = 2 end"), nil)
    T.ok(select(2, load("do local C <const> = 1; C = 2 end")):find("attempt to assign to const variable
'C'", 1, true))
  end},

  {name = "close attribute runs in reverse declaration order", fn = function()
    local log = {}
    do
      local a <close> = setmetatable({}, {__close = function() log[#log + 1] = "a" end})
      local b <close> = setmetatable({}, {__close = function() log[#log + 1] = "b" end})
      local n <close> = nil
      log[#log + 1] = "body"
    end
    T.eq(log, {"body", "b", "a"})
  end},

  {name = "close rejects values without a callable close", fn = function()
    T.throws(function() load("do local x <close> = 5 end")() end,
      "variable 'x' got a non-closable value")
    T.throws(function()
      load("do local x <close> = setmetatable({}, {__close = 3}) end")()
    end, "attempt to call a number value")
  end},
```

listing 30: closing order, the nil exemption, both failure modes (source: lua/samples/ch05_statements.lua, lines 39-61)

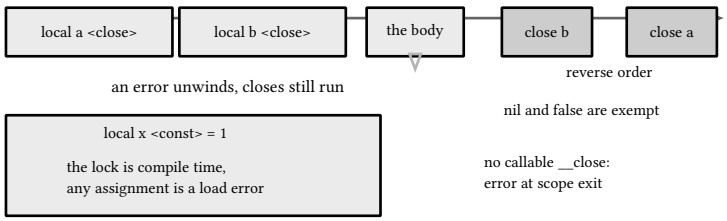


figure 21: the life of a block, const at compile time, closes at exit in reverse

NOTE: closes run on errors too

To-be-closed variables close when the block exits for any reason, including an error unwinding through it, which is what makes `<close>` the right tool for file and lock lifetimes. Chapter 7 revisits it from the coroutine side, where closing a suspended coroutine is how you cancel it.

5.2. loops

`while` tests before, `repeat` tests after, and the repeat test sees locals declared in the body, the one place a condition can reference something the body created. The numeric `for` evaluates start, limit, and step exactly once, before the first iteration, holds the loop variable as read-only in 5.5, and counts with integer arithmetic that does not wrap past `math.maxinteger`:

```
{name = "repeat conditions see the body's locals", fn = function()
  local rounds = 0
  repeat
    rounds = rounds + 1
    local done = rounds ≥ 3
  until done
  T.eq(rounds, 3)
end},

{name = "numeric for evaluates its bounds exactly once", fn = function()
  local calls = 0
  local function once() calls = calls + 1 return calls end
  local iterations = 0
  for i = once(), 3 * once(), once() do iterations = iterations + 1 end
  T.eq(calls, 3)
  T.eq(iterations, 2)
end},

{name = "the for control variable is read only in 5.5", fn = function()
  local f, e = load("for i = 1, 3 do i = i + 1 end")
  T.eq(f, nil)
  T.ok(e:find("attempt to assign to const variable 'i'", 1, true), e)
end},

{name = "numeric for near maxinteger terminates, floats step", fn = function()
  local n = 0
  for i = math.maxinteger - 2, math.maxinteger do n = n + 1 end
  T.eq(n, 3)
  local seen = {}
  for i = 1, 2, 0.25 do seen[#seen + 1] = i end
  T.eq(#seen, 5)
  T.eq(seen[5], 2.0)
end},
```

listing 31: repeat's late scope, single evaluation, read-only control, no wrap loop, float steps (source: lua/samples/ch05_statements.lua, lines 63-96)

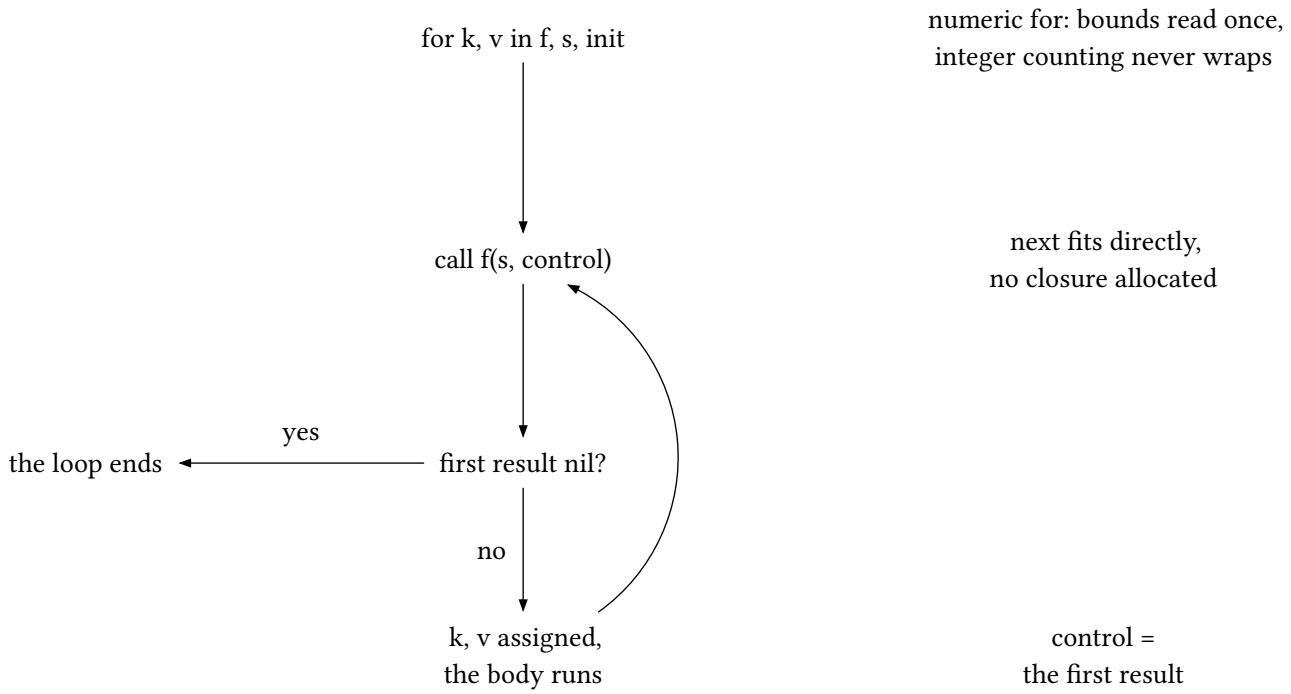


figure 22: the generic for as a call loop, the first nil terminates it

The generic `for` calls a function repeatedly with a state and a control value, stopping at the first nil. `next` fits directly, so `for k, v in next, t` iterates without allocating a closure, and a hand written iterator gets one last call that returns nil:

```lua
{name = "generic for over next directly and over closures", fn = function()
  local t = {10, 20, 30}
  local sum = 0
  for i, v in next, t do sum = sum + v end
  T.eq(sum, 60)
  local calls = 0
  local function iter(s, control)
    calls = calls + 1
    if control < s then return control + 1 end
  end
  local hits = 0
  for _ in iter, 3, 0 do hits = hits + 1 end
  T.eq(hits, 3)
  T.eq(calls, 4)
  local none = 0
  for _ in function() return nil end do none = none + 1 end
  T.eq(none, 0)
end},
```

listing 32: stateless next, closure iterator, nil terminates (source: lua/samples/ch05_statements.lua, lines 97-115)

When the `in` expression list carries a fourth value, the loop closes it as a to-be-closed variable on exit, break and error included, which is the half of the protocol `io.lines` rides in chapter 12.

5.3. goto and returns

`goto` jumps to a label forward in the same block, cannot enter the scope of a local, and powers the continue idiom lua otherwise lacks:

```lua
{name = "goto skips forward, continue idiom works", fn = function()
  local count = 0
  for i = 1, 3 do
    if i == 2 then goto continue end
    count = count + i
    ::continue::
  end
  T.eq(count, 4)
end},

{name = "goto cannot jump into a local's scope", fn = function()
  local f, e = load("do goto skip; local y = 1; ::skip:: print(y) end")
  T.eq(f, nil)
  T.ok(e:find("jumps into the scope of 'y'", 1, true), e)
end},
```

listing 33: continue by label, and the scope boundary error (source: lua/samples/ch05_statements.lua, lines 116-131)

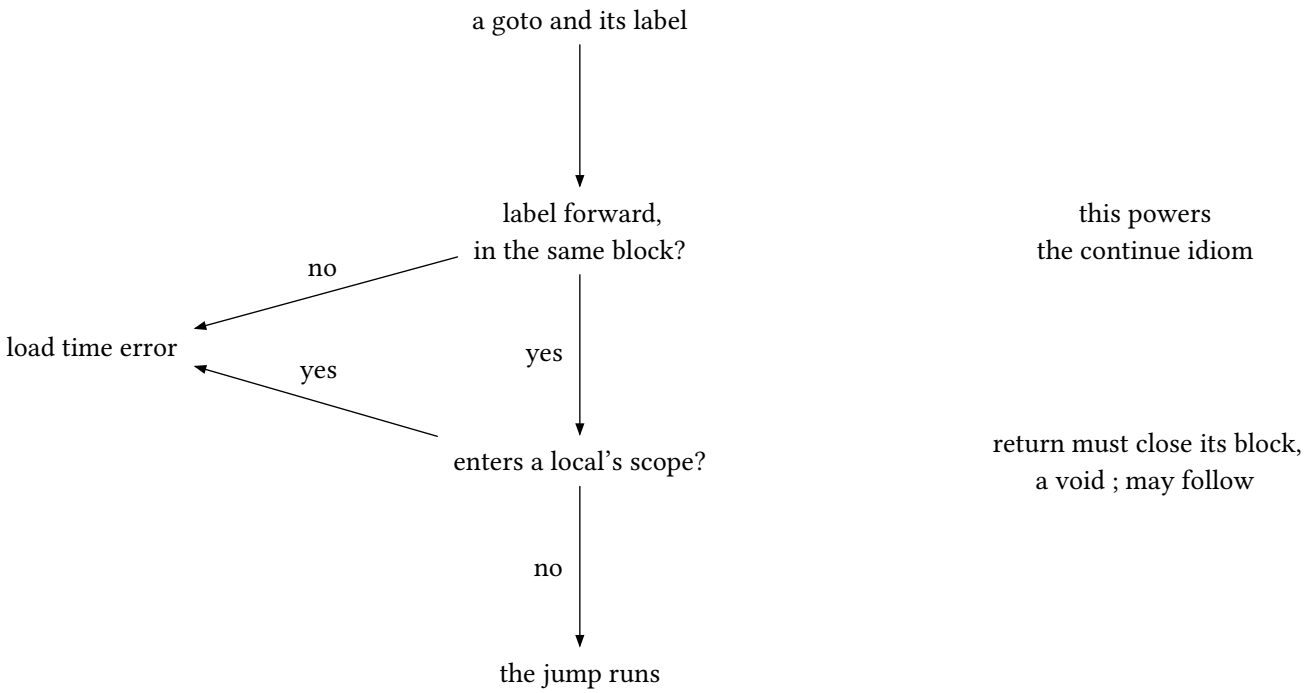


figure 23: goto validity, and return closing its block

`return` must close its block, with an optional semicolon after it; empty `;` statements are void anywhere. `break` exits the innermost loop. Both are statements, not expressions, and the grammar enforces the shape at load time.

```lua
{name = "return ends the block, semicolons are void", fn = function()
  T.eq(load("return 1;")(), 1)
  T.eq(load(";;; return 7")(), 7)
  T.eq(load("return 1 return 2"), nil)
  local function early(n)
    if n > 0 then return "pos" end
    return "nonpos"
  end
  T.eq(early(5), "pos")
  T.eq(early(-5), "nonpos")
end},
```

listing 34: return as block end, void semicolons (source: lua/samples/ch05_statements.lua, lines 132-143)

sources: lua.org manual 5.5 sections 2.2, 3.2, 3.3, accessed 2026-09-08. Close and goto error wording verified live with lua 5.5.1, 15 tests green through `make verify-lua`.

6. functions

Three definition sugars sit on one runtime fact, functions are values. `function f() end` assigns to the visible `f`, `local function f` expands to `local f; f = function` so the body can recurse, and 5.5 adds the third form, `global function f`, which declares then initializes the global, so redefining it is the initialization error:

```
{name = "local function supports recursion, global function is sugar", fn = function()
  local function fact(n) if n < 2 then return 1 end return n * fact(n - 1) end
  T.eq(fact(5), 120)
  local chunk = load([[
    global function greet() return "hi" end
    return greet()
  ]])
  T.eq(chunk(), "hi")
  local again = load("global function f() end; global function f() end")
  T.throws(again, "global 'f' already defined")
end},
```

listing 35: recursion through the local sugar, and the global redefinition error (source: lua/samples/ch06_functions.lua, lines 5-15)

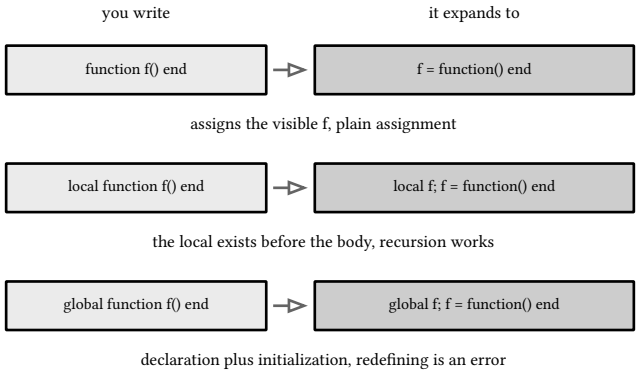


figure 24: the three definition sugars and what each expands to

The colon defines methods: `function t.a.b.c:m(...)` sugar-assigns `t.a.b.c.m` with `self` as the first parameter, and the call site `obj:m(x)` passes `obj` implicitly.

6.1. multiple results

Lua functions return lists, not tuples. The list expands only in tail positions: the last argument of a call, the last field of a constructor, the last element of a `return`. Anywhere else the call truncates to one value, and parentheses force that truncation explicitly:

```lua
{name = "multiple results expand only in last position", fn = function()
  local function three() return 1, 2, 3 end
  T.eq(#{three()}, 3)
  T.eq(#{three(), 10}, 2)
  T.eq(select("#", three()), 3)
  T.eq(select("#", (three())), 1)
  local a, b = three()
  T.eq(a, 1)
  T.eq(b, 2)
  local t = {n = 0, m = 0}
  local function sink(x, y, z) t.n, t.m = x, y end
  sink(10, three())
  T.eq(t.n, 10)
  T.eq(t.m, 1)
end},

{name = "parentheses truncate one call, not the last one", fn = function()
  local function three() return 1, 2, 3 end
  T.eq((three()), 1)
  local middle = {(three()), three()}
  T.eq(#middle, 4)
  T.eq(middle, {1, 1, 2, 3})
end},
```

listing 36: expansion rules and the parenthesis cut (source: lua/samples/ch06_functions.lua, lines 27-49)

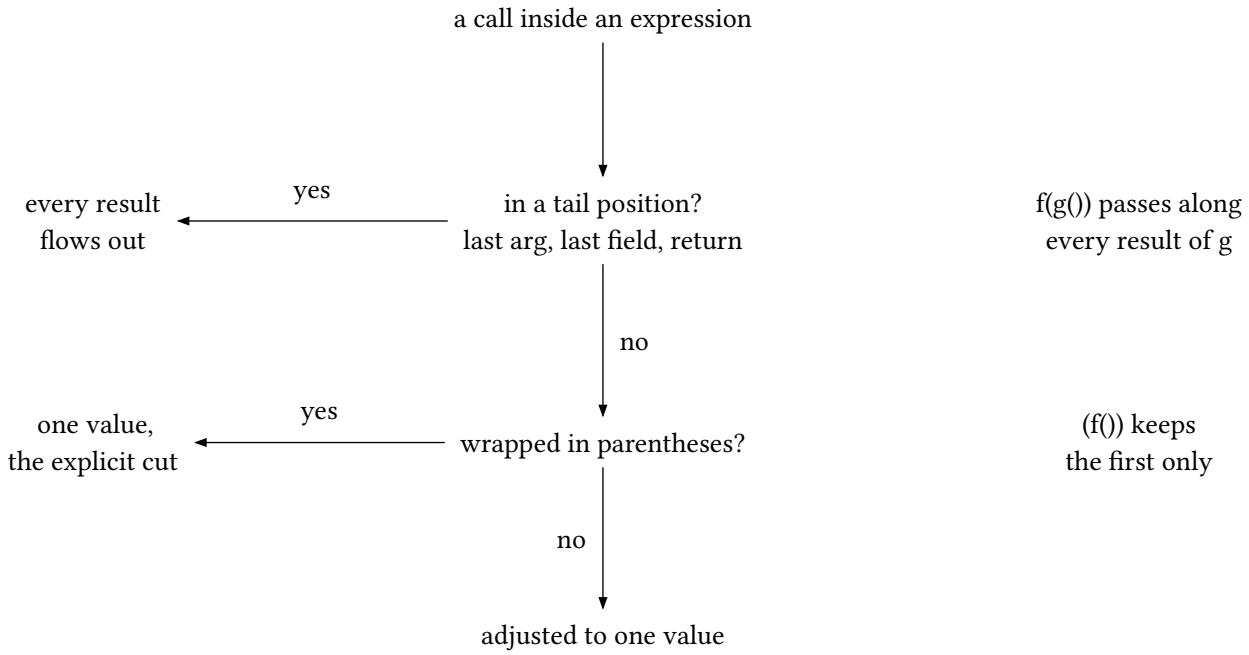


figure 25: when a call's results expand, and when they are cut

6.2. the 5.5 vararg model

This is the release's second language change. A variadic function no longer exposes only an expression: the extra arguments land in a vararg table whose field `n` is the count, and the parameter list can name it, `function g(a, b, ...rest)`. The name binds a read-only local; without a name, only the `...` expression reaches the table. The manual's own mapping table, with a multret call in the last argument position, is pinned directly:

```lua
{name = "a named vararg table carries the count at n", fn = function()
  local function g(a, b, ...rest)
    return a, b, type(rest), rest.n, rest[1], rest[2]
  end
  T.eq({g(3, 4, 5, 8)}, {3, 4, "table", 2, 5, 8})
  T.eq({g(3, 4)}, {3, 4, "table", 0, nil, nil})
end},

{name = "the manual's argument mapping table", fn = function()
  local function r() return 1, 2, 3 end
  local function g(a, b, ...rest)
    return a, b, rest.n, rest[1], rest[2]
  end
  T.eq({g(5, r())}, {5, 1, 2, 2, 3})
  local function f2(a, b) return a, b end
  T.eq({f2(r())}, {1, 2})
end},
```

listing 37: named vararg table, count at n, and the manual's mapping (source: lua/samples/ch06_functions.lua, lines 51-67)

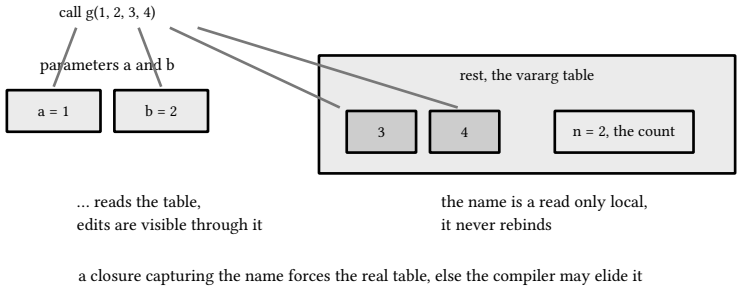


figure 26: where the extra arguments land in the 5.5 vararg model

Because the extras are a real table, code can edit it, and the vararg expression reads what the table now holds:

```lua
{name = "the vararg expression expands like multiple returns", fn = function()
  local function count(...) return select("#", ...) end
  T.eq(count(1, 2, 3), 3)
  T.eq(count(), 0)
  local function pass(...) return ... end
  T.eq({pass("x", "y")}, {"x", "y"})
  T.eq(select("#", pass()), 0)
end},

{name = "mutating the vararg table changes the vararg expression", fn = function()
  local function edited(...xs)
    xs[1] = 99
    xs.n = 1
    return ...
  end
  T.eq({edited(1, 2, 3)}, {99})
  local function trimmed(...xs)
    xs.n = 1
    return select("#", ...), ...
  end
  T.eq({trimmed(1, 2, 3)}, {1, 1})
end},

{name = "the vararg name is a read-only local", fn = function()
  local f, e = load("local function h(...xs) xs = {} end")
  T.eq(f, nil)
  T.ok(e:find("attempt to assign to const variable 'xs'", 1, true), e)
end},
```

listing 38: mutation through the table, and the compile time name lock (source: lua/samples/ch06_functions.lua, lines 69-96)

WARNING: the table is observable but not guaranteed

When the vararg table is anonymous, or the name is only ever used as `t[exp]` or `t.id` and never captured by a nested closure, the compiler skips building an actual table and compiles accesses to the internal vararg data. Capturing the name into a closure, as the escape test does, forces the real table. Programs should treat the table as a read window unless they deliberately mutate it.

6.3. closures and tail calls

A closure pairs a function prototype with the locals it captured. Each execution of a `local` creates a fresh variable, so the loop's closures each own their `own` while sharing the outer `shared`, and two counters from the same factory are independent:

```lua
{name = "named vararg tables can escape into closures", fn = function()
  local function capture(...xs)
    return function() return xs.n, xs[xs.n] end
  end
  local get = capture("a", "b", "c")
  T.eq({get()}, {3, "c"})
end},

{name = "closures share or capture per declared local", fn = function()
  local shared = 100
  local fns = {}
  for i = 1, 3 do
    local own = i
    fns[i] = function() return shared + own end
  end
  T.eq(fns[1](), 101)
  T.eq(fns[3](), 103)
  local function counter()
    local n = 0
    return function() n = n + 1 return n end, function() return n end
  end
  local inc, read = counter()
  local inc2, read2 = counter()
  inc()
  inc()
  T.eq(read(), 2)
  T.eq(read2(), 0)
end},
```

listing 39: per-declaration upvalues, shared versus independent (source: lua/samples/ch06_functions.lua, lines 98-125)

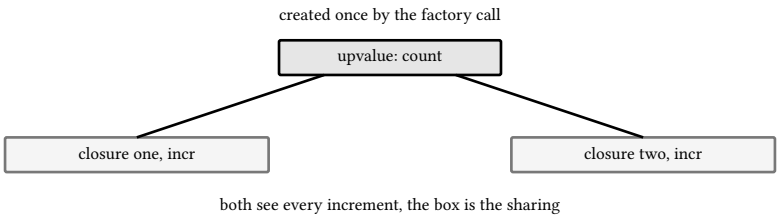


figure 27: two closures from one factory, one upvalue box between them

A proper tail call, `return f(...)`, reuses the frame, so a tail recursive loop of any depth runs in constant stack. A non tail recursion still consumes stack, but that stack is bounded by the lua stack's million slots, not by the 200 c call budget, which only counts nested c calls, so plain recursion runs hundreds of thousands deep before the overflow error:

```lua
{name = "proper tail calls do not grow the stack", fn = function()
  local function loop(n)
    if n == 0 then return "done" end
    return loop(n - 1)
  end
  T.eq(loop(200000), "done")
end},

{name = "non tail recursion is bounded by the lua stack size", fn = function()
  local function sum(n)
    if n == 0 then return 0 end
    return n + sum(n - 1)
  end
  T.eq(sum(100), 5050)
  T.eq(pcall(sum, 300000), true)
  local ok, err = pcall(sum, 1000000)
  T.eq(ok, false)
  T.ok(tostring(err):find("stack overflow"), err)
```

listing 40: constant stack tail loop, deep non tail recursion, the eventual overflow (source: lua/samples/ch06_functions.lua, lines 127-144)

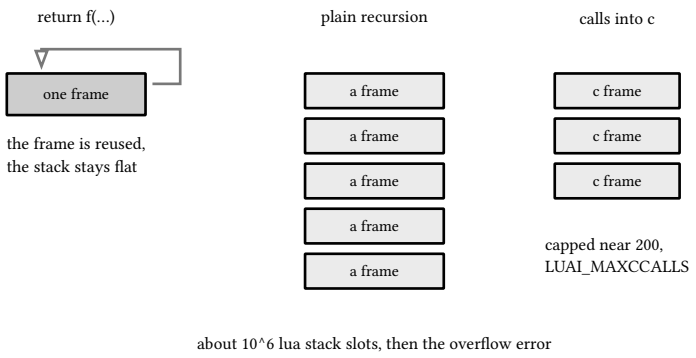


figure 28: tail calls reuse the frame, plain recursion pays a frame per level

sources: lua.org manual 5.5 sections 2.2, 3.4.11, 3.4.12, accessed 2026-09-08. Vararg and recursion limits verified live with lua 5.5.1, LUAI_MAXCCALLS confirmed at 200 in ldo.h for C calls only, 13 tests green through `make verify-lua`.

7. coroutines

A coroutine is a value of type thread, a collaborative execution context with its own stack. `create` only builds it, `resume` runs it, `yield` suspends it, and the status field walks `suspended`, `running`, `suspended`, `dead` across that life. A coroutine that resumes another is `normal` while the callee runs, and the main thread reports itself through `coroutine.running`'s second result:

```
{name = "status walks suspended running suspended dead", fn = function()
  local seen = {}
  local co
  co = coroutine.create(function()
    seen[#seen + 1] = coroutine.status(co)
    coroutine.yield()
    return "done"
  end)
  T.eq(coroutine.status(co), "suspended")
  coroutine.resume(co)
  seen[#seen + 1] = coroutine.status(co)
  T.eq({coroutine.resume(co)}, {true, "done"})
  seen[#seen + 1] = coroutine.status(co)
  T.eq(seen, {"running", "suspended", "dead"})
  T.eq({coroutine.resume(co)}, {false, "cannot resume dead coroutine"})
end},

{name = "resume and yield pass values both ways", fn = function()
  local co = coroutine.create(function(a, b)
    local c = coroutine.yield(a + b)
    local d, e = coroutine.yield(c * 2)
    return d - e
  end)
  T.eq({coroutine.resume(co, 3, 4)}, {true, 7})
  T.eq({coroutine.resume(co, 10)}, {true, 20})
  T.eq({coroutine.resume(co, 9, 4)}, {true, 5})
end},
```

listing 41: the status walk, dead resume (source: lua/samples/ch07_coroutines.lua, lines 5-31)

```
{name = "running distinguishes the main thread", fn = function()
  local mainthread, ismain = coroutine.running()
  T.eq(ismain, true)
  local inner
  inner = coroutine.create(function()
    local t, m = coroutine.running()
    return t == inner, m
  end)
  T.eq({coroutine.resume(inner)}, {true, true, false})
end},

{name = "a resumer is normal while its callee runs", fn = function()
  local outer, inner
  local seen
  inner = coroutine.create(function()
    seen = coroutine.status(outer)
  end)
  outer = coroutine.create(function()
    coroutine.resume(inner)
  end)
  coroutine.resume(outer)
  T.eq(seen, "normal")
end},
```

listing 42: normal status of a resumer (source: lua/samples/ch07_coroutines.lua, lines 33-55)

Values flow both ways: extra `resume` arguments enter the body and then each `yield`, and each `yield`'s arguments come back out of the matching `resume`:

```
{name = "resume and yield pass values both ways", fn = function()
  local co = coroutine.create(function(a, b)
    local c = coroutine.yield(a + b)
    local d, e = coroutine.yield(c * 2)
    return d - e
  end)
  T.eq({coroutine.resume(co, 3, 4)}, {true, 7})
  T.eq({coroutine.resume(co, 10)}, {true, 20})
  T.eq({coroutine.resume(co, 9, 4)}, {true, 5})
end},
```

listing 43: the resume yield channel (source: lua/samples/ch07_coroutines.lua, lines 22-31)

7.1. errors and the un-unwound stack

An unprotected error terminates the coroutine without unwinding its stack, on purpose, so the debug library can inspect the corpse. The consequence for to-be-closed variables is that they wait: nothing runs at error time, and the single call to `coroutine.close` both performs the closing and returns false plus the original error. Error objects cross the boundary by identity, tables included:

```lua
{name = "errors do not unwind, close performs the cleanup", fn = function()
  local log = {}
  local co = coroutine.create(function()
    local guard <close> = setmetatable({}, {__close = function() log[#log + 1] = "g" end})
    error("boom")
  end)
  local ok, err = coroutine.resume(co)
  T.eq(ok, false)
  T.ok(tostring(err):find("boom"), tostring(err))
  T.eq(log, {})
  T.eq(coroutine.status(co), "dead")
  local cok, cerr = coroutine.close(co)
  T.eq(cok, false)
  T.ok(tostring(cerr):find("boom"), tostring(cerr))
  T.eq(log, {"g"})
end},

{name = "non string error objects survive the resume boundary", fn = function()
  local sentinel = {code = 42}
  local co = coroutine.create(function() error(sentinel) end)
  local ok, got = coroutine.resume(co)
  T.eq(ok, false)
  T.eq(got, sentinel)
end},
```

listing 44: deferred cleanup, the close error report, object identity (source: lua/samples/ch07_coroutines.lua, lines 57-80)

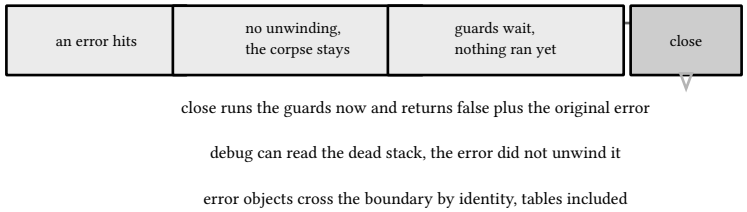


figure 29: an error kills the coroutine without unwinding, close settles it later

7.2. close as cancellation

`coroutine.close` accepts dead, suspended, or the running coroutine, and its default argument is the running one. On a suspended coroutine it runs the guards in reverse order and reports true. On the running coroutine it is a cancel: it does not return, not even through `pcall`, the pending guards run, the code after the call is unreachable, and the `resume` that started the body returns plain `true`:

```lua
{name = "self close cancels the coroutine without unwinding errors", fn = function()
  local log = {}
  local co
  co = coroutine.create(function()
    local guard <close> = setmetatable({}, {__close = function() log[#log + 1] = "g" end})
    pcall(coroutine.close, coroutine.running())
    log[#log + 1] = "unreachable"
  end)
  T.eq({coroutine.resume(co)}, {true})
  T.eq(log, {"g"})
  T.eq(coroutine.status(co), "dead")
end},
```

listing 45: self close as a non catchable cancel (source: lua/samples/ch07_coroutines.lua, lines 96-107)

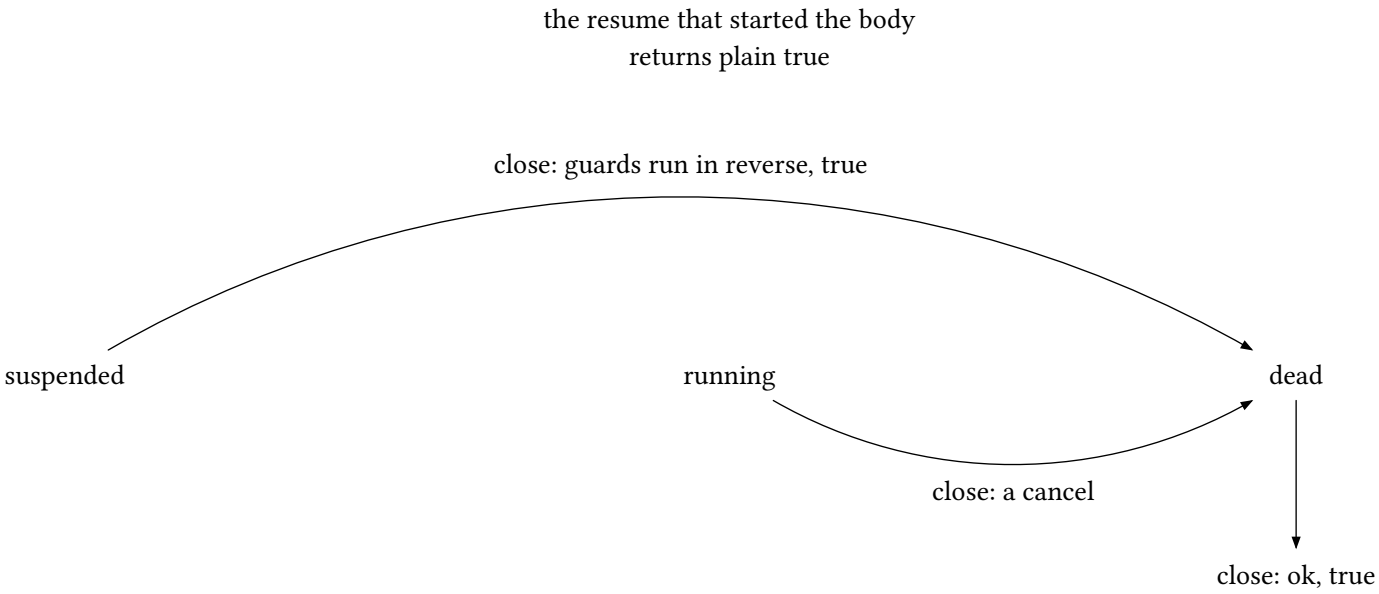


figure 30: close over each coroutine state

`coroutine.wrap` trades `resume`'s error tuple for exceptions: errors propagate to the caller and the coroutine gets closed on the way, guards and all:

```
{name = "wrap propagates errors and closes the coroutine", fn = function()
  local log = {}
  local f = coroutine.wrap(function()
    local guard <close> = setmetatable({}, {__close = function() log[#log + 1] = "g" end})
    coroutine.yield("first")
    error("wrapped")
  end)
  T.eq(f(), "first")
  local ok, err = pcall(f)
  T.eq(ok, false)
  T.ok(tostring(err):find("wrapped"), tostring(err))
  T.eq(log, {"g"})
end},
```

listing 46: wrap closing the coroutine while propagating (source: lua/samples/ch07_coroutines.lua, lines 109-121)

7.3. coroutines as iterators

`wrap` returns a plain function, which is exactly what a generic `for` wants, so a producer loop becomes an iterator with no state plumbing:

```lua
{name = "wrap drives a generic for producer", fn = function()
  local function range(n)
    return coroutine.wrap(function()
      for i = 1, n do coroutine.yield(i) end
    end)
  end
  local sum = 0
  for i in range(4) do sum = sum + i end
  T.eq(sum, 10)
end},
```

listing 47: range as a wrapped producer (source: lua/samples/ch07_coroutines.lua, lines 123-132)

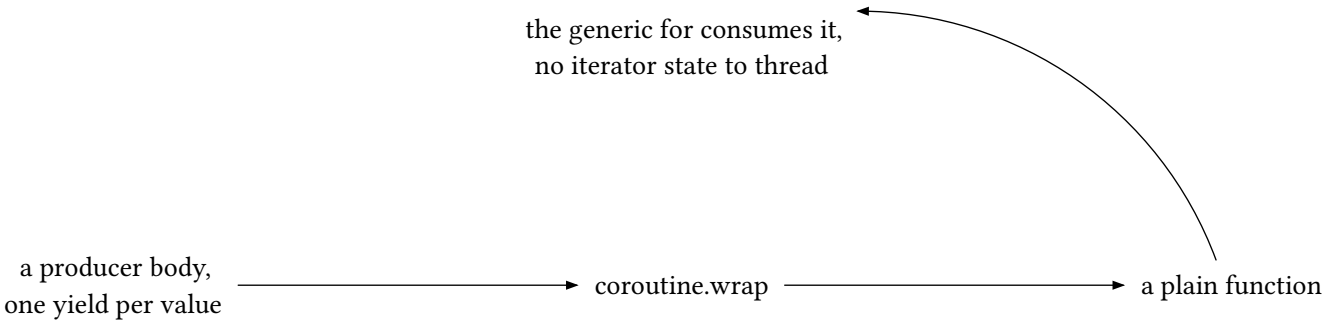


figure 31: wrap turns a producer into the function the generic for wants

7.4. yielding through C

A yield inside nested Lua calls resumes correctly, and 5.4's deeper guarantee holds in 5.5: yields cross `pcall` bodies and metamethods. An `__index` metamethod can suspend the table access mid-flight and the resumed value becomes the indexing result:

```lua
{name = "yield crosses pcall and __index metamethods", fn = function()
  local co = coroutine.wrap(function()
    local ok, v = pcall(function() return coroutine.yield("from pcall") end)
    return ok, v
  end)
  T.eq(co(), "from pcall")
  T.eq({co("resumed")}, {true, "resumed"})

  local t = setmetatable({}, {__index = function(_, k) return coroutine.yield("need " .. k) end})
  local getter = coroutine.create(function() return t.price end)
  T.eq({coroutine.resume(getter)}, {true, "need price"})
  T.eq({coroutine.resume(getter, 42)}, {true, 42})
end},
```

listing 48: yield inside pcall and inside a metamethod (source: lua/samples/ch07_coroutines.lua, lines 134-146)

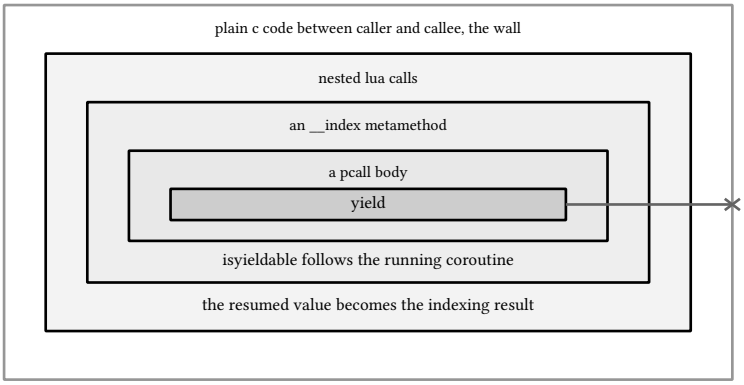


figure 32: the boundaries a yield crosses, and the one it cannot

The state machine of the whole system is small enough to draw:

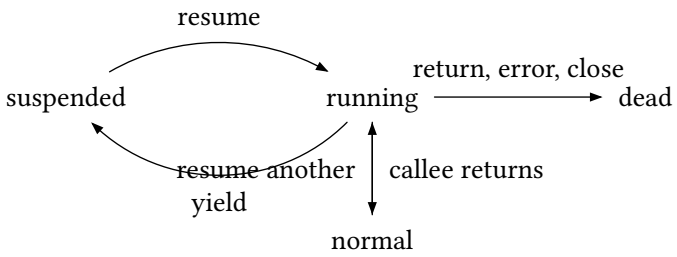


figure 33: coroutine states

sources: lua.org manual 5.5 sections 2.6 and 6.3, accessed 2026-09-08. Close and self close semantics verified live with lua 5.5.1, 12 tests green through `make verify-lua`.

8. environments, load, gc

There is no global scope, only a variable named `_ENV`. Every free name `var` compiles to `_ENV.var`, every chunk is compiled inside an external local named `_ENV`, and every function captures whichever `_ENV` was visible where it was defined. Shadow `_ENV` with a local and the code inside that block changes worlds, imports and all:

```
{name = "free names resolve through the visible _ENV", fn = function()
  local log = {}
  do
    local _ENV = {print = function(...) log[#log + 1] = table.concat({...}, " ") end, x = 10}
    x = x + 5
    print(x)
  end
  T.eq(log, {"15"})
end},
```

listing 49: a local env variable redirects every free name in the block (source: lua/samples/ch08_environments.lua, lines 5-13)

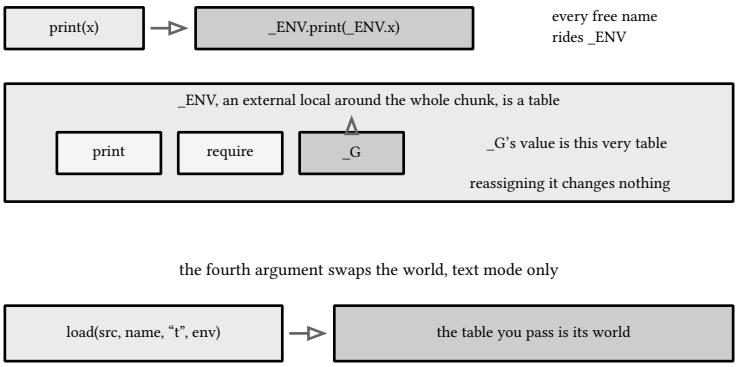


figure 34: how free names really resolve, and what load's fourth argument swaps

`_G` is not the mechanism. It is an ordinary global whose value happens to be the table the interpreter booted with, so reassigning it changes nothing about how globals resolve, and a sandboxed chunk cannot escape through it because it only sees what its environment table contains:

```
{name = "_G is a global, not the environment mechanism", fn = function()
  local old = _G
  _G = {z = 1}
  w = 2
  T.eq(old.w, 2)
  T.eq(rawget(old, "z"), nil)
  _G = old
  T.eq(_G.w, 2)
end},

{name = "load takes the environment as its fourth argument", fn = function()
  T.eq(load("return type(print)", "c", "t", {type = type, print = print})(), "function")
  T.eq(load("return type(print)", "c", "t", {type = type})(), "nil")
  T.eq(load("return io")() == io, true)
end},

{name = "a sandbox sees only what it is handed", fn = function()
  local sandbox = {string = string, type = type}
  local f = load("return string.rep('ab', 3) .. type(io)", "c", "t", sandbox)
  T.eq(f(), "abababnil")
end},

{name = "global declarations assign into the current env", fn = function()
  local env = {}
  local f = load("global x; x = 41; x = x + 1", "c", "t", env)
  f()
  T.eq(env.x, 42)
end},
```

listing 50: the global table as plain data, the load env argument, and a sandbox with no io (source: lua/samples/ch08_environments.lua, lines 15-43)

The 5.5 global declaration compiles into this same picture: it is syntactic only, the assignments still land in whatever `_ENV` is current, which the environment-injected declaration test pins. For loading untrusted code the working set is `load(src, name, "t",
env)`, text mode only, explicit environment, no bytecode accepted.

8.1. the collector

5.5's fourth headline change is here: major collections in generational mode are themselves incremental, and the default mode is now generational, where 5.4 defaulted to incremental. The mode options report the mode they replaced, and 5.5 removed their argument taking: extra arguments to `"incremental"` and `"generational"` are silently ignored rather than setting parameters, which is the incompatibility chapter's library item:

```
{name = "the default mode is generational, switches report previous", fn = function()
  T.eq(collectgarbage("incremental"), "generational")
  T.eq(collectgarbage("generational"), "incremental")
end},

{name = "extra arguments to mode options are ignored now", fn = function()
  collectgarbage("incremental")
  T.eq({collectgarbage("generational", 1, 2, 3)}, {"incremental"})
end},
```

listing 51: generational by default, switches report the previous mode (source: lua/samples/ch08_environments.lua, lines 56-64)

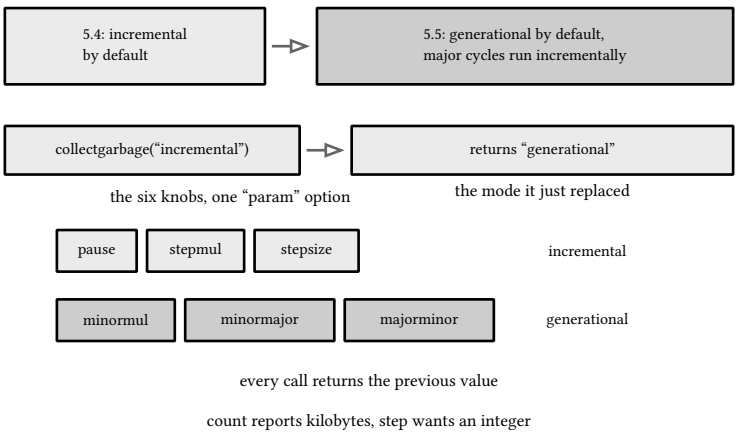


figure 35: the 5.5 collector, the default flip and where the knobs went

Parameters moved to one new option, `"param"`, with six names, `pause`, `stepmul`, `stepsize` for incremental work and `minormul`, `minormajor`, `majorminor` for generational. Every call returns the previous value, values are stored compressed so a read-back can round slightly, the manual's documented range is 0 to 100000, and this build accepts and stores values beyond it without complaint, a discrepancy the suite pins rather than repeats:

```
{name = "the param option reads and writes collector knobs", fn = function()
  local old = collectgarbage("param", "stepsize", 13)
  T.eq(type(old), "number")
  T.eq(collectgarbage("param", "stepsize"), 13)
  T.throws(function() collectgarbage("param", "bogus") end, "invalid option 'bogus'")
  local prev = collectgarbage("param", "pause", 200000)
  T.eq(type(prev), "number")
  T.ok(collectgarbage("param", "pause") ≥ 100000, "out of range value still stored")
  T.eq(type(collectgarbage("param", "pause", 250)), "number")
  for _, name in ipairs({"minormul", "minormajor", "majorminor", "pause", "stepmul"}) do
    T.eq(type(collectgarbage("param", name)), "number", name)
  end
end},

{name = "step with a non number argument is rejected", fn = function()
  T.throws(function() collectgarbage("step", "bogus") end, "number expected, got string")
  T.eq(type(collectgarbage("step", 0)), "boolean")
end},
```

listing 52: reading and writing the six knobs, the out of range fact (source: lua/samples/ch08_environments.lua, lines 66-83)

`"count"` reports kilobytes with a fraction, `"step"` takes an optional integer, and passing anything else to it is an argument error, not a silent ignore:

```
{name = "step with a non number argument is rejected", fn = function()
  T.throws(function() collectgarbage("step", "bogus") end, "number expected, got string")
  T.eq(type(collectgarbage("step", 0)), "boolean")
end},
```

listing 53: step's type gate (source: lua/samples/ch08_environments.lua, lines 80-83)

8.2. weak tables and finalizers

`__mode = "v"` or `"k"` makes values or keys weak, and a full collect clears entries whose only reference was the weak table. Finalization has three rules the tests hold in place: `__gc` marks the object only if present when `setmetatable` runs, finalizers run in reverse order of marking at the end of the cycle, and a finalizer that stores its object somewhere reachable resurrects it permanently:

```
{name = "weak values release entries on collect", fn = function()
  local t = setmetatable({}, {__mode = "v"})
  do
    local value = {"payload"}
    t.slot = value
  end
  collectgarbage("collect")
  T.eq(next(t), nil)
end},

{name = "weak keys release entries on collect", fn = function()
  local t = setmetatable({}, {__mode = "k"})
  do
    local key = {"key"}
    t[key] = "payload"
  end
  collectgarbage("collect")
  T.eq(next(t), nil)
end},

{name = "finalizers run in reverse marking order", fn = function()
  local log = {}
  do
    local a = setmetatable({}, {__gc = function() log[#log + 1] = "a" end})
    local b = setmetatable({}, {__gc = function() log[#log + 1] = "b" end})
  end
  collectgarbage("collect")
  T.eq(log, {"b", "a"})
end},

{name = "gc must be present when the metatable is set", fn = function()
  local log = {}
  do
    local mt = {}
    local early = setmetatable({}, mt)
    mt.__gc = function() log[#log + 1] = "late" end
    local proper = setmetatable({}, {__gc = function() log[#log + 1] = "proper" end})
  end
  collectgarbage("collect")
  T.eq(log, {"proper"})
end},
```

listing 54: weak entries, reverse order, the mark time rule (source: lua/samples/ch08_environments.lua, lines 85-126)

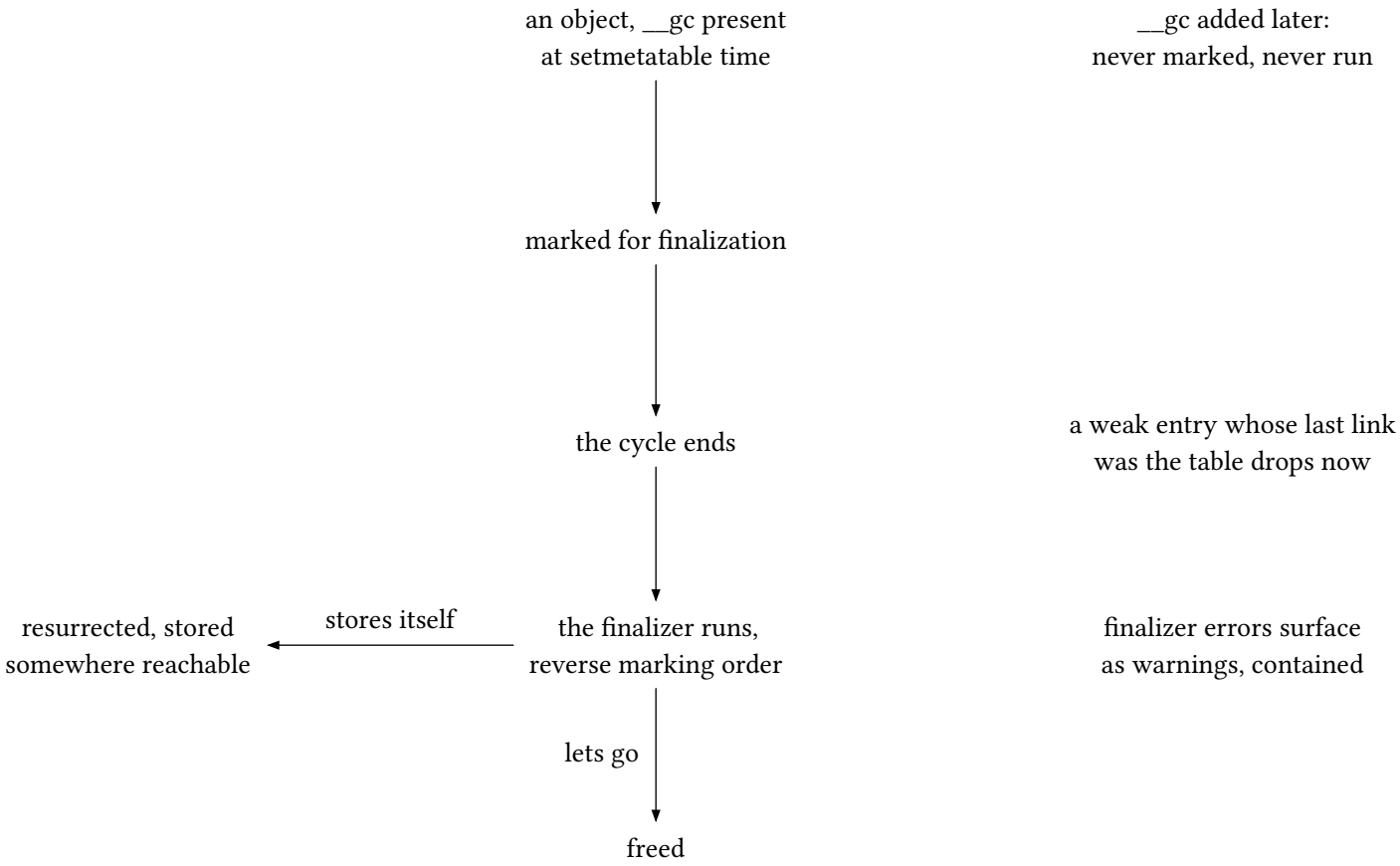


figure 36: an object with a finalizer, from marking to resurrection

WARNING: a classic self-deceiving probe

The mark time test first passed for the wrong reason: a `_ = early, proper` line to silence unused variable hints is a multiple assignment that stores `early` into the global `_` and drops `proper`, keeping exactly the wrong object alive. The tests now simply let the block end. Chapter 6's adjustment rules, biting their author.

Finalizer errors are contained, they surface as warnings, never as propagated errors, so a broken `__gc` cannot take down the process during collection:

```
{name = "finalizer errors do not propagate", fn = function()
  local log = {}
  do
    setmetatable({}, {__gc = function()
      log[#log + 1] = "attempted"
      error("finalizer boom")
    end})
  end
  T.eq(pcall(collectgarbage, "collect"), true)
  T.eq(log, {"attempted"})
end},

{name = "a finalizer can resurrect its object", fn = function()
  local saved
  do
    setmetatable({}, {__gc = function(self) saved = self end})
  end
  collectgarbage("collect")
  T.eq(type(saved), "table")
  collectgarbage("collect")
  T.eq(type(saved), "table")
end},
```

listing 55: errors contained, resurrection through a global (source: lua/samples/ch08_environments.lua, lines 127-148)

sources: lua.org manual 5.5 sections 2.2, 2.5, 3.2, 6.2, accessed 2026-09-08. Default mode, param behavior, and finalizer ordering verified live with lua 5.5.1, 16 tests green through `make verify-lua`.

9. inside the collector

Chapter 8 set the collector's knobs and measured what they return. This chapter opens the machine: what an incremental cycle actually does between two allocations, how the tricolor walk decides what dies, what the pause and step multipliers mean arithmetically, what separates a minor collection from a major one, and what the allocator hook below it all looks like. Stock lua offers no gc internals api, so everything here is observed from lua through the only instruments it ships: `collectgarbage("count")` as the oscilloscope, the boolean return of `"step"` as the cycle flag, and weak tables as the verdict on what survived. Every number in this chapter was measured on this machine against `tools/lua55/build/bin/lua.exe`, lua 5.5.1, on 2026-09-13.

9.1. the count as an oscilloscope

`collectgarbage("count")` reports kilobytes with a fraction, and the manual promises that the value times 1024 is the exact byte count in use. That precision is what makes it a usable probe: two full collects in a row differ by 0.000 KB on this build, so the floor after a collect is a stable baseline, and the delta over any allocation burst is the true retained cost of what the burst built:

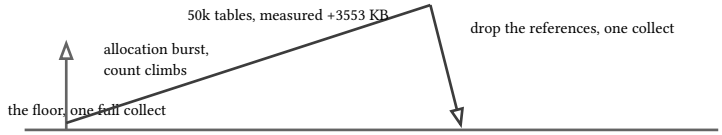
```lua
local function kb()
  return collectgarbage("count")
end

-- one boxed particle table t[1..8], the memory this chapter budgets
local function boxed_bytes(n)
  collectgarbage("collect")
  local base = kb()
  local keep = {}
  for i = 1, n do keep[i] = {1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5} end
  local grew = (kb() - base) * 1024 / n
  keep = nil
  collectgarbage("collect")
  return grew
end
```

listing 56: the two probes: kilobytes now, and bytes per boxed table (source: lua/samples/ch09_collector.lua, lines 5-19)

```lua
  {name = "a full collect returns to a repeatable floor", fn = function()
    collectgarbage("collect")
    local floor = kb()
    local junk = {}
    for i = 1, 50000 do junk[i] = {i, i} end
    local grown = kb()
    T.ok(grown > floor + 1000, "heap should grow past 1 MB, grew to " .. grown)
    junk = nil
    collectgarbage("collect")
    T.ok(kb() < floor + 64, "floor should return, at " .. kb() .. " from " .. floor)
  end},
```

listing 57: the floor returns to itself after the garbage dies (source: lua/samples/ch09_collector.lua, lines 30-40)



count * 1024 is exact bytes, the fraction is real

two collects in a row differ by 0.000 KB here

the delta over a burst is the retained cost

steps never sink below the floor: only garbage frees

figure 37: count as an oscilloscope, the floor, the ramp, the drop

The stop and restart pair completes the instrument panel: `"stop"` freezes the automatic collector but explicit calls still work, which is how a benchmark can hold the heap still while it runs.

```
{name = "stop and restart gate the automatic collector", fn = function()
  T.eq(collectgarbage("isrunning"), true)
  collectgarbage("collect")
  local floor = kb()
  collectgarbage("stop")
  T.eq(collectgarbage("isrunning"), false)
  local junk = {}
  for i = 1, 50000 do junk[i] = {i} end
  T.ok(kb() > floor + 1000,
    "a stopped collector lets the heap grow, at " .. kb())
  junk = nil
  collectgarbage("collect")
  T.ok(kb() < floor + 64,
    "an explicit collect works while stopped, at " .. kb())
  collectgarbage("restart")
  T.eq(collectgarbage("isrunning"), true)
end},
```

listing 58: stop freezes the automatic collector, explicit calls still run (source: lua/samples/ch09_collector.lua, lines 87-103)

9.2. incremental anatomy

In incremental mode a mark-and-sweep cycle is sliced into steps interleaved with the program's own execution. `collectgarbage("step")` returns true when a call finished a cycle. The measurement that shapes everything else in this section: garbage can outlive the cycle that should have freed it. A table allocated while a cycle is mid-flight is marked black as reachable, because at that moment it is, and the sweep at the end of that cycle lets it go. The freed bytes arrive one whole cycle later:

```
{name = "incremental steps reclaim a dropped graph, one cycle behind", fn = function()
  local was = collectgarbage("param", "stepsize", 9600)
  collectgarbage("incremental")
  collectgarbage("collect")
  local floor = kb()
  local graph = {}
  for i = 1, 100000 do graph[i] = {i, i, i, i, i, i, i, i} end
  local grown = kb()
  graph = nil
  local steps = 0
  local finishes = 0
  local lowest = math.huge
  repeat
    if collectgarbage("step") then finishes = finishes + 1 end
    steps = steps + 1
    if kb() < lowest then lowest = kb() end
  until (finishes ≥ 2 and kb() < floor + 2048) or steps > 4000
  T.ok(finishes ≥ 2, "the heap needed the cycle after the drop, saw "
    .. finishes .. " cycle ends in " .. steps .. " steps")
  T.ok(lowest ≥ floor - 1, "steps only free garbage, sank to " .. lowest
    .. " under floor " .. floor)
  T.ok(kb() < floor + 2048, "cycle end should be near the floor " .. floor)
  T.ok(grown - floor > 10000, "the heap really grew, by " .. (grown - floor))
  collectgarbage("param", "stepsize", was)
  collectgarbage("collect")
  collectgarbage("generational")
end},
```

listing 59: the drop needs the cycle after it, two cycle ends asserted (source: lua/samples/ch09_collector.lua, lines 59-85)

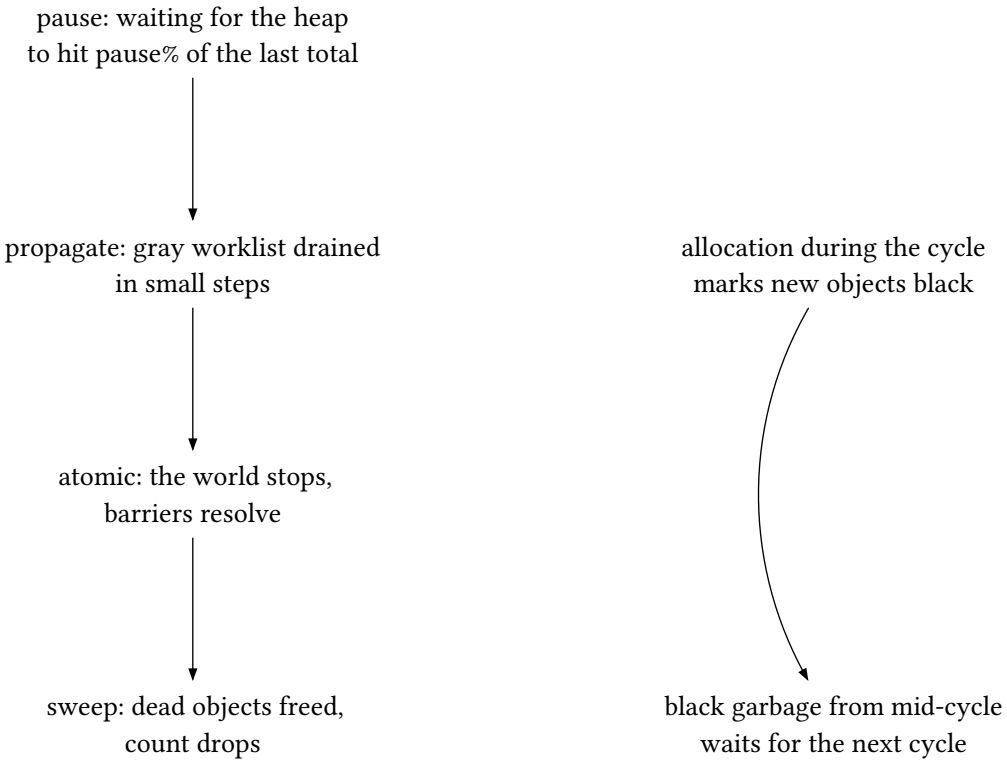


figure 38: one incremental cycle, sliced

How much work one step does is set by the step size knob, not by the step's argument. Measured on this machine, 2026-09-13: a fresh process clearing a 13 MB dropped heap at the 9600 KB default step size needed 4 steps and 2 cycle ends (`bench.lua trace`); the same heap and step size inside the full sample suite, after eight chapters of

allocation history, needed 409 steps and 2 cycle ends. The step count is a function of collector state, not a constant. An explicit 200 MB budget argument to "step" changed the count by nothing measurable in either state.

9.3. the tricolor walk

The marking phase is a tricolor walk over the object graph. Roots are grayed, a worklist drains: each gray object's children are grayed if still white, then the object turns black. Cycles are irrelevant to reachability, which is the whole point of tracing versus refcounting. The walk runs in pure lua on a toy graph, one rooted cycle and one unreachable island, and the real collector is then asked to agree through weak values:

```
{name = "the tricolor walk in pure lua marks the reachable set", fn = function()
  local a, b, c, d = {name = "a"}, {name = "b"}, {name = "c"}, {name = "d"}
  local x, y = {name = "x"}, {name = "y"}
  a.kids = {b, d}
  b.kids = {c}
  c.kids = {a}          -- a cycle back to the root
  x.kids = {y}
  y.kids = {x}          -- an unreachable island that also cycles
  local function tricolor(roots)
    local gray, black = {}, {}
    for _, r in ipairs(roots) do gray[#gray + 1] = r end
    while #gray > 0 do
      local n = table.remove(gray)
      black[n] = true
      for _, child in ipairs(n.kids or {}) do
        if not black[child] then gray[#gray + 1] = child end
      end
    end
    return black
  end
  local marked = tricolor({a})
  T.eq(marked[a], true)
  T.eq(marked[b], true)
  T.eq(marked[c], true)
  T.eq(marked[d], true)
  T.eq(marked[x], nil)
  T.eq(marked[y], nil)
end},
```

listing 60: the walk in pure lua, cycles and all (source: lua/samples/ch09_collector.lua, lines 205-232)

```
{name = "the real collector agrees with the walk through weak values", fn = function()
  local weak = setmetatable({}, {__mode = "v"})
  local root
  do
    root = {name = "root"}
    local chain = {name = "chain"}
    root.next = chain
    chain.next = root              -- reachable cycle
    local island = {name = "island"}
    island.next = island           -- unreachable self cycle
    weak.root, weak.chain, weak.island = root, chain, island
  end
  collectgarbage("collect")
  T.eq(weak.root.name, "root")
  T.eq(weak.chain.name, "chain")
  T.eq(weak.island, nil)
end},
```

listing 61: the real collector agrees, the island dies (source: lua/samples/ch09_collector.lua, lines 234-250)

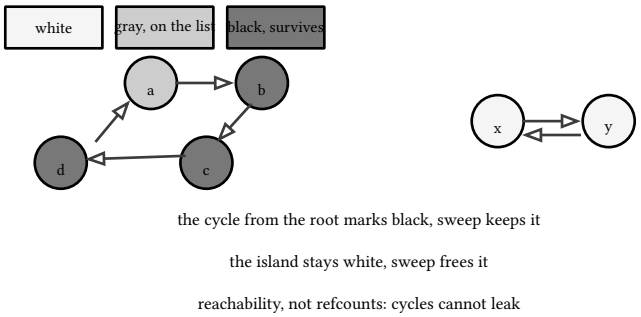


figure 39: three colors over the toy graph, one walk

9.4. the pause and step multipliers

Chapter 8 read and wrote the six knobs. The arithmetic behind the incremental three, measured as this build's defaults on 2026-09-13: pause 250, stepmul 200, stepsize 9600. Pause of n means a new cycle starts when live bytes hit n% of the total after the previous collection, so the default waits for a 2.5x heap (the manual's 200 example is the double). Step size n means roughly n bytes allocated between automatic steps. Step multiplier n means n% units of work per word allocated, and 0 is the special stop-the-world value. The values are stored compressed: setting 333 reads back 325, 256 reads back 250, and the documented 100000 ceiling reads back 99200:

```
{name = "param values are stored compressed", fn = function()
  local old = collectgarbage("param", "pause", 333)
  T.eq(collectgarbage("param", "pause"), 325)
  collectgarbage("param", "pause", 256)
  T.eq(collectgarbage("param", "pause"), 250)
  collectgarbage("param", "pause", 100000)
  T.ok(collectgarbage("param", "pause") ≥ 99000,
    "the documented ceiling should read back within compression: "
    .. collectgarbage("param", "pause"))
  collectgarbage("param", "pause", old)
end},
```

listing 62: compression: the stored value is not the written one (source: lua/samples/ch09_collector.lua, lines 167-177)

Pause maxed out is the cleanest behavioral demonstration of the arithmetic: with pause at 100000 the collector waits for a 1000x heap, so a garbage burst grows the count monotonically, 22.5 MB over a 24 KB floor measured in a fresh probe, a 943x ratio, with no reclaim until the knob comes home:

```
{name = "pause maxed defers the automatic cycle", fn = function()
  local old = collectgarbage("param", "pause", 100000)
  collectgarbage("incremental")
  collectgarbage("collect")
  local floor = kb()
  local junk = {}
  local monotonic = true
  local prev = floor
  for i = 1, 100000 do
    junk[i] = {i}
    if i % 20000 == 0 then
      local now = kb()
      if now < prev then monotonic = false end
      prev = now
    end
  end
  T.eq(monotonic, true)
  T.ok(kb() > floor * 20,
    "a 1000x pause should let the heap grow, ratio " .. kb() / floor)
  junk = nil
  collectgarbage("param", "pause", old)
  collectgarbage("collect")
  T.ok(kb() < floor + 64, "restored pause plus collect returns to the floor")
  collectgarbage("generational")
end},
```

listing 63: a 1000x pause defers the cycle, the heap only grows (source: lua/samples/ch09_collector.lua, lines 179-203)

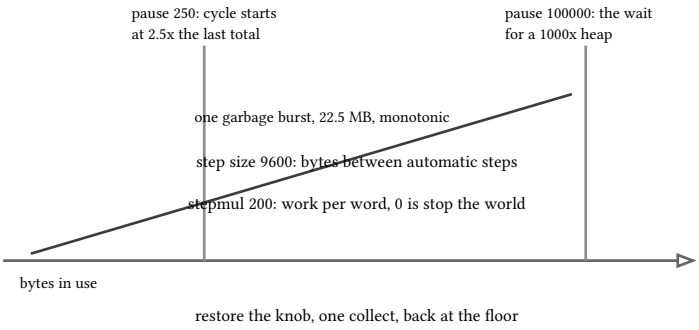


figure 40: the pause threshold against the heap

9.5. generational anatomy

5.5 defaults to generational mode, and its majors run incrementally. Minor collections traverse only objects born since the last one, which makes two facts observable from lua. Fresh young garbage dies to a single minor step. But a table that is alive across a minor, referenced at the moment the minor runs, is tenured into the old generation, and old garbage is invisible to minors: a 3.5 MB tenured cohort survived 301 consecutive minor steps at byte-for-byte the same count and was freed only by the explicit major that `"collect"` runs:

```lua
{name = "a minor collection frees fresh young garbage", fn = function()
  collectgarbage("generational")
  collectgarbage("collect")
  local floor = kb()
  local cohort = {}
  for i = 1, 200 do cohort[i] = {i} end
  local grown = kb()
  cohort = nil
  local finished = collectgarbage("step")
  T.ok(kb() < floor + 6,
    "one minor should free the young cohort, at " .. kb()
    .. " grown " .. grown .. " from floor " .. floor)
  T.eq(finished, false)
end},
```

listing 64: a fresh young cohort dies to one minor (source: lua/samples/ch09_collector.lua, lines 105-118)

```lua
{name = "tenured garbage is minor-proof, a major frees it", fn = function()
  collectgarbage("generational")
  collectgarbage("collect")
  local floor = kb()
  local junk = {}
  for i = 1, 50000 do junk[i] = {i} end
  collectgarbage("step")   -- a minor while everything is still referenced
  junk = nil
  local steps, saw_major = 0, false
  repeat
    saw_major = collectgarbage("step")
    steps = steps + 1
  until saw_major or steps > 120
  T.ok(kb() > floor + 1000,
    "minor steps should not free tenured garbage, at " .. kb()
    .. " from floor " .. floor)
  collectgarbage("collect")
  T.ok(kb() < floor + 64,
    "a major should free it, at " .. kb() .. " from floor " .. floor)
end},
```

listing 65: tenured garbage is minor-proof, the major frees it (source: lua/samples/ch09_collector.lua, lines 120-139)

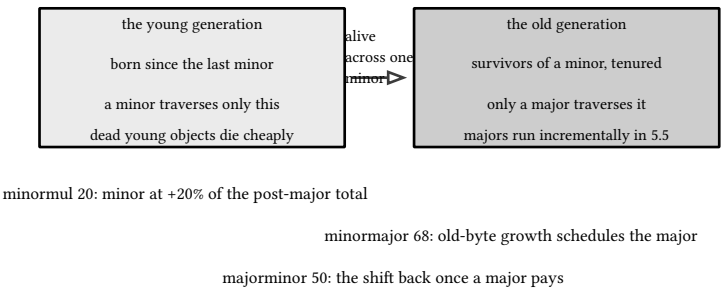


figure 41: two generations, promotion, and the three multipliers

A young object held alive through its cohort's death is the promotion proof from the other side: it survives forty minor steps and a full major with its payload intact while everything allocated beside it is gone. The three generational defaults measured on this build are minormul 20, minormajor 68, majorminor 50, and the same state dependence seen in incremental mode applies: in a fresh process the minor steps alone reclaimed a 3.5 MB young cohort, inside the full suite the tenured cohort needed the major. The suite pins both directions honestly, one per design.

9.6. the allocator hook, from lua

Every byte the collector manages arrives through one function pointer, the `lua_Alloc` hook a host passes to `lua_newstate`: `f(ud, ptr, osize, nsize)`. The contract is realloc-shaped but not realloc: `ud` is an opaque pointer the host chose, `ptr` is the block being created, grown, shrunk, or freed, `osize` its old size, `nsize` the requested one. When `nsize` is zero the hook frees and returns null. When `ptr` is null the hook creates, and `osize` then carries a type tag, `LUA_TSTRING`, `LUA_TTABLE`, and so on, for exactly the calls that create a new collectable object. The manual's own `luaL_alloc` is the whole thing in eight lines over `free` and `realloc`.

Chapter 14 and chapter 18 build hosts, but none swaps the allocator, so the hook is not demonstrated live from this book: the honest instrument is the count-delta census, which measures the same traffic the hook would see, allocations that grew the heap and the collect that returned it:

```
{name = "an allocation census through count deltas", fn = function()
  collectgarbage("collect")
  local base = kb()
  local batch = {}
  for i = 1, 10000 do batch[i] = {i} end
  local grew = kb() - base
  batch = nil
  collectgarbage("collect")
  local back = kb() - base
  T.ok(grew > 400, "10k one field tables should cost well past 400 KB, got " .. grew)
  T.ok(back < grew * 0.1, "after collect the delta should collapse, at " .. back)
end},
```

listing 66: the census: what the hook would have counted, measured by delta (source: lua/samples/ch09_collector.lua, lines 265-276)

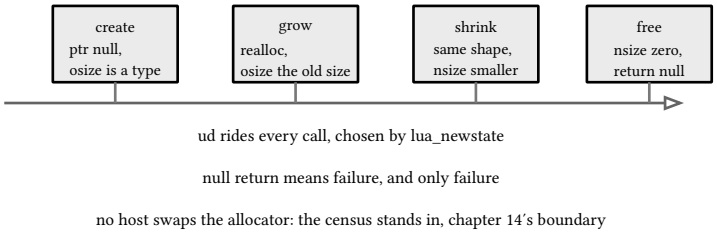


figure 42: one block's life through the hook

9.7. weak tables and finalizers, in one breath

The endgame machinery stays where chapter 8 measured it. Weak entries clear when the last strong reference elsewhere dies, `__mode` picking keys, values, or both, `__gc` marks only when present at setmetatable time, finalizers run in reverse marking order, and a finalizer that stores its object somewhere reachable resurrects it. One recap test holds the whole story in miniature:

```lua
{name = "weak entries clear only when no strong reference remains", fn = function()
  local weak = setmetatable({}, {__mode = "v"})
  local held
  do
    local a, b = {"a"}, {"b"}
    weak.a, weak.b = a, b
    held = b
  end
  collectgarbage("collect")
  T.eq(weak.a, nil)
  T.eq(weak.b[1], "b")
end},
```

listing 67: the held value survives, the dropped one clears (source: lua/samples/ch09_collector.lua, lines 252-263)

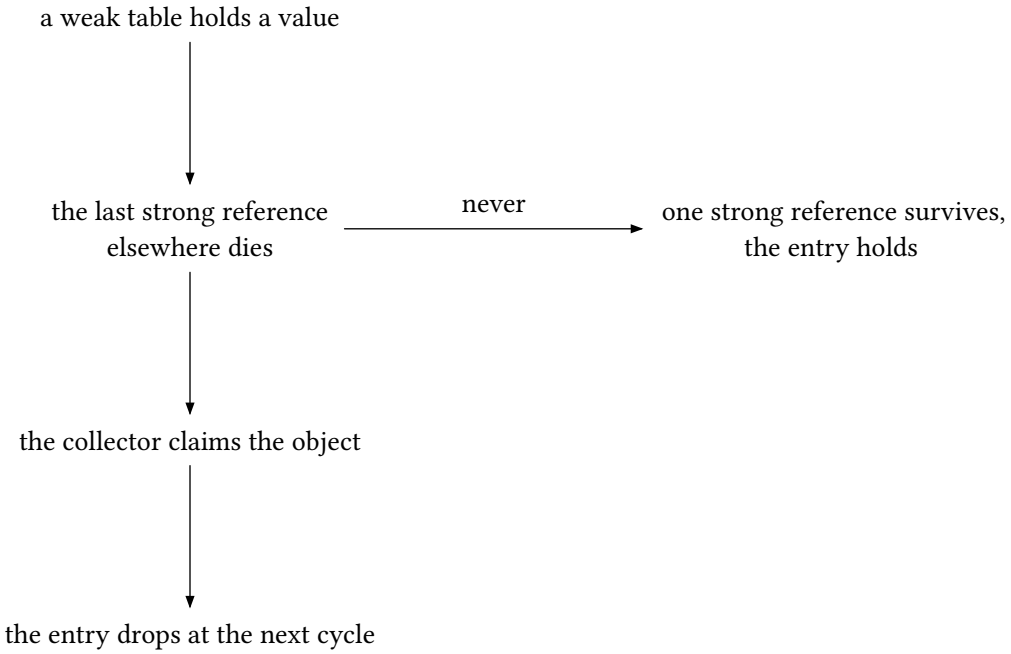


figure 43: a weak entry's life

9.8. the particle kernel, boxed

The corpus-wide particle kernel lands here in its densest lua form: n particles, eight float64 fields each, boxed as tables `t[1..8]`, a seeded `xorshift64`-star generator, and a pure arithmetic step with no allocation after init, `pos += vel*dt + 0.5*a*dt^2` under an analytic spring-and-field force, sixteen steps to a run:

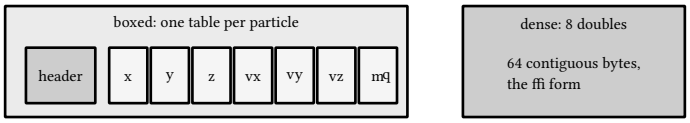
```
    x = x ~ (x >> 12)
    x = x ~ (x << 25)
    x = x ~ (x >> 27)
    return ((x * MUL) >> 11) * 0x1p-53
  end
end

function K.init(n, seed)
  local next01 = K.rng(seed or K.SEED)
  local ps = {}
  for i = 1, n do
    ps[i] = {
      next01() * 2 - 1,        -- 1 x
      next01() * 2 - 1,        -- 2 y
      next01() * 2 - 1,        -- 3 z
      next01() * 0.2 - 0.1,    -- 4 vx
      next01() * 0.2 - 0.1,    -- 5 vy
      next01() * 0.2 - 0.1,    -- 6 vz
      next01() + 0.5,          -- 7 m
      next01() * 2 - 1,        -- 8 q
    }
  end
  return ps
end
```

listing 68: the seeded generator and the boxed layout (source: lua/samples/kernel.lua, lines 18-41)

```
-- origin plus a constant field acting on the charge, divided by mass.
-- pos += vel*dt + 0.5*a*dt^2, vel += a*dt
local K0, E0 = 1.0, 0.5
function K.step(ps, dt)
  local h = 0.5 * dt * dt
  for i = 1, #ps do
    local p = ps[i]
    local x, y, z, vx, vy, vz, m, q = p[1], p[2], p[3], p[4], p[5], p[6], p[7], p[8]
    local c = q * E0 / m
    local ax = (c - K0 * x) / m
    local ay = (c - K0 * y) / m
    local az = (c - K0 * z) / m
    p[1] = x + vx * dt + 0.5 * ax * dt * dt
    p[2] = y + vy * dt + 0.5 * ay * dt * dt
    p[3] = z + vz * dt + 0.5 * az * dt * dt
    p[4] = vx + ax * dt
    p[5] = vy + ay * dt
    p[6] = vz + az * dt
  end
end
```

listing 69: the arithmetic step, allocation free (source: lua/samples/kernel.lua, lines 44-63)



measured: 133.4 bytes per boxed particle at 1m, 139.1 at 10m

payload is 64 bytes: boxing pays 2.1x, header and tagged slots

100m boxed is 13.9 gb by this arithmetic, not run on this machine

figure 44: one particle, boxed against dense

Measured on this machine, 2026-09-13, `samples/bench.lua scale`, a lua-5.5-only opt-in script the gates never run: at 1m particles init took 0.545 s and grew the heap 130310 KB, 133.4 bytes per particle, and sixteen steps took 1.825 s of `os.clock` cpu, 114.1 ns per particle per step, with the heap exactly flat across the loop, +0 KB, because the step allocates nothing and the collector has nothing to do. At 10m: init 6.169 s, 1358394 KB, 139.1 bytes per particle, the same sixteen steps 23.678 s, 148.0 ns per particle per step, slowest single step 1.577 s. Both runs return to the 31 KB floor after one collect. The 100m scale is infeasible by design in the boxed form: the same arithmetic prices it at 13.9 gb against the 14.4 gb of free memory measured on this machine, and that is the chapter's conclusion working as intended, the representation is the budget. The ffi path in the capstone carries 100m in 6.4 gb of cdata.

The suite keeps a 10k-particle smoke with both checksums pinned, fast enough for every gate run, and the seed is printed by every bench run:

```
{name = "the kernel smoke: 10k particles, 16 steps, fixed checksum", fn = function()
  local ps = K.init(10000)
  local initial = K.checksum(ps)
  for _ = 1, K.STEPS do K.step(ps, K.DT) end
  local sum = K.checksum(ps)
  T.near(sum, -167.9714928645, 1e-6)
  T.near(initial, -169.6501484565, 1e-6)
  T.eq(#ps, 10000)
end},
```

listing 70: the smoke: 10k particles, both checksums pinned (source: lua/samples/ch09_collector.lua, lines 278-286)

```
local function scale(n)
  print(string.format("scale n=%d seed=%d steps=%d dt=%.3f",
    n, K.SEED, K.STEPS, K.DT))
  collectgarbage("collect")
  local floor = kb()
  local t0 = os.clock()
  local ps = K.init(n)
  local t1 = os.clock()
  local grown = kb()
  print(string.format("init: %.3f s, heap %.0f KB (+%.0f KB), %.1f bytes/particle",
    t1 - t0, grown, grown - floor, (grown - floor) * 1024 / n))
  local slowest = 0
  local total = 0
  for s = 1, K.STEPS do
    local a = os.clock()
    K.step(ps, K.DT)
    local d = os.clock() - a
    total = total + d
    if d > slowest then slowest = d end
  end
  local after = kb()
  print(string.format("steps: %d in %.3f s, %.1f ns/particle/step (os.clock cpu)",
    K.STEPS, total, total / K.STEPS / n * 1e9))
  print(string.format("slowest step %.3f s, heap after loop %.0f KB (+%.0f KB)",
```

listing 71: the opt-in scale run every number above came from (source: lua/samples/bench.lua, lines 14-38)

sources: lua.org manual 5.5 sections 2.5.1 through 2.5.4, 4.6 (lua_Alloc, lua_gc, lua_newstate), 6.2 (collectgarbage), accessed 2026-09-13. All measurements from lua 5.5.1 built at tools/lua55, run on this machine, 18 tests green through `make verify-lua`.

10. profiling: hooks, clocks, and p99

Chapter 9 watched the collector. This chapter watches the program. Lua gives one debug hook, one cpu clock, and no sampling profiler, so a profiler here is built the honest way: exact hook counts over fixed workloads, an instruction census that says instruction census and not wall time, percentiles computed by hand, and on the LuaJIT side of the tree the jit's own profiler dumping real numbers. Every figure in this chapter is a real run on this machine, lua 5.5.1 and LuaJIT 2.1.1788856981 from `tools/`, dated 2026-09-13.

10.1. clocks, on this machine

`os.clock` is process cpu time, the iso C `clock()` under it, and on this windows build its smallest observed tick measured 1.000 ms over 200 changes: nothing faster than a millisecond can be timed by it, and shorter intervals read as zero. `os.time` carries whole seconds. Stock 5.5 ships no monotonic clock at all, that is the honest boundary, and one worth saying plainly: interval measurement below the tick, or across wall-time disturbances, needs a host. The ffi side of the tree has one, `QueryPerformanceCounter` declared through LuaJIT's ffi and measured at 10000000 Hz on this machine:

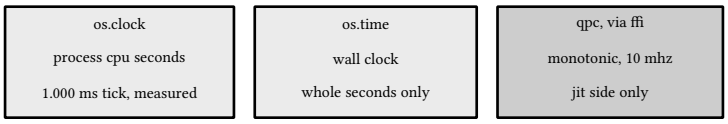
```
{name = "os.clock advances under cpu work", fn = function()
  local t0 = os.clock()
  local acc = 0
  for i = 1, 3000000 do acc = acc + i end
  local t1 = os.clock()
  T.eq(type(t0), "number")
  T.ok(acc > 0, "the work ran")
  T.ok(t1 ≥ t0, "os.clock is monotonic in the process")
  T.ok(t1 - t0 < 10, "three million adds should not take ten cpu seconds, took "
    .. (t1 - t0))
end},

{name = "os.time carries whole seconds only", fn = function()
  local now = os.time()
  T.eq(now, math.floor(now))
  T.eq(type(os.date("!%Y", now)), "string")
end},
```

listing 72: cpu time advances, wall time is whole seconds (source: lua/samples/ch10_profiling.lua, lines 19-35)

```
local function clockres()
  print("os.clock on this build:", os.clock())
  local best, last, changes = math.huge, os.clock(), 0
  local guard = 0
  while changes < 200 and guard < 200000000 do
    guard = guard + 1
    local now = os.clock()
    if now ~= last then
      local d = now - last
      if d > 0 and d < best then best = d end
      last = now
      changes = changes + 1
    end
  end
  print(string.format("smallest observed tick over %d changes: %.6f s (%.2f ms)",
    changes, best, best * 1000))
```

listing 73: measuring the tick: the smallest observed os.clock step (source: lua/samples/bench.lua, lines 76-92)



stock 5.5 has no monotonic clock, the host brings one

sub-millisecond intervals read zero under os.clock

every number here is cpu time unless it says wall

figure 45: three clocks, what each can and cannot see

10.2. the counting profiler

`debug.sethook` takes a mask of events and, for the count mask, a budget: the hook fires after every budget instructions. On a fixed workload the counts are exact, and that exactness is what makes a hook usable as a deterministic instrument rather than a timing guess. The 50 iteration loop below compiles to a known instruction stream on this build: a budget of 100 catches it once, a budget of 1 counts 112:

```
  {name = "a count hook fires an exact number of times", fn = function()
    local hits = 0
    debug.sethook(function() hits = hits + 1 end, "", 100)
    local r = fixed(50)
    debug.sethook()
    T.eq(r, 1275)
    T.eq(hits, 1)
  end},

  {name = "a per instruction budget counts the whole workload", fn = function()
    local hits = 0
    debug.sethook(function() hits = hits + 1 end, "", 1)
    local r = fixed(50)
    debug.sethook()
    T.eq(r, 1275)
    T.eq(hits, 112)
  end},
```

listing 74: exact counts on a fixed workload, no timing (source: lua/samples/ch10_profiling.lua, lines 37-53)

The line mask fires as the interpreter enters each new line, and the call and return masks bracket every lua call. A tail call replaces the frame, so it reports `tail call` and the caller's return never comes, which is the profiler's view of chapter 6's proper tail calls:

```
{name = "a line hook walks the body lines in order", fn = function()
  local three
  three = function()
    local a = 1
    a = a + 2
    return a
  end
  local first = debug.getinfo(three, "S").linedefined
  local lines = {}
  debug.sethook(function(_, l) lines[#lines + 1] = l end, "l")
  local x = three()
  debug.sethook()
  T.eq(x, 3)
  local body = {}
  for _, l in ipairs(lines) do
    if l ≥ first and l ≤ first + 3 then body[#body + 1] = l end
  end
  T.eq(body, {first + 1, first + 2, first + 3})
end},
```

listing 75: the line hook walks the three body lines (source: lua/samples/ch10_profiling.lua, lines 55-73)

```
{name = "call and return hooks see the nesting exactly", fn = function()
  local leaf = function() return 7 end
  local mid = function() return leaf() + 1 end
  local top = function() return mid() + 1 end
  local events = {}
  debug.sethook(function(ev)
    local info = debug.getinfo(2, "n")
    local name = info and info.name
    if name == "top" or name == "mid" or name == "leaf" then
      events[#events + 1] = ev .. ":" .. name
    end
  end, "cr")
  local y = top()
  debug.sethook()
  T.eq(y, 9)
  T.eq(events, {"call:top", "call:mid", "call:leaf",
    "return:leaf", "return:mid", "return:top"})
end},
```

listing 76: call and return events, the nesting exactly (source: lua/samples/ch10_profiling.lua, lines 75-92)

```lua
{name = "a tail call replaces the frame in the event stream", fn = function()
  local leaf = function() return 7 end
  local tf = function() return leaf() end
  local events = {}
  debug.sethook(function(ev)
    local info = debug.getinfo(2, "n")
    local name = info and info.name
    if name == "tf" or ev == "tail call" then
      events[#events + 1] = ev .. ":" .. tostring(name)
    end
  end, "cr")
  local y = tf()
  debug.sethook()
  T.eq(y, 7)
  T.eq(#events ≥ 2, true)
  T.eq(events[1], "call:tf")
  T.eq(events[2], "tail call:nil")
  local saw_return_tf = false
  for _, e in ipairs(events) do
    if e == "return:tf" then saw_return_tf = true end
  end
  T.eq(saw_return_tf, false)
end},
```

listing 77: a tail call erases the frame and its return (source: lua/samples/ch10_profiling.lua, lines 94-116)

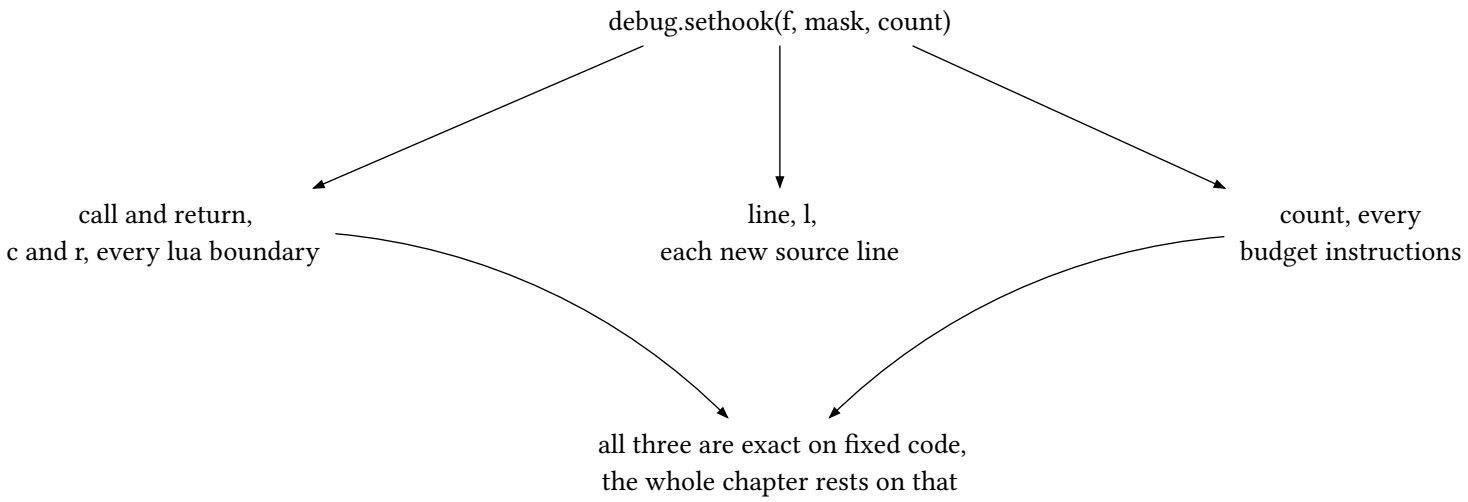


figure 46: the hook masks, what each one sees

10.3. the census, honestly labeled

A real profiler in this world is a count hook at every K instructions tallying `debug.getinfo` on the running line. Over a hot loop at K = 200 the tally is a cliff: 327 hits standalone, all of them on the loop line, none anywhere else. The honest label matters more than the code: this counts instructions, not time. A hook costs far more than the instruction it interrupts, samples are not uniform in wall time, and attribution is exact only because the workload is one line. It is an instruction census, and the chapter says so everywhere it quotes one:

```
{name = "a census profiler tallies one hot line", fn = function()
  local hot
  hot = function(n)
    local acc = 0
    for i = 1, n do acc = acc + i * 1.0001 end
    return acc
  end
  local loop_line = debug.getinfo(hot, "S").linedefined + 2
  local tally = {}
  local total = 0
  debug.sethook(function()
    total = total + 1
    local ln = debug.getinfo(2, "l").currentline
    tally[ln] = (tally[ln] or 0) + 1
  end, "", 200)
  local z = hot(20000)
  debug.sethook()
  T.near(z, 200030001, 1)
  T.ok(total ≥ 320 and total ≤ 335,
    "the census count is stable within caller context, got " .. total)
  T.eq(tally[loop_line], total)
  T.eq(next(tally, loop_line), nil)
end},
```

listing 78: the census: one hot line takes every sample (source: lua/samples/ch10_profiling.lua, lines 118-140)

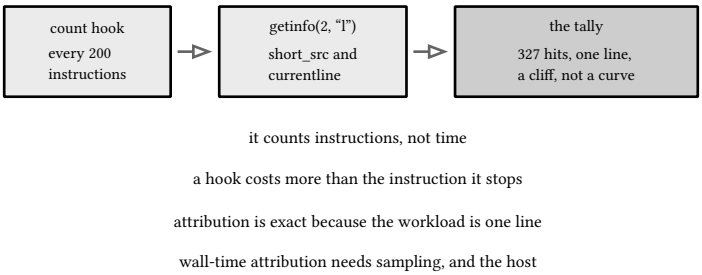


figure 47: the census pipeline and its honest limits

10.4. what a collector step costs

The collector's explicit step carries a measurable price. On the fixed 13 MB dropped heap of chapter 9's bench, step size 9600, a fresh process cleared it in 4 steps and 2 cycle ends for 0.0050 s of cpu, 1250 us per step; inside the full sample suite the same heap needed 409 steps, the state dependence chapter 9 measured. The number to carry is the shape, not a constant: step cost scales with the step size knob and the collector's backlog:

```lua
local function stepcost()
  -- per-step collector cost on a fixed workload: clear the same
  -- dropped heap with explicit steps and time the loop
  local was = collectgarbage("param", "stepsize", 9600)
  collectgarbage("incremental")
  collectgarbage("collect")
  local graph = {}
  for i = 1, 100000 do graph[i] = {i, i, i, i, i, i, i, i} end
  local bytes = (collectgarbage("count")) * 1024
  graph = nil
  local steps, finishes = 0, 0
  local t0 = os.clock()
  repeat
    if collectgarbage("step") then finishes = finishes + 1 end
    steps = steps + 1
  until finishes ≥ 2 or steps > 8000
  local dt = os.clock() - t0
  print(string.format("heap=%.0f KB steps=%d cycle-ends=%d wall=%.4f s, %.1f us/step",
    bytes / 1024, steps, finishes, dt, dt / steps * 1e6))
  collectgarbage("collect")
  collectgarbage("param", "stepsize", was)
  collectgarbage("generational")
end
```

listing 79: timing the step loop at a fixed workload (source: lua/samples/bench.lua, lines 120-143)

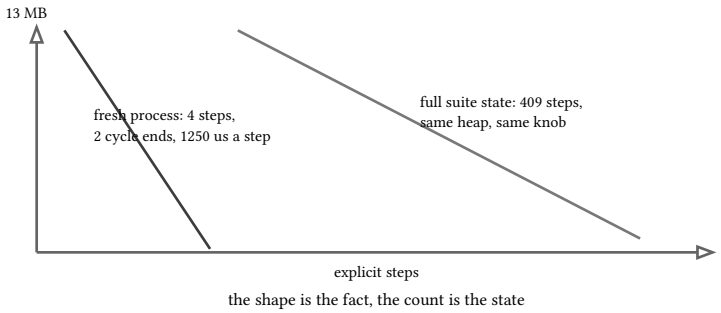


figure 48: one step's cost, measured against the heap it clears

10.5. percentiles by hand

Nearest-rank is the whole definition: sort, take the value at `ceil(p/100 * n)`. No interpolation, nothing to get wrong. The suite pins it against a fixed 1000 element vector with all four ranks as literals, then measures 1024 real chunk latencies, one kernel step of 10000 particles each, sized above the 1 ms clock floor so the table resolves. Measured on this machine, 2026-09-13: p50 1 ms, p95 2 ms, p99 3 ms, p99.9 3 ms, max 3 ms, and 98 of 1024 chunks still read zero at the floor, the quantization stated rather than smoothed away:

```lua
-- nearest-rank percentile: the smallest value whose cumulative count
-- reaches ceil(p/100 * n) in the sorted vector
local function nearest_rank(sorted, p)
  return sorted[math.ceil(p / 100 * #sorted)]
end

-- the fixed workload every exact-count test runs: a 50 iteration loop
local function fixed(n)
  local s = 0
  for i = 1, n do s = s + i end
  return s
end
```

listing 80: the definition and the fixed workload (source: lua/samples/ch10_profiling.lua, lines 5-16)

```lua
  {name = "nearest-rank percentiles on a fixed vector", fn = function()
    local v = {}
    for i = 1, 1000 do v[i] = (i * 997) % 101 + (i % 7) * 0.125 end
    table.sort(v)
    T.near(nearest_rank(v, 50), 50.375, 1e-12)
    T.near(nearest_rank(v, 95), 95.75, 1e-12)
    T.near(nearest_rank(v, 99), 99.75, 1e-12)
    T.near(nearest_rank(v, 99.9), 100.75, 1e-12)
    T.eq(#v, 1000)
  end},

  {name = "measured chunk latencies order their percentiles", fn = function()
    local lat = {}
    for c = 1, 256 do
      local ps = K.init(500, K.SEED + c)
      local t0 = os.clock()
      K.step(ps, K.DT)
      lat[c] = os.clock() - t0
      ps = nil
    end
    table.sort(lat)
    local p50 = nearest_rank(lat, 50)
    local p99 = nearest_rank(lat, 99)
    local p999 = nearest_rank(lat, 99.9)
    T.ok(p50 ≥ 0, "latencies are non negative")
    T.ok(p50 ≤ p99, "p50 should not exceed p99")
    T.ok(p99 ≤ p999, "p99 should not exceed p99.9")
    T.ok(p999 ≥ 0, "p99.9 exists")
  end},
```

listing 81: pinned literals, then measured latencies ordered (source: lua/samples/ch10_profiling.lua, lines 142-170)

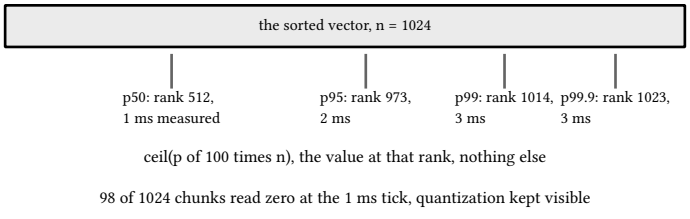


figure 49: the nearest-rank ladder over the sorted vector

10.6. the jit side of the tree

LuaJIT brings its own profiler, `jit.p`, and it speaks in compiled traces, not interpreted instructions. Over the ffi particle loop at 1m the line mode and the function mode each collapse to one entry, quoted exactly as the dump printed it, 2026-09-13, LuaJIT 2.1.1788856981, x64:

```
line profile of the loop (jit.p mode l):
100%\x20 bench.lua:125

function profile of the loop (jit.p mode f):
100%\x20 step
```

fragment, illustrative, not executed

The kernel itself is the same eight fields and the same sixteen steps, but dense: eight cdata double arrays, 64 bytes per particle, seeded by a 32-bit xorshift because lua 5.1 numbers have no 64-bit integer arithmetic. Measured alone on this machine: 1m particles at 9.9 ns per particle per step, 10m at 7.1 ns, and the one-off 100m run at 8.2 ns in 6.4 gb of cdata, init 1.259 s, slowest step 0.965 s, run alone and sequentially, state discarded after. Against chapter 9′s boxed tables the ratios are 11.5x at 1m, 114.1 against 9.9, and 20.8x at 10m, 148.0 against 7.1: the boxed form pays twice over, 2.1x the bytes and several times the arithmetic, and the gap widens with scale as the boxed heap outruns cache:

```
-- kernel32 queryperformancecounter: the monotonic clock stock 5.5 lacks
ffi.cdef[[
int QueryPerformanceCounter(long long *out);
int QueryPerformanceFrequency(long long *out);
]]
local k32 = ffi.load("kernel32")
local qc, qf = ffi.new("long long[1]"), ffi.new("long long[1]")
k32.QueryPerformanceFrequency(qf)
local freq = tonumber(qf[0])
local function wall()
  k32.QueryPerformanceCounter(qc)
  return tonumber(qc[0]) / freq
end

-- two extra sqlite calls the teaching wrapper does not declare
local dll = here .. "/../../../tools/sqlite/build/bin/sqlite3.dll"
local lib = ffi.load(dll)
ffi.cdef[[
int sqlite3_bind_double(void *stmt, int idx, double value);
int sqlite3_reset(void *stmt);
]]
-- one cast hoisted out of the row loop: a cast per row overflows
-- luajit's cdata intern table ("table overflow") around a few million
local TRANSIENT = ffi.cast("void (*)(void *)", -1)
```

listing 82: the monotonic clock and the two extra ffi calls the bench declares (source: lua/jit/bench.lua, lines 21-44)

```
-- seeded xorshift32 (the jit side has no 64 bit integer arithmetic in
-- lua numbers; the 5.5 side uses xorshift64, streams differ by design)
local function rng(seed)
  local x = seed
  return function()
    x = bit.bxor(x, bit.lshift(x, 13))
    x = bit.bxor(x, bit.rshift(x, 17))
    x = bit.bxor(x, bit.lshift(x, 5))
    return (x % 4294967296) / 4294967296
  end
end
```

listing 83: the jit side generator, 32-bit and honest about it (source: lua/jit/bench.lua, lines 49-59)

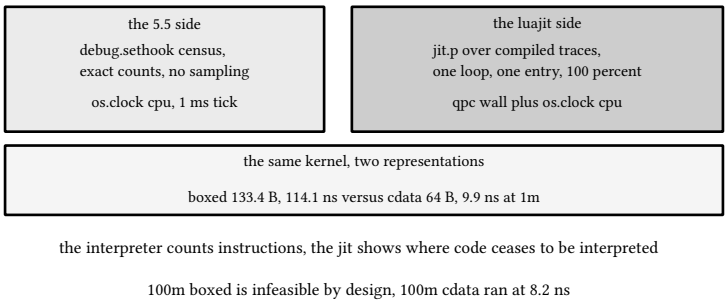


figure 50: two profilers over one tree of kernels

10.7. the row kernel, ten million rows

The second corpus workload rides the ffi sqlite wrapper chapter 17 pinned, 3.53.4 asserted at require time, with two extra declarations the teaching wrapper does not carry, `bind_double` and `reset`. The schema is the shared contract, one station key and eight real columns; 412 stations st000 through st411; seeded values; 10k-row batched transactions over one prepared statement. Measured alone on this machine, 2026-09-13: 10m rows inserted at 504,765 rows per second, the 24-aggregate SQL GROUP BY at 573,915 rows per second, the manual cursor walk at 580,353 rows per second, and the two aggregation paths agree, min and max exactly, averages within 1e-9 scaled by the column max because the scan orders differ. The 100m-row one-off ran alone and sequentially the same day: insert 100,000,000 rows in 203.886 s, 490,469 rows per second, 10000 batches, the insert rate holding its 10m pace within 3 percent, while the aggregations paid for the scale, SQL group by 313.430 s at 319,050 rows per second and the manual walk 346.036 s at 288,987 rows per second, roughly half their 10m rates as the table, past 8 gb on disk, outran cache. Agreement held intact, and the database was deleted after the numbers were read off. The one billion row scale stays by math only, not measured on this machine:

```lua
local function rows(n)
  n = n or 10000000
  local path = (os.getenv("TEMP") or ".") .. "\\lua-bench-rows.sqlite"
  os.remove(path)
  local seed = 2463534242
  local next01 = rng(seed)
  print(string.format("ffi sqlite rows n=%d seed=%d dll=%s",
    n, seed, sqlite.version()))
  local conn = sqlite.open(path)
  conn:exec("create table readings (station text not null, v0 real, " ..
    "v1 real, v2 real, v3 real, v4 real, v5 real, v6 real, v7 real)")
  local ins = conn:prepare("insert into readings values (?, ?, ?, ?, ?, ?, ?, ?, ?)")
  local handle = ins.handle
  local stations = {}
  for s = 0, 411 do stations[#stations + 1] = string.format("st%03d", s) end
  local t0 = wall()
  local batch = 10000
  local done = 0
  while done < n do
    conn:exec("begin")
    local take = math.min(batch, n - done)
    for _ = 1, take do
      local si = math.floor(next01() * 412) + 1
      lib.sqlite3_bind_text(handle, 1, stations[si], 5, TRANSIENT)
      for k = 0, 7 do
        lib.sqlite3_bind_double(handle, k + 2, (next01() * 2 - 1) * (1 + k / 8))
      end
      if lib.sqlite3_step(handle) ~= 101 then error("insert step failed") end
      lib.sqlite3_reset(handle)
    end
    conn:exec("commit")
    done = done + take
  end
  local t1 = wall()
  print(string.format("insert: %d rows in %.3f s, %.0f rows/s (%d batches of %d)",
    n, t1 - t0, n / (t1 - t0), math.floor((n + batch - 1) / batch), batch))
```

listing 84: schema, stations, prepared insert, batched transactions (source: lua/jit/bench.lua, lines 142-177)

```lua
  -- aggregation (a): one sql group by, 24 aggregates
  local t2 = wall()
  local agg = {}
  for row in conn:rows(
      "select station, min(v0), max(v0), avg(v0), min(v1), max(v1), avg(v1), " ..
      "min(v2), max(v2), avg(v2), min(v3), max(v3), avg(v3), " ..
      "min(v4), max(v4), avg(v4), min(v5), max(v5), avg(v5), " ..
      "min(v6), max(v6), avg(v6), min(v7), max(v7), avg(v7) " ..
      "from readings group by station order by station") do
    agg[#agg + 1] = row
  end
  local t3 = wall()
  print(string.format("sql group by: %d stations in %.3f s, %.0f rows/s",
    #agg, t3 - t2, n / (t3 - t2)))
```

listing 85: aggregation a, one group by with 24 aggregates (source: lua/jit/bench.lua, lines 178-191)

```
-- aggregation (b): manual cursor walk folding in lua
local t4 = wall()
local walk = {}
local stmt = conn:prepare("select station, v0, v1, v2, v3, v4, v5, v6, v7 " ..
  "from readings order by station")
while stmt:step() == "row" do
  local key = stmt:value(0)
  local acc = walk[key]
  if acc == nil then
    acc = {}
    for k = 0, 7 do
      local v = stmt:value(k + 1)
      acc[k * 4 + 1] = v acc[k * 4 + 2] = v acc[k * 4 + 3] = v
      acc[k * 4 + 4] = 1
    end
    walk[key] = acc
  else
    for k = 0, 7 do
      local v = stmt:value(k + 1)
      local lo, hi = k * 4 + 1, k * 4 + 2
      if v < acc[lo] then acc[lo] = v end
      if v > acc[hi] then acc[hi] = v end
      local sumi = k * 4 + 3
      local cnti = k * 4 + 4
      acc[sumi] = acc[sumi] + v
      acc[cnti] = acc[cnti] + 1
    end
  end
end
stmt:finalize()
local t5 = wall()
print(string.format("manual walk: %d stations in %.3f s, %.0f rows/s",
  412, t5 - t4, n / (t5 - t4)))
```

listing 86: aggregation b, the manual cursor walk (source: lua/jit/bench.lua, lines 192-224)

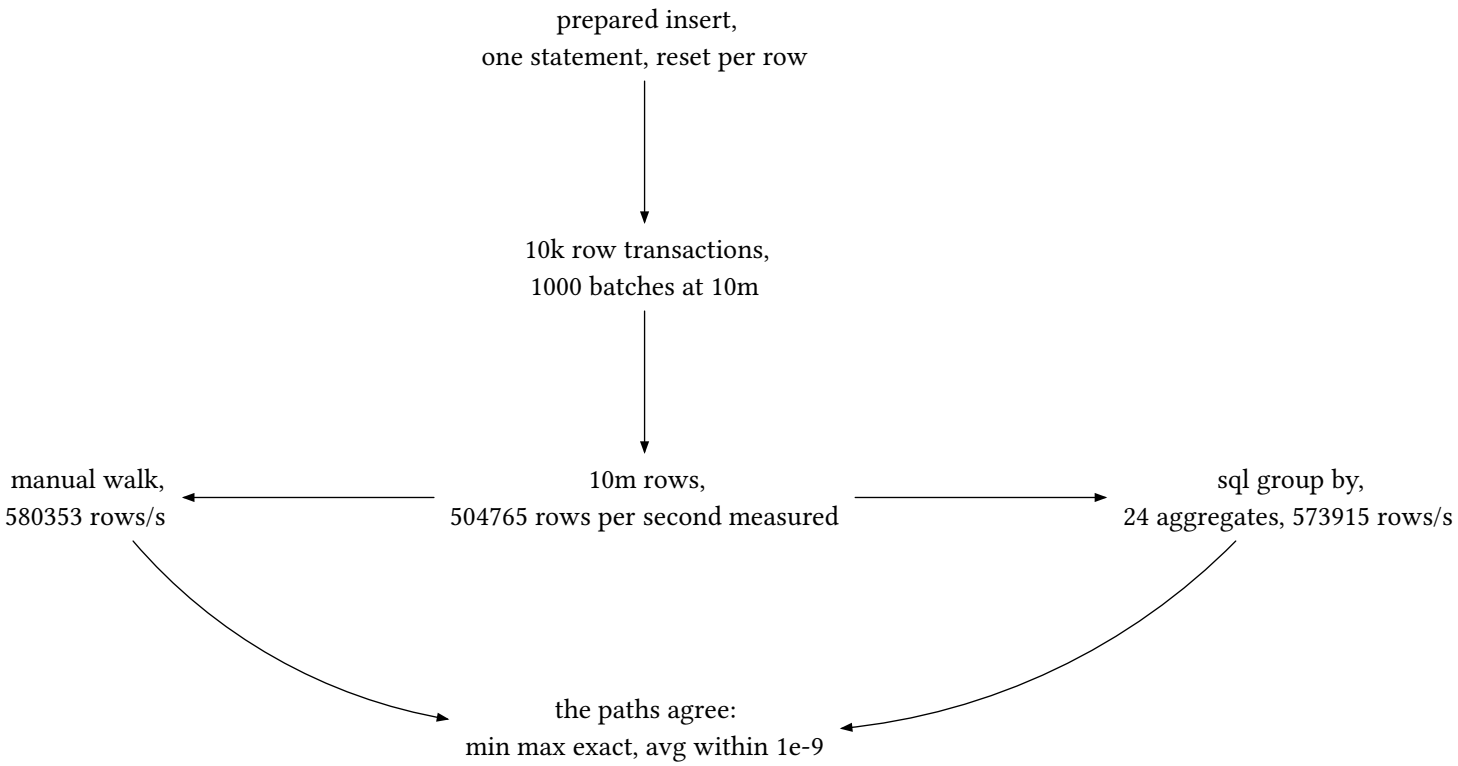


figure 51: the row kernel, ingestion to two agreeing aggregates

10.8. across the six books

The same two kernels ran in all six language books under one contract, N particles of 8 float64 fields stepped 16 times and 10 million readings rows of one station key plus 8 real columns, every number below quoted from the chapter that measured it on this machine, 2026-09-13. The first two columns are the particle step at 1m in each language's row-of-objects repr and its flat column repr, and they disagree on purpose: the flat repr wins 1.4x in c, 2.6x in javascript, and 21x in python, the object repr wins 2.3x in c# and 3.8x in go because the step touches all eight fields of every particle and one cache line per particle beats eight streams in flight, and lua's ffi cdata wins 11.5x over boxed tables. The last four columns are the shared sqlite contract at 10m rows, insert through 10k-row batched transactions, the sql group by, the manual cursor walk, and the p99 of one 10k-row commit. Lua's p99 cell sits empty on purpose: its os.clock ticks at 1 ms and the chapter refuses to quote a percentile it cannot read, the boundary every book states in its own clock.

book	objects ns	flat ns	insert/s	group/s	walk/s	p99 ms
c, clang -O2	6.15 to 6.75	4.53 to 4.91	468,803	584,696	2,242,885	28.3
c#, .net 11	5.7	13.1	339,091	723,248	501,820	48.5
go 1.27	8.2	30.9	267,942	324,614	610,720	67.957
javascript, node 26	12.4	4.72	530.42k	851k	562.3k	17.914
python 3.14	345.7	16.18	327,703	536,378	307,199	39.60
lua 5.5 + luajit	114.1	9.9	504,765	573,915	580,353	

Table 1: the two kernels across the six language books: particle step ns at 1m in each book's object and flat repr, then the 10m row kernel with insert, group by, and walk in rows per second and p99 in milliseconds per 10k-row commit, every cell quoted from the chapter that measured it

The ladder above the table stays in its own book: the 100m particle one-off measured 5.768 ns flat and 7.229 ns object in c, 21.815 ns flat in c# (memory bound, soa only), 8.898 ns flat in go, 4.63 ns flat in javascript, 16.88 ns flat in python, and 8.2 ns ffi in lua, with the boxed 100m priced at 13.9 GB and refused. The 100m row one-off held its 10m insert rate within 5 percent in c, javascript, and lua, gained 4 percent in python, and paid 8 percent in go and

21 percent in c#. One billion is arithmetic in every book, 64 GB of particle state and roughly 81 to 87 GB of rows, not measured on this machine.

sources: lua.org manual 5.5 sections 6.10 (os.clock, os.time), 6.11 (debug.sethook, getinfo), accessed 2026-09-13; luajit.org profiler page for jit.p semantics, accessed 2026-09-13. All measurements from lua 5.5.1 and luajit 2.1.1788856981 built under `tools/`, run on this machine, 11 tests green through `make verify-lua`.

11. stdlib 1: basic, string patterns, utf8

The basic library is the prelude that needs no prefix. The chapter picks up the pieces not already spent in earlier chapters: `select` from both ends, `tonumber` with a base, `ipairs` versus `pairs` on a holed table, `xpcall` with a handler, `rawlen` and `rawequal`, and `tostring` honoring the `__tostring` metamethod. Every string method call in this book, like `("abc"):upper()`, is the string metatable from book 9, ch 3 (lua 5.5, values, metatables, metamethods) at work.

```
{name = "tonumber parses bases and trims spaces", fn = function()
  T.eq(tonumber("ff", 16), 255)
  T.eq(tonumber("10", 2), 2)
  T.eq(tonumber("Z", 36), 35)
  T.eq(tonumber(" 3 "), 3)
  T.eq(tonumber("0x1p4"), 16.0)
  T.eq(tonumber("3."), 3.0)
  T.eq(tonumber("hello"), nil)
  T.throws(function() return tonumber("10", 37) end, "base out of range")
end},

{name = "select indexes from both ends", fn = function()
  T.eq(select("#", "a", nil, "c"), 3)
  T.eq(select(-1, "a", "b", "c"), "c")
  T.eq({select(2, "a", "b", "c")}, {"b", "c"})
end},

{name = "ipairs stops at the first nil, pairs does not", fn = function()
  local t = {1, nil, 3, x = "field"}
  local icount = 0
  for _ in ipairs(t) do icount = icount + 1 end
  T.eq(icount, 1)
  local seen = {}
  for k, v in pairs(t) do seen[tostring(k)] = v end
  T.eq(seen["1"], 1)
  T.eq(seen["3"], 3)
  T.eq(seen["x"], "field")
end},
```

listing 87: base parsing, select, the two iteration protocols (source: lua/samples/ch11_stdlib1.lua, lines 130-158)

```
{name = "xpcall catches through a handler", fn = function()
  local ok, msg = xpcall(function() error("inner") end,
    function(e) return "handled: " .. e end)
  T.eq(ok, false)
  T.ok(msg:find("handled: .+inner"), msg)
  local ok2, val = xpcall(function() return 7 end, error)
  T.eq({ok2, val}, {true, 7})
end},

{name = "tostring honors the tostring metamethod", fn = function()
  local t = setmetatable({}, {__tostring = function() return "CUSTOM" end})
  T.eq(tostring(t), "CUSTOM")
  T.eq(rawlen({1, 2, 3}), 3)
  T.eq(rawlen("abcd"), 4)
  T.eq(rawequal(1, 1.0), true)
end},
```

listing 88: xpcall handlers, the tostring metamethod, and raw access (source: lua/samples/ch11_stdlib1.lua, lines 159-174)

call	what it does
select	counts from either end, −1 is the last
tonumber	a base argument reads 2 through 36
ipairs, pairs	ipairs stops at the first nil, pairs walks every key
xpcall	the handler runs at the error site, stack still intact
the raw family	rawequal, rawlen, rawget, rawset bypass metatables
tostring	honors __tostring, else the type and address format

figure 52: the basic library's leftover surface, one row each

11.1. the string library

`sub` indexes negatively from the end, `rep` takes a separator, `byte` and `char` are byte level, and `string.format` carries the c printf surface with one lua addition, `%q`, which quotes a string so `load` can read it back:

```
{name = "sub takes negative indices from the end", fn = function()
  local s = "abcdef"
  T.eq(s:sub(-2), "ef")
  T.eq(s:sub(2, -2), "bcde")
  T.eq(s:sub(1, -1), s)
  T.eq(s:sub(4, 3), "")
  T.eq(s:sub(10), "")
end},

{name = "byte and char are byte level round trips", fn = function()
  T.eq(string.char(string.byte("A")), "A")
  T.eq(string.byte("\255"), 255)
  T.eq(#string.char(0, 65, 255), 3)
  T.eq(("lua"):reverse(), "aul")
end},

{name = "format covers the c printf surface", fn = function()
  T.eq(("%d %i %u"):format(7, 7, 7), "7 7 7")
  T.eq(("%5.2f|%-6s|"):format(3.14159, "ab"), " 3.14|ab    |")
  T.eq(("%x %X %o %c"):format(255, 255, 8, 65), "ff FF 10 A")
  T.eq(("%g %e"):format(1 / 3, 1 / 3), "0.333333 3.333333e-01")
  T.eq(("%s and %s"):format("str", 1.0), "str and 1.0")
  T.eq(("%q"):format('he said "hi"\n'), "\"he said \\\"hi\\\"\\\n\"")
  T.eq(("%q"):format("\0"), '"\\0"')
end},
```

listing 89: indexing, bytes, and the format surface (source: lua/samples/ch11_stdlib1.lua, lines 12-36)

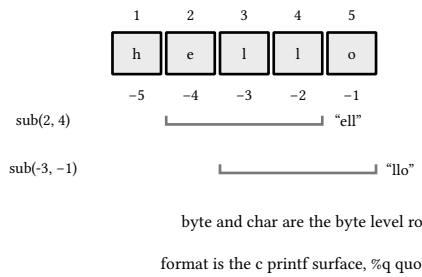


figure 53: the string as one byte line, indexed from both ends

11.2. patterns

Patterns are not regular expressions. They are a smaller matcher with single character classes, `%d` `%a` `%s`, sets and negated sets, and four repetitions: `+` one or more greedy, `*` zero or more greedy, `-` zero or more lazy, `?` optional. Anchors `^` and `$` apply at pattern position, so `^%d+$` means the whole string is digits:

```
{name = "classes, sets, and negated sets", fn = function()
  T.eq(("order 42x"):match("%d+"), "42")
  T.eq(("  spaced  "):match("^%s*(%S*)%s*$"), "spaced")
  T.eq(("a1b2"):match("%a+"), "a")
  T.eq(("b7c"):match("[abc]+"), "b")
  T.eq(("abc123"):match("[^%d]+"), "abc")
end},

{name = "the minus quantifier is lazy, star is greedy", fn = function()
  local html = "one <b>bold</b> two <i>it</i> end"
  T.eq((html:gsub("<.->", "")), "one bold two it end")
  T.eq(html:match("<.*>"), "<b>bold</b> two <i>it</i>")
  T.eq(("aaaa"):match("a-"), "")
  T.eq(("uuu"):match("u+"), "uuu")
  T.eq(("color"):match("colou?r"), "color")
end},

{name = "anchors pin the whole string", fn = function()
  T.eq(("123"):match("^%d+$"), "123")
  T.eq(("a123"):match("^%d+$"), nil)
  T.eq({("key=value"):match("^(%w+)=(%w+)$")}, {"key", "value"})
end},
```

listing 90: classes, sets, lazy versus greedy, whole string anchors (source: lua/samples/ch11_stdlib1.lua, lines 38-59)

kind	spells and reads
classes	%d %a %s %w, one character each, uppercase negates
sets	[aeiou] any of, [^aeiou] none of
greedy	1. one or more, * zero or more, eat all they can
lazy	• zero or more, stop at the first fit
optional	? zero or one
anchors	^ and $, ^%d+$ demands all digits
captures	(p) the match, () the position
specials	%bxy a balanced pair, %f[set] a frontier into it

figure 54: the whole pattern matcher in one table

Captures come in two flavors. Parenthesized patterns capture what they match, empty parentheses capture the position, and `find`, `match`, and `gmatch` all return them:

```
  {name = "position captures return indexes", fn = function()
    T.eq({("hello world"):find("()wor()")}, {7, 9, 7, 10})
  end},

  {name = "balanced and frontier patterns", fn = function()
    T.eq(("(a(b)c)d"):match("%b()"), "(a(b)c)")
    local fixed = ("hi 42there and 7up"):gsub("%f[%a]%a+", string.upper)
    T.eq(fixed, "HI 42THERE AND 7UP")
  end},
```

listing 91: position captures, balanced pairs, frontier (source: lua/samples/ch11_stdlib1.lua, lines 61-69)

Two special patterns have no regex equivalent worth confusing them with: `%bxy` matches balanced pairs, and `%f[set]` matches a frontier transition into the set:

```
  {name = "balanced and frontier patterns", fn = function()
    T.eq(("(a(b)c)d"):match("%b()"), "(a(b)c)")
    local fixed = ("hi 42there and 7up"):gsub("%f[%a]%a+", string.upper)
    T.eq(fixed, "HI 42THERE AND 7UP")
  end},
```

listing 92: balanced match, frontier uppercasing (source: lua/samples/ch11_stdlib1.lua, lines 65-69)

`gsub` takes its replacement as a string with `%n` captures and `%0`, a table keyed by the match, or a function called per match. Both the table and the function keep the original text when they yield nil or false, and the second return counts matches:

```
  {name = "gsub replaces with strings, tables, or functions", fn = function()
    T.eq({("hello world"):gsub("(%w+)", "<%1>")}, {"<hello> <world>", 2})
    local map = {hello = "goodbye"}
    T.eq({("hello world"):gsub("%w+", map)}, {"goodbye world", 2})
    local calls = {}
    local r = ("a b c"):gsub("%a", function(m)
      calls[#calls + 1] = m
      return m:upper()
    end)
    T.eq(r, "A B C")
    T.eq(calls, {"a", "b", "c"})
    T.eq({("keep"):gsub("k", function() return nil end)}, {"keep", 1})
    T.eq({("keep"):gsub("k", function() return false end)}, {"keep", 1})
    T.eq({("ab"):gsub("%a", "%0%0")}, {"aabb", 2})
  end},

  {name = "gmatch walks matches, captures included", fn = function()
    local words = {}
    for w in ("one two three"):gmatch("%a+") do words[#words + 1] = w end
    T.eq(words, {"one", "two", "three"})
    local pairs = {}
    for k, v in ("a=1, b=2"):gmatch("(%w+)=(%w+)") do pairs[k] = v end
    T.eq(pairs, {a = "1", b = "2"})
  end},
```

listing 93: the three replacement forms and the keep rule (source: lua/samples/ch11_stdlib1.lua, lines 71-95)

`find` grows a plain flag for literal searches and an init position that can be negative, and `gmatch` walks matches as an iterator that fits the generic for directly.

```
{name = "gmatch walks matches, captures included", fn = function()
  local words = {}
  for w in ("one two three"):gmatch("%a+") do words[#words + 1] = w end
  T.eq(words, {"one", "two", "three"})
  local pairs = {}
  for k, v in ("a=1, b=2"):gmatch("(%w+)=(%w+)") do pairs[k] = v end
  T.eq(pairs, {a = "1", b = "2"})
end},

{name = "find takes a plain flag and an init", fn = function()
  T.eq(("a.b.c"):find(".", 1, true), 2)
  T.eq(("a.b.c"):find(".", 3), 3)
  T.eq(("abc"):find("b", 3), nil)
  T.eq(("hello"):find("l", -2), 4)
end},
```

listing 94: plain find with init, gmatch walking captures (source: lua/samples/ch11_stdlib1.lua, lines 87-101)

11.3. binary packing

`string.pack` and `string.unpack` implement the manual's binary formats: endian prefixes, integer widths, `z` for zero terminated strings, `s` for size prefixed ones:

```
{name = "string.pack and unpack round trip with endianness", fn = function()
  local little = string.pack("<i4", 0x01020304)
  T.eq(#little, 4)
  T.eq(string.unpack("<i4", little), 0x01020304)
  T.eq({string.byte(little, 1, 4)}, {4, 3, 2, 1})
  local big = string.pack(">i4", 0x01020304)
  T.eq({string.byte(big, 1, 4)}, {1, 2, 3, 4})
  T.eq(string.unpack("z", string.pack("z", "hi")), "hi")
  T.eq(#string.pack("s1", "abcd"), 5)
end},
```

listing 95: round trip, byte order, sized strings (source: lua/samples/ch11_stdlib1.lua, lines 103-112)

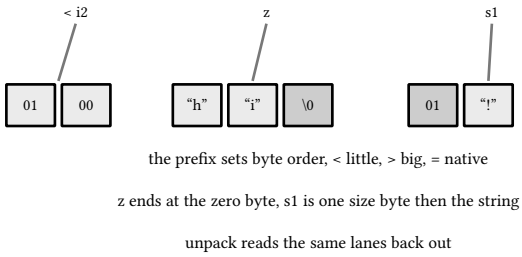


figure 55: a pack format string over the byte lanes it produces

11.4. utf8

The `utf8` library counts code points while `#` counts bytes, exposes `charpattern` as a plain string, itself usable inside larger patterns, and reports bad encodings by returning nil plus the byte position of the trouble:

```
{name = "utf8 counts code points, len counts bytes", fn = function()
  local s = "héllo"
  T.eq(#s, 6)
  T.eq(utf8.len(s), 5)
  T.eq(utf8.char(0x263A), "\u{263A}")
  T.eq({utf8.codepoint(s, 1, 3)}, {104, 233})
  T.eq(utf8.offset(s, 3), 4)
  local codes = {}
  for p, c in utf8.codes(s) do codes[#codes + 1] = p * 1000 + c end
  T.eq(codes, {1000 + 104, 2000 + 233, 4000 + 108, 5000 + 108, 6000 + 111})
  local bad, pos = utf8.len("ok\xffbad")
  T.eq(bad, nil)
  T.eq(pos, 3)
  T.eq(utf8.charpattern, "[\x00-\x7F\xC2-\xFD][\x80-\xBF]*")
end},
```

listing 96: code points, offsets, codes, invalid encoding (source: lua/samples/ch11_stdlib1.lua, lines 114-128)

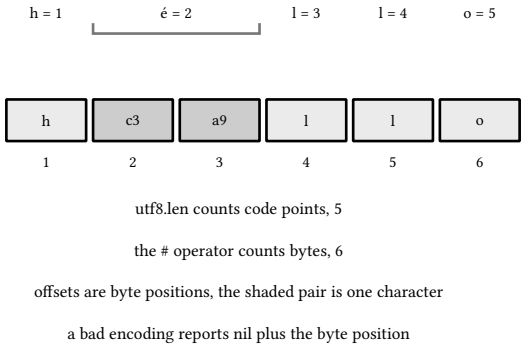


figure 56: the same text as six bytes and as five code points

sources: lua.org manual 5.5 sections 6.2, 6.5, 6.6, accessed 2026-09-08. Pattern semantics and replacement rules verified live with lua 5.5.1, 19 tests green through `make verify-lua`.

12. stdlib 2: table, math, io, os

The table library is five functions over the one data structure. `insert` appends or shifts, `remove` returns what it removed, `concat` joins a range and errors on holes with the index in the message, `move` copies a range that may overlap itself, and `pack`/`unpack` are the table shaped view of an argument list, with `n` as the only honest length when the list holds nil:

```
{name = "insert and remove shift and return", fn = function()
  local t = {1, 2}
  table.insert(t, 3)
  table.insert(t, 1, 0)
  T.eq(t, {0, 1, 2, 3})
  T.eq(table.remove(t, 1), 0)
  T.eq(t, {1, 2, 3})
  T.eq(table.remove(t), 3)
  T.eq(table.remove({}), nil)
end},

{name = "concat takes a range and rejects holes", fn = function()
  T.eq(table.concat({1, 2, 3}, ",", 2, 3), "2,3")
  T.eq(table.concat({"a", "b"}, ""), "ab")
  T.throws(function() table.concat({1, nil, 3}, ",", 1, 3) end, "invalid value (nil) at index 2")
end},

{name = "move handles overlap and empty ranges", fn = function()
  local a = {1, 2, 3, 4}
  local r = table.move(a, 1, 3, 2)
  T.eq(a, {1, 1, 2, 3})
  T.eq(r, a)
  local src, dst = {7, 8}, {}
  table.move(src, 1, 2, 1, dst)
  T.eq(dst, {7, 8})
  T.eq(table.move({1, 2, 3}, 3, 1, 1), {1, 2, 3})
end},

{name = "pack keeps the count, unpack restores the list", fn = function()
  local p = table.pack(1, nil, 3)
  T.eq(p.n, 3)
  T.eq(select("#", table.unpack(p, 1, p.n)), 3)
  local q = table.pack()
  T.eq(q.n, 0)
  T.eq(select("#", table.unpack(q, 1, q.n)), 0)
end},
```

listing 97: the five table operations (source: lua/samples/ch12_stdlib2.lua, lines 5-40)

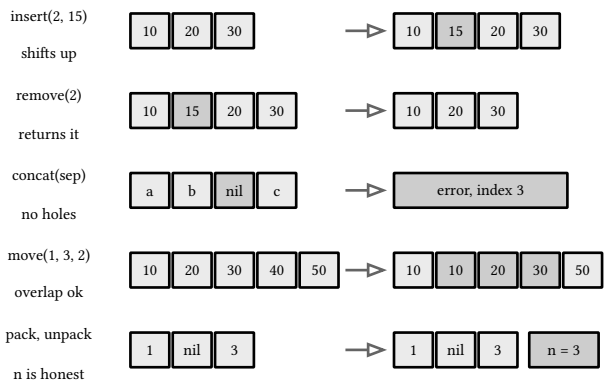


figure 57: the five table operations over the array part

`sort` takes a comparator or defaults to <, is not stable, and detects inconsistent comparators, an always true function is an error, though small inputs can slip through undetected, which is why the probe sorts twenty elements:

```
{name = "sort takes a comparator and detects invalid ones", fn = function()
  local st = {"3", "10", "2"}
  table.sort(st, function(x, y) return tonumber(x) < tonumber(y) end)
  T.eq(st, {"2", "3", "10"})
  local n = {}
  for i = 1, 20 do n[i] = 20 - i end
  T.throws(function()
    table.sort(n, function() return true end)
  end, "invalid order function for sorting")
end},
```

listing 98: numeric sort of strings, invalid order detection (source: lua/samples/ch12_stdlib2.lua, lines 42-51)

12.1. math

The subtype rules here changed in 5.5 and the suite pins them: `math.floor`, `math.ceil`, and `math.modf` return an integer when the result fits the integer range and a float otherwise, where 5.4 always returned floats. `math.abs` follows its argument's subtype. `math.fmod` truncates toward zero while the `%` operator floors, so `fmod(-7, 3)` is `-1` against `2`:

```
{name = "floor and ceil return integers when the result fits", fn = function()
  T.eq(math.floor(3.5), 3)
  T.eq(math.type(math.floor(3.5)), "integer")
  T.eq(math.type(math.floor(2.0)), "integer")
  T.eq(math.ceil(-0.5), 0)
  T.eq(math.type(math.ceil(3.2)), "integer")
  T.eq(math.type(math.floor(1e100)), "float")
  local intpart, frac = math.modf(3.7)
  T.eq(intpart, 3)
  T.near(frac, 0.7)
  T.eq(math.type(intpart), "integer")
end},

{name = "fmod truncates where modulo floors", fn = function()
  T.eq(math.fmod(-7, 3), -1)
  T.eq(-7 % 3, 2)
  T.eq(math.type(math.fmod(-7, 3)), "integer")
  T.eq({math.frexp(3.5)}, {0.875, 2})
end},
```

listing 99: integer returning floors, modf, fmod versus percent (source: lua/samples/ch12_stdlib2.lua, lines 53-71)

call	5.5 rule
floor, ceil, modf	integer when the result fits, 5.4 always floated
abs	follows its argument's subtype
fmod	truncates toward zero, −7 fmod 3 is −1, % floors to 2
tointeger	nil on failure, numeric strings coerce
ult	the unsigned view, 1 is below −1 under it
min, max	compare with <, reject non numbers

figure 58: the math library's subtype rules, one call each

`math.tointeger` and `math.type` answer with nil on failure, and `tointeger` also accepts numeric strings, following the language's string to number coercion. `math.ult` is the one place lua exposes the unsigned view of integers, so `-1` is above everything: `ult(1, -1)` is true. `max` and `min` compare with the language's `<` and reject non numbers outright. `math.random` returns a float in `[0, 1)` bare, an integer with bounds given, and `randomseed` makes runs reproducible:

```
{name = "tointeger and type use nil for failure", fn = function()
  T.eq(math.tointeger(3.0), 3)
  T.eq(math.tointeger(3.5), nil)
  T.eq(math.tointeger("7"), 7)
  T.eq(math.tointeger("x"), nil)
  T.eq(math.tointeger(2.0 ^ 63), nil)
  T.eq(math.type("x"), nil)
end},

{name = "ult compares integers as unsigned", fn = function()
  T.eq(math.ult(-1, 1), false)
  T.eq(math.ult(1, -1), true)
  T.eq(math.ult(1, 2), true)
end},

{name = "max and min compare by the lua operator", fn = function()
  T.eq(math.max(1, 2.5), 2.5)
  T.eq(math.min(3, 2, 2.0), 2)
  T.eq(math.max(-1, -2), -1)
  T.throws(function() math.max(1, "x") end, "attempt to compare number with string")
end},

{name = "random is seedable and range bound", fn = function()
  math.randomseed(42)
  local first = {math.random(1000), math.random(1000), math.random(1000)}
  math.randomseed(42)
  local second = {math.random(1000), math.random(1000), math.random(1000)}
  T.eq(first, second)
  local ok = true
  for _ = 1, 100 do
    local v = math.random(3, 4)
    if v ~= 3 and v ~= 4 then ok = false end
    local f = math.random()
    if f < 0 or f ≥ 1 then ok = false end
  end
  T.eq(ok, true)
end},
```

listing 100: tointeger coercion, ult, min max, seeded random (source: lua/samples/ch12_stdlib2.lua, lines 73-109)

```lua
{name = "random is seedable and range bound", fn = function()
  math.randomseed(42)
  local first = {math.random(1000), math.random(1000), math.random(1000)}
  math.randomseed(42)
  local second = {math.random(1000), math.random(1000), math.random(1000)}
  T.eq(first, second)
  local ok = true
  for _ = 1, 100 do
    local v = math.random(3, 4)
    if v ~= 3 and v ~= 4 then ok = false end
    local f = math.random()
    if f < 0 or f ≥ 1 then ok = false end
  end
  T.eq(ok, true)
end},
```

listing 101: seeded reproducibility and bounds (source: lua/samples/ch12_stdlib2.lua, lines 95-109)

The trigonometric and logarithmic surface is complete with `math.atan(y, x)` as the two argument arc tangent, `math.log` with a base, and hyperbolic and `atan2` functions gone, removed back in 5.4.

12.2. io

Two styles: implicit handles through `io.read` and `io.write`, and explicit handles from `io.open`, which is everything this book uses. Read formats are `"n"` number, `"l"` line without newline, `"L"` line with, `"a"` rest of file, and nil at end. `io.type` reports `"file"` or `"closed file"`, and file handles carry a `__close` metamethod, so `<close>` manages them:

```lua
{name = "trig, log, and pi", fn = function()
  T.near(math.atan(1, 1), math.pi / 4)
  T.near(math.log(8, 2), 3)
  T.near(math.exp(math.log(7)), 7)
  T.ok(math.sqrt(-1) ~= math.sqrt(-1), "sqrt of negative is nan")
end},

{name = "io write, read formats, and seek", fn = function()
  local path = "ch10-tmp.txt"
  local w = assert(io.open(path, "w"))
  w:write("10 apples\nline two\n")
  w:close()
  local r = assert(io.open(path, "r"))
  T.eq({r:read("n", "l")}, {10, " apples"})
  T.eq(r:read("L"), "line two\n")
  T.eq(r:read("a"), "")
  T.eq(r:read("l"), nil)
  T.eq(r:seek("end"), 21)
  T.eq(r:seek("set", 0), 0)
  r:close()
  T.eq(io.type(r), "closed file")
  T.eq(io.type(io.stdin), "file")
  os.remove(path)
  T.eq(io.open(path, "r"), nil)
end},
```

listing 102: read formats, seek positions, closed handles (source: lua/samples/ch12_stdlib2.lua, lines 111-136)

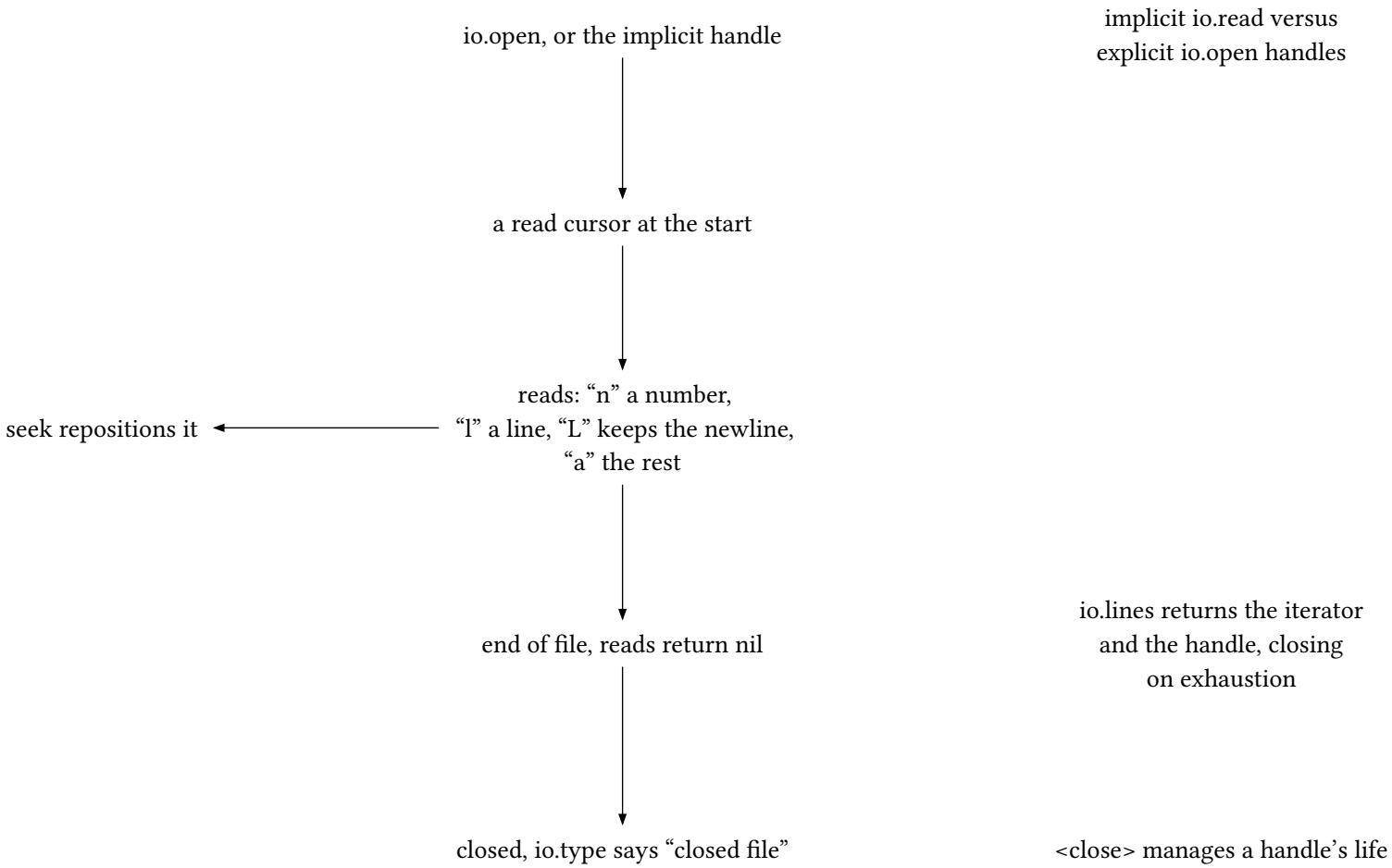


figure 59: a file handle’s life, from open cursor to closed

```
{name = "files are closable values", fn = function()
  local path = "ch10-close.txt"
  local handle
  do
    local f <close> = assert(io.open(path, "w"))
    handle = f
  end
  T.eq(io.type(handle), "closed file")
  os.remove(path)
end},
```

listing 103: files as to be closed values (source: lua/samples/ch12_stdlib2.lua, lines 137-146)

`io.lines` returns four values, the iterator, two nil placeholders for generic for compatibility, and the file handle as the closing value, so the file closes when the loop ends, a broken loop included:

```
{name = "io.lines closes the file and returns four values", fn = function()
  local path = "ch10-lines.txt"
  local w = assert(io.open(path, "w"))
  w:write("a\nb\n")
  w:close()
  local ret = table.pack(io.lines(path))
  T.eq(ret.n, 4)
  ret[4]:close()
  local got = {}
  for l in io.lines(path) do got[#got + 1] = l end
  T.eq(got, {"a", "b"})
  local n = 0
  for _ in io.lines(path, "L") do n = n + 1 end
  T.eq(n, 2)
```

listing 104: lines closes its file, the four value return (source: lua/samples/ch12_stdlib2.lua, lines 148-161)

12.3. os

`os.time` is an integer now and builds from a date table with hour defaulting to noon, `os.date` formats with c strftime rules plus the `"!*t"` table forms, `!` forcing utc, `difftime` returns a float, and `os.getenv` yields nil for missing variables rather than an error. `os.rename` and `os.remove` report true or nil plus a message, the file error convention `assert` pairs with:

```
end},

{name = "os time, date, and environment", fn = function()
  T.eq(math.type(os.time()), "integer")
  local t = os.date("!*t", 0)
  T.eq(t.year, 1970)
  T.eq(t.month, 1)
  T.eq(t.hour, 0)
  T.eq(t.isdst, false)
  T.eq(os.date("!%Y-%m-%d", 0), "1970-01-01")
  T.eq(math.type(os.time({year = 2000, month = 1, day = 1})), "integer")
  T.eq(os.difftime(100, 40), 60.0)
  T.eq(type(os.getenv("PATH")), "string")
  T.eq(os.getenv("NO_SUCH_VAR_XYZ"), nil)
end},

{name = "rename and remove round trip", fn = function()
  local a, b = "ch10-a.txt", "ch10-b.txt"
  local w = assert(io.open(a, "w"))
  w:write("x")
  w:close()
  T.eq(os.rename(a, b), true)
  T.eq(os.remove(b), true)
```

listing 105: time and date forms, environment, filesystem round trip (source: lua/samples/ch12_stdlib2.lua, lines 163-185)

call	shape
os.time	an integer, built from a date table, hour defaults to noon
os.date	strftime rules, ! forces utc, !*t builds a table
os.difftime	a float difference
os.getenv	nil for a missing variable, never an error
rename, remove	true, or nil plus a message, what assert pairs with

figure 60: the os library's wall clock surface

sources: lua.org manual 5.5 sections 6.7, 6.8, 6.9, 6.10, accessed 2026-09-08. Floor and ceil integer returns, tointeger string coercion, and io.lines arity verified live with lua 5.5.1, 17 tests green through `make verify-lua`.

13. stdlib 3: coroutine, debug, package

The coroutine half of this chapter's territory lives in book 9, ch 7 (lua 5.5, coroutines), lifecycle, close semantics, yields through metamethods and pcall. What remains here is the debug library, lua's reflection surface, and the package machinery behind `require`.

13.1. the debug library

`debug.getinfo` classifies activation records: `"Lua"` for interpreted functions with a source and a defined line, `"C"` for host functions like `print` or the `pcall` frame sitting under any test, and `"main"` for chunk frames. `debug.traceback` is the message handler that turns an error object into an error plus stack, which is the `xpcall` idiom:

```
{name = "getinfo classifies lua, c, and main", fn = function()
  local function marked()
    local info = debug.getinfo(1, "S")
    return info.what, info.short_src, info.linedefined > 0
  end
  local what, src, defined = marked()
  T.eq(what, "Lua")
  T.ok(src:find("ch13_stdlib3"), src)
  T.eq(defined, true)
  T.eq(debug.getinfo(print, "S").what, "C")
  T.eq(debug.getinfo(2, "S").what, "C")
end},

{name = "traceback strings through xpcall", fn = function()
  local ok, msg = xpcall(function() error("tb") end, debug.traceback)
  T.eq(ok, false)
  T.ok(tostring(msg):find("stack traceback:"), msg)
  T.ok(tostring(msg):find("ch13_stdlib3"), msg)
end},
```

listing 106: classification by what field, traceback through xpcall (source: lua/samples/ch13_stdlib3.lua, lines 5-23)

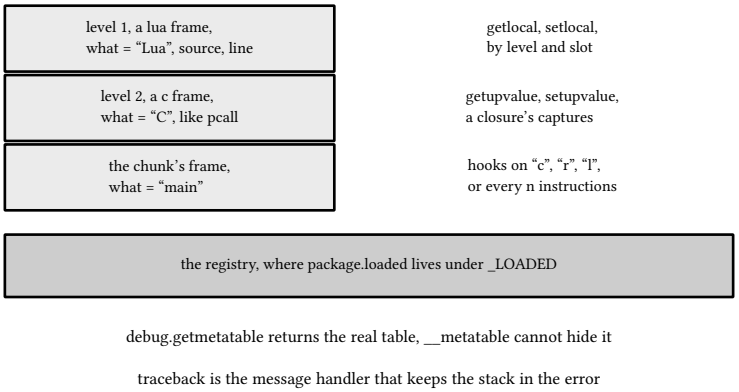


figure 61: the call stack the debug library reads, level by level

`debug.getlocal` and `debug.setlocal` reach into a live frame by level and slot, names and values both, and `debug.getupvalue` with `debug.setupvalue` do the same for a closure's captured variables. This is the standard library's escape hatch from immutability, and the tests use it to rewrite a counter mid-closure:

```
{name = "getlocal and setlocal touch live frames", fn = function()
  local function probe()
    local alpha, beta = 1, 2
    local n1, v1 = debug.getlocal(1, 1)
    local n2, v2 = debug.getlocal(1, 2)
    debug.setlocal(1, 1, 100)
    return n1, v1, n2, v2, alpha, beta
  end
  T.eq({probe()}, {"alpha", 1, "beta", 2, 100, 2})
end},

{name = "getupvalue and setupvalue edit closures", fn = function()
  local count = 0
  local function tick() count = count + 1 return count end
  tick()
  local name, value = debug.getupvalue(tick, 1)
  T.eq(name, "count")
  T.eq(value, 1)
  debug.setupvalue(tick, 1, 41)
  T.eq(tick(), 42)
end},
```

listing 107: reading and writing locals and upvalues (source: lua/samples/ch13_stdlib3.lua, lines 25-45)

Hooks instrument execution. The "c", "r", and "l" events fire on call, return, and line, the count form fires every n vm instructions, and debug.sethook() with no arguments unhooks. The call hook can ask debug.getinfo(2) who called:

```
{name = "count hooks fire per instruction budget", fn = function()
  local hits = 0
  debug.sethook(function() hits = hits + 1 end, "", 1)
  local sum = 0
  for i = 1, 50 do sum = sum + i end
  debug.sethook()
  T.eq(sum, 1275)
  T.ok(hits > 50, "expected many hook hits, got " .. hits)
end},

{name = "call and return hooks observe nesting", fn = function()
  local events = {}
  debug.sethook(function(what)
    local info = debug.getinfo(2, "nS")
    if info.short_src:find("ch13_stdlib3") then
      events[#events + 1] = what .. ":" .. (info.name or info.what)
    end
  end, "cr")
  local function inner() return 1 end
  local function outer() return inner() + 1 end
  outer()
  debug.sethook()
  local joined = table.concat(events, " ")
  T.ok(joined:find("call:outer"), joined)
  T.ok(joined:find("call:inner"), joined)
  T.ok(joined:find("return:inner"), joined)
end},
```

listing 108: count budget and call return nesting (source: lua/samples/ch13_stdlib3.lua, lines 47-73)

Two registry level operations complete the set: the registry itself, where `package.loaded` actually lives under the key `"_LOADED"`, and `debug.getmetatable`, which returns the real metatable even when `__metatable` protection lies to ordinary `getmetatable`:

```
{name = "the registry holds the loaded table", fn = function()
  local lib = require("lib")
  T.eq(rawget(debug.getregistry(), "_LOADED"), package.loaded)
  T.eq(package.loaded.lib, lib)
end},

{name = "debug.getmetatable sees through protection", fn = function()
  local mt = {__index = function() return "hidden" end, __metatable = "locked"}
  local t = setmetatable({}, mt)
  T.eq(getmetatable(t), "locked")
  local seen = debug.getmetatable(t)
  T.eq(seen, mt)
  T.eq(t.anything, "hidden")
end},
```

listing 109: the registry table and the metatable bypass (source: lua/samples/ch13_stdlib3.lua, lines 75-88)

13.2. package and require

`require` walks `package.searchers`, first the preload searcher, which consults `package.preload`, then the path searcher over `package.path`, then the c path over `package.cpath`, and fourth a c-root searcher that probes the module's first component in the cpath, unused in this book. A searcher returns a loader plus a provenance string on success, or a message string explaining its part of the failure, and the require error concatenates the whole trail, which is why a missing module's error lists every directory lua looked in:

```
  {name = "require loads, caches, and reports identity", fn = function()
    local path = "ch13-mod.lua"
    local w = assert(io.open(path, "w"))
    w:write("return {marker = 'loaded-once'}\n")
    w:close()
    local m1 = require("ch13-mod")
    T.eq(m1.marker, "loaded-once")
    local m2 = require("ch13-mod")
    T.eq(rawequal(m1, m2), true)
    T.eq(rawequal(package.loaded["ch13-mod"], m1), true)
    package.loaded["ch13-mod"] = nil
    os.remove(path)
    T.eq(package.loaded["ch13-mod"], nil)
  end},

  {name = "require errors list the search trail", fn = function()
    local ok, msg = pcall(require, "no-such-module-xyz")
    T.eq(ok, false)
    T.ok(tostring(msg):find("module 'no%-such%-module%-xyz' not found"), msg)
    T.ok(tostring(msg):find("no field package%.preload"), msg)
  end},

  {name = "preload registers loaders without files", fn = function()
    package.preload["virtual-mod"] = function()
      return {origin = "preload"}
    end
    T.eq(require("virtual-mod").origin, "preload")
    T.eq(rawequal(require("virtual-mod"), require("virtual-mod")), true)
    package.preload["virtual-mod"] = nil
    package.loaded["virtual-mod"] = nil
  end},

  {name = "searchpath and the searcher chain", fn = function()
    local path = "ch13-find.lua"
    local w = assert(io.open(path, "w"))
    w:write("return 1\n")
    w:close()
    local found = package.searchpath("ch13-find", package.path)
    T.ok(found and found:find("ch13%-find%.lua"), tostring(found))
    T.eq(select(2, package.searchpath("no-such-module-xyz", package.path)) ~= nil, true)
    os.remove(path)
    T.eq(type(package.cpath), "string")
    package.preload["s1probe"] = function() end
    T.eq(type(package.searchers[1]("s1probe")), "function")
    local miss = package.searchers[1]("lib")
    T.eq(type(miss), "string")
    T.ok(miss:find("no field package%.preload"), miss)
    T.eq(type(package.searchers[2]("lib")), "function")
    package.preload["s1probe"] = nil
  end},
```

listing 110: file based require with caching, the search trail, preload, and the searcher contract (source: lua/samples/ch13_stdlib3.lua, lines 90-139)

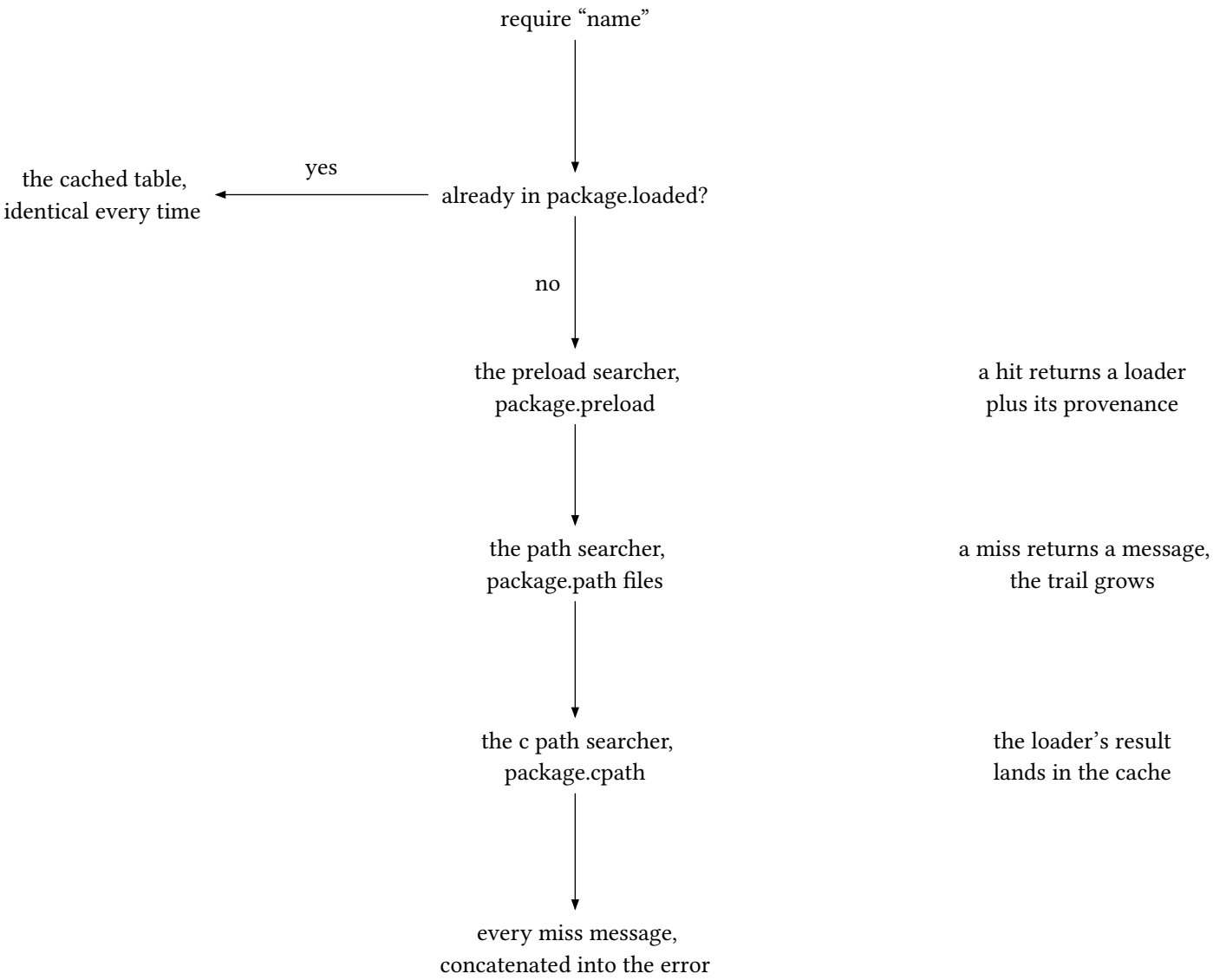


figure 62: the require pipeline, searchers in order, the cache in front

The cache is `package.loaded`: a second `require` of the same name returns the identical table, clearing the entry forces a reload, and the suite builds and removes a real module file to watch all three steps happen. `package.searchpath` is the path searcher’s probe exported, returning the resolved file or nil plus the same trail message.

NOTE: why the samples runner prepends the path

The stock `package.path` on this build points into the interpreter’s install tree first and the current directory last. The runner sets `package.path = "./?.lua;" .. package.path` so chapter modules and the assertion library resolve from the samples directory regardless of the caller’s location, the same trick `make verify-lua` relies on by always running from inside the project.

sources: lua.org manual 5.5 sections 6.3, 6.4, 6.11, accessed 2026-09-08. Searcher return contract and registry layout verified live with lua 5.5.1, 12 tests green through `make verify-lua`.

14. the c api, hands on

Lua is a library first and a language second. The interpreter this book runs is one host program among possible ones, linking `lua55.dll` and driving it through a c api of roughly two hundred functions, and pure lua code can see the seam from both sides: 127 of the standard library's functions are c functions, identified by `debug.getinfo`'s `"C"` classification, and the host's compiler, `luac`, is a separate executable that shares the same front end.

```lua
{name = "the standard library is 127 c functions", fn = function()
  local libs = {_G = _G, string = string, table = table, math = math,
    io = io, os = os, debug = debug, coroutine = coroutine,
    utf8 = utf8, package = package}
  local n = 0
  for _, lib in pairs(libs) do
    for _, v in pairs(lib) do
      if type(v) == "function" and debug.getinfo(v, "S").what == "C" then
        n = n + 1
      end
    end
  end
  T.eq(n, 127)
end},
```

listing 111: counting the c functions behind the lua libraries (source: lua/samples/ch14_capi.lua, lines 70-83)

This tree now builds hosts of its own: six c programs under `samples-c/src/Ch14/`, compiled with the documented mingw gcc against `lua55.dll` and run by `make verify-lua-c`, 89 checks green. The same chapter walks them in order, a first host, the stack, protected calls, c functions for lua, userdata, and references.

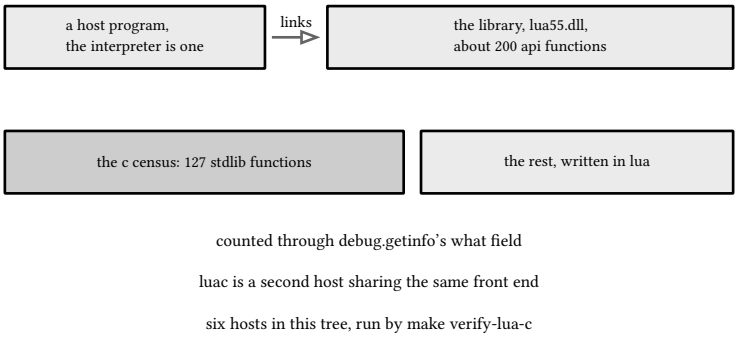


figure 63: the seam from both sides, a host linking the library

14.1. luac

`luac` precompiles and disassembles. Its output binary is exactly what `string.dump` produces, loadable with mode `"b"`, and its `-l` listing is the register machine made visible: `VARARGPREP` opens vararg functions, `MMBIN` is the metamethod fallback arm beside each arithmetic op, `TAILCALL` and the `RETURN` family carry result counts, and `LOADI` packs small integers into the instruction:

```
{name = "luac compiles and the binary executes", fn = function()
  local src, luo = "ch12-src.lua", "ch12-out.luo"
  local w = assert(io.open(src, "w"))
  w:write("local function add(a, b) return a + b end\nreturn add(1, 2)\n")
  w:close()
  local _, ok = run(LUAC .. " -o " .. luo .. " " .. src)
  T.eq(ok, true)
  local r = assert(io.open(luo, "rb"))
  local bin = r:read("a")
  r:close()
  local f = assert(load(bin, luo, "b"))
  T.eq(f(), 3)
  os.remove(src)
  os.remove(luo)
end},

{name = "the disassembly names 5.5 opcodes", fn = function()
  local src = "ch12-src.lua"
  local w = assert(io.open(src, "w"))
  w:write("local function add(a, b) return a + b end\nreturn add(1, 2)\n")
  w:close()
  local out = run(LUAC .. " -p -l " .. src)
  for _, op in ipairs({"VARARGPREP", "CLOSURE", "TAILCALL", "MMBIN", "ADD", "RETURN1"}) do
    T.ok(out:find(op, 1, true), "missing opcode " .. op)
  end
  os.remove(src)
end},
```

listing 112: compile to binary and execute it, then read the opcode names (source: lua/samples/ch14_capi.lua, lines 20-46)

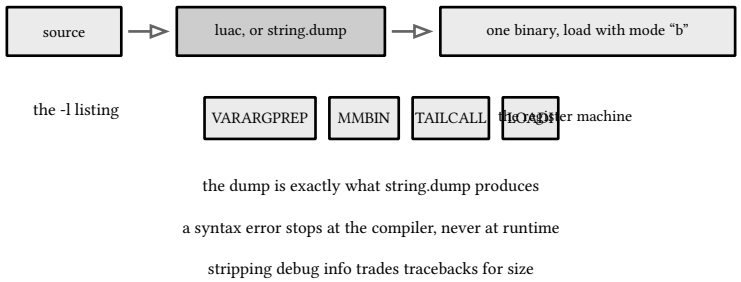


figure 64: source to binary through luac, and the register machine it shows

```
{name = "luac rejects a broken chunk", fn = function()
  local src = "ch12-bad.lua"
  local w = assert(io.open(src, "w"))
  w:write("local x = = 3\n")
  w:close()
  local out, ok = run(LUAC .. " -o ch12-bad.luo " .. src .. " 2>&1")
  T.eq(not ok, true)
  T.ok(out:find("unexpected symbol"), out)
  os.remove(src)
end},
```

listing 113: a syntax error stops at the compiler, not at runtime (source: lua/samples/ch14_capi.lua, lines 85-94)

`string.dump` takes a strip flag that removes debug information, the difference between the 98 byte dump of a one line function and its 73 byte stripped form, trading traceback quality for size:

```
{name = "string.dump strips debug info when asked", fn = function()
  local f = function(x) return x + 1 end
  local full = string.dump(f)
  local stripped = string.dump(f, true)
  T.eq({#full, #stripped}, {98, 73})
  T.eq(load(stripped, "s", "b")(41), 42)
end},
```

listing 114: full and stripped dumps (source: lua/samples/ch14_capi.lua, lines 48-54)

14.2. a first host

The smallest host is three calls: `luaL_newstate` builds a state, `luaL_openlibs` stocks the standard libraries, and `luaL_dostring` runs one chunk. Everything else this chapter does is the same three moves with more values crossing the boundary:

```
lua_State *L = luaL_newstate();
CHECK(L != NULL, "luaL_newstate returns a state");

luaL_openlibs(L);
lua_getglobal(L, "math");
CHECK(lua_type(L, -1) == LUA_TTABLE,
      "openlibs stocked the globals table");
lua_pop(L, 1);

// dostring compiles and runs one chunk, LUA_OK means it returned a value
CHECK(luaL_dostring(L, "return 6 * 7") == LUA_OK,
      "luaL_dostring runs a chunk");
CHECK(lua_type(L, -1) == LUA_TNUMBER, "the result waits on the stack");
CHECK((int)lua_tointeger(L, -1) == 42, "the result is 42");
lua_pop(L, 1);
CHECK(lua_gettop(L) == 0, "the stack is empty again");
```

listing 115: newstate, openlibs, dostring, and the value it leaves behind (source: lua/samples-c/src/Ch14/firsthost.c, lines 27-42)

Errors come back as return codes, never crashes. One 5.5 trap hides in the convenience macro: `luaL_dostring` expands to `luaL_loadstring(L, s) || lua_pcall(...)`, so its result is a bare truth value, 0 for success, 1 for any failure, and a host that wants the specific code calls the two steps itself:

```
// a syntax error is a return code plus a message, never a crash
CHECK(luaL_dostring(L, "return 6 *") != LUA_OK,
      "a syntax error returns nonzero");
CHECK(lua_type(L, -1) == LUA_TSTRING, "the error object is a string");
CHECK(strstr(lua_tostring(L, -1), "near") != NULL,
      "the message names the offending token");
lua_pop(L, 1);

// a runtime error reaches pcall only after loadstring succeeds; its code
// is LUA_ERRRUN, where the dostring macro only says nonzero
CHECK(luaL_loadstring(L, "error('boom')") == LUA_OK,
      "loadstring compiles the runtime-error chunk");
CHECK(lua_pcall(L, 0, 1, 0) == LUA_ERRRUN,
      "a runtime error is LUA_ERRRUN");
CHECK(strstr(lua_tostring(L, -1), "boom") != NULL,
      "the error text crossed the boundary");
lua_pop(L, 1);
```

listing 116: syntax errors from the macro, runtime errors from the two steps (source: lua/samples-c/src/Ch14/firsthost.c, lines 44-60)

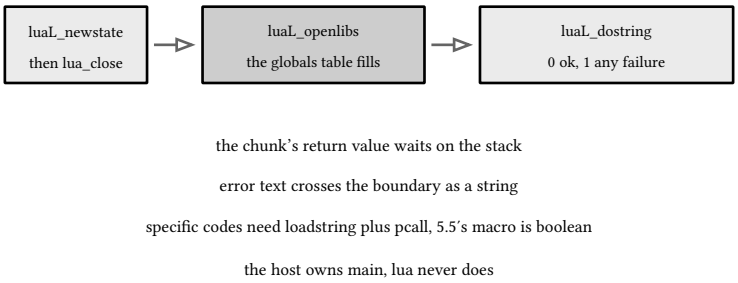


figure 65: a host's first minute, state, libraries, one chunk, error codes home

14.3. the stack

Every value crosses the boundary on one growable stack, pushed by the host or by lua, read by index. Positive indices count from the bottom, negative from the top, and the type predicates split the number subtypes exactly the way book 9, ch 3 (lua 5.5, values, metatables, metamethods) taught them:

```
// five pushes of five different types
lua_pushinteger(L, 7);
lua_pushnumber(L, 2.5);
lua_pushstring(L, "hi");
lua_pushboolean(L, 0);
lua_pushnil(L);
CHECK(lua_gettop(L) == 5, "five pushes, five slots");

// reading: positive from the bottom, negative from the top
CHECK(lua_type(L, 1) == LUA_TNUMBER, "slot 1 holds a number");
CHECK(lua_type(L, -3) == LUA_TSTRING, "negative indices count from the top");
CHECK(lua_type(L, -1) == LUA_TNIL, "nil is pushed like any value");
CHECK(lua_isnumber(L, 1) && lua_isnumber(L, 2),
      "lua_isnumber accepts both subtypes");
CHECK(lua_isinteger(L, 1), "isinteger sees the integer subtype");
```

listing 117: five pushes, both index directions, the subtype predicates (source: lua/samples-c/src/Ch14/stack.c, lines 27-41)

Stack discipline is explicit: `lua_pop` takes from the top, `lua_settop` truncates, `lua_pushvalue` duplicates, and `lua_copy` overwrites one slot with another without moving anything else:

```
  lua_pop(L, 2);
  CHECK(lua_gettop(L) == 3, "pop removes from the top only");
  lua_settop(L, 1);
  CHECK(lua_gettop(L) == 1, "settop truncates in place");

  // duplication and copy, the moves that built-in functions are made of
  lua_pushvalue(L, 1);
  CHECK(lua_gettop(L) == 2 && (int)lua_tointeger(L, -1) == 7,
        "pushvalue duplicates a slot");
  lua_pushinteger(L, 99);
  lua_copy(L, 1, -1);
  CHECK((int)lua_tointeger(L, -1) == 7, "lua_copy overwrites in place");
  CHECK(lua_gettop(L) == 3 && (int)lua_tointeger(L, 3) == 7,
        "the overwritten 99 is gone from that slot");
  lua_pop(L, 3);
```

listing 118: pop, settop, pushvalue, copy (source: lua/samples-c/src/Ch14/stack.c, lines 50-64)

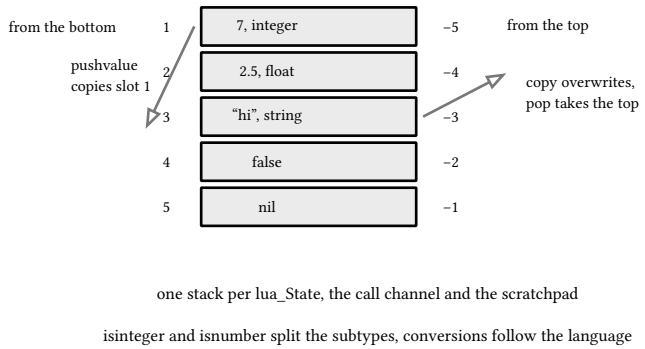


figure 66: one stack, two index directions, the moves between slots

14.4. calling lua protected

`lua_call` jumps unprotected and takes the host down with an error. Every robust host, the stock interpreter included, runs `lua_pcall` instead, with a message handler sitting below the function whose absolute index the call receives. The handler is the one place to build a traceback, because the error is still on the stack and the frames are still alive:

```
// the message handler the interpreter itself uses: stringify the error
// object, then append a traceback below it, and let the handler's return
// replace the error object the host receives
static int msghandler(lua_State *L) {
  const char *msg = lua_tostring(L, 1);
  if (msg == NULL) {
    if (luaL_callmeta(L, 1, "__tostring") &&
        lua_type(L, -1) == LUA_TSTRING)
      return 1;
    msg = lua_pushfstring(L, "(error object is a %s value)",
                          luaL_typename(L, 1));
    lua_remove(L, -2);
  }
  luaL_traceback(L, L, msg, 1);
  return 1;
}
```

listing 119: the message handler, stringify then traceback (source: lua/samples-c/src/Ch14/protected.c, lines 23-38)

```c
// the handler goes below the function, pcall receives its absolute index,
// and the host removes the handler afterwards: pcall never pops it
static int docall(lua_State *L, const char *chunk, int nresults) {
  lua_pushcfunction(L, msghandler);
  int base = lua_gettop(L); // the handler's absolute index
  if (luaL_loadstring(L, chunk) ≠ LUA_OK) {
    lua_remove(L, base); // drop the handler, the message is on top
    return LUA_ERRSYNTAX;
  }
  int status = lua_pcall(L, 0, nresults, base);
  lua_remove(L, base);
  return status;
}
```

listing 120: docall, handler below, absolute index, host cleans up (source: lua/samples-c/src/Ch14/protected.c, lines 40-52)

Results and errors come back on the same stack. A runtime error arrives with the traceback appended, a non-string error object is stringified by the handler instead of lost, and a syntax error never reaches the handler at all:

```c
  // a runtime error through the handler: traceback appended, host alive
  CHECK(docall(L, "local t = nil; t.x = 1", 1) == LUA_ERRRUN,
        "the runtime error returns LUA_ERRRUN");
  const char *msg = lua_tostring(L, -1);
  CHECK(msg ≠ NULL && strstr(msg, "attempt to index a nil value") ≠ NULL,
        "the message names the fault");
  CHECK(msg ≠ NULL && strstr(msg, "traceback") ≠ NULL,
        "the handler appended a traceback");
  CHECK(msg ≠ NULL && strstr(msg, "local 't'") ≠ NULL,
        "the traceback names the local by role");
  lua_pop(L, 1);

  // a non-string error object: the handler stringifies it, nothing is lost
  CHECK(docall(L, "error(setmetatable({}, {__name = 'Token'}))", 1) ==
            LUA_ERRRUN,
        "a table error object is LUA_ERRRUN too");
  msg = lua_tostring(L, -1);
  CHECK(msg ≠ NULL && strstr(msg, "(error object is a table value)") ≠ NULL,
        "the handler stringified the object");
  lua_pop(L, 1);
```

listing 121: the three failure flavors, all survived (source: lua/samples-c/src/Ch14/protected.c, lines 72-91)

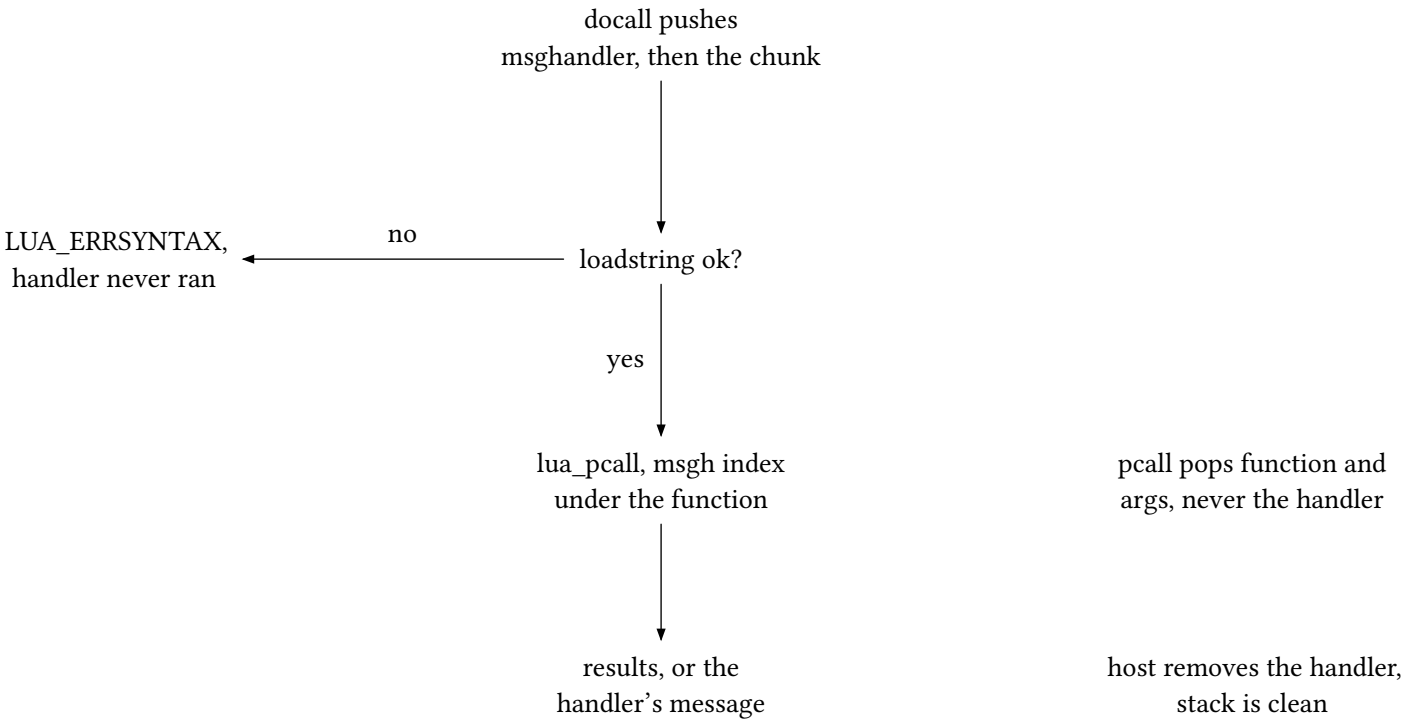


figure 67: one protected call, handler under function, results or one error

14.5. c functions for lua

The seam runs the other way too: a c function is a `lua_CFunction` that receives the state, reads its arguments from the stack, and returns how many results it pushed. `luaL_checkinteger` and `luaL_optinteger` do the argument validation with the stdlib's own error wording, and `lua_gettop` is the arity:

```c
static int l_add(lua_State *L) {
  lua_Integer a = luaL_checkinteger(L, 1);
  lua_Integer b = luaL_checkinteger(L, 2);
  lua_pushinteger(L, a + b);
  return 1;
}

// gettop is the argument count: sum accepts any number of arguments
static int l_sum(lua_State *L) {
  int n = lua_gettop(L);
  lua_Integer s = 0;
  for (int i = 1; i ≤ n; i++)
    s += luaL_checkinteger(L, i);
  lua_pushinteger(L, s);
  lua_pushinteger(L, n); // two results, the sum and the arity
  return 2;
}
```

listing 122: three c functions, checked args, arity, multiple results (source: lua/samples-c/src/Ch14/cfuncs.c, lines 24-40)

Registration is one array. `luaL_newlib` builds the table, registers every `luaL_Reg` pair, and leaves the table on the stack for `lua_setglobal`; `lua_register` plants a single function without any array at all:

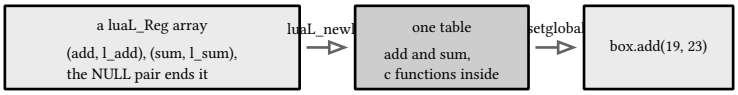
```c
static const luaL_Reg boxlib[] = {
    {"add", l_add},
    {"sum", l_sum},
    {"echo", l_echo},
    {NULL, NULL},
};

int main(void) {
  lua_State *L = luaL_newstate();
  luaL_openlibs(L);

  // luaL_newlib builds the table and registers the array in one call
  luaL_newlib(L, boxlib);
  lua_setglobal(L, "box");

  // lua_register plants a single function without any array
  lua_pushcfunction(L, l_add);
  lua_setglobal(L, "plain_add");
```

listing 123: the luaL_Reg array and both registration forms (source: lua/samples-c/src/Ch14/cfuncs.c, lines 48-65)



arguments at 1..n, the return counts what was pushed

checkinteger raises with the stdlib wording, optinteger defaults

getinfo classifies the newcomer exactly like print, what C

this is how all 127 stdlib c functions are made

figure 68: registration, the seam from lua's side

14.6. userdata in outline

Userdata is raw c memory wearing a lua value: `lua_newuserdata` allocates, the host writes its struct, and a metatable named in the registry guards every later access. `luaL_checkudata` compares the name and refuses the wrong type with a loud error, which is the entire type safety story:

```c
#define COUNTER_NAME "book.counter"

typedef struct {
  lua_Integer n;
  char owner[16];
} Counter;

static Counter *check_counter(lua_State *L) {
  return luaL_checkudata(L, 1, COUNTER_NAME);
}
```

listing 124: the struct, and checked access through the metatable name (source: lua/samples-c/src/Ch14/userdata.c, lines 24-34)

Methods are ordinary c functions whose first argument is the value itself, the self convention, and `__gc` is the one hook into the collector: the finalizer runs when the last reference drops, once per value, and `lua_close` finalizes whatever survived:

```c
static int counter_bump(lua_State *L) {
  Counter *c = check_counter(L);
  c->n += luaL_optinteger(L, 2, 1);
  lua_settop(L, 1);
  return 1; // return self, calls chain
}

static int counter_get(lua_State *L) {
  Counter *c = check_counter(L);
  lua_pushinteger(L, c->n);
  return 1;
}

// __gc sees the value one last time before the memory goes back
static int counter_gc(lua_State *L) {
  Counter *c = check_counter(L);
  if (strcmp(c->owner, "suite") == 0)
    finalized++;
  return 0;
```

listing 125: bump returns self, get reads, gc counts (source: lua/samples-c/src/Ch14/userdata.c, lines 36-54)

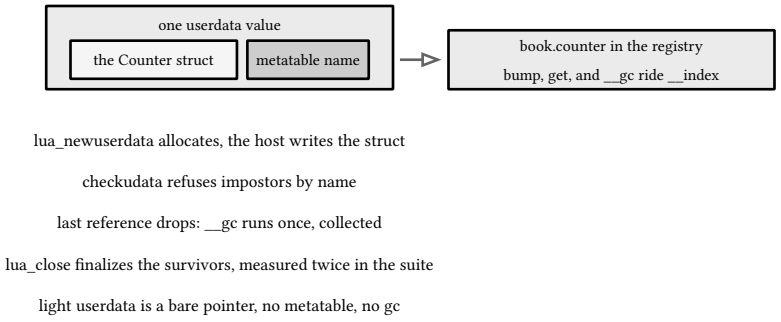


figure 69: a counter value, c memory behind a lua face

14.7. references

The registry is a table the host reaches through the `LUA_REGISTRYINDEX` pseudo-index. In 5.5 its fixed integer slots are exactly two, `LUA_RIDX_GLOBALS` at 2 and `LUA_RIDX_MAINTHREAD` at 3, and `package.loaded` lives at the `"_LOADED"` string key, verified by identity against the global it publishes:

```c
// the registry is a table behind a pseudo-index the api reserves
CHECK(lua_type(L, LUA_REGISTRYINDEX) == LUA_TTABLE,
      "the registry sits behind its pseudo-index");

// its fixed integer slots in 5.5: the globals table and the main thread
lua_rawgeti(L, LUA_REGISTRYINDEX, LUA_RIDX_GLOBALS);
CHECK(lua_type(L, -1) == LUA_TTABLE, "LUA_RIDX_GLOBALS is a table");
lua_getglobal(L, "math");
CHECK(lua_rawequal(L, -1, -3) != 1,
      "the registry globals slot is not the math table");
lua_pop(L, 2);
lua_rawgeti(L, LUA_REGISTRYINDEX, LUA_RIDX_MAINTHREAD);
CHECK(lua_type(L, -1) == LUA_TTHREAD, "LUA_RIDX_MAINTHREAD is a thread");
lua_pop(L, 1);

// package.loaded lives at the "_LOADED" string key, not a fixed slot
lua_getfield(L, LUA_REGISTRYINDEX, LUA_LOADED_TABLE);
CHECK(lua_type(L, -1) == LUA_TTABLE, "the _LOADED key holds a table");
lua_getglobal(L, "package");
lua_getfield(L, -1, "loaded");
CHECK(lua_rawequal(L, -1, -3) == 1,
      "that table is exactly package.loaded");
lua_pop(L, 3);
```

listing 126: the registry, its two fixed slots, and the _LOADED key (source: lua/samples-c/src/Ch14/refs.c, lines 27-49)

`luaL_ref` pins any value at a fresh integer key and pops it, `lua_rawgeti` pushes it back, and `luaL_unref` releases the pin. The 5.5 release keeps a freelist: the freed slot reads back as the next-free integer rather than nil, and slot 1 heads the list, so the same integer is reused before any new one is minted:

```c
// a reference pins a value with an integer key, then unpins it
(void)(luaL_dostring(L, "return function(x) return x * 2 end"));
int ref = luaL_ref(L, LUA_REGISTRYINDEX);
CHECK(ref ≥ 0, "luaL_ref returns an integer key");
CHECK(lua_gettop(L) == 0, "luaL_ref pops what it pinned");

lua_rawgeti(L, LUA_REGISTRYINDEX, ref);
CHECK(lua_isfunction(L, -1), "rawgeti pushes the pinned function back");
lua_pushinteger(L, 21);
CHECK(lua_pcall(L, 1, 1, 0) == LUA_OK, "the pinned value still calls");
CHECK((int)lua_tointeger(L, -1) == 42, "the closure kept its upvalue");
lua_pop(L, 1);

// after unref the pinned value is gone: 5.5 keeps a freelist, the freed
// slot reads back as the next-free integer and t[1] heads the list
luaL_unref(L, LUA_REGISTRYINDEX, ref);
lua_rawgeti(L, LUA_REGISTRYINDEX, ref);
CHECK(lua_type(L, -1) ≠ LUA_TFUNCTION, "unref removed the pinned value");
lua_pop(L, 1);
lua_rawgeti(L, LUA_REGISTRYINDEX, 1);
CHECK(lua_type(L, -1) == LUA_TNUMBER && (int)lua_tointeger(L, -1) == ref,
      "the freed ref now heads the freelist at slot 1");
```

listing 127: pin, call through, release, and the freelist it joins (source: lua/samples-c/src/Ch14/refs.c, lines 51-72)

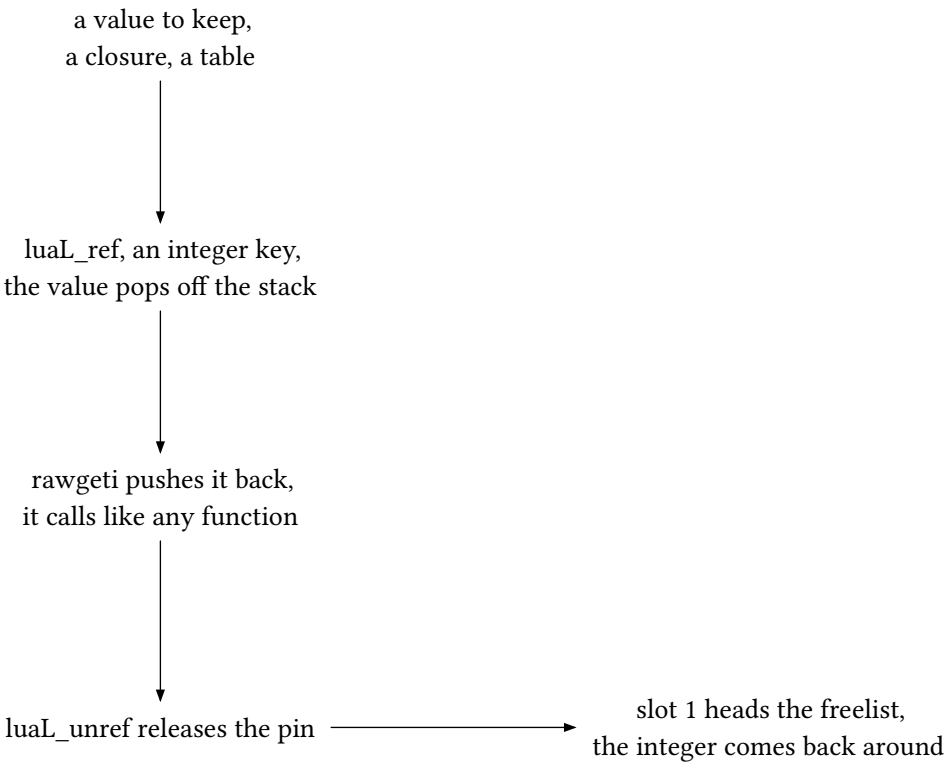


figure 70: a reference's life, pin, use, freelist

14.8. the state model

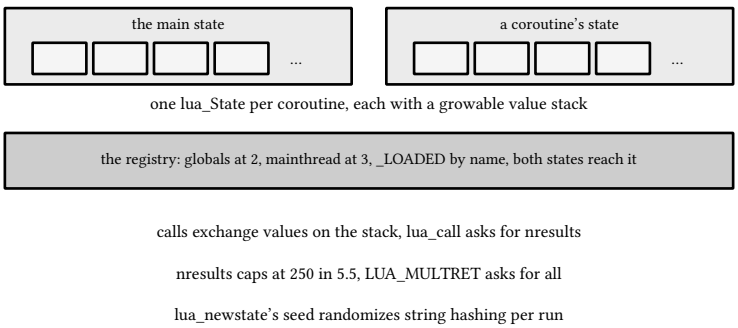


figure 71: what a host drives, states, their stacks, and the registry

The api the hosts use is one `lua_State` per coroutine plus the main state, a growable value stack per state, and the registry, measured above as globals at slot 2, the main thread at 3, and `package.loaded` behind the `"_LOADED"` string key that book 9, ch 13 (lua 5.5, stdlib 3: coroutine, debug, package) visited. Calls exchange values over that stack, `lua_call` and friends requesting `nresults` return slots, and 5.5 specifies the limit book 9, ch 15 (lua 5.5, incompatibilities with 5.4) lists among incompatibilities: at most 250, with `LUA_MULTRET` for everything. `lua_newstate` gained a string hashing seed parameter, which is why table iteration order differs between runs and why `math.randomseed` exists separately for reproducible sequences.

NOTE: what stays invisible

The fourth 5.5 headline, more compact internal arrays, is an implementation change with no observable language semantics, and this book's binary is the only lua in the tree, so there is no 5.4 baseline to measure memory against. The claim is documented, not tested, and the coverage matrix says so.

14.9. chunk names

A chunk's name decides its error prefix and tracebacks. Source from `loadfile` gets the file path, a string from `load` defaults to a `[string "..."]` form, and the `"=name"` form uses the name literally, the way to make generated code debuggable:

```
{name = "chunk names pick the error prefix", fn = function()
  local named = select(2, pcall(load("error('e')", "=NAME")))
  T.ok(named:find("^NAME:1:"), named)
  local anon = select(2, pcall(load("error('e')")))
  T.ok(anon:find('^%[string "error%(\'e\'%)"%]:1:'), anon)
  local path = "ch12-err.lua"
  local w = assert(io.open(path, "w"))
  w:write("error('fromfile')\n")
  w:close()
  local filed = select(2, pcall(assert(loadfile(path))))
  T.ok(filed:find("ch12%-err%.lua:1:"), filed)
  os.remove(path)
end},
```

listing 128: the three prefix forms (source: lua/samples/ch14_capi.lua, lines 56-68)

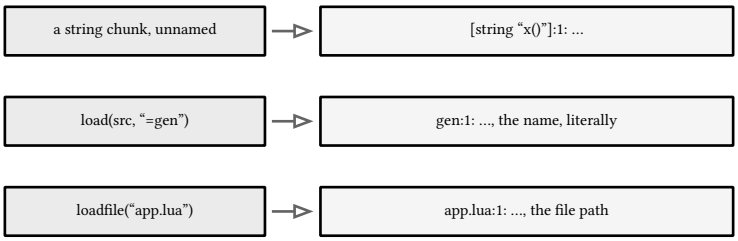


figure 72: three source name forms and the prefixes they buy

sources: lua.org manual 5.5 sections 1 and 4, the auxiliary library sections on tracebacks, references, and userdata, and the luac man page in the 5.5.1 distribution, accessed 2026-09-08 and 2026-09-21. Opcode names, dump sizes, the c census, the dostring macro shape, the registry slots, and the ref freelist verified live with lua 5.5.1: 7 tests green through `make verify-lua`, 89 c checks across 6 hosts green through `make verify-lua-c`.

15. incompatibilities with 5.4

The manual's section 8 lists four language breaks and one library break. This chapter pins each against the binary, then adds three more observable changes the manual made outside section 8, each verified against 5.5.1 and compared with the 5.4 documentation.

15.1. the language list

`global` is reserved, softened in the stock build by the `LUA_COMPAT_GLOBAL` default documented in book 9, ch 2 (lua 5.5, lexical structure): the word stays usable as a name, but the declaration statement still enforces itself. The `for` control variable is read only, and the migration is the manual's own suggestion, shadow it with a local. A chain of `__call` metamethods caps at 15 objects. An error whose object is nil gets the string `"<no error object>"` instead:

```
  {name = "language: global is a reserved word, softened by compat", fn = function()
    T.eq(load("local global = 1") ~= nil, true)
    T.eq(load("local end = 1"), nil)
    T.eq(load("global x; x = 1; y = 2"), nil)
    T.ok(tostring(select(2, load("global x; x = 1; y = 2"))):find("variable 'y' not declared"), "declared
check")
  end},

  {name = "language: the for control variable is read only", fn = function()
    local f, e = load("for i = 1, 3 do i = i + 1 end")
    T.eq(f, nil)
    T.ok(e:find("attempt to assign to const variable 'i'", 1, true), e)
    local ok = 0
    for i = 1, 3 do
      local i = i + 10
      ok = ok + i
    end
    T.eq(ok, 36)
  end},

  {name = "language: __call chains cap at 15 objects", fn = function()
    T.eq(buildchain(15)(), "done")
    local ok, err = pcall(buildchain(16))
    T.eq(ok, false)
    T.eq(err, "'__call' chain too long")
  end},

  {name = "language: a nil error object becomes a string", fn = function()
    local ok, err = pcall(function() error(nil) end)
    T.eq(ok, false)
    T.eq(type(err), "string")
    T.eq(err, "<no error object>")
```

listing 129: reserved global under compat, read only loop variables, the call chain cap, nil error objects (source: lua/samples/ch15_incompat.lua, lines 16-46)

break	what 5.5 does
global reserved	compat un-reserves it, the declaration stands
the for variable	read only, shadow it with a local
__call chains	capped at 15 objects
a nil error object	becomes the string “<no error object>”
each pinned against the binary in the listing above	

figure 73: the four language breaks of manual section 8

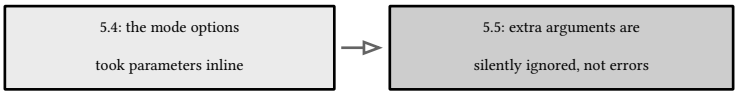
15.2. the library list

The garbage collector no longer takes parameters through the `"incremental"` and `"generational"` options. Extra arguments are not an error, they are ignored, and the parameters moved to `"param"` as book 9, ch 8 (lua 5.5, environments, load, gc) showed:

```
{name = "library: collectgarbage takes params through param only", fn = function()
  T.eq({collectgarbage("incremental", 200, 100, 13)}, {"generational"})
  T.eq(collectgarbage("param", "pause"), 250)
  collectgarbage("incremental")
end},

{name = "beyond the list: the default mode is generational", fn = function()
  T.eq(collectgarbage("generational"), "incremental")
  collectgarbage("generational")
```

listing 130: ignored arguments, the param option (source: lua/samples/ch15_incompat.lua, lines 49-57)



the knobs moved to one “param” option, chapter 8’s figure

the manual lists this as the release’s one library break

figure 74: collectgarbage’s options before and after the release

15.3. beyond the list

Three observable deltas sit outside section 8. The default collector mode is generational, where 5.4 defaulted to incremental, and the manual states it only in its introduction. `math.floor`, `math.ceil`, and `math.modf` return integers when the result fits, a spec change in their manual entries. And the arithmetic error messages were reworded to name the operation and both operand types, where 5.4 said “attempt to perform arithmetic on a”:

```
{name = "beyond the list: floor, ceil, and modf return integers", fn = function()
  T.eq(math.type(math.floor(3.5)), "integer")
  T.eq(math.type(math.ceil(-0.5)), "integer")
  T.eq(math.type(math.modf(9.9)), "integer")
  T.eq(math.type(math.floor(1e100)), "float")
end},

{name = "beyond the list: division error messages were reworded", fn = function()
  T.throws(function() return 1 // 0 end, "attempt to divide by zero")
  T.throws(function() return 1 % 0 end, "attempt to perform 'n%0'")
  T.throws(function() return "a" + 1 end, "attempt to add a 'string' with a 'number'")
  T.throws(function() return {} < 1 end, "attempt to compare table with number")
end},

{name = "beyond the list: varargs are tables with names", fn = function()
  local function f(...rest) return rest.n, rest[1] end
  T.eq({f("a", "b")}, {2, "a"})
  local g = load("local function h(...xs) xs = 1 end")
  T.eq(g, nil)
```

listing 131: default mode, integer floors, the new message wording (source: lua/samples/ch15_incompat.lua, lines 60-78)

delta	where the manual says it
the default mode	generational now, stated only in the introduction
floor, ceil, modf	integer when the result fits, a change in their entries
arithmetic errors	name the operation and both operand types
the vararg rework	compatible, but the largest semantic change, chapter 6

figure 75: three deltas outside manual section 8

The vararg rework, the tables and names of book 9, ch 6 (lua 5.5, functions), is technically compatible, old … code keeps working, but it is the largest semantic change of the release and belongs in any migration review:

```
{name = "beyond the list: varargs are tables with names", fn = function()
  local function f(...rest) return rest.n, rest[1] end
  T.eq({f("a", "b")}, {2, "a"})
  local g = load("local function h(...xs) xs = 1 end")
  T.eq(g, nil)
```

listing 132: vararg tables as the release's real headline (source: lua/samples/ch15_incompat.lua, lines 74-78)

15.4. the api list

call	the change
nresults	capped at 250, LUA_MULTRET for everything
lua_newstate	a third parameter seeds string hashing
lua_closethread	replaces lua_resetthread, from = null
lua_dump	the writer runs one extra time to signal the end
LUA_GCPARAM	replaces LUA_GCINC and LUA_GCGEN
lua_pushvfstring	reports errors instead of raising them
lua_setcstacklimit	deprecated, removable

figure 76: the c side changes, documented only, not exercised by the chapter 14 hosts

The c side changes, unverifiable from pure lua and recorded here as documentation: `nresults` in `lua_call` and related functions is capped at 250, with `LUA_MULTRET` for more. `lua_newstate` takes a third parameter, a seed for string hashing. `lua_resetthread` is deprecated in favor of `lua_closethread` with a null `from`, `lua_setcstacklimit` is deprecated and removable, `lua_dump` now specifies its stack behavior and calls the writer one extra time to signal the end, `LUA_GCINC` and `LUA_GCGEN` gave way to `LUA_GCPARAM`, and `lua_pushvfstring` reports errors instead of raising them.

sources: lua.org manual 5.5 section 8 in full, sections 1, 2.5, 3.4, 6.8 for the beyond-the-list items, accessed 2026-09-08, plus the 5.5.1 source tree's luaconf.h, ldo.h, lstate.c for limits. 10 tests green through `make verify-lua`.

16. idioms

Lua's object model is three moves: a table of methods, `__index` pointing there, and constructor sugar. The chapter's vector class puts every metamethod family to work at once, arithmetic, equality, ordering, length, and printing, and the inheritance pattern is one more line, a class whose metatable's `__index` is the superclass:

```lua
-- chapter 16 probes: idiomatic lua, class patterns, error handling, forwarding
local T = require("lib")

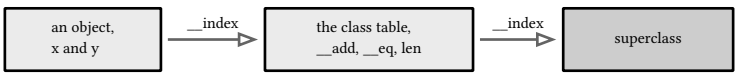
local Vec = {}
Vec.__index = Vec
Vec.__add = function(a, b) return Vec.new(a.x + b.x, a.y + b.y) end
Vec.__eq = function(a, b) return a.x == b.x and a.y == b.y end
Vec.__len = function() return 2 end
Vec.__tostring = function(v) return "Vec(" .. v.x .. "," .. v.y .. ")" end
Vec.__lt = function(a, b) return (a.x ^ 2 + a.y ^ 2) < (b.x ^ 2 + b.y ^ 2) end

function Vec.new(x, y) return setmetatable({x = x, y = y}, Vec) end
function Vec:norm2() return self.x ^ 2 + self.y ^ 2 end
function Vec:scale(k)
  self.x, self.y = self.x * k, self.y * k
  return self
end

local Shape = {}
Shape.__index = Shape
function Shape.new(kind) return setmetatable({kind = kind}, Shape) end
function Shape:name() return "shape:" .. self.kind end

local Circle = setmetatable({}, {__index = Shape})
Circle.__index = Circle
function Circle.new(r)
  return setmetatable({r = r}, Circle)
end
function Circle:name() return "circle(" .. self.r .. ")" end
```

listing 133: the vector class and the shape to circle chain (source: lua/samples/ch16_idioms.lua, lines 1-30)



a constructor is a function that sets the metatable and returns

mutation methods return self, that is the fluent chain

inheritance is the second link, one line of setup

figure 77: the object model as one chain of index links

```
{name = "classes are metatables plus sugar", fn = function()
  local v = Vec.new(1, 2)
  T.eq(v:norm2(), 5)
  T.eq(Vec, getmetatable(v))
  T.eq(tostring(Vec.new(3, 4)), "Vec(3,4)")
  T.eq(#Vec.new(0, 0), 2)
  T.eq(Vec.new(1, 1) + Vec.new(2, 3), Vec.new(3, 4))
  T.eq(Vec.new(1, 1) == Vec.new(1, 1), true)
  T.eq(Vec.new(1, 1) < Vec.new(2, 2), true)
end},

{name = "mutation methods return self for chaining", fn = function()
  local v = Vec.new(1, 2):scale(2):scale(0.5)
  T.eq(v, Vec.new(1, 2))
end},
```

listing 134: operator classes, chaining through returned self (source: lua/samples/ch16_idioms.lua, lines 32-46)

Mutation methods return `self`, which is what makes fluent apis readable, and constructors are just functions that set a metatable.

16.1. errors

`error` with level 2 blames the caller, which is what argument checking wants, with one caveat the suite tripped over: a tail call erases the frame the blame reads, so the level 2 position comes back empty when the checked call sits in `return` position. Keep the call out of a tail or the message loses its location:

```
{name = "inheritance is a one line index chain", fn = function()
  local c = Circle.new(5)
  T.eq(c:name(), "circle(5)")
  T.eq(getmetatable(c).__index, Circle)
  T.eq(getmetatable(Circle).__index, Shape)
  local s = Shape.new("plain")
  T.eq(s:name(), "shape:plain")
end},

{name = "error levels blame the caller", fn = function()
  local function check_positive(n)
    if n ≤ 0 then error("expected positive, got " .. n, 2) end
    return n
  end
  local ok, err = pcall(function()
    local r = check_positive(-1)
    return r
  end)
  T.eq(ok, false)
  T.ok(err:find("ch16_idioms%.lua:%d+: expected positive"), err)
end},
```

listing 135: the one line index chain, level 2 blame (source: lua/samples/ch16_idioms.lua, lines 48-68)

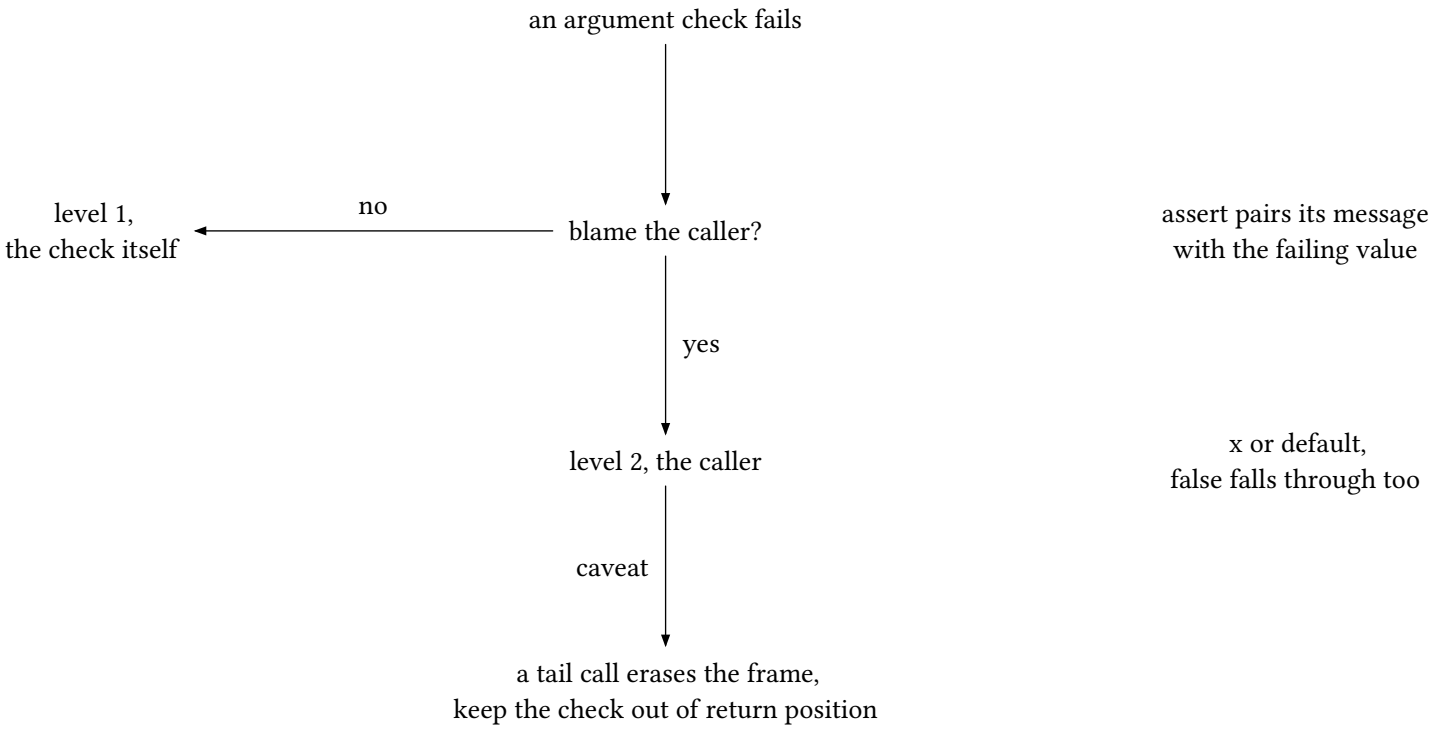


figure 78: choosing the error level, with the tail call caveat

`x or default` is the default parameter idiom, with the standing caveat that false falls through to the default too, which for boolean flags is a bug and for absent values is exactly right.

16.2. iteration and forwarding

Two iterator shapes cover most needs: the stateless triple that avoids allocating anything per step, and the coroutine wrap that can express arbitrarily complex producers as straight line code:

```lua
{name = "assert pairs with custom messages", fn = function()
  local ok, err = pcall(assert, false, "value must be a number")
  T.eq(ok, false)
  T.eq(err, "value must be a number")
  T.eq(assert(42, "never shown"), 42)
end},

{name = "or covers defaults, carefully", fn = function()
  local function greet(name) name = name or "world" return "hi " .. name end
  T.eq(greet(), "hi world")
  T.eq(greet("lua"), "hi lua")
  T.eq(greet(false), "hi world")
end},

{name = "stateless and closure iterators", fn = function()
  local function iter(state, control)
    if control < state.stop then return control + 1 end
  end
  local collected = {}
  for i in iter, {stop = 3}, 0 do collected[#collected + 1] = i end
  T.eq(collected, {1, 2, 3})
  local function words(s)
    return coroutine.wrap(function()
      for w in s:gmatch("%a+") do coroutine.yield(w) end
    end)
  end
  local got = {}
  for w in words("one two") do got[#got + 1] = w end
  T.eq(got, {"one", "two"})
end},
```

listing 136: stateless triple, coroutine producer, assert with messages, defaults through or (source: lua/samples/ch16_idioms.lua, lines 70-99)

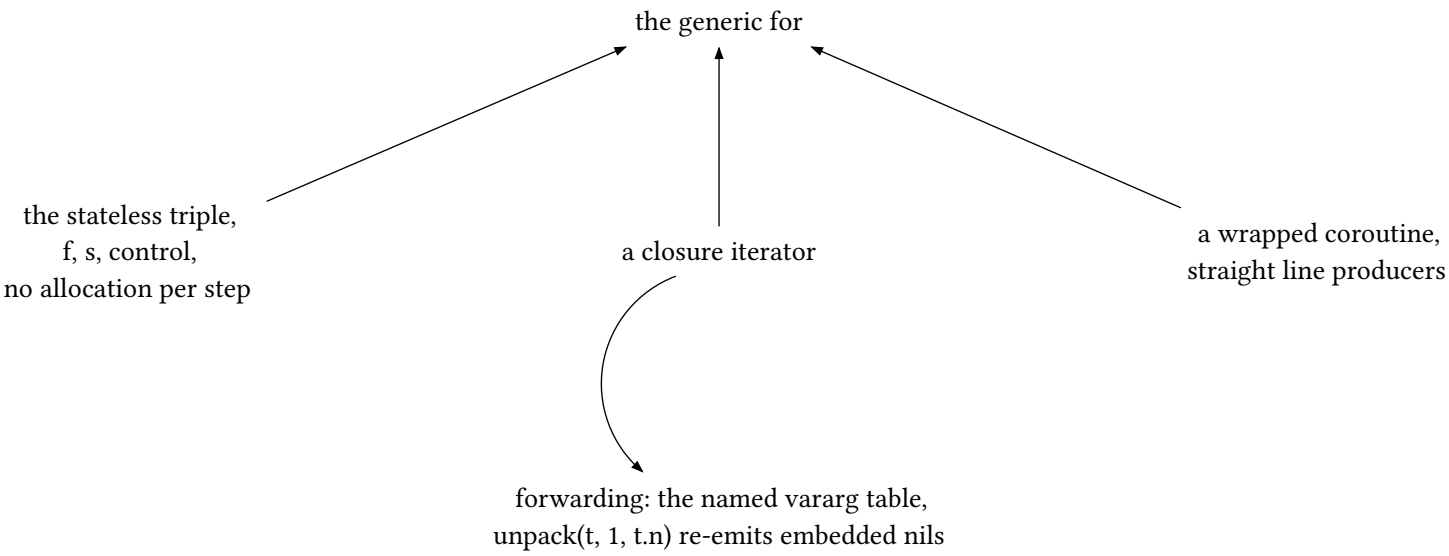


figure 79: three producers, one loop, and 5.5 forwarding

Forwarding varargs in 5.5 reads better than any earlier release: the named table carries the count, `unpack` to position `n` re-emits the list including embedded nils.

16.3. state and caching

Private state is a closure over locals, two counters from one factory share nothing. Memoization is a table and a call counter, and the protected call wrapper returns its success flag first, which composes into the result type shape the javascript book built with generics, here in seven lines:

```
{name = "vararg forwarding through the table", fn = function()
  local function logged(prefix, ...args)
    return prefix .. " " .. select("#", ...) .. " " .. tostring(args[1]),
      table.unpack(args, 1, args.n)
  end
  T.eq({logged("got", "a", "b")}, {"got 2 a", "a", "b"})
  local function passthrough(...) return ... end
  T.eq({passthrough(table.unpack({"x", "y"}, 1, 2))}, {"x", "y"})
end},

{name = "memoization counts and caches", fn = function()
  local calls = 0
  local fib
  local slow = function(n)
    calls = calls + 1
    if n < 2 then return n end
    return fib(n - 1) + fib(n - 2)
  end
  local cache = {}
  fib = function(n)
    local hit = cache[n]
    if hit then return hit end
    local v = slow(n)
    cache[n] = v
    return v
  end
  T.eq(fib(10), 55)
  T.eq(fib(10), 55)
  T.ok(calls < 20, "memoized calls: " .. calls)
end},

{name = "private state lives in closure upvalues", fn = function()
  local function new_counter()
    local n = 0
    return {
      inc = function() n = n + 1 return n end,
      get = function() return n end,
    }
  end
  local a, b = new_counter(), new_counter()
  a.inc()
  a.inc()
  T.eq(a.get(), 2)
  T.eq(b.get(), 0)
end},
```

listing 137: vararg forwarding detail, memoization, closure state, pcall as result (source: lua/samples/ch16_idioms.lua, lines 101-145)

```
  {name = "memoization counts and caches", fn = function()
    local calls = 0
    local fib
    local slow = function(n)
      calls = calls + 1
      if n < 2 then return n end
      return fib(n - 1) + fib(n - 2)
    end
    local cache = {}
    fib = function(n)
      local hit = cache[n]
      if hit then return hit end
      local v = slow(n)
      cache[n] = v
      return v
    end
    T.eq(fib(10), 55)
    T.eq(fib(10), 55)
    T.ok(calls < 20, "memoized calls: " .. calls)
  end},
```

listing 138: memoization with call counting (source: lua/samples/ch16_idioms.lua, lines 111-131)

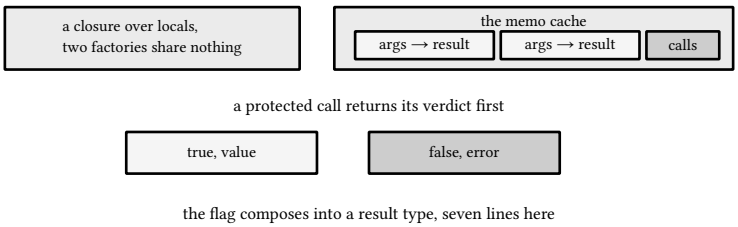


figure 80: closure state, the memo cache, and pcall as a result type

16.4. the sandbox shape

The load environment from book 9, ch 8 (lua 5.5, environments, load, gc) is the whole sandboxing story: hand in exactly the functions the embedded code may use, and nothing else exists, not even `_G`:

```
  {name = "sandboxes stay closed without _g", fn = function()
    local env = {load = load, math = math, type = type}
    local f = load("return math.floor(2.7) .. type(_G)", "c", "t", env)
    T.eq(f(), "2nil")
  end},
}
```

listing 139: a three function environment (source: lua/samples/ch16_idioms.lua, lines 160-165)

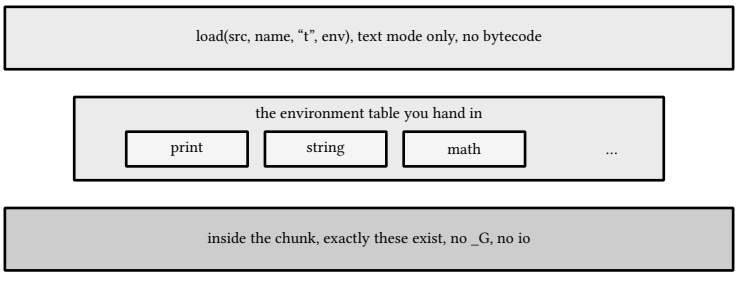


figure 81: the sandbox, an environment holding exactly what you grant

sources: lua.org manual 5.5 sections 2.4, 3.4.11, 6.2, programming in lua fourth edition idioms as background, accessed 2026-09-08. Verified live with lua 5.5.1, 12 tests green through `make verify-lua`.

17. ffi: c without the api

Chapter 14 showed the c api from the host's side, a stack protocol the embedding program drives. LuaJIT's ffi library is the other road to c: lua code declares the signatures, loads the library, and calls the functions directly, with no binding module and no glue c compiled against anything. The tree keeps a second interpreter for exactly this, the LuaJIT rolling 2.1 that `make sqlite-tools` builds beside the sqlite 3.53.4 dll this chapter calls, and the ffi lane under `books/lua/jit/` uses the pair to give the editor modules a real database file to persist into.

The price is printed in the interpreter's own banner: LuaJIT speaks lua 5.1, no `goto`, no integer division operator, none of the 5.2 and later stdlib. That constraint is not incidental here, it is the load bearing wall between the two runtimes, and this chapter ends with the rule the repo enforces to keep them apart.

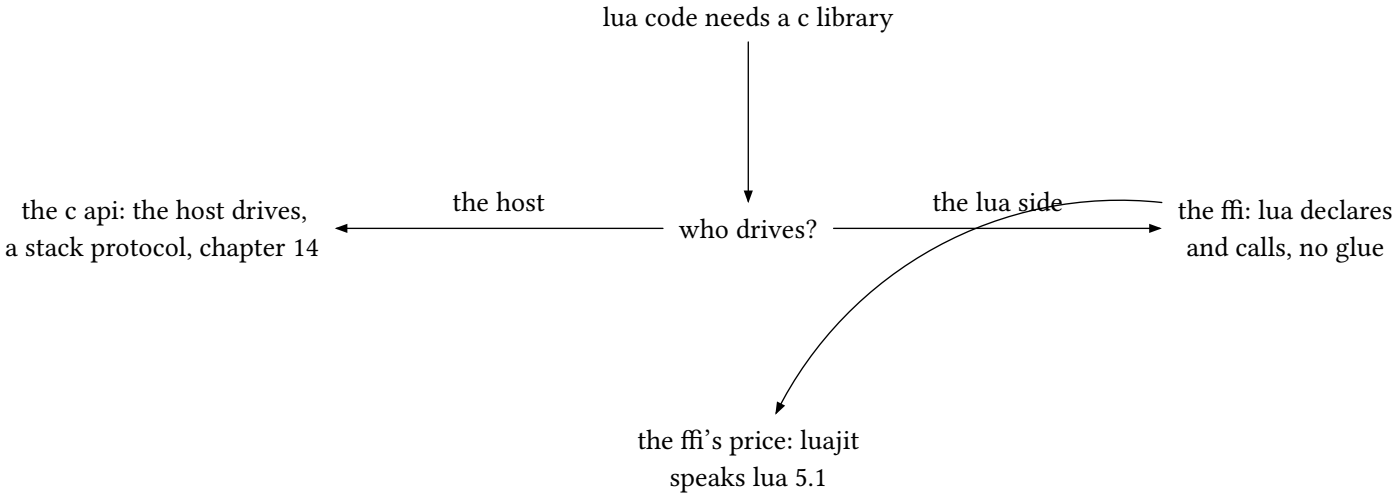


figure 82: the two roads to c, who drives the calls

17.1. the boundary

`ffi.cdef` parses c declarations. Nothing is compiled and nothing is verified against the dll: cdef lines are a promise the caller makes, and the first call through a wrong promise is memory corruption, not an error message. The wrapper's promise covers the seventeen functions the editor lane needs, no more:

```
ffi.cdef[[
const char *sqlite3_libversion(void);
const char *sqlite3_errmsg(void *db);
int sqlite3_open(const char *filename, void **db);
int sqlite3_close(void *db);
int sqlite3_exec(void *db, const char *sql,
                 int (*cb)(void *, int, char **, char **), void *arg, char **err);
void sqlite3_free(void *p);
int sqlite3_prepare_v2(void *db, const char *sql, int nbyte, void **stmt, const char **tail);
int sqlite3_bind_text(void *stmt, int idx, const char *text, int n, void *destructor);
int sqlite3_bind_int64(void *stmt, int idx, long long value);
int sqlite3_step(void *stmt);
int sqlite3_finalize(void *stmt);
int sqlite3_column_count(void *stmt);
const char *sqlite3_column_name(void *stmt, int idx);
int sqlite3_column_type(void *stmt, int idx);
const char *sqlite3_column_text(void *stmt, int idx);
long long sqlite3_column_int64(void *stmt, int idx);
double sqlite3_column_double(void *stmt, int idx);
]]
```

listing 140: the whole cdef block, the signatures the wrapper promises (source: lua/jit/sqlite.lua, lines 5-24)

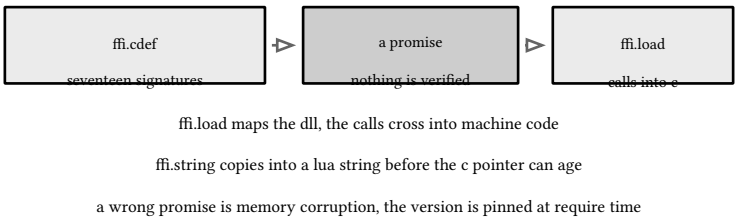


figure 83: the boundary, declarations as promises, then live calls

`ffi.load` maps the dll into the process, `lib.sqlite3_libversion()` is then an ordinary lua function call that crosses into machine code from the dll, and `ffi.string` copies the returned `const char *` into a lua string before the c pointer can age. The module refuses to load against any other build:

```
local lib = ffi.load(((arg and arg[0] or "run-jit.lua"):match("^(.*)[/\\]") or ".")
  .. "/../../../tools/sqlite/build/bin/sqlite3.dll")

function M.version()
  return ffi.string(lib.sqlite3_libversion())
end

assert(M.version() == "3.53.4",
  "sqlite3.dll reports " .. M.version() .. ", this wrapper pins 3.53.4" ..
  " (rebuild with: make sqlite-tools)")
```

listing 141: load the dll and pin the version at require time (source: lua/jit/sqlite.lua, lines 34-43)

17.2. the statement lifecycle

The c api's shape survives the wrapper untouched: open returns a connection, statements are prepared, bound, stepped, and finalized, and every non-zero result code becomes a lua error through `sqlite3_errmsg`. Connections and statements are thin tables whose only job is holding the cdata handles where the collector can see them:

```lua
function M.open(path)
  local box = ffi.new("void *[1]")
  if lib.sqlite3_open(path, box) ~= SQLITE_OK then
    error("sqlite3_open failed: " .. ffi.string(lib.sqlite3_errmsg(box[0])), 2)
  end
  -- the table pins the cdata handle: lua holds the reference, sqlite owns the memory
  return setmetatable({ db = box[0] }, { __index = Conn })
end

function Conn:exec(sql)
  local err = ffi.new("char *[1]")
  if lib.sqlite3_exec(self.db, sql, nil, nil, err) ~= SQLITE_OK then
    local msg = "sqlite3_exec failed: " .. ffi.string(err[0])
    lib.sqlite3_free(err[0])
    error(msg, 2)
  end
  return true
end

function Conn:prepare(sql)
  local box = ffi.new("void *[1]")
  if lib.sqlite3_prepare_v2(self.db, sql, #sql, box, nil) ~= SQLITE_OK then
    fail(self.db, "sqlite3_prepare")
  end
  return setmetatable({ handle = box[0], db = self.db }, { __index = Stmt })
end
```

listing 142: open with an out pointer, exec with the error slot, prepare (source: lua/jit/sqlite.lua, lines 51-76)

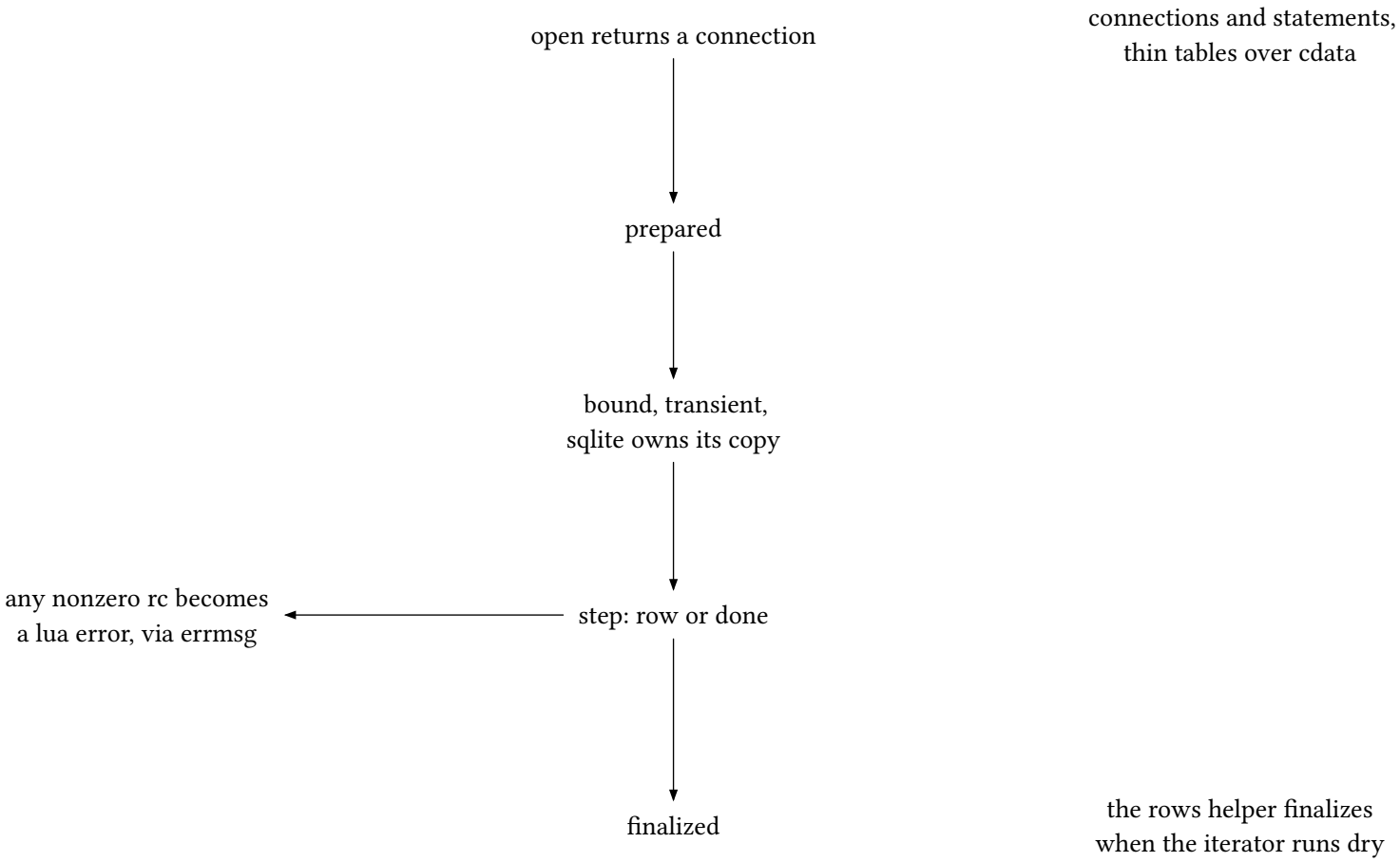


figure 84: a statement's life, prepare to finalize

The destructor argument of `sqlite3_bind_text` is where ownership is decided. SQLite defines two constants that are function pointer casts of 0 and −1: static promises sqlite the bytes stay valid until the statement is rebound, transient tells it to copy at bind time. A lua string cannot promise its bytes to a c library, the collector is free to reuse them once lua forgets the reference, so the wrapper always hands over transient and lets sqlite own its copy:

```
function Stmt:bind_text(i, s)
  s = tostring(s)
  if lib.sqlite3_bind_text(self.handle, i, s, #s, TRANSIENT) ~= SQLITE_OK then
    fail(self.db, "sqlite3_bind_text")
  end
  return true
end

function Stmt:bind_int(i, n)
  if lib.sqlite3_bind_int64(self.handle, i, n) ~= SQLITE_OK then
    fail(self.db, "sqlite3_bind_int")
  end
  return true
end

function Stmt:step()
  local rc = lib.sqlite3_step(self.handle)
  if rc == SQLITE_ROW then return "row" end
  if rc == SQLITE_DONE then return "done" end
  fail(self.db, "sqlite3_step")
end
```

listing 143: bind text and int, step reporting row or done (source: lua/jit/sqlite.lua, lines 105-125)

The query helper wraps that lifecycle in one closure. It finalizes the statement exactly when the iterator runs dry, which makes breaking out of a `rows` loop early a leak, a leak the suite found for real on its first red run: a lookup that returned from inside the loop left the statement alive, and sqlite refused the close with `unable to close due to unfinalized statements`, exactly as the api documents. Plain `sqlite3_close`, not the `close_v2` variant that defers the problem to the collector, is the deliberate choice:

```
function Conn:rows(sql, params)
  local stmt = self:prepare(sql)
  if params then
    for i, v in ipairs(params) do
      if type(v) == "string" then stmt:bind_text(i, v) else stmt:bind_int(i, v) end
    end
  end
  local names = {}
  for i = 0, stmt:column_count() - 1 do names[#names + 1] = stmt:name(i) end
  return function()
    if stmt:step() ~= "row" then
      stmt:finalize()
      return nil
    end
    local row = {}
    for i = 0, #names - 1 do row[names[i + 1]] = stmt:value(i) end
    return row
  end
end

function Conn:close()
  if lib.sqlite3_close(self.db) ~= SQLITE_OK then
    fail(self.db, "sqlite3_close")
  end
  return true
end
```

listing 144: the rows helper finalizes on exhaustion, close stays strict (source: lua/jit/sqlite.lua, lines 78-103)

Column values come back through `sqlite3_column_type`, integers and floats through `tonumber`, text through `ffi.string`, null as nil, and the index is 0-based because the wrapper does not hide which convention it stands on:

```
-- column index i is 0-based, the c convention the wrapper deliberately does not hide
function Stmt:value(i)
  local kind = lib.sqlite3_column_type(self.handle, i)
  if kind == SQLITE_NULL then return nil end
  if kind == SQLITE_INTEGER then return tonumber(lib.sqlite3_column_int64(self.handle, i)) end
  if kind == SQLITE_FLOAT then return tonumber(lib.sqlite3_column_double(self.handle, i)) end
  return ffi.string(lib.sqlite3_column_text(self.handle, i))  -- text and blob
end
```

listing 145: typed column reads (source: lua/jit/sqlite.lua, lines 142-149)

17.3. who owns what

Every c value in the process is cdata, a lua value carrying a c type identity. The collector tracks cdata like any other value, and that is the whole gc story: handles live in lua tables that outlive the calls made through them, `ffi.string` copies c memory into lua memory instead of aliasing it, and memory sqlite allocates is returned to sqlite, the error string from `exec` is read then freed in the same breath:

```lua
function Conn:exec(sql)
  local err = ffi.new("char *[1]")
  if lib.sqlite3_exec(self.db, sql, nil, nil, err) ~= SQLITE_OK then
    local msg = "sqlite3_exec failed: " .. ffi.string(err[0])
    lib.sqlite3_free(err[0])
    error(msg, 2)
  end
  return true
end
```

listing 146: borrow, copy, free back to the owner (source: lua/jit/sqlite.lua, lines 60-68)

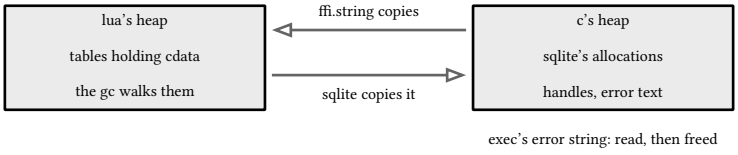


figure 85: who owns what, two heaps and the arrows between them

The rule the wrapper teaches is the ffi's one law: whoever allocates memory frees it, lua borrows and copies but never keeps a c pointer it did not pin in a table.

17.4. two runtimes, one tree

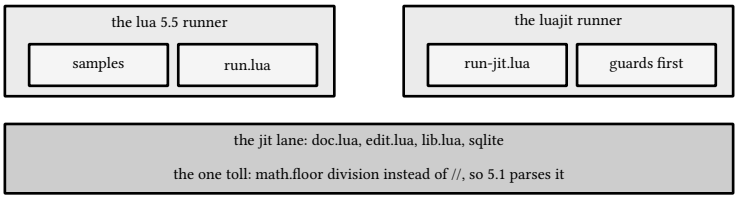
The split rule is absolute in this repo: the lua 5.5 runner executes the sample suites and the capstone's `run.lua`, the auto chess game, and never loads ffi code, the LuaJIT runner executes `run-jit.lua` from `books/lua/jit/` and never runs the 5.5 modules. `make verify` runs them as two separate targets, `verify-lua` with the 5.5 binary, `verify-lua-jit` with the LuaJIT binary, so a green build proves both worlds, never a merged one.

The editor modules themselves, `doc.lua`, `edit.lua`, `lib.lua`, live in the jit lane now, and they still pay the one line their shared past cost them: the binary search in `doc.lua` uses `math.floor` division instead of the 5.3 `//` operator so LuaJIT can parse the module. Everything else in them was already 5.1 syntax. The jit suite guards its own side of the boundary before any test runs:

```lua
-- luajit test suite: editor documents persisted to sqlite through the ffi wrapper
-- run with tools/luajit/build/bin/luajit.exe from this directory; the lua 5.5
-- interpreter has no ffi library and must never load this file or its modules
if jit == nil then
  error("this suite needs the luajit interpreter, not lua 5.5:\n" ..
    "  tools/luajit/build/bin/luajit.exe run-jit.lua\n" ..
    "build it with: make sqlite-tools", 0)
end

local dll = (((arg and arg[0] or "run-jit.lua"):match("^(.*)[/\\]") or ".")
  .. "/../../../tools/sqlite/build/bin/sqlite3.dll")
local probe = io.open(dll, "rb")
if not probe then
  error("missing " .. dll .. "\nbuild it with: make sqlite-tools", 0)
end
probe:close()
```

listing 147: refuse the wrong interpreter, refuse the missing dll, in plain english (source: lua/jit/run-jit.lua, lines 1-16)



make verify runs two targets, verify-lua and verify-lua-jit

a green build proves both worlds, never a merged one

figure 86: two runtimes over one tree of shared modules

17.5. the editor lane gains a database

`persist-sqlite.lua` keeps the file store's one guarantee, bodies always carry a final newline, and adds what a file cannot give: documents by name, replacement in place, and a modification timestamp, one table with a primary key and an upsert:

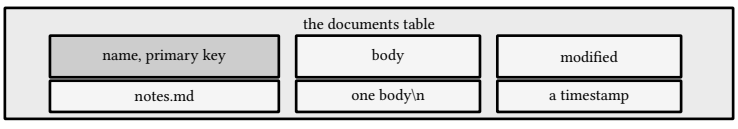
```
local SCHEMA = [[create table if not exists documents (
  name text primary key,
  body text not null,
  updated integer not null)]]

local function body_of(editor)
  local text = editor:text()
  if text:sub(#text) ~= "\n" then text = text .. "\n" end
  return text
end

function persist.open(path)
  local conn = sqlite.open(path)
  conn:exec(SCHEMA)
  return setmetatable({ conn = conn }, { __index = persist })
end

function persist:save(name, editor)
  local stmt = self.conn:prepare(
    "insert or replace into documents(name, body, updated) values(?, ?, ?)")
  stmt:bind_text(1, name)
  stmt:bind_text(2, body_of(editor))
  stmt:bind_int(3, os.time())
  stmt:step()
  stmt:finalize()
  return true
end
```

listing 148: schema, the final newline rule, the upsert save (source: lua/jit/persist-sqlite.lua, lines 8-34)



upsert replaces in place, the key decides

bodies always carry a final newline, carried over from the file store

lookups run their iterator to the end, the lesson of the red run

figure 87: the documents table, the store the file could not be

The lookups learned from the red run and run their iterator to the end, then reopen semantics are the test's job: close the store, open the same file again, and demand the same documents back, empty document included:

```
-- the rows iterator finalizes its statement only when exhausted, so these
-- primary key lookups run the loop to its end instead of returning mid-loop
function persist:load(name)
  local body = nil
  for row in self.conn:rows("select body from documents where name = ?", { name }) do
    body = row.body
  end
  if body == nil then return nil end
  return doc.new(body)
end

function persist:updated(name)
  local stamp = nil
  for row in self.conn:rows("select updated from documents where name = ?", { name }) do
    stamp = row.updated
  end
  return stamp
end
```

listing 149: primary key lookups that never break the loop (source: lua/jit/persist-sqlite.lua, lines 36-53)

```
function tests.documents_round_trip_through_a_reopened_file()
  local path = "capstone-tmp.sqlite"
  os.remove(path)
  local store = persist.open(path)
  local notes = edit.new(doc.new(""))
  notes:insert(1, "alpha\nbeta")
  T.eq(store:save("notes", notes), true)
  T.eq(store:save("other", edit.new(doc.new("second"))), true)
  store:close()
  local reopened = persist.open(path)
  T.eq(reopened:load("notes"):text(), "alpha\nbeta\n")
  T.eq(reopened:load("notes"):line_count(), 2)
  T.eq(reopened:load("other"):text(), "second\n")
  T.eq(reopened:names(), {"notes", "other"})
  reopened:close()
  os.remove(path)
end
```

listing 150: save, close, reopen the file, demand equality (source: lua/jit/run-jit.lua, lines 91-107)

NOTE: what the ffi does not cover

This wrapper is a teaching surface, not a driver. There is no blob binding, no collation hooks, no progress handler, no `busy_timeout` wiring, and integers round through lua doubles, so 64-bit values beyond 2^53 lose digits on their way back. The suites never store such values, and a real driver would keep int64 columns as cdata end to end instead of converting.

sources: luajit.org luajit ffi semantics and tutorial pages, sqlite.org c interface reference for the seventeen declared functions, accessed 2026-09-10, plus this tree's `tools/sqlite/probe.lua` from the build phase. All behavior verified live with luajit 2.1 rolling and sqlite 3.53.4, 10 tests green through `make verify-lua-jit`, now from `books/lua/jit/`, the auto chess capstone's 34 tests green under 5.5 through `make verify-lua`.

18. a c host: allegro 5 with lua embedded

Chapter 14 drove lua from c on a console. A game wants the same embedding with a clock, a window, and an event source, and that is what a host library is for: this chapter embeds lua 5.5 in an allegro 5 program, the c side owning time and input, the lua side owning the game. Everything is built by the tree itself, one mingw gcc 15.2 building `lua55.dll` and linking the same-flavored allegro prebuilt, and the contract the two sides speak is designed to run with no window at all.

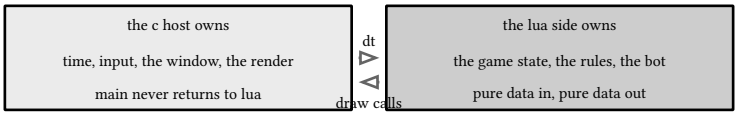
book 9, ch 14 (lua 5.5, the c api, hands on) built the vocabulary, the stack, protected calls, registered functions, userdata, references. This chapter spends it.

18.1. why a c host owns time

Lua has no clock of its own and no opinion about frame rates. The language does not even guarantee that a busy loop yields, so scheduling belongs to the embedder: the c host creates a timer, waits for it, and only then tells lua a tick happened. The lua side receives a fixed `dt`, never measures wall time, and stays deterministic, the property book 9, ch 10 (lua 5.5, profiling: hooks, clocks, and p99) needed and the capstone depends on:

```
{name = "one update and one draw per tick, fixed dt", fn = function()
  local h = fake_host{max_ticks = 5}
  local updates, draws, dt_sum = 0, 0, 0
  h.run(function(host, dt) updates = updates + 1; dt_sum = dt_sum + dt end,
       function() draws = draws + 1 end)
  T.eq({updates, draws}, {5, 5})
  T.near(dt_sum, 5 / 60)
  T.eq(h.ticks, 5)
end},
```

listing 151: one update per tick, the fixed dt asserted, no wall clock anywhere (source: lua/samples/ch18_allegro.lua, lines 98-106)



a fixed dt and a seeded rng make runs reproducible

lua never waits, it is always the host that wakes it

the same game code runs headless, the capstone's whole test story

chapter 14's stack protocol carries every crossing

figure 88: who owns what in an embedded game

18.2. the vendored toolchain

Allegro ships official mingw prebuilts. `make allegro-tools` fetches the pinned asset, checks its sha256, unpacks the monolith dll, its import library, and the headers into gitignored `tools/allegro/build/`, then compiles and runs a probe that asserts the linked version:

```
asset  allegro-x86_64-w64-mingw32-gcc-15.2.0-posix-seh-dynamic-5.2.11.3.zip
sha256 f9a02e841d956b05f032f8dca70dab3972ffd0972a94ea0c4453e0190c0429a0
release  5.2.11.3, 2026-02-09, github.com/liballeg/allegro5/releases
probe    linked allegro 5.2.11.4
```

fragment, illustrative, not executed

The version byte is a measured oddity worth pinning: the 5.2.11.3 release asset reports itself as 5.2.11.4 through `al_get_allegro_version()`, the packed integer's low revision byte set to 4 by the release tooling, so the probe asserts the top three bytes, 5.2.11, and the acquisition script records the fourth as printed. The asset was built by gcc 15.2.0 posix-seh, the same runtime flavor as this repo's documented compiler, which is why one compiler serves both libraries:

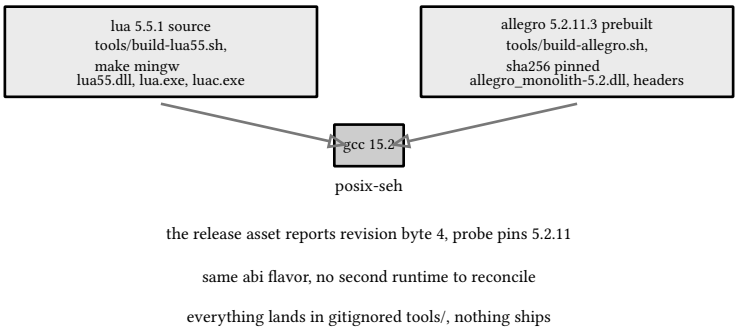


figure 89: the vendored toolchain, two libraries one compiler

18.3. linking two mingw libraries

The host links both libraries in one command, `-llua55 -lallegro_monolith`, against `lua55.dll` built in-tree and the monolith import library `liballegro_monolith.dll.a` from the prebuilt. The monolith folds allegro's addons into one dll, so one flag covers core, primitives, font, and image. The runtime dll set is short because every dependency, freetype, zlib, png, the audio codecs, is statically baked into the monolith already; what loads besides it is only the mingw runtime:

```
allegro_monolith-5.2.dll
libgcc_s_seh-1.dll
libwinpthread-1.dll
libstdc++-6.dll
```

fragment, illustrative, not executed

Those three runtime dlls are not in the allegro zip and scoop's gcc ships none, so the acquisition script locates them on the mingw runtime already on PATH and stages them beside the monolith, which also sidesteps a windows trap this repo has met before: the loader searches the exe's own directory first, so no PATH edit is needed at all. `make verify-lua-c` runs every sample with the two bin directories prepended in PowerShell, where the semicolon PATH needs no cygpath games.

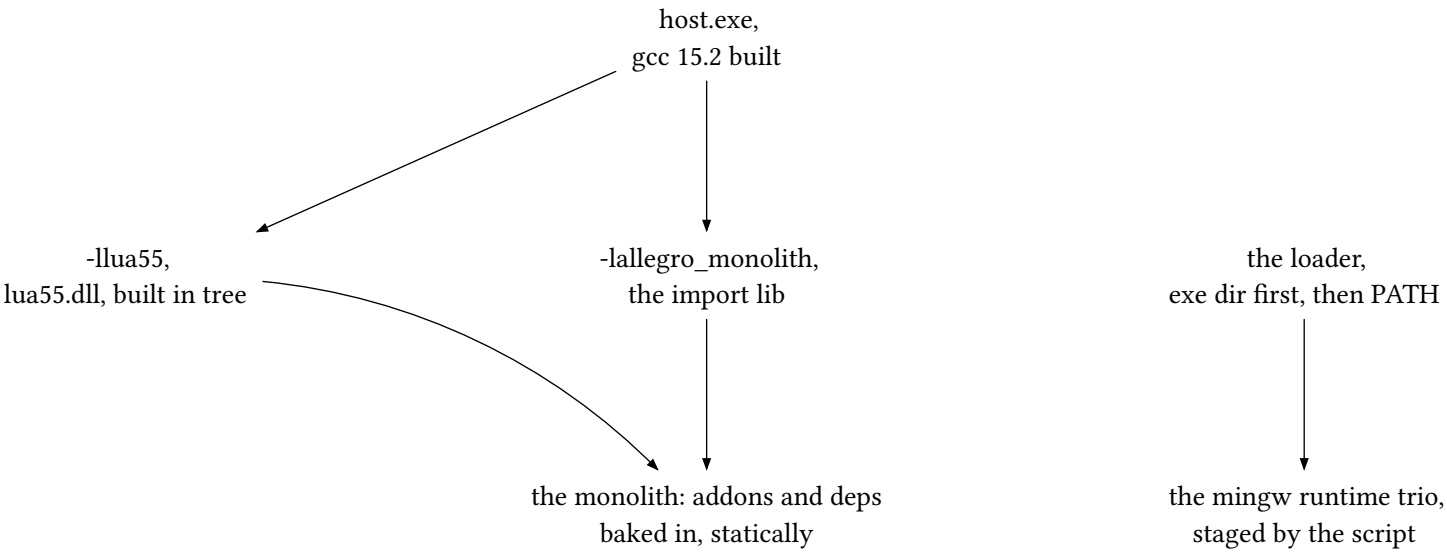


figure 90: one host binary, two libraries, the dlls it loads

18.4. the event loop skeleton

The loop is a timer, a queue, and a wait. `al_create_timer` fixes the tick rate, the timer's event source is registered on one queue, and `al_wait_for_event` blocks the host until the next tick, which is the entire scheduling story. No display is needed for any of it, the skeleton below runs in the verify suite with no window:

```
ALLEGRO_TIMER *timer = al_create_timer(1.0 / 64.0);
CHECK(timer != NULL, "a 64 hz timer exists");
double dt = al_get_timer_speed(timer);
CHECK(dt > 1.0 / 64.001 && dt < 1.0 / 63.999, "the timer speed is the dt");

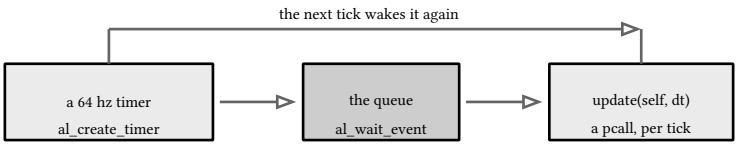
ALLEGRO_EVENT_QUEUE *queue = al_create_event_queue();
CHECK(queue != NULL, "an event queue exists");
al_register_event_source(queue, al_get_timer_event_source(timer));
al_start_timer(timer);
```

listing 152: the clock, its queue, and the registration (source: lua/samples-c/src/Ch18/host_skeleton.c, lines 37-46)

Each tick the host digs the lua game table out of the registry, the chapter 14 reference trick, and calls its `update` with `self` and the fixed `dt`, protected:

```
while (ticks < 64) {
  ALLEGRO_EVENT ev;
  al_wait_for_event(queue, &ev); // blocks until the next tick
  if (ev.type == ALLEGRO_EVENT_TIMER) {
    ticks++;
    lua_rawgeti(L, LUA_REGISTRYINDEX, game); // the table
    lua_getfield(L, -1, "update");
    lua_pushvalue(L, -2);      // self
    lua_pushnumber(L, dt);     // the fixed dt
    if (lua_pcall(L, 2, 0, 0) != 0) {
      printf("FAIL lua update raised: %s\n", lua_tostring(L, -1));
      return 1;
    }
    lua_pop(L, 1); // the table
  }
}
```

listing 153: wait, drain, call lua per tick (source: lua/samples-c/src/Ch18/host_skeleton.c, lines 48-64)



64 events became 64 updates, asserted in the suite

the game table is pinned with luaL_ref between ticks

the timer's own count is the cross check, 64 and 64

no display anywhere, a window is an addon to this loop

figure 91: the loop's shape, timer to queue to lua and back

18.5. events to commands

Input is the second thing the host owns, and the design decision that makes the capstone testable is translating events into plain command tables the moment they arrive. A command is a table with a string `type` and a fixed shape per type, and it enters the game through one fifo queue with validation at the door. The pure lua sample below is the whole contract, written and run before any windowed code exists, so the c host must satisfy lua's tests rather than the other way around:

```lua
local shapes = {
  keydown = {key = "string"},
  keyup = {key = "string"},
  click = {x = "number", y = "number"},
  quit = {},
}

local function new_queue()
  local q = {items = {}, first = 1, last = 0}
  function q.push(cmd)
    if type(cmd) ~= "table" or type(cmd.type) ~= "string" then
      error("commands are tables carrying a string type", 2)
    end
    local shape = shapes[cmd.type]
    if not shape then error("unknown command type: " .. cmd.type, 2) end
    for field, want in pairs(shape) do
      if type(cmd[field]) ~= want then
        error(cmd.type .. " wants " .. field .. ": " .. want, 2)
      end
    end
    q.last = q.last + 1
    q.items[q.last] = cmd
    return cmd
  end
```

listing 154: the command shapes, and the push that validates them (source: lua/samples/ch18_allegro.lua, lines 9-32)

```lua
local function fake_host(opts)
  local h = {
    dt = opts.dt or 1 / 60,
    ticks = 0,
    draws = 0,
    seen = {},
    queue = new_queue(),
    running = true,
  }
  function h.push(cmd) h.queue.push(cmd) end
  function h.stop() h.running = false end
  function h.run(update, draw)
    local script = opts.events or {}
    local next_event = 1
    while h.running and h.ticks < opts.max_ticks do
      h.ticks = h.ticks + 1
      while next_event ≤ #script and script[next_event].at == h.ticks do
        h.push(script[next_event].cmd)
        next_event = next_event + 1
      end
      update(h, h.dt)
      draw(h)
    end
    return h
  end
  return h
end
```

listing 155: the fake host, scripted events, one update and one draw a tick (source: lua/samples/ch18_allegro.lua, lines 47-73)

The automation seam falls out for free: whatever pushes a command, an allegro event translated in c, or a scripted driver in lua, the game cannot tell apart, and the suite pins exactly that:

```lua
  {name = "commands are identical whatever pushed them", fn = function()
    -- the automation seam: a scripted driver and the host's own event
    -- translation push the same tables, the game cannot tell them apart
    local scripted, translated = new_queue(), new_queue()
    scripted.push({type = "keydown", key = "d"})
    translated.push({type = "keydown", key = "d"})
    T.eq(scripted.drain(), translated.drain())
  end},
```

listing 156: identical commands, whatever pushed them (source: lua/samples/ch18_allegro.lua, lines 129-136)

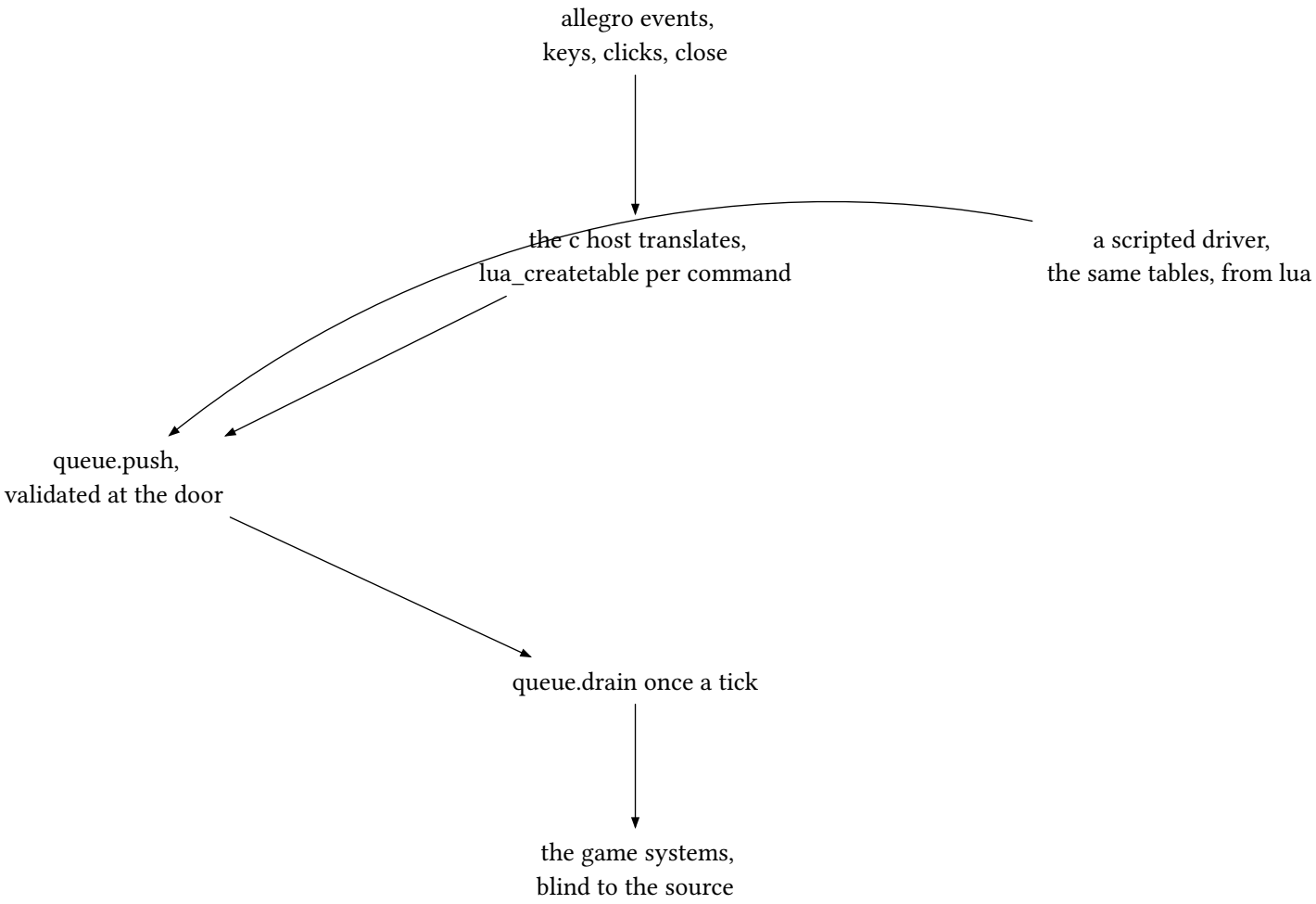


figure 92: two producers, one queue, one consumer

18.6. lua draws while c renders

Drawing is the mirror of commands: the host owns the target and the frame, lua receives primitives as registered c functions, exactly the `luaL_Reg` table of book 9, ch 14 (lua 5.5, the c api, hands on). Each primitive draws onto whatever target the host set, counts as it crosses, and knows nothing about displays:

```c
static int api_clear(lua_State *L) {
  al_clear_to_color(al_map_rgb((int)luaL_optinteger(L, 1, 0),
                               (int)luaL_optinteger(L, 2, 0),
                               (int)luaL_optinteger(L, 3, 0)));
  draw_calls++;
  return 0;
}

static int api_rect(lua_State *L) {
  float x = (float)luaL_checknumber(L, 1);
  float y = (float)luaL_checknumber(L, 2);
  float w = (float)luaL_checknumber(L, 3);
  float h = (float)luaL_checknumber(L, 4);
  al_draw_filled_rectangle(x, y, x + w, y + h,
                           al_map_rgb((int)luaL_optinteger(L, 5, 250),
```

listing 157: clear and rect, drawing is registered functions (source: lua/samples-c/src/Ch18/draw_host.c, lines 32-46)

```c
static int api_text(lua_State *L) {
  const char *s = luaL_checkstring(L, 1);
  float x = (float)luaL_checknumber(L, 2);
  float y = (float)luaL_checknumber(L, 3);
  al_draw_text(font, al_map_rgb(255, 255, 255), x, y, 0, s);
  draw_calls++;
  return 0;
}

static int api_size(lua_State *L) {
  lua_pushinteger(L, al_get_bitmap_width(target));
  lua_pushinteger(L, al_get_bitmap_height(target));
  return 2;
}

static const luaL_Reg apilib[] = {
    {"clear", api_clear},
    {"rect", api_rect},
    {"text", api_text},
    {"size", api_size},
    {NULL, NULL},
};
```

listing 158: text and size, the api table lua sees (source: lua/samples-c/src/Ch18/draw_host.c, lines 53-74)

The frame is the host's to finish, `al_flip_display` is c's call, and the sample proves the drawing really happened by reading pixels back from the target and comparing exact bytes:

```c

  // the pixels prove the drawing happened: exact bytes back from the bitmap
  unsigned char r, g, b, a;
  al_unmap_rgba(al_get_pixel(target, 1, 1), &r, &g, &b, &a);
  CHECK(r == 20 && g == 40 && b == 90, "the clear color is in the pixels");
  al_unmap_rgba(al_get_pixel(target, 6, 6), &r, &g, &b, &a);
  CHECK(r == 250 && g == 80 && b == 80, "the rect color is in the pixels");
```

listing 159: exact colors read back from the target (source: lua/samples-c/src/Ch18/draw_host.c, lines 104-110)

18.7. headless by design

Nothing above needs a window. The draw sample targets a memory bitmap, the loop sample needs only a timer, and the fake host sample runs the entire contract in pure lua with a tick counter for a clock. That is a design rule, not an accident: game logic never touches allegro, draw primitives degrade to cheap calls on an offscreen target, and the same lua runs in the verify suite and in front of a display. The capstone leans on this completely, its whole test story is the game headless while the human side is automated through the command queue.

```
{name = "a game loop with no window runs to completion", fn = function()
  local h = fake_host{max_ticks = 120, dt = 1 / 64}
  local clock = 0
  local gold = 0
  h.run(function(host, dt)
    clock = clock + dt
    for _, cmd in ipairs(host.queue.drain()) do
      if cmd.type == "keydown" and cmd.key == "g" then gold = gold + 1 end
    end
    if clock ≥ 1 then host.stop() end
  end, function() end)
  T.ok(clock ≥ 1, "simulated one second")
  T.eq(h.ticks, 64)
  T.eq(gold, 0)
end},
```

listing 160: a full loop with no window, one simulated second (source: lua/samples/ch18_allegro.lua, lines 138-152)

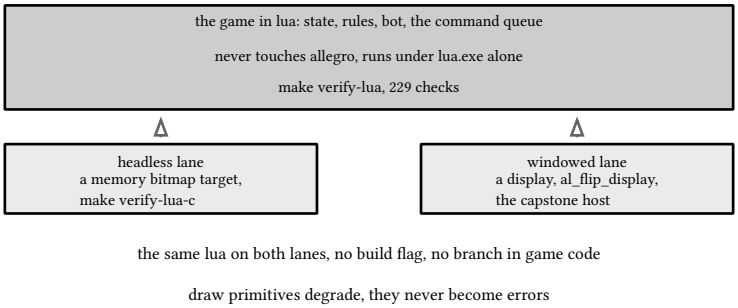


figure 93: headless by design, what a window adds and what it does not

sources: liballeg.org allegro 5 api docs and the 5.2.11.3 release page at github.com/liballeg/allegro5/releases, accessed 2026-09-21; the lua.org manual sections chapter 14 already pinned. All facts measured in-tree: the asset name and sha256 frozen in `tools/build-allegro.sh`, the linked version, the dll set, and the loop counts printed by the probe and the verify suite. 6 tests in `samples/ch18_allegro.lua` green through `make verify-lua`, the samples suite at 229 checks, 2 c samples with 20 checks green through `make verify-lua-c`.

19. the http kernel

The game backend part of this book stands on one small module set: the http kernel. The go book built its kernel over `net/http`, a server the platform hands over, and lua has no such platform. What it has is the c host from the capstone's own pattern: winsock owns the listening socket and one server thread, everything past the wire is lua. The host owns sockets, the kernel owns bytes: the request line, the headers, the body, the route table, the error envelope, the one json codec, the one response writer, and the composition root each later family splices into.

19.1. the seam and the handler shape

The seam between c and lua is five functions on a global table. `net.recv` pulls up to 4096 bytes and answers an empty string both for a closed peer and for an error, the one honest simplification a blocking edge allows, and the kernel treats that empty string as connection end. `net.send` pushes bytes and answers how many went, `net.close` returns the socket to the os, and `net.addr` reads the listen address. The fifth call grew with the part: `net.peer` answers the connection's remote ip once per connection, and the kernel stamps it on the request so the register route's rate key reads an address without lua touching a socket. The host accepts one connection at a time on its single server thread, so the service is sequential by construction: one vm, no locks anywhere in the part, stated once here and never defended again.

The handler shape follows from the seam. A handler is one function of a request value returning a response value, and both are plain tables. Because the response is a value, everything downstream reads exactly what goes on the wire, and the go lane's status-capturing writer wrapper, the trick an `http.ResponseWriter` architecture needs, has no lua counterpart to get wrong. Handlers and layers return responses or fail values, and the dispatch base renders fail values through the one envelope writer, so outcomes ride values and only bugs raise:

```
function app.dispatch(req)
  if req.request_id == nil then
    req.request_id = envelope.resolve_id(req, app.cfg.new_id)
  end
  local base = function(r)
    local out = app.router.dispatch(r)
    if envelope.is_fail(out) then
      return envelope.failure(out, r.request_id, out.headers)
    end
    return out
  end
  local h = base
  for i = #app.layers, 1, -1 do
    h = app.layers[i](h)
  end
  return h(req)
end
```

listing 161: dispatch folds the layers over a base that renders failures inside the fold (source: lua/service/main.lua, lines 80-96)

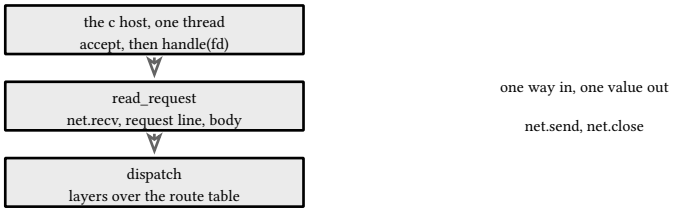


figure 94: the host hands over an fd, the kernel turns bytes into a value and back

19.2. bytes into a request

The parser reads from a callable, not a socket: `read_request(read)` takes any function answering the next chunk, which is what makes the whole kernel testable without a port, and the connection loop is the only place that hands it the real seam. The grammar it accepts is a deliberate subset. The request line must spell method, target, and an `HTTP/1.0` or `HTTP/1.1` version, uppercase method letters, a target that starts with a slash or is the asterisk, anything else answered as a 400 envelope and a closed connection. Headers lowercase their names and trim their whitespace.

Two rulings inside the head are worth their own sentences. A repeated `Content-Length` or `Transfer-Encoding` is refused on sight, agreeing or not, because a pair that disagrees is a request smuggling attempt. And `Transfer-Encoding` present at all is refused: this service speaks content-length bodies only, and a 400 envelope with a clear message beats a half implemented chunked decoder. The body ladder then runs in a fixed order: an unparsable length is a 400, a length past the contract's 1 MiB cap is a 413 answered without draining a byte, because reading a body you already refused is work for the attacker, and a connection that ends mid-body closes silently, a truncated request has no answer worth sending:

```
if headers["transfer-encoding"] ~= nil then
  return envelope.failure(envelope.fail("invalid_json", "chunked bodies are not supported"),
    new_id())
end
local raw_len = headers["content-length"] or "0"
if raw_len:match("^%d+$") == nil then
  return envelope.failure(envelope.fail("invalid_json", "malformed content length"),
    new_id())
end
local length = tonumber(raw_len)
if length > envelope.max_body then
  return envelope.failure(envelope.fail("payload_too_large", "request body exceeds 1 MiB"),
    new_id())
end
while #rest < length do
  local chunk = read()
  if chunk == "" then return nil end
  rest = rest .. chunk
end
```

listing 162: the framing ladder: chunked refused, lengths validated, oversize never drained (source: lua/service/main.lua, lines 168-186)

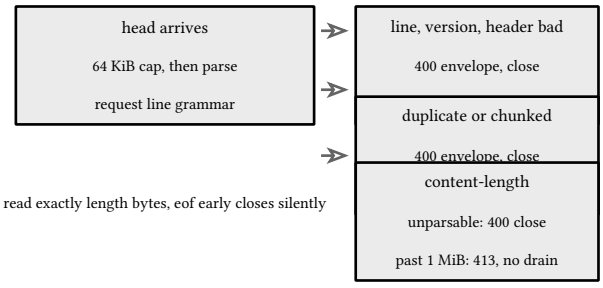


figure 95: the framing decision tree, every leaf an envelope or a closed connection

19.3. the route table

The router is hand rolled because lua has no mux to wrap, and the decomposition is the lesson. A pattern compiles into a list of segments, each a literal or a named param, whole segments only: a param takes exactly one non-empty path segment, so a trailing slash never reads as a blank id and `/a/b` never matches `/a/b/`. Percent escapes decode after the split, per segment, so an encoded slash cannot manufacture a separator the router never saw. Registration time is where ambiguity dies: two same-method patterns of equal length with equal per-side literal

counts can both match some request with neither strictly more specific, and that pair is refused the moment it registers, exact duplicates included.

Matching walks the table once. Every entry whose segments fit the path contributes, the fittest same-method entry wins, most literals first, and the methods that fit the path but not the request collect into the sorted allow set the 405 answer carries. The winning entry stamps the request with its own pattern string and its params, the part the authz family needs later: policies key on `/api/users/{id}`, not on whatever path shape arrived. The 404 and the 405 are values returned, never errors raised:

```
local function compile(pattern)
  local segs, literals = {}, 0
  for _, raw in ipairs(segments_raw(pattern)) do
    local seg = percent_decode(raw)
    if seg:sub(1, 1) == "{" and seg:sub(-1) == "}" and #seg > 2 then
      segs[#segs + 1] = { name = seg:sub(2, -2) }
    elseif seg:find("[{}]") then
      error("router: params are whole segments: " .. pattern, 3)
    else
      segs[#segs + 1] = { literal = seg }
      literals = literals + 1
    end
  end
  return segs, literals
end
```

listing 163: compile: patterns decompose into literal and param segments (source: lua/service/router.lua, lines 34-48)

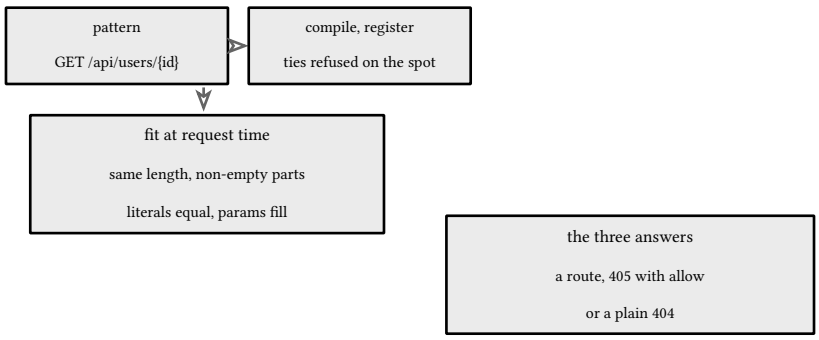


figure 96: compile once at registration, fit every request, refuse the ties

19.4. the error envelope

The contract freezes one failure shape for the whole api, two levels deep: an error object with a code from a closed set of eleven, one human sentence, optional details on `validation_failed` only, and the request id. The kernel owns the set and the mapping in one table, code to status, and the two split codes pass their status explicitly at the split site: `invalid_json` rides 400 for a malformed body and 415 for a wrong media type, `not_found` rides 404 and 405. The `fail` constructor refuses an unknown code and refuses details off `validation_failed`, both at construction, so a bad fail cannot reach the wire. Unknown failures are coerced to `internal` with the detail kept for the log, because a leaked store error tells a client more than it should know.

One writer renders every failure body, member order fixed by construction: code, message, details when present, request_id. The request id resolves once at the edge: a value already carried wins, else a client header that parses as a uuid echoes canonicalized to lower case, else the injected source mints one. Garbage is replaced, never echoed:

```
function M.render(f, request_id)
  local inner = { "code", f.code, "message", f.message }
  if f.details ~= nil then
    inner[#inner + 1] = "details"
    inner[#inner + 1] = f.details
  end
  inner[#inner + 1] = "request_id"
  inner[#inner + 1] = request_id
  return json.encode(json.object({ "error", json.object(inner) }))
end

-- the response value: a status, headers as a flat name value list so
```

listing 164: render: the one envelope writer, member order fixed by construction (source: lua/service/envelope.lua, lines 68-79)

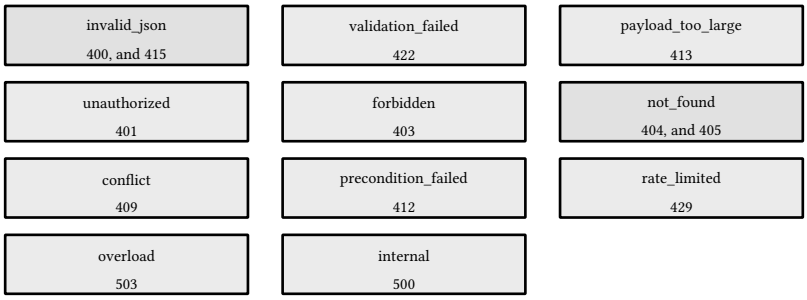


figure 97: eleven codes, one table, two split sites pass their status explicitly

19.5. json in and out

Book 9 at 5.0 owns no json encoder, so the service rolls its own and this chapter is the book's home for encoding discipline. The decoder is a position scanner, recursive descent over the string, strict the way input deserves: duplicate members, trailing garbage, control bytes, and lone surrogates all refused, a depth cap at 100 nesting levels, every error carrying its byte offset. Numbers keep lua's integer subtype, `1790830800` decodes as an integer and `1e3` as a float, and integers that overflow int64 land as floats, the only honest reading.

The encoder's discipline is construction. A lua table's string keys carry no iteration order, so bodies are built as ordered member lists, `json.object` with a flat name value sequence, and the encoder renders exactly that order. The same metatable that carries the order marks decoded objects, so re-encoding a decoded body is a fixed point, the canonical idempotence the vector tests run: encode of decode of encode is byte stable. A plain string keyed table is refused at the encoder, not silently reordered, and the empty plain table is refused as ambiguous. Stock 5.5 ships no `string.buffer`, that is the luajit extension, so both directions build over `table.concat`, one join:

```
function M.object(flat)
  if #flat % 2 ~= 0 then error("json: object members come in name value pairs", 2) end
  local t, order = {}, {}
  for i = 1, #flat, 2 do
    local name = flat[i]
    if type(name) ~= "string" then error("json: member names are strings", 2) end
    if t[name] ~= nil then error("json: duplicate member " .. name, 2) end
    t[name] = flat[i + 1]
    order[#order + 1] = name
  end
  return tag_object(t, order)
end

-- array(t) tags a sequence so the empty array stays distinct from the
```

listing 165: object: ordered members by construction, duplicates refused here too (source: lua/service/json.lua, lines 24-37)

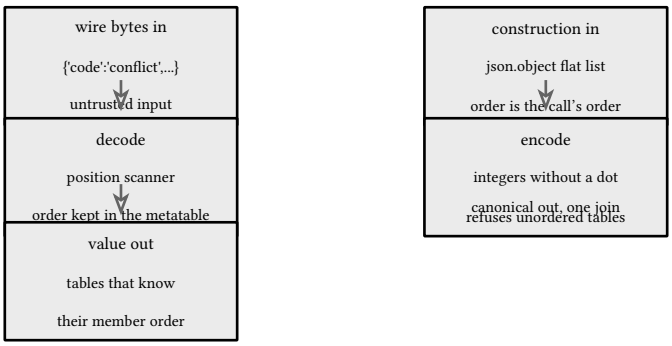


figure 98: decode tags what it builds, encode renders only what construction ordered

19.6. writing the response

One site writes every response. `render_response` takes the response value and the connection's keep decision and answers bytes: a status line with the standard reason phrase, the response's own headers in construction order, a content length except on the two bodyless statuses, and an explicit `Connection` line on every answer so the lifecycle is stated, never implied. The 204 and the 304 carry no body and no length, and the byte shape of a whole answer is asserted exactly in the tests:

```
function M.render_response(resp, keep)
  local bodyless = resp.status == 204 or resp.status == 304
  local out = {
    string.format("HTTP/1.1 %d %s\r\n", resp.status, reasons[resp.status] or ""),
  }
  for i = 1, #resp.headers, 2 do
    out[#out + 1] = resp.headers[i] .. ": " .. tostring(resp.headers[i + 1]) .. "\r\n"
  end
  if not bodyless then
    out[#out + 1] = "Content-Length: " .. #resp.body .. "\r\n"
  end
  out[#out + 1] = "Connection: " .. (keep and "keep-alive" or "close") .. "\r\n"
  out[#out + 1] = "\r\n"
  if not bodyless then
    out[#out + 1] = resp.body
  end
```

listing 166: render_response: the single write site, the lifecycle always stated (source: lua/service/main.lua, lines 215-230)

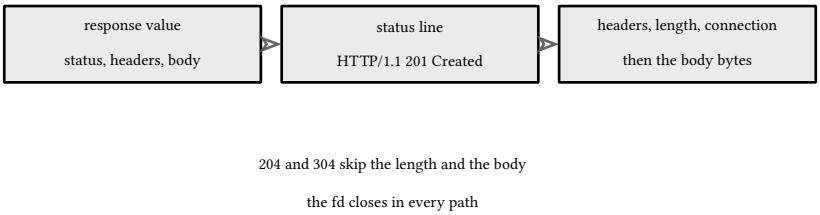


figure 99: a response value becomes bytes once, in one place

19.7. the kernel routes and the root

The kernel owns two routes. `GET /healthz` answers liveness with the status and the build version, injected through the app config so a deployment can prove what it runs. `GET /readyz` answers readiness, ok while the injected probe says ready and the envelope's `overload` code at 503 the moment it does not, the hook the drain wiring in the load chapter flips.

The composition root is `main.lua`, and this chapter's commit is the last time this file changes shape under this stream. It is the only file in the service tree that reads the wall clock or draws randomness: the clock is `os.time`, whole seconds, and the id mint is a uuid v7 built from that clock times 1000 plus random bits, so a minted timestamp ticks once a second and the random half carries the uniqueness. Every module below the root receives clock and id callables, and no test ever sleeps. The four middleware layers of chapter 20 wire here in request order, and the families have been landing below their markers as their chapters ship: the metrics layer and the limiter sit between the access log and the deadline, the identity layer and the resource families sit innermost with their routes, and the rail closed at the ship marker, the drain that flips the readyz probe and the reports pair among them:

```
function M.build(cfg)
  cfg = cfg or {}
  local app = M.new_app(cfg)
  local svc = {}

  -- the middleware stack, wired in request order, first used outermost.
  -- the four kernel layers are chapter 20's, the ops families splice
  -- their layers between the access log and the deadline, and every
  -- later family lands its wiring below its own marker, closing over
  -- app, clock, and new_id
  app:use(middleware.request_id(app.cfg.new_id))
  app:use(middleware.access_log(app.cfg.sink, app.cfg.clock))
```

listing 167: the root: the builder over injected seams, the two edge layers inside it (source: lua/service/main.lua, lines 242-253)

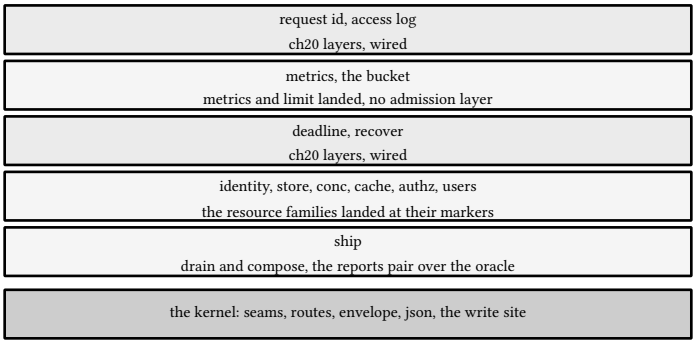


figure 100: the root as a wiring rail: the families land at their markers in request order

19.8. what the kernel proves

The numbers first, from the tree this chapter landed on: 12 module tests in `main.lua`, 9 in json, 9 in envelope, 10 in router, and 29 checks in the wire driver against the compiled host, every lane green. What the kernel proves

is that the part's shape is fixed: the envelope renders every failure, the decode ladder hands every write route its refusals, the route table refuses its own ambiguities and carries its patterns, and the root's markers say where every family lands. From here the part is additive, and the driver is the proof that runs on every commit:

```
-- healthz: exact status, exact body, content length present
do
  local status, headers, body = parse(roundtrip(
    "GET /healthz HTTP/1.1\r\nHost: t\r\nConnection: close\r\n\r\n"))
  C.eq(status, 200, "healthz status")
  C.eq(body, '{"status":"ok","version":"dev"}', "healthz body bytes")
  C.eq(headers["content-length"], "31", "healthz content length")
  C.eq(headers["content-type"], "application/json", "healthz content type")
end
```

listing 168: the wire driver: exact status, exact bytes, over the real host (source: lua/service/tests/drivers/kernel.lua, lines 32-41)

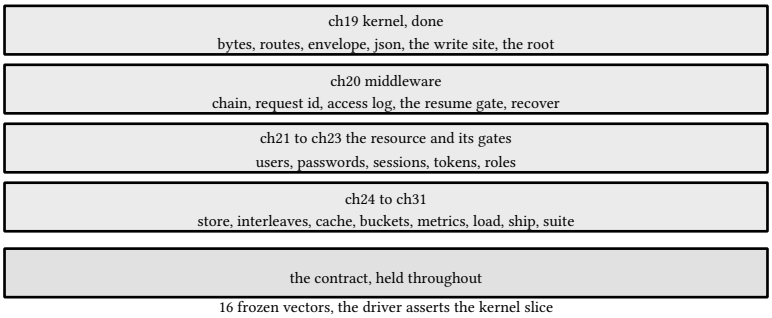


figure 101: every later chapter adds routes or layers, none rewrites the kernel

sources: the lua 5.5 manual at lua.org/manual/5.5, sections 3.1 on the integer and float subtypes, 6.1 on tostring number formatting, 6.4.1 on string patterns, 6.5 on utf8.char, 6.7 on math.random, and 6.9 on os.time, accessed 2026-09-27. RFC 9112 sections 2.2, 2.3, and 6.3 on the request line, message parsing, and content length, RFC 9110 section 15.5.5 on 405 and the allow header, RFC 8259 on the json grammar, and RFC 9562 section 5.7 on the v7 layout, accessed 2026-09-27. Verified against `go/api/internal/httpx/kernel.go`, `python/api/pyapi/kernel.py`, and `c-os-cloud/api/api.h`, and by the kernel family's own gates: 40 module tests across json, envelope, router, and main, and 44 host lane checks across the kernel, mw, and smoke drivers, both lua lanes green.

20. the middleware chain

Cross-cutting behavior does not belong inside handlers, and lua does not make anyone put it there. This chapter composes the service's middleware stack over the kernel: the chain fold that turns layers into one handler, the request id born once at the edge and quoted everywhere, the access log that reads the answer off the response value, the deadline whose whole authority is owning the resume, and recover, which keeps a handler bug from killing the connection. The wiring order at the end is contract, and the stack test is the property that enforces it.

20.1. the chain fold

One layer is one function: given the next handler, return the handler that wraps it. That is the whole shape, and because it is a plain function contract, anything written against it composes with anything else. The fold runs right to left so the first argument ends up outermost, sees the request first and the response last, the onion picture every middleware chapter draws because it is the true one. A chain is itself a layer, chains compose into bigger chains, and the kernel's `app:use` appends onto the same fold, first used outermost. The loop direction is worth one sentence out loud: folding left to right would put the last layer outside, and the wiring order in this chapter's sixth section is a contract, not an accident:

```
function M.chain(...)
  local layers = { ... }
  return function(next_h)
    local h = next_h
    for i = #layers, 1, -1 do
      h = layers[i](h)
    end
    return h
  end
end
```

listing 169: chain: one layer wraps the next, first argument outermost (source: lua/service/middleware.lua, lines 17-26)

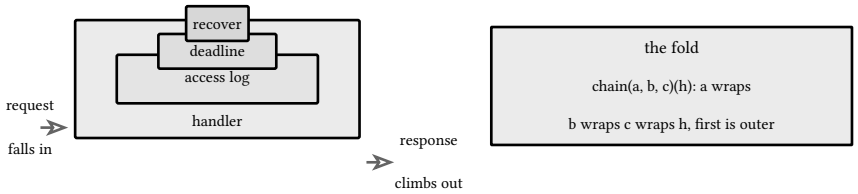


figure 102: the onion: first argument outside, request falls in, response climbs out

20.2. request id

Every request needs one id every artifact agrees on: the access log line, the error envelope, the recover line, and later the metrics. The layer resolves it exactly once, at the edge, using the kernel's three-way order: a value already carried wins, else a client header that parses as a uuid echoes canonicalized to lower case, else the injected source mints a fresh v7. Parsing is not politeness, it is contract: the envelope promises the member is a uuid, and a client supplied free-text header echoed into logs is log injection. Garbage is replaced, never echoed.

The layer then does two small things with the value. It sets `req.request_id`, and lua's answer to a context is right there: the request is a table the whole stack already passes by reference, so an id carried on it needs no context library and no closure tricks. And it appends the id to the response headers, after whatever the inner stack built, so a client can correlate its own logs with the server's. The dispatch base resolves an id too when this layer has not run, which is why every envelope in the kernel tests carries one even without the stack:

```
function M.request_id(new_id)
  return function(next_h)
    return function(req)
      if req.request_id == nil then
        req.request_id = envelope.resolve_id(req, new_id)
      end
      local resp = next_h(req)
      resp.headers = resp.headers or {}
      resp.headers[#resp.headers + 1] = "X-Request-ID"
      resp.headers[#resp.headers + 1] = req.request_id
      return resp
    end
  end
end
```

listing 170: request_id: resolve at the edge, carry on the request, echo on the response (source: lua/service/middleware.lua, lines 31-44)

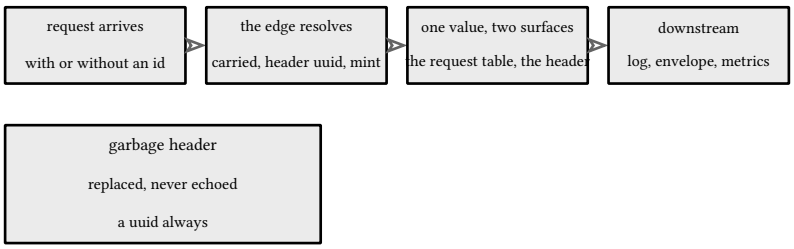


figure 103: one id, born at the edge, quoted by everything downstream

20.3. the access log

The go lane's access log wraps the `ResponseWriter` in a capturing proxy because a go handler writes the answer instead of returning it, and that wrapper costs: first-write rules, forwarded optional interfaces, the double-write trap. The lua kernel returns the answer as a value, so the layer reads it directly, and every one of those costs evaporates before it exists. Method, path, status, bytes, duration, and the request id go out as one json line through the service's own codec, which is the discipline doing double duty: the log line is built exactly the way a wire body is, ordered members by construction.

Two honesty notes ride with the layer. The sink and the clock are injected: tests collect lines into a table and step time by hand, and the production sink writes to stderr because the host owns the process. And the duration comes from the injected clock, which in production is `os.time` in whole seconds, so a fast request logs a duration of 0 and that reading is honest, not a bug: the stdlib has no monotonic clock, the profiling chapter measured that boundary, and finer timing is a c seam question, not a lua one:

```
function M.access_log(sink, clock)
  return function(next_h)
    return function(req)
      local started = clock()
      local resp = next_h(req)
      sink(json.encode(json.object({
        "method", req.method,
        "path", req.path or req.target,
        "status", resp.status,
        "bytes", #resp.body,
        "duration", clock() - started,
        "request_id", req.request_id or "-",
      })))
      return resp
    end
  end
end
```

listing 171: access_log: the value goes on the wire, so the log reads it directly (source: lua/service/middleware.lua, lines 51-67)

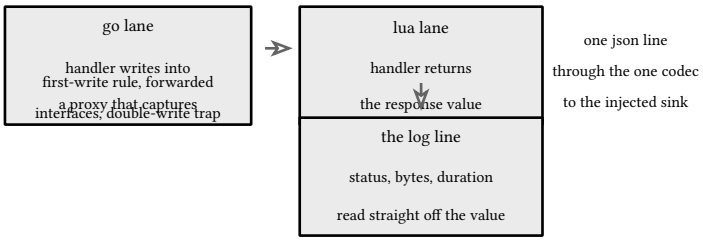


figure 104: no capture wrapper exists to get wrong: the value is the record

20.4. the deadline owns the resume

A deadline in go runs the handler on another goroutine and abandons it on expiry. A lua vm cannot do that: nothing preempts running lua code, and pretending otherwise would be the dishonest sentence this chapter refuses to write. What the layer owns instead is every resume. The inner stack runs inside a coroutine, and a coroutine only continues when someone resumes it, so the layer turns each yield into a checkpoint: resume, check the injected clock against the budget, resume again while time is left. On expiry it abandons the coroutine unresumed, answers the overload envelope, and the suspended routine becomes garbage the collector reclaims.

The boundary is stated as a fact and tested as one. A handler that never yields cannot be interrupted, and the test proves it by burning three times the budget with no yield and getting its answer. The blocking half of the story lives below lua: a client that stalls mid-body holds the connection inside `net.recv`, and the answer there is a socket timeout at the c seam, taught as a host concern. Errors do cross the coroutine boundary, resume reports them as a false return, and the layer rethrows for whoever guards it, the one place the go lane's goroutine rule inverts:

```
function M.deadline(clock, budget)
  return function(next_h)
    return function(req)
      local expires = clock() + budget
      local co = coroutine.create(next_h)
      local ok, out = coroutine.resume(co, req)
      while coroutine.status(co) == "suspended" do
        if clock() ≥ expires then
          return envelope.failure(envelope.fail("overload", "request timed out"),
            req.request_id)
        end
        ok, out = coroutine.resume(co, out)
      end
      if not ok then error(out, 0) end
      return out
    end
  end
end
```

listing 172: deadline: the resume is the checkpoint, expiry abandons the coroutine (source: lua/service/middleware.lua, lines 78-95)

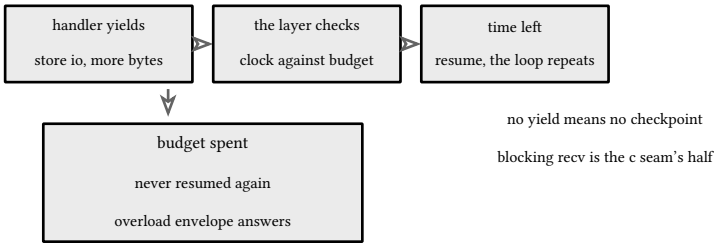


figure 105: each yield is a checkpoint, expiry is a refusal to resume

20.5. recover

A handler bug must not kill the connection, and the host makes that doubly true: a lua error escaping `handle` is fatal for the whole process, the c host exits on it by design. The recover layer stands innermost and converts the failure instead. `xpcall` runs the inner stack with `debug.traceback` as the handler, so the captured value is the error and its stack in one string, one json line goes to the sink with the request id, and the client sees the internal envelope through the one writer, message hidden because a leaked `attempt to index a nil value` tells a client more than it should know.

The go lane rethrows one sentinel, `ErrAbortHandler`, the server's own drop-the-connection signal. There is no lua counterpart to rethrow: the host's fatal errors live outside `handle` entirely, and the kernel's outer pcall is the last resort under this layer, which is also why recover can sit inside the deadline without a goroutine rule. Placement is still mechanical: the deadline runs the stack in a coroutine, and recover inside it catches handler bugs where they happen, the rethrow only ever carrying a bug in a layer itself:

```
function M.recover(sink)
  return function(next_h)
    return function(req)
      local ok, out = xpcall(next_h, debug.traceback, req)
      if ok then return out end
      sink(json.encode(json.object({
        "msg", "panic recovered",
        "request_id", req.request_id or "-",
        "error", tostring(out),
      })))
      return envelope.failure(envelope.fail("internal", "internal error"),
        req.request_id)
    end
  end
end
```

listing 173: recover: xpcall captures the traceback, the envelope hides the detail (source: lua/service/middleware.lua, lines 103-117)

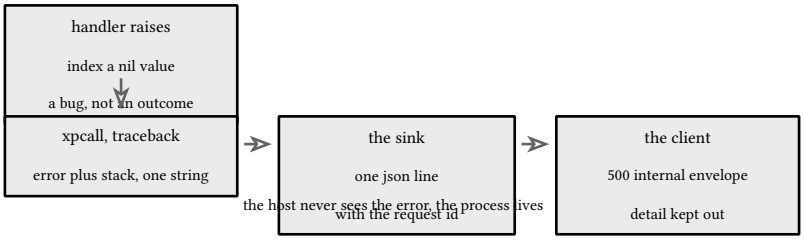


figure 106: a bug becomes one log line and one envelope, never a dead host

20.6. wiring order and what it proves

The order is contract, and each position is a sentence. Request id outermost, because every other artifact quotes it: the access log line, the recover line, the envelope. Access log second, outside recover, so a recovered 500 is logged as the 500 the client saw rather than a status no one answered. Then the families that have landed, in the order the request meets them: the metrics layer, outside the bucket so a refused request still counts, then the limiter, which rejects a 429 before the deadline budget spends anything. The deadline is next, recover innermost beside the handlers it guards, and the identity layer sits below recover with the concurrency chapter's idempotency guard inside it, the innermost family layer. No admission gate ever joined this chain: the sequential vm and the readiness gate at the ship seam own overload, the bucket is the per-route budget, and a hypothetical gate would slot between the metrics counter and the bucket so a refusal still counts. The composition root reads top to bottom as the request flows, and the wrong-order failure mode is exactly the go lane's: a log inside recover records nothing on a panic while the client still gets its 500, two artifacts disagreeing about one request:

```
-- load splice: the admission gate, later the drain that flips readyz
-- limit splice: the token bucket layer
local limit = require("limit")
local limiter = limit.limiter({ now = app.cfg.clock })
for _, name in ipairs(cfg.rules or { "login", "register" }) do
  if name == "login" then
    limit.attach(limiter, "/api/auth/login", limit.login_rule())
  elseif name == "register" then
    limit.attach(limiter, "/api/users", limit.register_rule())
  end
end
app:use(onion(limit.layer(limiter, {
  envelope = function(req, code, message)
    return envelope.failure(envelope.fail(code, message), req.request_id)
  end,
})))
app:use(middleware.deadline(app.cfg.clock, 5))
app:use(middleware.recover(app.cfg.sink))
```

listing 174: the real order on disk: the rules seam naming what attaches, the budget, recover innermost (source: lua/service/main.lua, lines 287-304)

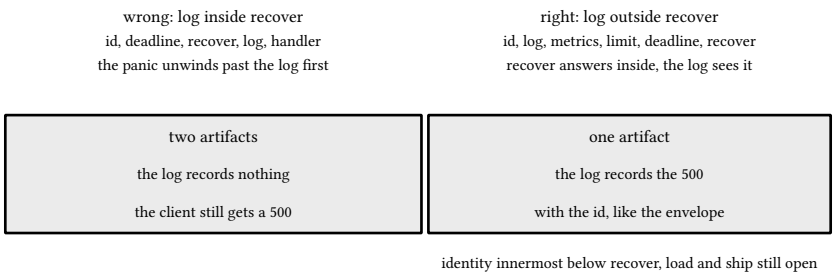


figure 107: wrong order versus right: where the log sits decides what it sees

20.7. testing the chain

Every clock in this chapter's tests is a stepped callable: it advances when the test says so, a fixed increment per call, and no test sleeps anywhere in the part. The fast paths compose recording layers around tiny handlers and read the pass order off the recorded table, the log layers capture their lines into a table and decode them with the service's own codec before asserting, and the deadline tests script the yield sequence so expiry lands on a named resume. The stack test is the one that fails when anyone reorders the wiring: a handler that raises runs under the full four-layer chain, and the assertions demand one recover line, one access line recording the 500, and the id on both the header and the envelope:

```
M[#M + 1] = {
  name = "the stack order: a recovered 500 is logged as the 500 the client saw",
  fn = function()
    local lines = {}
    local clock = stepped(1)
    local stack = M.chain(
      M.request_id(function() return "01a0f5d5-aaaa-7aaa-8aaa-bbbbbbbbbbbb" end),
      M.access_log(function(line) lines[#lines + 1] = line end, clock),
      M.deadline(clock, 5),
      M.recover(function(line) lines[#lines + 1] = line end)
    )(function() error("boom") end)
    local resp = stack({ method = "GET", target = "/healthz", headers = {} })
    T.eq(resp.status, 500)
    T.eq(resp.headers[#resp.headers - 1], "X-Request-ID")
    T.eq(resp.headers[#resp.headers], "01a0f5d5-aaaa-7aaa-8aaa-bbbbbbbbbbbb")
    T.eq(#lines, 2, "the recover log and the access log, one each")
    local access = json.decode(lines[2])
    T.eq(access.status, 500, "the log records the recovered 500")
    T.eq(access.request_id, "01a0f5d5-aaaa-7aaa-8aaa-bbbbbbbbbbbb")
  end,
}
```

listing 175: the stack test: wrong order fails here, log and envelope agree on the 500 (source: lua/service/middleware.lua, lines 277-297)

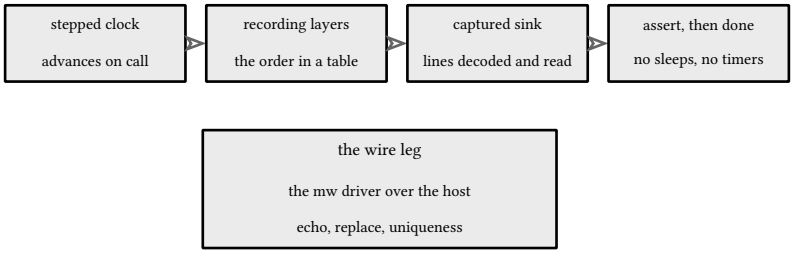


figure 108: stepped clocks, recording layers, captured sinks: the harness has no timers

sources: the lua 5.5 manual at lua.org/manual/5.5, sections 2.6 on coroutines and symmetric control transfer, 6.2 on the coroutine library's resume and status semantics, 6.1 on xpcall message handlers, 6.10 on debug.traceback, and 6.9 on os.time, accessed 2026-09-27. Verified against the go lane's `go/api/internal/middleware/chain.go`, `timeout.go`, and `middleware_test.go`, the python lane's `python/api/pyapi/middleware.py` worker-thread deadline, and the c lane's four-layer continuation stack in `c-os-cloud/api/api.h`, and by the service suites: the 10 middleware module tests, 12 kernel module tests, and the 10 wire checks in the mw driver.

21. the user resource

The service's first real resource is the user: a record with an id, an email, a display name, roles, a creation time, and a version. This chapter builds the resource layer on the chapter 19 substrate: the record and its two projections, validation from first principles, merge-patch semantics, keyset pagination, the in-memory store the store chapter later swaps for sqlite behind the same verbs, and the golden vector replay that pins all of it against the frozen bytes.

21.1. the record and its etag

A lua table is the record, and the wire form is the same table walked in a fixed member order, because a lua table's string keys hash into no order at all. Two order lists name the members, and the stored password hash appears in neither, which is the lua answer to the go lane's dash tag: a field that no order list names never serializes, and there is no second record type to drift. The etag is not stored anywhere, it is computed, the sha256 hex of the exact body bytes the codec produced, quoted:

```
function M.etag_of(body)
  return "\"" .. crypto.hex(crypto.sha256(body)) .. "\""
end

-- the cursor payload is the frozen two-member envelope
```

listing 176: the strong etag, a function of the body bytes (source: lua/service/users.lua, lines 141-145)

Because the etag is a function of the body, the two can never disagree, and a client that saves the etag has saved the body's fingerprint. The handler marshals once, hashes those exact bytes, and writes those same bytes, so the etag on the wire always describes the body on the wire. Creation time is stored as unix seconds and rendered at the edge by `os.date` with the bang prefix, the utc form with no locale and no zone arithmetic, one format string and nothing else. The conditional read answers before any body work repeats: an If-None-Match that equals the computed etag gets the header and an empty 304, which is the whole point of having saved the fingerprint.

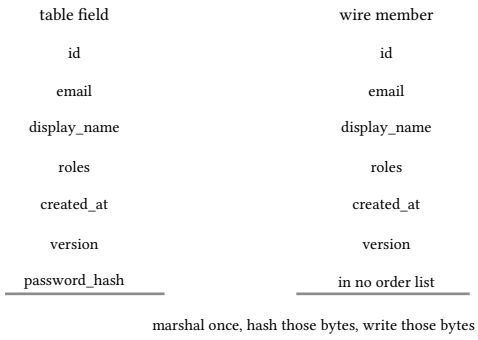


figure 109: the record against its wire members, one field in no order list

21.2. validation from first principles

Validation rules come from the contract, not from a library, and lua's stdlib needs no help stating them: passwords 12 to 128 bytes, display names 1 to 64, one email shape with the 254 ceiling folded in. The bounds live in named constants at the top of the module so the handlers, the tests, and this chapter quote one set of numbers. The email check is a handful of plain finds over bytes:

```lua
function M.valid_email(email)
  local n = #email
  if n == 0 or n > M.EMAIL_MAX then return false end
  for _, ws in ipairs({ " ", "\t", "\r", "\n" }) do
    if email:find(ws, 1, true) then return false end
  end
  local at = email:find("@", 1, true)
  if not at or at == 1 or at == n then return false end
  if email:find("@", at + 1, true) then return false end
  local domain = email:sub(at + 1)
  if not domain:find(".", 1, true) then return false end
  if domain:sub(1, 1) == "." or domain:sub(-1) == "." then return false end
  return true
end
```

listing 177: one email shape, the 254 cap folded in (source: lua/service/users.lua, lines 41-55)

The folding is a contract of details, not of behavior: an overlong address answers the same shape detail a missing dot answers, because the ceiling is part of what a valid address is, never a third tier of mistake. Emptiness is its own tier. A missing email and a malformed email are different failures, `email is required` against `email is not a valid address`, and the same split runs for the password and the display name, so a 422 details array tells the whole story of one request in field order. Length is bytes, the `#` operator, so a multi-byte name counts its utf-8 bytes exactly as the ceilings mean. The normalize step trims and lower-cases before any store contact.

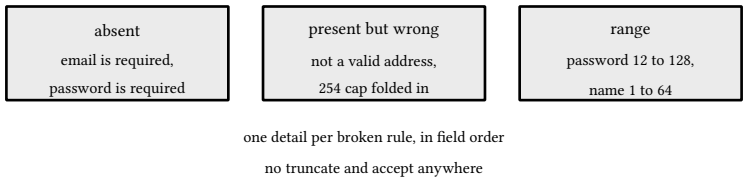


figure 110: the two tiers of wrong, one details array per request

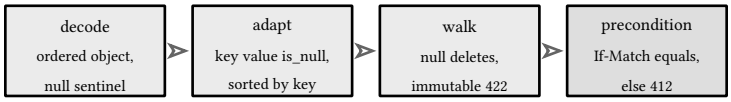
21.3. merge patch semantics

RFC 7386 merge-patch gives an update three-way semantics: an absent member leaves the field alone, a null deletes it, a value replaces it. A lua table cannot hold absent and null apart, so the distinction cannot live in the decoded table, it lives in the adapter. The codec hands back an ordered object with a null sentinel, and `patch_members` walks the codec's own order list into triples, key, value, and the null flag, before any rule runs. The walk itself sorts by member name so the details array is deterministic no matter how the body ordered its members:

```
    if m.key == "display_name" then
      if m.is_null then
        add("display_name cannot be removed")
      elseif type(m.value) ~= "string" then
        add("display_name must be a string")
      else
        local bad = M.validate_display_name(m.value)
        if bad then
          add(bad[1])
        else
          name = m.value
        end
      end
    elseif IMMUTABLE[m.key] then
      add(m.key .. " is immutable")
    else
      add("unknown field " .. m.key)
    end
  end
  return name, details
end
```

listing 178: the member walk: null deletes, immutables refuse on presence (source: lua/service/users.lua, lines 117-137)

The contract makes one member writable, the display name, and the walk shows the whole policy in one branch chain. A null on it answers 422, because deleting the display name would leave the record invalid, so the delete semantics run and the range rules answer. The immutable members, id, email, created_at, version, roles, refuse the moment they appear, null or not: patching one is not a typo, it is an attempt to write a field the contract closed. Unknown members refuse too, the same strictness the register bind enforced with its accepted-member set, enforced here by the walk because a member map cannot carry a set. After the walk the precondition runs, the client's If-Match against the etag of the current body, else 412, and the store's update carries the expected version so a lost race answers 412 as well. The empty patch is legal and answers 200 with the current representation.



absent members never reached the walk at all

a lua table cannot hold absent and null apart, the adapter can

figure 111: absent, null, value: the three-way walk before any write

21.4. keyset cursors

Offset pagination asks for row 86041 onward and fails twice: the store walks and discards every skipped row, and offsets are positions in a moving file, so one registration between requests makes a row appear twice or vanish. Keyset pagination remembers where it stopped. The order is the pair, creation time then id as the tiebreaker, and uuid v7 ids sort by their own embedded time, so the composite order is total. The cursor is the position encoded, base64url without padding over a two-member envelope fixed by the frozen vectors, rendered with one `string.format` call rather than through the json codec, because the shape is frozen and lives entirely in this file, the same ruling the c lane made for its cursor. The decode is strict in layers, the b64url codec first with its own refusals, the padding character, the wrong alphabet, impossible lengths, then the exact envelope shape, then the id's uuid shape:

```lua
function M.decode_cursor(s)
  local raw = crypto.b64url_decode(s)
  if not raw then return nil, "bad base64url" end
  local t, id = raw:match('^{"t":(%d+),"i":"([%da-f%-]+)"}$')
  if not t or #t > 19 or not uuid_shape(id) then return nil, "bad envelope" end
  local n = 0
  for i = 1, #t do n = n * 10 + (t:byte(i) - 48) end
  if n == 0 then return nil, "bad envelope" end
  return n, id
end
```

listing 179: strict decode: any miss is garbage, never a missing page (source: lua/service/users.lua, lines 168-178)

The milliseconds parse as integer arithmetic, multiply by ten and add, which lua's 64-bit integers hold exactly where a float would blur at the thirteenth digit. A cursor the service never issued is garbage, so every miss answers 422, never a 404, and the limit must be a whole number in 1..100 with 20 the default. One unit seam is stated where it lives: the cursor carries milliseconds, the store keys seconds, and the handler divides once at the boundary.

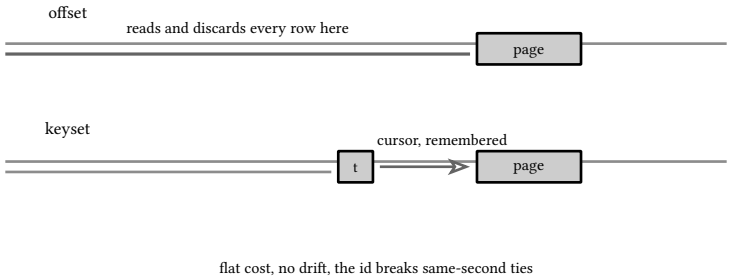


figure 112: offset walks the prefix, keyset jumps to the remembered position

21.5. the store as three tables

The in-memory store is three tables behind one seam: records keyed by id, ids keyed by normalized email for the login lookup, and the ids in keyset order so a list page walks a sorted array and reads the live records through the map, never a stale copy. Every read returns a copy, field by field and roles array by roles array, so a handler that mutates what it read cannot corrupt the store, the value semantics the go lane got from its structs, stated here as a function. There is no lock anywhere, and that is not an omission: the vm runs one instruction stream, a table cannot be mid-mutation under a second reader, and the honest statement is that the single thread is the lock. Errors are strings, `not_found`, `conflict`, `version`, `last_admin`, mapped to the envelope by the handlers, so the store speaks outcomes and never statuses:

```
function store.create(u)
  if by_email[u.email] then return "conflict" end
  if not u.roles or #u.roles == 0 then
    if #order == 0 then
      u.roles = { M.ROLE_ADMIN, M.ROLE_USER }
    else
      u.roles = { M.ROLE_USER }
    end
  end
  u.version = 1
  by_id[u.id] = u
  by_email[u.email] = u.id
  local lo, hi = 1, #order
  while lo ≤ hi do
    local mid = (lo + hi) // 2
    if key_less(u, by_id[order[mid]]) then hi = mid - 1 else lo = mid + 1 end
  end
  table.insert(order, lo, u.id)
  return nil
end
```

listing 180: create: bootstrap roles, then the binary-search insert (source: lua/service/users.lua, lines 298-318)

The insert position comes from a binary search over the order array, so the array stays sorted across inserts. The bootstrap rule lives here: the first registration in an empty store holds both roles, admin and user, every later one holds user alone. The store chapter swaps this object for sqlite over the ffi behind the same verbs, and the contract it must match row for row is this one's behavior.

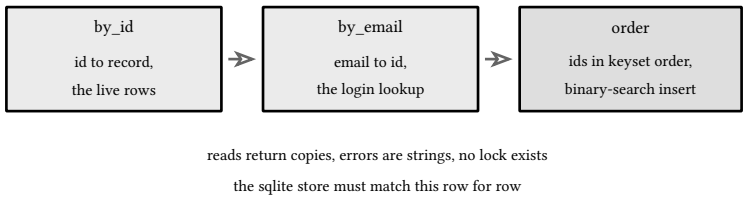


figure 113: three tables, one seam: by id, by email, and the ordered ids

21.6. projections and the admin list

Field-level authorization lives in one function, the projection. An admin reader gets the full order, everyone else gets the public four, id, display name, created_at, version, and no handler decides what to hide because the decision is not in any handler:

```
function M.project(u, admin)
  local order = admin and M.FULL_ORDER or M.PUBLIC_ORDER
  local flat = {}
  for _, k in ipairs(order) do
    local v = u[k]
    if k == "created_at" then v = M.rfc3339(u.created_at) end
    flat[#flat + 1] = k
    flat[#flat + 1] = v
  end
  return json.object(flat)
end

-- the decoded patch body as the walk's members: the codec's ordered
-- object plus the null sentinel become {key, value, is_null} triples,
```

listing 181: the projection: one function, two orders (source: lua/service/users.lua, lines 198-211)

The admin list is where this earns its keep. List rows for an admin are full records with email and roles, the same representation the register route answers, and that is a lesson a sibling lane paid for: its admin list once hid email behind the public projection, a field-level bug that survived every route-level test because the route was allowed and only the row was wrong. The frozen vector walks the public side instead, a non-admin reader listing three users and seeing exactly the four members, and the replay asserts both projections' exact bytes. The roles array rides the record in sorted order, admin before user, so the wire order is the store order and grant answers sort before they persist.

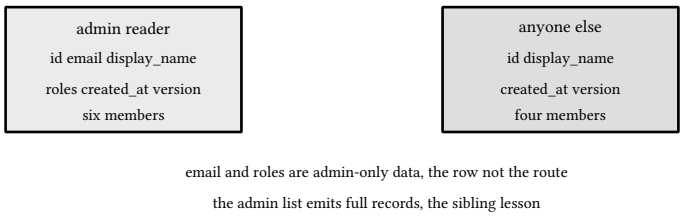


figure 114: two orders out of one record, the reader decides nothing

21.7. golden vector replay

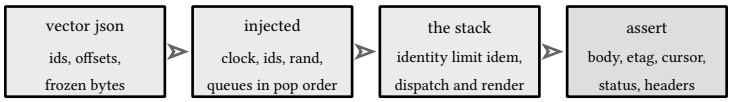
The frozen vectors under `contract/testdata` are the fixture source of truth, and the replay reads them as expectations, never as hints. One fact the fixture had to face: the vectors are scenarios, not one story. Decoding every bearer's sid shows vectors 09 through 12 acting with the very session family vector 08 revokes, so the replay runs four fresh states, the happy walk, the login limits, the reuse attack, and the roles chain, each seeded by its own setup the way each vector's setup field describes. Every time-dependent byte comes from an injected clock ticked to the vector's offset, an id queue loaded with the vector's own uuids in pop order, and a rand queue carrying the pinned refresh bytes:

```
M[#M + 1] = {
  name = "vectors 09 and 10: the etag read and its 304",
  fn = function()
    a.t = harness.T0 + 30
    local auth = { ["Authorization"] = "Bearer " .. a_access,
                   ["X-Request-ID"] = quiet() }
    local res = harness.call(a, "GET", "/api/users/" .. ADA, auth)
    T.eq(res.status, 200)
    T.eq(harness.header(res, "ETag"), ADA_ETAG)
    T.eq(res.body, ADA_BODY)
    a.t = harness.T0 + 31
    res = harness.call(a, "GET", "/api/users/" .. ADA, {
      ["Authorization"] = "Bearer " .. a_access,
      ["If-None-Match"] = ADA_ETAG,
      ["X-Request-ID"] = quiet(),
    })
    T.eq(res.status, 304)
    T.eq(res.body, "")
    T.eq(harness.header(res, "ETag"), ADA_ETAG)
  end,
}

M[#M + 1] = {
```

listing 182: the etag pair: 200 with the body, 304 with only the header (source: lua/service/tests/vectors_test.lua, lines 189-211)

Exactness is the point. The etag assertion proves the body bytes byte for byte, because the etag is the sha256 of those bytes, the cursor assertion proves the encoding down to the missing padding, and the rate header arithmetic proves the bucket's refill to the second. The render point sits between dispatch and the limit layer, so a handler's fail value becomes the envelope response the rate headers stamp. When the vectors change the code is wrong.



four scenarios, fresh state each, no shared story

drift means the code is wrong, never the vector

figure 115: the replay: frozen bytes in, frozen bytes judged

sources: RFC 7386, JSON Merge Patch, and RFC 9110 sections 8.8.3 and 13.1.1 on etags and conditional requests, both at rfc-editor.org. lua.org manual 5.5 sections 6.4 for string.find's plain mode, string.format, and gsub, 6.6 for table.insert, table.sort, and table.concat, and 6.9 for os.date's utc form, all verified against the tools/lua55 pin. The structural mirror is books/go/api/internal/ user, and the cursor's frozen-envelope ruling follows books/ c-os-cloud/api/src/users_cursor.c. Accessed 2026-09-27. Verified by the service suite through `make verify-lua`: users 18 checks, the family replay 13, all green on the pinned runner.

22. authentication

Authentication answers one question, who is calling, and this chapter builds every piece of the answer in pure lua: sha-256 from the fips spec over the bitwise operators, hmac and pbkdf2 over it, login flattened so every miss costs and answers the same, sessions in a fixed preallocated table with rotation and family revocation, the cookie as exact bytes, and HS256 tokens from the primitives rather than a library.

22.1. sha-256 from the spec

Lua's stdlib ships no crypto, so the hash is written from fips 180-4: the message schedule, the 64 rounds, the length as a big endian 64-bit count. Everything runs on 5.3-era integers, 64-bit words masked back to 32 bits after every add, because lua integers do not wrap at 32 and sums drift high without the mask. Rotation composes from the two shifts, the one place the shift-only set needs help:

```
    local t1 = (i8 + s1 + ch + K[i + 1] + w[i]) & 0xffffffff
    local s0 = ror(a, 2) ~ ror(a, 13) ~ ror(a, 22)
    local maj = (a & e) ~ (a & cc) ~ (e & cc)
    local t2 = (s0 + maj) & 0xffffffff
    i8, hh, g = hh, g, f
    f = (d + t1) & 0xffffffff
    d, cc, e = cc, e, a
    a = (t1 + t2) & 0xffffffff
  end
  return (h1 + a) & 0xffffffff, (h2 + e) & 0xffffffff,
    (h3 + cc) & 0xffffffff, (h4 + d) & 0xffffffff,
    (h5 + f) & 0xffffffff, (h6 + g) & 0xffffffff,
    (h7 + hh) & 0xffffffff, (h8 + i8) & 0xffffffff
end

local function word(x)
  return string.char(x >> 24, (x >> 16) & 0xff, (x >> 8) & 0xff, x & 0xff)
end
```

listing 183: the compression rounds, masked to 32 bits after every add (source: lua/service/crypto.lua, lines 58-75)

The one-shot form pads with `string.pack`, a 1 bit, zeros to 56 mod 64, then the bit length as `>I8`, and walks whole 64-byte blocks. Ground truth is never this prose, it is rfc 6234's vectors: the empty string, `abc`, the 56-byte message, the 448-bit message, and the million-a message, all asserted as hex literals in the module's own test list, and they said no more than once while this module grew.

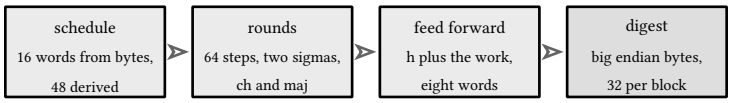


figure 116: one block through the machine: schedule, 64 rounds, eight words out

22.2. hmac and pbkdf2

Hmac is rfc 2104's two-hash construction: a key longer than the block hashes down first, then the key xor one constant pads to 64 bytes and the inner hash feeds the outer. Both pads build with one `gsub` over the key's bytes, and the rfc 4231 cases 1 through 7 pin the result, including the two cases with 131-byte keys that force the hash-down. Pbkdf2 is the same hmac in rfc 8018's loop, each output block chaining the previous block's macs:

```
function M.pbkdf2_hmac_sha256(password, salt, iterations, dklen)
  local parts = {}
  for i = 1, math.ceil(dklen / 32) do
    local u = M.hmac_sha256(password, salt .. string.pack(">I4", i))
    local t = u
    for _ = 2, iterations do
      u = M.hmac_sha256(password, u)
      t = xor_bytes(t, u)
    end
    parts[#parts + 1] = t
  end
  return table.concat(parts):sub(1, dklen)
end
```

listing 184: pbkdf2: the block counter rides the salt, the chain xors every round (source: lua/service/crypto.lua, lines 111-123)

The rfc 7914 one-iteration vector plus two literals cross-checked against python's hashlib pin the loop. The tiers are named numbers, never magic. The issue tier is the 210000-iteration recommendation, and its cost was measured on the tools/lua55 pin on 2026-09-27: one derive takes 19.811 s of cpu. That is the chapter's honest platform fact, a pure-lua kdf prices in seconds on this vm, so the tier is pinned as a constant, structurally asserted, never run by the suite. Tests hash at 1000 iterations, 0.093 s per derive measured the same day, and the encoded string carries the tier that wrote it, so verification replays the writer's parameters, never today's defaults.

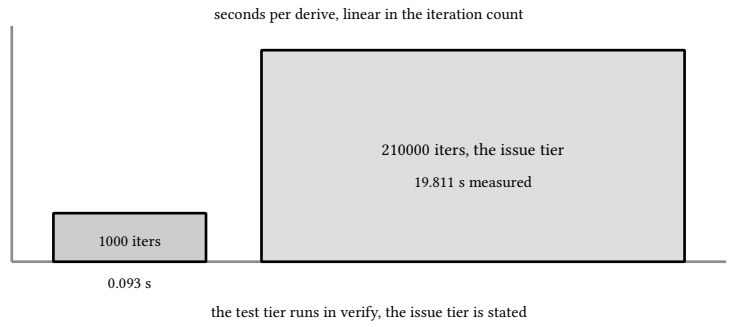


figure 117: the measured ladder: cost against iteration count, 2026-09-27 pin

22.3. login and the dummy verify

A login route that answers fast for unknown emails and slowly for wrong passwords hands out an account census to anyone with a stopwatch. The flatten sits in every miss: the hash of a fixed dummy password is computed once at wire time, and each failure branch runs the same verify against it, discards the result, and answers the identical bytes, empty member, unknown email, wrong password alike:

```
    -- every miss past the wire is the same 401 after the same work:
    -- an empty member, an unknown email, a wrong password, so the
    -- stopwatch learns nothing about which branch ran
    if type(body.email) ~= "string" or type(body.password) ~= "string"
      or body.email == "" or body.password == "" then
      M.verify_password(dummy_hash, tostring(body.password))
      return envelope.fail("unauthorized", "invalid credentials")
    end
    local u = deps.users.get_by_email(users.normalize_email(body.email))
    if not u then
      M.verify_password(dummy_hash, body.password)
      return envelope.fail("unauthorized", "invalid credentials")
    end
    if not M.verify_password(u.password_hash, body.password) then
      return envelope.fail("unauthorized", "invalid credentials")
```

listing 185: every miss past the wire pays the same work and answers the same 401 (source: lua/service/authn.lua, lines 307-321)

The ruling this lane hardened past its go mirror: any miss after the decode ladder is the 401, including an empty member, because a missing email that answered 422 would itself be a distinguishable branch. The wire ladder still runs first, a body that is not json at all is the 400 of the decode step, and the rate bucket is drawn exactly once by the limit layer upstream. No handler here draws a bucket or stamps a rate header, and the replay proves it: vectors 04, 05, and 06 carry the three headers with remaining 4 on both allowed draws and the 429 on the sixth.

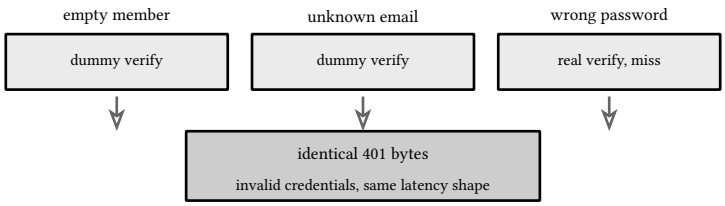


figure 118: three misses, one lane: the stopwatch learns nothing

22.4. sessions in a fixed table

A session is the server's own record of one login, and the table that holds them is fixed and preallocated: a pool of slot tables built once at construction, recycled through a freelist, so a login mutates existing tables instead of allocating. There is no lock anywhere. The vm runs one instruction stream, no handler can observe a table mid-mutation, and the single thread is the lock, stated as the platform's own fact rather than apologized for:

```
function M.new_sessions(capacity)
  local free, by_id, by_refresh = {}, {}, {}
  for _ = 1, capacity do free[#free + 1] = {} end

  local function copy(slot)
    local out = {
      id = slot.id, user_id = slot.user_id, refresh_hash = slot.refresh_hash,
      expires_at = slot.expires_at, revoked = slot.revoked,
    }
    -- reads the live slot, so a family revoked after the copy still
    -- answers dead
    function out.alive(now)
      return not slot.revoked and now < slot.expires_at
    end
    return out
  end
```

listing 186: the preallocated pool, and reads that close over the live slot (source: lua/service/authn.lua, lines 81-97)

Reads return copies, but the copy's `alive` closes over the slot, not over the copied fields, so a family revoked after the copy still answers dead on the same clock tick. The refresh index maps every hash this family ever held to its slot, and retired hashes stay forever, because presenting one is the theft signal the next section builds on. The pool has a hard capacity and answers `full` rather than growing, an in-memory service's honest bound, and the wiring refuses with it: a login at a full pool answers 503 overload instead of minting a token no table backs, so the bound lives on the wire.

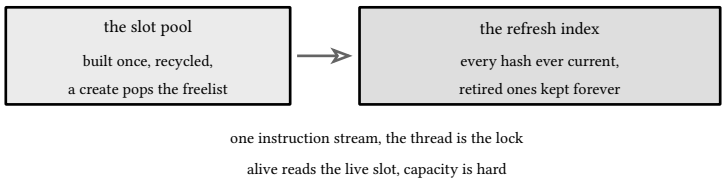


figure 119: the session pool: slots from a freelist, one refresh index, no lock

22.5. the cookie line

The browser carries the refresh token in a cookie, and the cookie's attributes are its entire protection. The line is rendered as exact bytes, not assembled by a library, and the frozen vector pins it:

```
function M.set_cookie_line(token)
  return M.COOKIE_NAME .. "=" .. token
    .. "; Path=/api; HttpOnly; Secure; SameSite=Lax"
end

function M.clear_cookie_line()
  return M.COOKIE_NAME .. "=; Path=/api; Max-Age=0; HttpOnly; Secure; SameSite=Lax"
end

-- the read side of the cookie header: first goapi_session member, no
-- attribute grammar, because the value is a hex token with no semicolons
function M.cookie_value(cookie_header)
  if not cookie_header then return nil end
  for part in cookie_header:gmatch("[^;]+") do
    local v = part:match("^%s*" .. M.COOKIE_NAME .. "=(%S+)$")
    if v then return v end
  end
  return nil
end

-- the only header this service ever writes: the alg is pinned before
```

listing 187: the set line, the clear line, and the first-member reader (source: lua/service/authn.lua, lines 158-178)

Path scopes the cookie to the api tree so it never rides on requests to other origins hosted beside the service. HttpOnly keeps scripts from reading it, so an injected snippet cannot exfiltrate what it cannot see. Secure keeps it off plain http. SameSite Lax blocks the cookie on cross-site posts while allowing ordinary top-level navigation. Logout clears it with Max-Age=0 rendered exactly, the attributes staying set so the deletion overwrites the exact cookie it replaces. The reader side is deliberately small, the first goapi_session member of a Cookie header and no attribute grammar. The bearer lane is accepted everywhere the cookie is, and both resolve to one actor, one identity either way.

attribute	stops	scope
Path /api	leak on sibling paths	one tree
HttpOnly	script exfiltration	read denied
Secure	plaintext transit	https only
SameSite Lax	cross-site post	top nav only

logout renders Max-Age=0 with the attributes kept

bearer and cookie resolve to one actor

figure 120: four attributes, four attacks, zero script visibility

22.6. HS256 from scratch

A json web token is three base64url parts, and the whole format fits in two functions over the chapter's primitives. The header and the claims render as fixed-order bytes with `string.format`, member order frozen by construction, so the same facts always produce the same token and the golden test reproduces the vector's token byte for byte, signature included:

```
-- any payload byte is trusted
local JWT_HEADER = '{"alg":"HS256","typ":"JWT"}'

local function claims_bytes(c)
  return string.format('{"exp":%d,"iat":%d,"iss":"%s","sid":"%s","sub":"%s"}',
    c.exp, c.iat, c.iss, c.sid, c.sub)
end

function M.sign_hs256(secret, c)
  local hs = crypto.b64url_encode(JWT_HEADER)
  local ps = crypto.b64url_encode(claims_bytes(c))
  local sig = crypto.b64url_encode(crypto.hmac_sha256(secret, hs .. "." .. ps))
  return hs .. "." .. ps .. "." .. sig
end

-- three strict segments, the signature before any claim is read, then
-- issuer and expiry. the claims shape is the frozen member order, so
-- the parse is a pattern over the exact bytes this service writes
local CLAIMS_PAT = '^{"exp":(%d+),"iat":(%d+),"iss":"([%w%-]+)",'
  .. '"sid":"([%da-f%-]+)","sub":"([%da-f%-]+)"}$'
```

listing 188: fixed-order bytes in, one hmac, base64url unpadded out (source: lua/service/authn.lua, lines 179-198)

The parse is the mirror discipline. Three strict segments, the signature decoded and compared in constant time before any payload byte is read, the algorithm pinned by comparing the header against the exact bytes this service writes, so the alg none forgery dies at the header. One bug the suite caught while this grew is worth naming: the signed message is header dot claims with the trailing dot excluded, and one early draft hashed one byte more than it verified. Issuer is checked, expiry always runs, and there is no roles claim, because authorization reads the store, which keeps every outstanding token revocable through its sid. The asymmetric variant is the one thing this platform cannot hand-roll at book scale: the stdlib has no curves, ed25519 would be a chapter of its own, and the dependency ruling forbids a crypto library. So the split-key argument stays prose, whoever can verify can also mint under HS256, and one binary with one signer makes that fine.

header	claims	signature
alg typ, fixed bytes b64url unpadded	exp iat iss sid sub rendered by string.format	hmac of header dot claims, the trailing dot excluded

alg pinned first, expiry always runs, no roles claim

figure 121: three segments, the signature covers exactly the first two

22.7. rotation and reuse detection

The access token lives fifteen minutes. The refresh token is the long-lived credential, 32 opaque bytes hex-rendered to the client and stored only as its sha256, so a stolen store dump cannot be replayed against the live service. It rotates on every use, and the store retires the presented hash rather than deleting it, because presenting a retired hash is the theft signal:

```
function sessions.rotate(current, next_hash, expires_at)
  local slot = by_refresh[current]
  if not slot then return nil, "unknown_refresh" end
  if (slot.retired and slot.retired[current]) or slot.revoked then
    slot.revoked = true
    return nil, "revoked"
  end
  slot.retired = slot.retired or {}
  slot.retired[current] = true
  slot.refresh_hash = next_hash
  slot.expires_at = expires_at
  by_refresh[next_hash] = slot
  return copy(slot)
end
```

listing 189: rotate: a retired or revoked presentation kills the family (source: lua/service/authn.lua, lines 128-142)

The vector pair pins the whole story. Refresh at plus twenty retires the first token and answers a new cookie and a new access token. Replay the retired token at plus twenty-one and both die: the replay answers 401 session revoked, and the successor, still perfectly formed, answers 401 session revoked one second later. The freshly minted next token on the refusal path never enters the store. A thief holding the token from before the holder refreshed gets one use, then loses the family along with the victim.

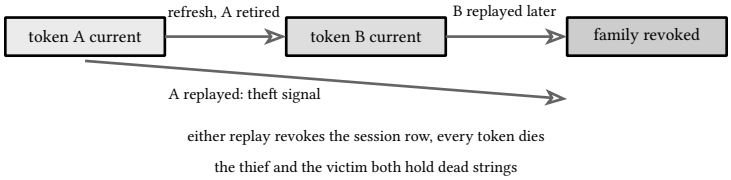


figure 122: the family: rotation forks, either replay revokes everything

22.8. golden vector replay

The login scenario replays vectors 04, 05, and 06 on one fresh state, and the reuse scenario replays 07 and 08 on another. The clock ticks to each offset, the queues hold the vector's session id and pinned refresh bytes, so both cookies and tokens reproduce exactly as the files carry them:

```
M[#M + 1] = {
  name = "vector 08: reuse revokes the family, both tokens die",
  fn = function()
    c.t = harness.T0 + 21
    local res = harness.call(c, "POST", "/api/auth/refresh", {
      ["Cookie"] = "goapi_session=" .. c_cookie,
    })
    T.eq(res.status, 401)
    T.eq(res.body, '{"error":{"code":"unauthorized","message":"session revoked",'
      .. '"request_id":"01a0f5d5-be88-7315-ae13-93688719ada4"}}')
    c.t = harness.T0 + 22
    res = harness.call(c, "POST", "/api/auth/refresh", {
      ["Cookie"] = "goapi_session=" .. c_rotated_cookie,
    })
    T.eq(res.status, 401)
    T.eq(res.body, '{"error":{"code":"unauthorized","message":"session revoked",'
      .. '"request_id":"01a0f5d5-c270-74e6-af08-2f1338edb7e0"}}')
  end,
}
```

listing 190: the reuse pair: both 401 bodies byte exact (source: lua/service/tests/vectors_test.lua, lines 344-362)

The 429 is the middleware layer's answer, not a handler's, and the replay proves the layer drew once: five allowed logins at one tick show remaining 4, 3, 2, 1, 0, and the sixth answers the envelope with Retry-After 1 and Reset on the next whole second. When the vectors change the code is wrong, and nothing in this chapter has a second source of truth to drift toward.

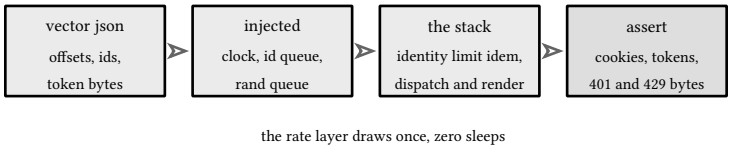


figure 123: two scenarios, five vectors, every byte from the fixture queues

sources: fips 180-4 for the sha-256 specification, rfc 6234 for the sha and hmac test vectors, rfc 4231 for the hmac cases, rfc 8018 for pbkdf2, rfc 7914 for its one-iteration vector, rfc 7519 for jwt claims, and rfc 6265 for cookie attributes, all at rfc-editor.org. lua.org manual 5.5 section 3.4.2 for the bitwise operators and their 64-bit integer domain, 6.4 for string.pack, gsub, and format, all verified against the tools/lua55 pin. The structural mirror is books/ go/api/internal/authn, whose argon2id contrast the pbkdf2 tier measurement replaces with this platform's own numbers, and books/ c-os-cloud/api/src/authn_sha.c for the hand-rolled discipline. Accessed 2026-09-27. Verified by the service suite through `make verify-lua`: authn 10 checks, the family replay 13, all green.

23. authorization and roles

Authentication settled who is calling. Authorization decides what that caller may do, and this chapter builds the whole answer as one data table and one walk over it: the two questions kept apart with their two codes, the policy rows keyed by method and pattern, the object routes asked by their pattern and never a raw path, roles as store data with the never-zero-admins guard, the two-direction audit that keeps the route table and the policy table honest, and the matrix replay that runs every route against every actor kind.

23.1. two questions, two codes

The identity layer resolves one actor from the credentials and installs it on the request, or installs nothing. Everything past that point reads the actor and never parses a token, checks a signature, or touches a cookie. Two failure codes carry the two questions: 401 means no proof of identity arrived, try again with credentials, and 403 means the identity is known and still may not, do not bother retrying. Conflating them produces the classic broken answer, a 403 for an anonymous request that leaks what standing the route wants:

```
function M.decide(pattern, has_actor, is_admin, is_owner)
  local row = M.POLICY[pattern]
  if not row then return "no_row" end
  if row.auth and not has_actor then return "unauthenticated" end
  if row.admin and not is_admin then return "forbidden" end
  if row.self_or_admin and not is_owner and not is_admin then
    return "forbidden"
  end
  return "allow"
end

-- extract the value under one {name} by walking path and pattern in
```

listing 191: the walk: 401 before any 403, no row the loud failure (source: lua/service/authz.lua, lines 64-75)

The order inside is policy, not accident. An anonymous caller learns nothing about a route's rules, because the 401 answers before any rule is consulted, and a present actor that fails a rule gets the 403 with the rule's own words, a statement about the actor and never about the resource. No handler re-derives any of this. The handlers read an actor or return the 401 fail value, and the walk is the only place a verdict is computed in the whole vehicle.

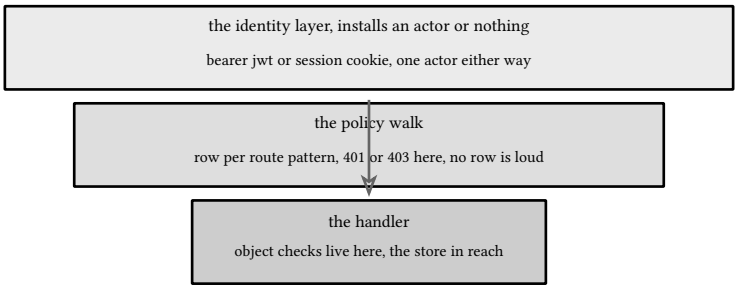


figure 124: two layers in wiring order: identity outside, the walk, the handler last

23.2. the policy table

The whole surface is fifteen rows of data, method and pattern as the key, three flags as the value, readable in one screen and greppable in one search. Plain data beats a rules dsl or annotation mining at this size because the audit test can walk it, a new route is a row next to its siblings, and nothing about the table needs a second parser:

```
M.POLICY = {
  ["GET /healthz"] = {},
  ["GET /readyz"] = {},
  ["POST /api/users"] = {},
  ["POST /api/auth/login"] = {},
  ["POST /api/auth/refresh"] = {},
  ["POST /api/auth/logout"] = { auth = true },
  ["GET /api/users"] = { auth = true },
  ["GET /api/users/{id}"] = { auth = true, self_or_admin = true },
  ["PATCH /api/users/{id}"] = { auth = true, self_or_admin = true },
  ["DELETE /api/users/{id}"] = { auth = true, self_or_admin = true },
  ["POST /api/admin/users/{id}/roles"] = { auth = true, admin = true },
  ["DELETE /api/admin/users/{id}/roles/{role}"] = { auth = true, admin = true },
  ["POST /api/reports"] = { auth = true },
  ["GET /api/reports/{id}"] = { auth = true },
  ["GET /metrics"] = {},
}
```

listing 192: the contract's whole route surface as fifteen rows (source: lua/service/authz.lua, lines 42-58)

The flags read as demands. `auth` demands a resolved actor, `admin` demands the admin role, `self_or_admin` accepts the object's owner or an admin. The object routes carry the owner flag in the row, but the check itself runs in the ladder the next section builds, because ownership needs the record. Later chapters append rows for the reports pair, and none ever edit an existing one, the same append-only discipline the route table keeps.

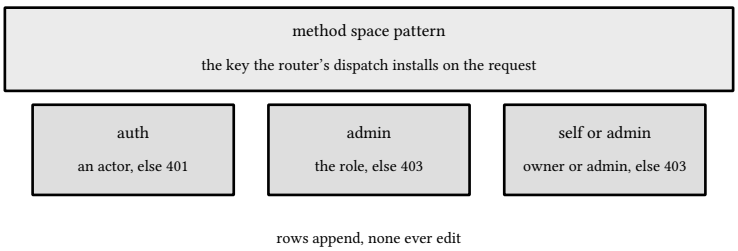


figure 125: rows as data: one key, three flags, the walk reads them in order

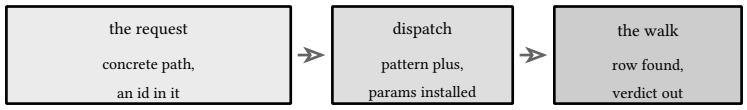
23.3. the object routes and the id

Route rules cannot guard a route whose danger lives in the path parameter. Reading `/api/users/` followed by someone else's id is the IDOR class, and it survives every route-level check because the route is the same for everyone. The check needs the record, which is why the object routes run a ladder: the 401 before anything touches the store, the store's 404 for an unknown id, then the walk asked with the pattern and the fetched record's ownership:

```lua
function M.load_target(req, store)
  local actor = req.actor
  if not actor then
    return nil, envelope.fail("unauthorized", "authentication required")
  end
  local u = store.get(req.params.id)
  if not u then
    return nil, envelope.fail("not_found", "user not found")
  end
  local outcome = M.decide(req.method .. " " .. req.pattern, true,
    actor.admin, actor.user_id == u.id)
  if outcome == "forbidden" then
    return nil, envelope.fail("forbidden", "not the owner")
  end
  if outcome == "no_row" then
    return nil, envelope.fail("internal", "route without a policy row")
  end
  return u
end
```

listing 193: the ladder: 401, then the store's 404, then the walk's 403 (source: lua/service/authz.lua, lines 127-145)

The walk is asked with the route's pattern, never the raw path, and that distinction is the biggest catch of this whole part's review history. The table keys on patterns, `GET /api/users/{id}` with braces, while a request carries a concrete path with a real uuid in it, so a walk fed the raw path answers `no_row` for every object request and the owner check silently never runs, leaving any authenticated actor free to touch any user. The router seam hands the pattern over at dispatch, `req.pattern` installed beside the params, so the ladder reads what the table is keyed by. The 404 before the 403 is a stated tradeoff: this service lets an authenticated stranger distinguish an unknown id from a forbidden one, and an api that prefers hiding existence answers 404 for both, a policy choice the audit records either way.



a raw path fed to the table answers no row, the check never runs

the c lane's review catch, restated as this lane's test

figure 126: pattern in, verdict out: the raw path never reaches the table

23.4. roles as data

Roles are rows, not magic. The closed set is two names, the grant and revoke verbs live in the user store where the data lives, and the one invariant that matters is never zero admins, enforced on the post state of both verbs: the scan substitutes the candidate role set for the stored one, and only a post state with some admin standing persists, so a rejected demotion leaves no trace:

```lua
local function set_roles(id, role, adding)
  local u = by_id[id]
  if not u then return nil, "not_found" end
  local next_roles = {}
  for _, r in ipairs(u.roles or {}) do
    if not (not adding and r == role) then next_roles[#next_roles + 1] = r end
  end
  if adding and not has_role(u, role) then
    next_roles[#next_roles + 1] = role
    table.sort(next_roles)
  end
  local candidate_admin = has_role({ roles = next_roles }, M.ROLE_ADMIN)
  if not admins_remain(id, candidate_admin) then return nil, "last_admin" end
  u.roles = next_roles
  local answer = {}
  for i, r in ipairs(next_roles) do answer[i] = r end
  return answer
end

function store.grant_role(id, role)
  return set_roles(id, role, true)
```

listing 194: both verbs run the same guard on the post state (source: lua/service/users.lua, lines 370-390)

The frozen vector walks the story at store level and through the routes: demoting the only admin answers 409 conflict, last admin, granting admin to a second user succeeds, and revoking that second admin then succeeds too, because the first stands again. The bootstrap rule makes the lockout unreachable in practice, the first registration in an empty store holds admin, and the guard makes it unreachable in theory. A plain user cannot self-grant admin for a simpler reason: the roles routes' rows demand the admin flag, and ownership of the target id means nothing there, the matrix section pins that cell at 403.

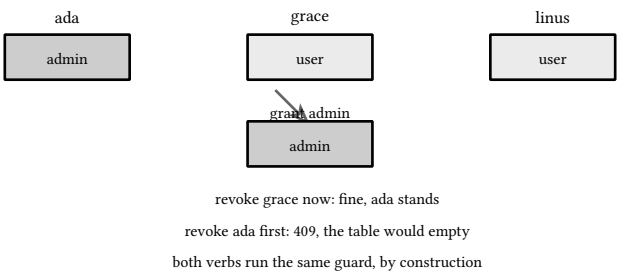


figure 127: the grant table: the admin row must never empty, both verbs checked

23.5. deny by default and the audit

One table maps every registered route to its row, and the audit faces the two against each other in both directions. A route without a row cannot pass: the walk answers `no_row`, the guards treat it as a wiring bug and answer the internal envelope, and the build itself runs this audit over the real route table and refuses to start on a mismatch, so the failure is loud at every layer. A row without a route is dead, and the audit names it too, because a policy for a route nobody registered is a lie about the surface:

```
function M.audit(routes)
  local problems
  local function add(msg)
    problems = problems or {}
    problems[#problems + 1] = msg
  end
  local seen = {}
  for _, r in ipairs(routes) do
    local key = r.method .. " " .. r.pattern
    if not M.POLICY[key] then
      add("route " .. key .. " has no policy row")
    end
    seen[key] = true
  end
  local keys = {}
  for k in pairs(M.POLICY) do keys[#keys + 1] = k end
  table.sort(keys)
  for _, k in ipairs(keys) do
    if not seen[k] then add("policy row " .. k .. " matches no route") end
  end
  return problems
end

-- the object-route ladder, the one arrangement that answers all three
```

listing 195: both directions: no row and no route are both named (source: lua/service/authz.lua, lines 98-121)

The closed decision surface is the deeper point. One function computes every verdict in the vehicle, handlers ask it or run the ladder that asks it, and no second decision path exists anywhere, so there is exactly one place to audit, one place to test, and one place a mistake can hide. The suite runs the audit against the shipped route list and it answers clean, and it answers with named problems the moment either side drifts.

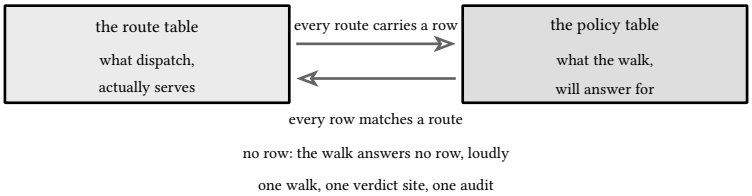


figure 128: the audit faces the tables: both directions, both named

23.6. the matrix replay

The grid is the proof. Every route against every actor kind, anonymous, owner, stranger, admin, one expected status per cell, asserted through the walk itself, and the vector's last-admin chain replayed through the real routes beside it:

```lua
M[#M + 1] = {
  name = "the matrix: every route against every actor kind",
  fn = function()
    local grid = {
      { "GET /api/users", "401", "200", "200", "200" },
      { "GET /api/users/{id}", "401", "200", "403", "200" },
      { "PATCH /api/users/{id}", "401", "200", "403", "200" },
      { "DELETE /api/users/{id}", "401", "204", "403", "204" },
      { "POST /api/admin/users/{id}/roles", "401", "403", "403", "200" },
      { "DELETE /api/admin/users/{id}/roles/{role}", "401", "403", "403", "200" },
    }
    for _, row in ipairs(grid) do
      local key = row[1]
      T.eq(M.decide(key, false, false, false), verdict(row[2]), key .. " anon")
      T.eq(M.decide(key, true, false, true), verdict(row[3]), key .. " owner")
      T.eq(M.decide(key, true, false, false), verdict(row[4]), key .. " stranger")
      T.eq(M.decide(key, true, true, false), verdict(row[5]), key .. " admin")
    end
  end,
```

listing 196: the grid: six routes, four actors, one status each (source: lua/service/authz.lua, lines 303-321)

Each cell fails with a name when it fails. The stranger column is the IDOR discipline, 403 on every object verb whatever etag the stranger presents, and the admin column on the roles rows is the self-grant refusal, 403 for a plain user even on their own id. The vector chain pins the store-level guard the same way, the 409 and the two 200s around it arriving as frozen bytes through dispatch. One stated divergence lives here: the frozen register vectors answer without rate headers, so the replay harness attaches only the login rule while the production root composes both, the bytes outranking the prose for replay. Adding a route costs one policy row and one matrix cell, and the audit plus the grid together answer the two questions a reviewer actually asks: is every route covered, and does every covered route behave for every kind of caller.

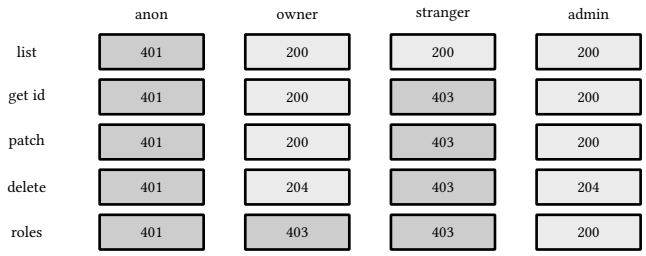


figure 129: routes one way, actors the other, statuses in the cells

sources: OWASP, Insecure Direct Object Reference Prevention Cheat Sheet and Access Control Cheat Sheet, at cheatsheetseries.owasp.org, for the IDOR class and the deny-by-default guidance. RFC 9110 sections 15.5.1 through 15.5.4 for the 401, 403, and 404 semantics. lua.org manual 5.5 section 6.6 for table.sort's stable use as a deterministic ordering over the policy keys, verified against the tools/lua55 pin. The structural mirror is books/go/api/internal/authz, and the pattern-not-path ruling follows the c lane's books/c-os-cloud/api/src/authz_policy.c and its review history. Accessed 2026-09-27. Verified by the service suite through `make verify-lua`: authz 9 checks, the family replay 13, all green.

24. the store

The service has run on tables in memory, which tests love and restarts hate. The platform ruling gives this lane a different answer than the other books took: the logic half stays pure lua 5.5, and the store rides the jit lane built in chapter 17, sqlite 3.53.4 through the book's own ffi wrapper against the mingw dll. No driver is translated, no engine is rebuilt, and no community dependency enters the graph, because the wrapper already exists as book content and the engine arrives as a pinned binary. What this chapter owns is the surface the service codes against: users, roles, sessions with their refresh history, events, and the analytics reports, every statement parameterized, every write group one `BEGIN IMMEDIATE` transaction, and every verdict read from a row.

The lane boundary is stated where it lives. The plain 5.5 interpreter has no ffi library, so `store.lua` never enters the plain runner's table: its suite rides `run-jit.lua` and the `verify-lua-jit` gate runs it green, a boundary the suite chapter owns, not a skip.

24.1. the ffi lane to the pinned engine

The wrapper is chapter 17's own `books/lua/jit/sqlite.lua`, reused without duplication. It pins the dll by `arg[0]` depth and asserts the engine version at require time, and the service tree sits at the same depth the wrapper was built for, so the same line resolves from here. The wrapper's surface is deliberately small: `open`, `exec`, `rows`, `close`, and the statement methods behind `rows`. What the store needs beyond that surface it composes over the wrapper's own contract rather than editing frozen book content.

That contract has a sharp edge the store's first red run found twice. The `rows` iterator finalizes its statement exactly when it observes a non-row step. A caller that abandons the iterator early leaks an unfinalized statement, and the close then refuses with `unable to close due to unfinalized statements`. A caller that resumes the iterator after the `nil` steps a handle sqlite already freed, and the process dies with a segmentation fault, no error message anywhere. Both exits sit one call apart, and the discipline that avoids both is the same: run every iterator exactly to its first `nil`:

```
-- the wrapper's iterator owns the statement's lifetime and finalizes
-- exactly when it observes a non-row step: a caller that abandons it
-- early leaks an unfinalized statement and the close refuses, a
-- caller that resumes it after nil steps a freed handle. so first
-- takes the leading row, and only when there was one keeps stepping
-- to the nil that finalizes
local function first(conn, sql, params)
  local it = conn:rows(sql, params)
  local row = it()
  if row == nil then return nil end
  while it() do end
  return row
end
```

listing 197: first takes the leading row, then steps to the nil that finalizes (source: lua/service/store.lua, lines 130-143)

`first` answers the leading row of a query and, only when there was one, keeps stepping to the `nil` that finalizes. `run` executes a write, whose single step observes `done` and finalizes on the spot. Every read and write in the module goes through one of the two, and the one place the walk needs an early exit, the list's `has_more` probe, drives its iterator by hand for exactly this reason: a `return` inside a generic `for` breaks the loop before the finalizing step.

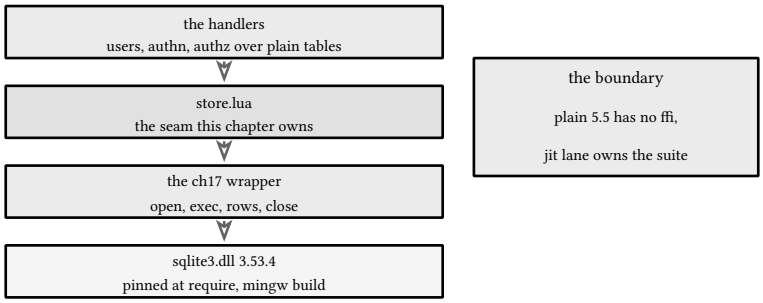


figure 130: the lane the store rides, and the one it never crosses

24.2. migrations as append-only records

Schema changes are code changes, so they ship as records. The registry is the `PRAGMA user_version` integer the database keeps about itself, each record is a numbered entry in a lua table, and the history is append-only by contract: a shipped version is frozen, a correction is a new numbered version, and no record is ever edited. Two records ship with the store. The first is the whole base vocabulary, users with their email uniqueness index, roles as the one many-to-many edge, sessions with their refresh token history, events as one presence row per user per day, and reports with their body claim. The second is the optimistic currency substrate, the `version` column the guarded update checks and bumps, and the keyset index in the exact order the list walk sorts on.

Each record applies inside its own `BEGIN IMMEDIATE` group with the version bump in the same group, so a record that fails halfway leaves nothing durable and can simply be retried. A pragma value cannot be bound, the one formatted integer in the module, and it is the module's own loop counter rather than caller data. The reopen test pins the ladder end to end: a file that closes at version 2 reopens at version 2, asserted on the pragma itself, and applies nothing again.

```lua
local function migrate(conn)
  local v = first(conn, "PRAGMA user_version").user_version
  for i = v + 1, SCHEMA_VERSION do
    tx(conn, function()
      conn:exec(MIGRATIONS[i])
      -- a pragma value cannot be bound; i is our own loop counter
      conn:exec(string.format("PRAGMA user_version = %d", i))
    end)
  end
  return true
end
```

listing 198: one record, one group, one bump, applied at most once (source: lua/service/store.lua, lines 163-173)

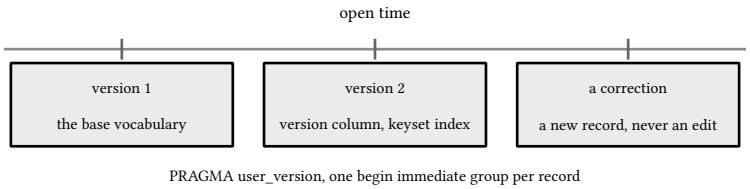


figure 131: the version ladder only grows right

24.3. the register transaction

Registration is the one write that decides two facts, a user exists and that user's roles, and the two must land together or not at all. The whole exchange runs inside one `BEGIN IMMEDIATE` group: the email claim, the insert, the bootstrap admin decision, and both grants. `BEGIN IMMEDIATE` takes the write lock up front, so two first registrations serialize at the lock and exactly one of them ever sees an empty admin count, the count-then-insert race closed

by construction rather than by luck. The count reads the roles table before this row's grants land, so a store with no admin crowns the row and a store with one does not, and the granted set rides home in the same committed fact that answers 201.

The email verdict is a select inside the group. A read decides, the unique index behind it is the invariant's seat, and the test that inserts a duplicate past the checked path still trips the index, because the schema, not the read, owns the claim. The failed group returns a sentinel, `nil, "conflict"`, that the handler maps to the 409 the contract freezes, and the rollback test stages a row mid group and proves the exit discards it whole.

```
function M:user_register(row)
  return tx(self.conn, function()
    if first(self.conn, SQL.email_taken, { row.email }) then
      return nil, "conflict"
    end
    run(self.conn, SQL.insert_user,
      { row.id, row.email, row.display_name, row.password_hash, row.created_ms, 1 })
    local granted = {}
    -- the count reads the roles table before this row's grants land:
    -- a store with no admin crowns this row, a store with one does not
    if first(self.conn, SQL.admin_count).n == 0 then
      granted[#granted + 1] = "admin"
    end
    granted[#granted + 1] = "user"
    for _, role in ipairs(granted) do
      run(self.conn, SQL.grant, { row.id, role })
    end
    return granted
  end)
end
```

listing 199: one group: claim, insert, count, grant, commit (source: lua/service/store.lua, lines 190-210)

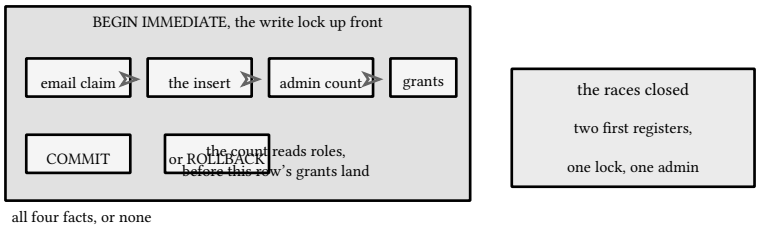


figure 132: the register group, one fence around four facts

24.4. guarded statements that keep an admin

The contract's never-zero-admins invariant rides the statement's own predicate, so the count and the write cannot be raced apart even in principle. A grant passes when the granted role is admin, the post state keeps an admin by construction, or an admin already stands. A revoke passes when the row is not an admin row, or another user holds admin and survives the delete. Both are single statements, `INSERT OR IGNORE ... SELECT` and `DELETE ... AND` a guard subquery, and a refused statement touches nothing, so a rejected change leaves no trace.

The verdict then reads the row itself. Standing after the grant means the guard refused it, gone after the revoke means it landed or never existed, and both are the caller's ok. Row existence is the discriminator for a reason the wrapper makes plain: a change count belongs to the connection rather than to the statement, and this wrapper does not even surface one, so the row is the only honest witness. A revoke of a role nobody holds is an idempotent no-op, and the last admin revoke answers the sentinel the handler renders as the 409 the contract freezes.

```lua
-- one statement, one serialization. the never-zero-admins invariant
-- rides the statement's own predicate, so the count and the write
-- cannot be raced apart, and the verdict reads the row itself:
-- standing after the grant means the guard refused it, gone after
-- the revoke means it landed or never existed
function M:role_set(user_id, role, grant)
  run(self.conn, grant and SQL.grant_guarded or SQL.revoke_guarded,
    { user_id, role })
  local held = first(self.conn, SQL.role_held, { user_id, role }) ~= nil
  if grant then
    if role == "admin" or held then return true end
    return nil, "conflict"
  end
  if role ~= "admin" then return true end
  if held then return nil, "conflict" end
  return true
end
```

listing 200: the guarded pair, then the row itself decides (source: lua/service/store.lua, lines 276-292)

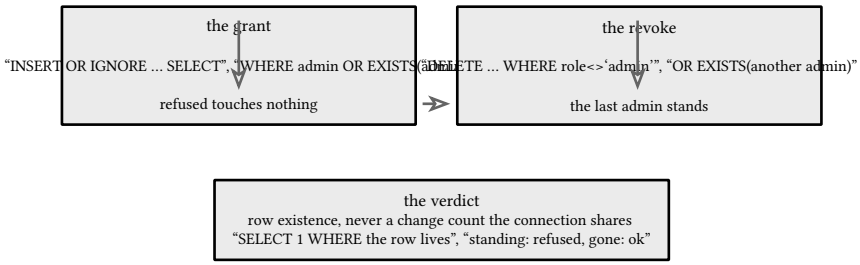


figure 133: two guards, one verdict

24.5. reads, the keyset walk, the version guard

Reads are one statement each, by id, by email, the roles of a user, and all of them through `first`. The list is the one read with mechanics worth their own section. The cursor the contract defines is the `(created_at, id)` pair, and the walk compares it as a sqlite row value, `(created_ms,id) > (?1,?2)`, one comparison that covers the first page, whose floor is the smallest time with an empty id, and every continuation. The fetch asks for `limit+1` rows, and the extra row is only the `has_more` observation: the answer carries the items and the cursor of the last returned item, or no cursor when the walk saw nothing beyond.

The walk drives its iterator by hand because its early exit is the exact shape the generic `for` cannot express safely here: the moment the extra row arrives, the walk has its answer, and breaking the loop then would leak the statement. The version guard is the write side of the same currency. `user_patch_cas` reads the row, refuses a missing row as `not_found` and a stale version as `version`, then runs the guarded update whose where clause still carries the version check, the seat the invariant keeps on any host that ever threads this code, because in this vm one instruction stream means nothing can move the row between the read and the write at all.

```
function M:user_list(limit, after)
  limit = math.max(1, math.min(limit or 20, 100))
  local floor_t = after and after.t or -9223372036854775808
  local floor_id = after and after.id or ""
  local items = {}
  local it = self.conn:rows(SQL.list, { floor_t, floor_id, limit + 1 })
  local row = it()
  while row ~= nil do
    if #items == limit then
      while it() do end -- drain to the nil that finalizes
      local last = items[#items]
      return { items = items, next = { t = last.created_ms, id = last.id } }
    end
    items[#items + 1] = row
    row = it()
  end
  return { items = items, next = nil }
end

-- the guarded update. one instruction stream runs this store, so
-- nothing can move the row between the read and the write, and the
-- version guard stays in the where clause anyway: it is the seat the
```

listing 201: the walk: row-value floor, limit+1 probe, a hand-driven iterator (source: lua/service/store.lua, lines 225-246)

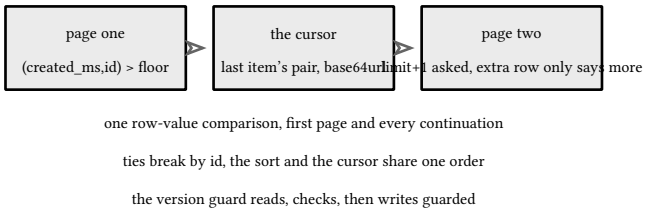


figure 134: pages chained by the same pair the sort uses

24.6. the analytics report as committed queries

The report is where sql earns its seat back. Presence collapses events to one row per user per day because actives count users, not actions, and the primary key absorbs the duplicate write. The series is one committed query: `ROW_NUMBER` over each user's days in day order marks the first day in the window, the newcomer, daily counts aggregate the marks, and `LAG` carries yesterday's count with today's as its default so the first delta is zero, window functions doing in one statement what the pre-store engines of the other books walked in loops. Leaders rank by day count descending then user id ascending, `RANK` over a total order, so a replayed report cannot differ run to run.

The compute and the done flip share one write group, so a reader never sees a done report with no computed rows behind it. The store surface keeps the contract's shape: create stores a pending row, the first read computes and flips, and a replay recomputes the same deterministic series, which is why the reports pair in the frozen vectors upgrades a replayed 202 to a 200 with identical bytes. The seeded test walks the vector's fixture, three events over two days and two users plus one duplicate write that presence absorbs, and asserts the series rows the frozen body carries.

```lua
-- the series, the leaders, and the done flip share one write group,
-- so a reader never sees a done report with no computed rows behind
-- it, and a replay recomputes the same deterministic series
function M:report_compute(id, from_day, to_day)
  return tx(self.conn, function()
    local rows = {}
    for row in self.conn:rows(SERIES_SQL, { from_day, to_day }) do
      rows[#rows + 1] = { day = row.day, active = row.active,
        newcomers = row.newcomers, delta = row.delta }
    end
    local leaders = {}
    for row in self.conn:rows(LEADERS_SQL, { from_day, to_day }) do
      leaders[#leaders + 1] = { user_id = row.user_id, days = row.days, rank = row.rnk }
    end
    run(self.conn, SQL.report_done, { id })
    return rows, leaders
  end)
end
```

listing 202: the series, the leaders, and the flip, one committed group (source: lua/service/store.lua, lines 356-373)

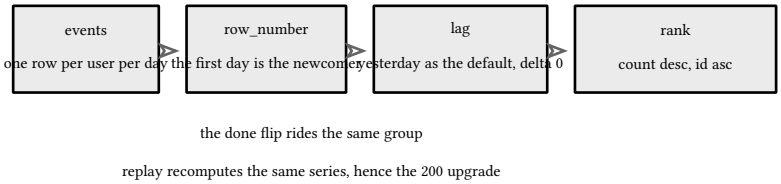


figure 135: events to series, one committed walk

sources: sqlite.org for the statement lifecycle (c3ref/prepare.html, c3ref/step.html, c3ref/finalize.html, c3ref/close.html), the transaction semantics (lang_transaction.html, BEGIN IMMEDIATE takes the reserved lock before the first read), the versioned schema (pragma.html carrying PRAGMA user_version), row values (rowvalue.html, the pair comparison the cursor and the keyset index share), the upsert clause (lang_conflict.html, INSERT OR IGNORE and ON CONFLICT DO
NOTHING), and the window functions (windowfunctions.html, ROW_NUMBER, LAG, RANK), all accessed 2026-09-27, the luajit ffi semantics from luajit.org/ext_ffi_semantics.html accessed 2026-09-27, and the wrapper contract from this book's own chapter 17. Verified by the jit store suite at books/lua/service/store_jit.lua, 18 checks including the reopen ladder, the duplicate email, the last admin, the keyset walk, the rotation family, and the report pair shape, run green by make verify-lua-jit from the repo root.

25. concurrency control

The store chapter's guard gave one writer the win and the loser the fact. This chapter moves the fact toward the wire and builds the platform's own version of everything around it: the etag duel two writers settle on one version, the conditional read that answers 304 before a body is spent, and the idempotency store that turns a retried register into a replay instead of a second create. The other lanes buy determinism with machinery, barriers to release writers together, singleflight to collapse a herd, locks to keep tables honest. This vm has one instruction stream, so those problems do not arise here, and the chapter says so plainly where the machinery is absent, as the platform's own lesson rather than an apology.

25.1. the duel: two writers, one version

Two clients read the same user, both hold the etag of version 1, and both patch. The wire answers in layers, and the store's guard is the layer that is atomic. The pure precondition ladder this chapter lands refuses a missing or stale `If-Match` before any decode burns, the same decision the middleware chain will carry at the edge, and a handler can check again because a route can exist without the edge. The store's `user_patch_cas` is the guarantee: the read that decides and the guarded write share one instruction stream here, and the where clause still carries the version, the seat the invariant keeps on any host that ever threads this code.

The duel rides the real store, so its suite lives in the jit lane where the store lives. There is no barrier object, because the vm never preempts: the test's own resume order is the schedule, every order is a legal one, and the guard is the arbiter. The choreography is three writer coroutines created around one shared read, resumed in the order the test names, and the outcome is a fact about the guard, never about the schedule:

```
local function duel(order)
  local s = store.new(":memory:")
  s:user_register({ id = "u-a", email = "a@example.org",
    display_name = "a", password_hash = "h", created_ms = 1 })
  local shared = s:user_get("u-a") -- both writers read here, at v1
  local writers = {}
  for i = 1, 3 do
    writers[i] = coroutine.create(function()
      return s:user_patch_cas("u-a", shared.version, "writer " .. i)
    end)
  end
  local results = {}
  for _, i in ipairs(order) do
    local ok, a, b = coroutine.resume(writers[i])
    results[i] = { ok, a, b }
  end
  local winners, refusals = 0, 0
  for i = 1, 3 do
```

listing 203: three writers around one shared read, resumed in the test's order (source: lua/service/store_jit.lua, lines 336-353)

Exactly one writer carries home the bumped row, the other two answer the version sentinel the handler renders as the 412 the contract freezes, and the row stands at version 2 after every run.

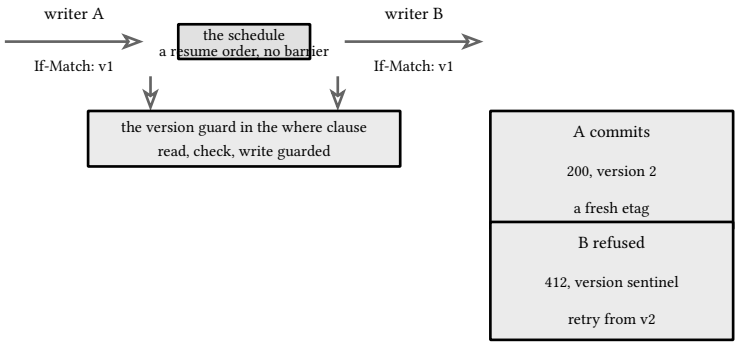


figure 136: two etags read at v1, the guard crowns one

25.2. the conditional read answers 304

The read side of the same coin costs nothing when nothing changed. A GET carrying an `If-None-Match` that matches the representation the resource wears right now answers 304 Not Modified, the body is never marshaled, and the client keeps the copy the etag names. Both preconditions are pure functions over two strings, the header the client sent and the etag the resource wears, so the whole wire decision is testable as arithmetic:

```
-- If-None-Match: absent is an ordinary read, a match answers the 304
-- that saves the body, a list matches when any member equals the
-- current etag. no header state invents a resource: a nil current
-- etag passes through so the handler's own 404 answers
function M.check_if_none_match(header, current)
  if header == nil or current == nil then return "fresh" end
  if header == "*" then return "not_modified" end
  for candidate in header:gmatch("[^,%s]+") do
    if candidate == current then return "not_modified" end
  end
  return "fresh"
end
```

listing 204: the 304 ladder: absent, star, list membership, fresh (source: lua/service/idem.lua, lines 22-33)

Four decisions in order. No header means an ordinary read. A header against a resource that is gone passes through, so the handler's own 404 answers and the precondition never invents a status. A list matches when any member equals the current etag. Only the match answers 304, with the etag stamped and an empty body, because a 304 with a body would teach the client to cache the wrong thing for the wrong reason. The write side is the same shape one section up: `check_if_match` answers `missing` for an absent header, accepts * against any live representation, and answers `stale` for a value the row no longer wears and for any value on a gone row, because the precondition is about the write, never the read.

One line on the wire itself: the contract and the frozen vectors pin single exact etags, and the handlers implement exactly that equality, the go mirror byte-identical on every pinned case – a list-bearing `If-None-Match` reads as no match and answers the full 200, a star `If-Match` fails the equality and answers 412 – while the two ladders above remain the vocabulary a contract with lists and stars would wire.

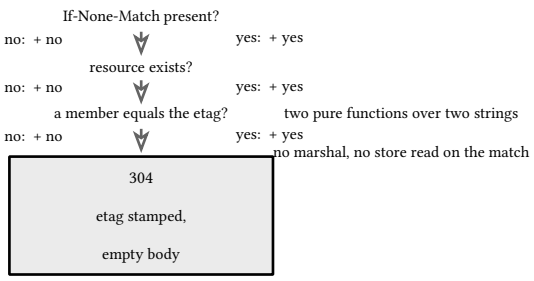


figure 137: the conditional GET decision

25.3. the idem store and the replay contract

A client that times out retries, and a register route that recreates on retry is a double-charge machine. The contract's answer is the `Idempotency-Key` header, and the composition ruling for this service is that one layer owns every idempotency decision: the scope triple, route, identity, key, is composed inside `idem.lua` and nowhere else, and no handler keeps an inline check. The exact request bytes are hashed, whitespace drift is a different request, and the verdict is one of three answers. A miss runs the handler. A replay returns the stored snapshot at the stored status, the retried register answers its original 201 with the same bytes, `Location` and `ETag` included. A mismatch, the same key under a different hash, is the 422 with details, the client misusing the contract, and the stored snapshot is untouched by the refusal:

```
-- one of "miss", "mismatch", or "replay" plus the snapshot row
function layer:check(route, identity, key, body_sha)
  local row = rows[at(route, identity, key)]
  if row == nil then return "miss" end
  if clock() ≥ row.expires_at then
    rows[at(route, identity, key)] = nil
    return "miss"
  end
  if row.body_sha ~= body_sha then return "mismatch" end
  return "replay", row
end
```

listing 205: three verdicts, and the expired row deletes itself on contact (source: lua/service/idem.lua, lines 55-66)

Two guardrails ride the store. Only outcomes under 400 become snapshots, because a retry after a validation error deserves a fresh run, never a replayed failure. And expiry is lazy against the injected clock: an expired row deletes itself on contact, so correctness never waits for a sweep, and no sweep exists anywhere, because the vm has no background thread to run one. The rows are a lua table, and the boundary is stated rather than hidden: replays live exactly as long as the vm, this lane's answer where the c lane persists idem rows in sqlite. The backstop behind that boundary is the store itself, the register group's email claim, so a key lost to a restart degrades to a correct 409, never a double create.

The reports routes replay through the same layer with one sanctioned divergence, and the divergence stays handler logic, never store logic: the routes live in their own module beside the store, the idem row carries the 202 a first submission answered, the report row the snapshot names decides the replay answer, pending replays the 202 and a finished job upgrades to the 200 of the completed body, exactly the pair the frozen vectors pin, byte for byte over the store's deterministic series.

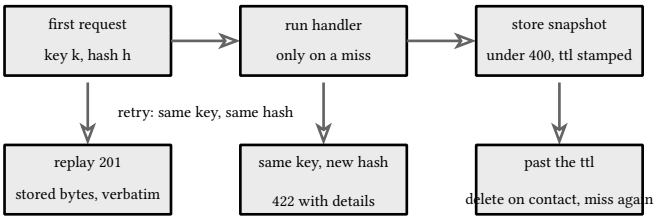


figure 138: one key's life: first run, snapshot, replay

25.4. what the single vm buys

The c lane had to build the unlocked-table race before it could lock it, and the go lane parks a herd on a channel inside singleflight. Neither problem can exist here, and the reason is not discipline but physics: one instruction stream means no two bytecodes interleave, so a table the vm reads and writes cannot be read by anyone else mid-thought. What the other lanes buy with locks, this platform buys by having no second reader. The chapter states this as the honest divergence, singleflight is unnecessary, and the test suite never spends a sleep proving it.

The gap is still real in one specific sense, and the module stages it rather than denying it. Coroutines are cooperative, so a yield placed between the idem lookup and the save opens exactly the window singleflight exists to close: a second caller sees the same miss, and the layer has no defense of its own. The vm never places that yield, no handler yields inside the window, and the seam contract says so. If the contract is ever broken, the backstop answers, the store's register group refuses the second create with the email conflict, so the worst case is a correct 409 and a lost replay, never a double create:

```
    -- yield inside the check-to-save window, and the store's own
    -- uniqueness claim is the backstop if it is ever broken
    local layer = M.new_layer({ clock = function() return 0 end })
    local a = coroutine.create(function()
      T.eq(layer:check("POST /api/users", "u", "k", "sha"), "miss")
      coroutine.yield("parked mid-window")
      layer:save("POST /api/users", "u", "k", "sha",
        201, "application/json", "{}")
      return "a saved"
    end)
    local ok, parked = coroutine.resume(a)
    T.eq(ok, true)
    T.eq(parked, "parked mid-window")
    -- the second caller runs while a is parked: it sees the same miss
    T.eq(layer:check("POST /api/users", "u", "k", "sha"), "miss")
    local ok2, done = coroutine.resume(a)
    T.eq(ok2, true)
    T.eq(done, "a saved")
    T.eq(layer:size(), 1)
  end,
}
```

listing 206: the yield placed in the gap: the window exists only if code opens it (source: lua/service/idem.lua, lines 256-276)

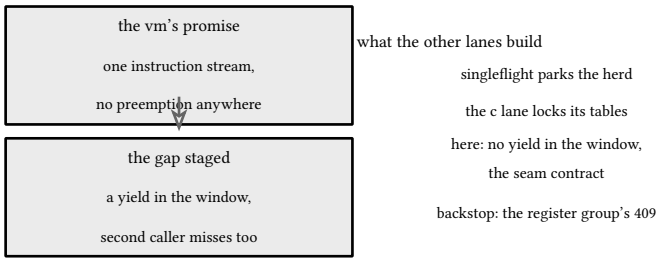


figure 139: one stream versus a parked coroutine

25.5. interleavings as scripted resume orders

Every interleaving the other lanes stage with barriers, channels, and bubbles, this lane stages as a resume order, and the staging is deterministic by construction: the test names the schedule, the vm runs it, and no two runs can differ because there is nothing to differ. The matrix test makes that a named invariant over a replay: for every legal resume order of the same three writers, the outcome set is identical, one bump and two refusals, because the schedule an order picks cannot move a guarded row. Six orders, six identical verdicts, zero sleeps, and the property is the chapter's whole argument in one loop:

```lua
M[#M + 1] = {
  name = "every scripted resume order crowns exactly one winner",
  fn = function()
    -- named invariant, deterministic replay: for every legal resume
    -- order the outcome set is identical, one bump and two refusals,
    -- because the schedule an order picks cannot move a guarded row
    for _, order in ipairs({
      { 1, 2, 3 }, { 3, 2, 1 }, { 2, 1, 3 }, { 3, 1, 2 },
      { 1, 3, 2 }, { 2, 3, 1 },
    }) do
      duel(order)
    end
  end,
}
```

listing 207: six legal orders, one invariant, the schedule cannot move a guarded row (source: lua/service/store_jit.lua, lines 378-391)

The replay property runs against the idem layer too, on the plain lane: a seeded op stream over three keys, and the invariant that the layer is a pure function of its ops, replaying the stream twice and asserting every step's verdict matches. Seeded streams, fixed invariants, no framework, the discipline the testing spine named and this family keeps.

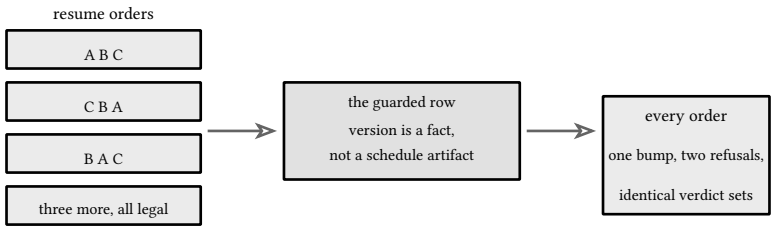


figure 140: orders in, the same verdict out

sources: rfc-editor.org for RFC 9110 section 13.1.1 If-Match and section 13.1.2 If-None-Match with the strong validator semantics of section 8.8.3, the 304 semantics of section 15.4.5, and the idempotency method property of section 9.2.2, accessed 2026-09-27, lua.org manual 5.5 sections 2.6 and 6.3 for coroutines, resume, and yield, accessed 2026-09-27, the singleflight registration semantics restated from the x/sync v0.22.0 source as the go lane's chapter 24 quotes it, and the c lane's unlocked-table lesson from this book's c-os-cloud chapter 34. Verified by `books/lua/service/idem.lua`, 9 checks on the plain pin, and the duel and order matrix in `books/lua/service/store_jit.lua`, 18 checks on the jit pin, both gates green with zero sleeps.

26. caching

A cache is a promise to answer from a copy, and every promise has a price. This chapter prices it in the platform's own currency: the lru as two plain tables, because lua has no container library to lean on, the ttl as arithmetic on an injected clock, the vary pair in the key so two identities never share an entry, the guarded fill that memoizes successes only, invalidation as delete after write scoped by path, and the exclusion table that makes caching opt-in rather than a default someone must remember to turn off.

26.1. the lru in two tables

Least recently used needs two structures that agree. The map finds any key in constant time, the links carry the recency order, and in lua both are tables: every entry is a node with `prev` and `next` fields pointing at its neighbors, the map points at nodes, and each node carries its own key so eviction can drop it from the map without walking the map. Two sentinels, `head` and `tail`, close the list so the front and back cases need no branches of their own. A write inserts at the front, a read moves its node to the front, and when capacity is exceeded the node at `tail.prev` is the eviction, found and removed in constant time.

The proof is the scramble test: fill to capacity, touch the oldest insert so it becomes the hottest, and the next insert evicts the true least recently used entry instead. The count rides a plain field, maintained at every insert, evict, and lazy expiry, because lua's `#` operator on a hash of nodes is a length nobody promised, and the op-stream test pins both invariants at once, the count never exceeds capacity across two hundred seeded operations and a walk over the links from `head` to `tail` visits every key the map holds exactly once:

```
function methods:put(key, value)
  local node = self.map[key]
  if node ~= nil then
    node.value = value
    if self.ttl then node.expires_at = self.clock() + self.ttl end
    unlink(node)
    push_front(self, node)
    return
  end
  if self.capacity ≤ 0 then return end
  while self.n ≥ self.capacity do
    self:delete(self.tail.prev.key) -- the true lru, not the oldest insert
  end
  local fresh = { key = key, value = value }
  if self.ttl then fresh.expires_at = self.clock() + self.ttl end
  self.map[key] = fresh
  push_front(self, fresh)
  self.n = self.n + 1
end
```

listing 208: the pairing: the map finds, the links order, the node carries its key (source: lua/service/cache.lua, lines 71-89)

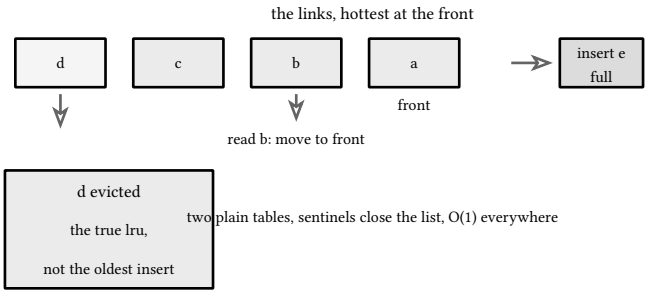


figure 141: a read moves its node to the front, an insert evicts the back

26.2. ttl with the injected clock

The lru answers crowding, not age. An entry nobody reads can be least recently used forever and still hold its bytes, and an entry everybody reads can stay hot long after the origin moved on. The ttl stamps an expiry from the injected clock at write time, and reads check it lazily: an expired entry deletes itself on contact, so every read is its own eviction decision. A re-put refreshes both the value and the expiry, and the arithmetic is exact, one millisecond short of the ttl the entry still answers, at the ttl it is gone.

No sweep exists anywhere, and that is a platform fact rather than a design choice: the vm has no background thread to run a janitor on, so an entry nobody reads leaves either by contact, when some request finally asks for it, or under capacity pressure, when crowding evicts it like any other cold entry. The test states the honest cost in two lines, both entries aged out and unread still hold their bytes, and the next insert evicts one while the other waits for a contact that deletes it. Nothing waits for a timer, and no test sleeps:

```
-- a read moves its node to the front, an expired entry deletes itself
-- on contact, and no sweep exists anywhere: the vm has no background
-- thread, so an entry nobody reads leaves by contact here or under
-- capacity pressure, the honest cost stated in the chapter
function methods:get(key)
  local node = self.map[key]
  if node == nil then return nil end
  if self.ttl and self.clock() ≥ node.expires_at then
    self:delete(key)
    return nil
  end
  unlink(node)
  push_front(self, node)
  return node.value
end
```

listing 209: the read: lazy expiry, delete on contact, move to front (source: lua/service/cache.lua, lines 55-69)

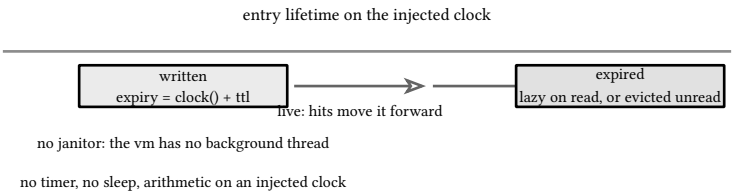


figure 142: two clocks push entries out: age and crowding

26.3. vary and the cache key

A private api's representation depends on who is asking, and the vary pair names that dependency: the authorization header and the session cookie. The cache key carries the path plus each vary value in fixed order, so two identities never share an entry and the same identity shares across requests that present the same pair. The key uses the values themselves rather than a derived identity, because the promise is about what was asked, not about who the router decided was asking, and the fixed order makes the key a pure function of its inputs, the same request always building the same key.

The wire states the same contract through the `Vary` response header, and the two must agree: the header tells any client-side cache which request headers the representation varies by, the layer's key list is the same fact stated in code. A body that varies by identity without a vary key is a cross-user leak waiting for traffic, and the pairing test pins the shape, two tokens on one path, two entries, no bleed-through either way:

```lua
M.VARY = { "authorization", "cookie" }

-- the cache key: the path plus each vary value in fixed order, two
-- identities never sharing an entry
function M.key(path, vary)
  local parts = { path }
  for _, name in ipairs(M.VARY) do
    parts[#parts + 1] = vary[name] or ""
  end
  return table.concat(parts, "|")
end
```

listing 210: the vary pair and the key, fixed order, values not identities (source: lua/service/cache.lua, lines 11-21)

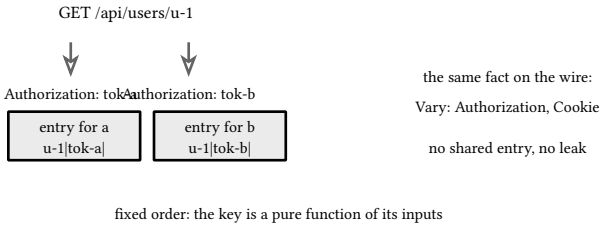


figure 143: one path, two identities, two entries

26.4. the guarded fill

A miss runs the loader, and the guard decides what the miss may leave behind: only a 200-shaped answer memoizes, so an origin error never becomes a sticky failure, a refused fill misses again on the next ask and the origin gets another chance. The verdicts are named at the call, `hit`, `filled`, `refused`, and the hit answers from the copy without the loader running at all.

The other lanes wrap this fill in singleflight because a hot expiry means a herd of identical origin reads. This vm cannot have that herd: one instruction stream, no preemption point between the miss and the fill, so the first request to miss is necessarily the only one running until it lands. The chapter states the divergence rather than building the missing machinery, and the single-writer discipline survives it, the fill's result is the only thing that ever writes the entry, so an error can never be parked in the copy by a second hand:

```lua
function methods:fill(key, loader)
  local hit = self:get(key)
  if hit ~= nil then return hit, "hit" end
  local res = loader()
  if type(res) == "table" and res.status == 200 then
    self:put(key, res)
    return res, "filled"
  end
  return res, "refused"
end
```

listing 211: the fill: a hit answers, a 200 stores, anything else refuses (source: lua/service/cache.lua, lines 97-106)

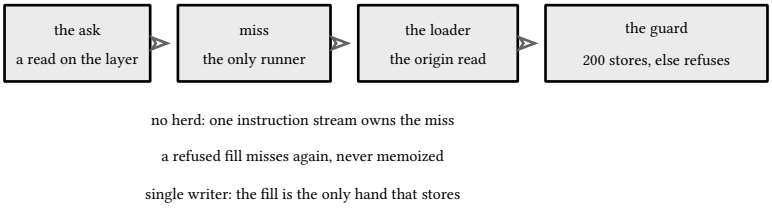


figure 144: one miss, one fill, the guard at the exit

26.5. path-scoped invalidation on write

A write changes the origin, and the copies of what it changed are now wrong. The discipline is delete after write, never update: the writer drops the affected entries and the next read refills through the guard. Deleting is idempotent and always safe, updating instead pushes the writer's view into the entry, sounds cheaper, and leaves the wrong value carrying a fresh timestamp the moment any raced ordering lands, the worst of both worlds. The refilled copy always reads the origin's current truth, and the staged test spells the whole arc, a stale copy answers, the write drops it, the refill reads new.

The rule is path plus collection. A write to a row drops the row entry, the collection that reads it, and any child under it. A write to a collection drops the collection and everything beneath it. The sibling row survives a row write, the reports entry survives a users write, and the drop count rides home so the caller can state what it invalidated. The staleness window is stated honestly and not apologized for: a role grant leaves the member's cached object stale to ttl, because the grant writes roles and the cache holds the user representation, and a revalidating client can loop on the stale copy until age takes it. The other lanes pay the same price with the same scoping, and the fix is the same everywhere, invalidate what the write touched and accept the window on what it did not:

```
-- delete after write, never update: the writer drops the affected
-- entries and the next read refills through the guard. the rule is
-- path plus collection: a write to a row drops the row, the
-- collection that reads it, and any child under it, a write to a
-- collection drops the collection and all its children
function methods:invalidate(path)
  local drop = {}
  for key in pairs(self.map) do
    local p = key:match("^([^|]+)")
    if p == path
      or p:sub(1, #path + 1) == path .. "/"
      or path:sub(1, #p + 1) == p .. "/" then
      drop[#drop + 1] = key
    end
  end
  for _, key in ipairs(drop) do self:delete(key) end
  return #drop
end
```

listing 212: delete after write: the row, its collection, its children, nothing else (source: lua/service/cache.lua, lines 108-125)

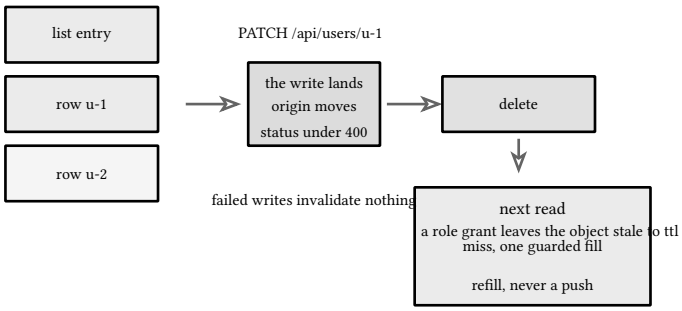


figure 145: write to a row: the row and the list go, the refill is the only writer

26.6. what never caches

The last decision is the one that prevents the incidents, and the shape is a table that denies by default: a route is cacheable only where the predicate says so, and nothing else in the service can flip that by accident. The predicate governs the object read route only. The list family is not acted on by the running service, the row policy states

that boundary instead of hiding it, and a future wave that wants the list cached changes one predicate and owns the invalidation that comes with it.

The exclusions carry reasons. Credential routes carry secrets and must never be replayable from a copy, and the policy already refuses them, they are not GET. Rate-limited answers must always reach the client, because a cached budget answer is a lie about money, and the fill's guard is the second wall, a 429 never memoizes. A personalized body without a vary key is a cross-user leak, and the vary section closed that door. Everything else answers from the origin every time, and the cheap check is the one worth keeping in the gate, because the day someone widens the predicate by accident, this is the test that names the lane they put it in:

```
-- deny by default: a route is cacheable only where this predicate
-- says so, and it governs the object read route only. the list family
-- is not acted on by the running service, a boundary stated in the
-- chapter, not hidden in the table
function M.cacheable(method, path)
  if method ~= "GET" then return false end
  return path:match("^/api/users/[^/]+$") ~= nil
end
```

listing 213: the policy: one object route opts in, everything else denies (source: lua/service/cache.lua, lines 23-30)

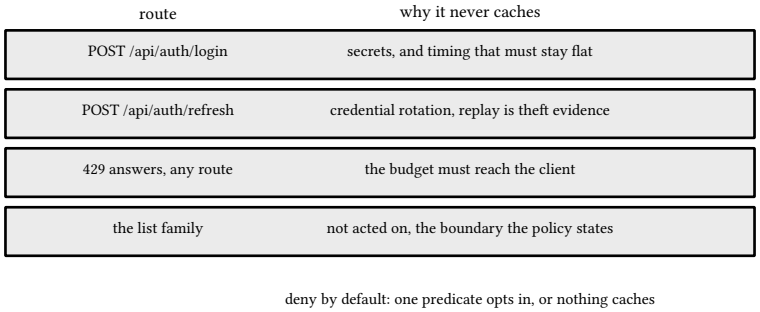


figure 146: the exclusion rows, each with its reason

sources: rfc-editor.org for RFC 9111 section 5.2 Cache-Control directives and RFC 9110 section 12.5.5 Vary, with the no-store promise of RFC 9111 section 5.2.1.5, accessed 2026-09-27, lua.org manual 5.5 sections 2.1 and 3.4.9 for tables as references and the length operator's promise on sequences, accessed 2026-09-27, and the staleness window statement restated from this book's c-os-cloud chapter 35 as the cross-lane ruling it is. Verified by `books/lua/service/cache.lua`, 11 checks on the plain pin, the scramble, the ttl arithmetic, the vary pairing, the guarded fill, the two invalidation rules, the policy table, and the op-stream capacity invariant, green in the plain lane's runner.

27. rate limiting

A service that answers every request it receives eventually answers none of them: password guessing arrives as a flood of logins, a runaway client loop arrives as the same request thousands of times, and a retry storm arrives the moment a slow route teaches every caller to resend. This chapter builds the service's rate limiting in `service/limit.lua`: the token bucket derived from first principles with the clock injected, one keyed table whose idle sweep is amortized into the draw, the 429 contract with its headers and envelope, what to key a limit on and what the host seam actually provides, and the fixed-clock test discipline that keeps every assertion off the wall clock.

27.1. why limits exist

Capacity is finite and failure is nonlinear. Past some offered load a service does not degrade gently, it falls off a cliff: queues grow, latency climbs past every client timeout, memory fills with requests waiting on answers that can no longer arrive, and the health check itself starts timing out. A limit is the refusal to stand on that cliff. It converts an overload that would take the whole service down into a fast, honest rejection of the excess, and the client that honors it gets a working service at one request per second instead of a dead one at zero.

The two limited routes carry numbers chosen for what they protect. Login allows a burst of five per address, then one per second, because every login attempt pays the password verification budget, the pbkdf2 ladder the authentication chapter measures, and a credential guesser does not need more than five tries to be dangerous. Register allows ten per client ip for the same reason plus its idempotency bookkeeping. Both refill one token per second, so a client that trips the limit is back to full capacity five and ten seconds later, which is the difference between a limit and a lockout:

```
-- the contract's two rules, verbatim numbers: login bursts 5 per
-- lower(email), register bursts 10 per client ip, both refill one
-- token per second
M.login_burst = 5
M.register_burst = 10
M.refill_per_second = 1.0
M.login_message = "too many login attempts"
M.register_message = "too many registration attempts"
M.default_idle_ttl = 600 -- seconds, the idle retirement age

M.header_limit = "X-RateLimit-Limit"
M.header_remaining = "X-RateLimit-Remaining"
M.header_reset = "X-RateLimit-Reset"
M.header_retry_after = "Retry-After"
```

listing 214: the contract's two rules as one constant block, numbers and messages frozen by the vectors (source: lua/service/limit.lua, lines 11-24)

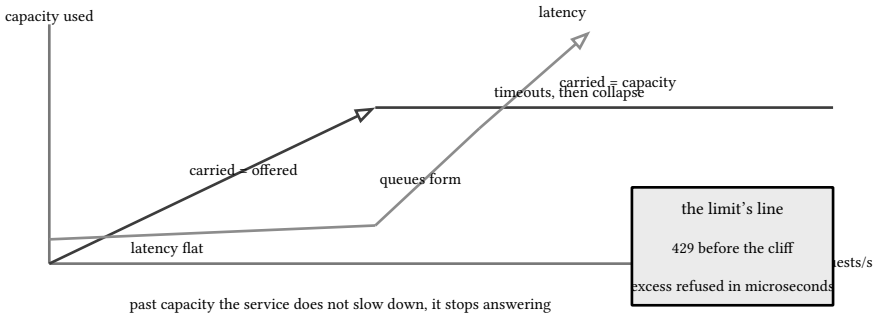


figure 147: offered load versus what the service can carry, the cliff between them

27.2. the token bucket from first principles

A token bucket holds a floating level, a capacity, and a refill rate. It starts full, every allowed request removes one token, and time adds tokens continuously at the configured rate until the capacity caps them, one `math.min` call. That is the entire algorithm, and its two knobs split the two jobs cleanly: capacity is the burst allowance, how much may arrive at once, and the refill rate is the sustained allowance, how much may arrive per second forever. The contract's login rule, capacity five at one per second, permits five requests in the first instant, a sixth at the one-second mark, and then one per second for as long as the client cares to wait.

The arithmetic of a denial is the part worth deriving rather than reciting. After the sixth same-second draw the level is exactly zero, so the next token exists one full second in the future, and that instant is both the `X-RateLimit-Reset` stamp and the `Retry-After` value. Half a second later the level is 0.5: still not spendable, and the next whole token is now only half a second away. A whole level waits a full period, a fraction waits its complement over the refill rate, and a denial never spends anything:

```
-- one draw at now. an allow costs exactly one token, a denial costs
-- nothing and reports the wait to the next whole token, which is both
-- the Reset stamp and the Retry-After the 429 carries: a whole
-- fraction of the level waits a full period, a fraction waits its
-- complement over the refill rate
function M.allow(b, now)
  refill(b, now)
  local d = { allowed = false, limit = b.capacity }
  if b.level ≥ 1 then
    b.level = b.level - 1
    d.allowed = true
  end
  d.remaining = math.floor(b.level)
  local frac = b.level - d.remaining
  local wait = (1 - frac) / b.per_second
  d.reset_at = now + wait
  if not d.allowed then
    d.retry_after = wait
  end
  return d
end
```

listing 215: allow: one draw, one decision, the wait computed from the fractional level (source: lua/service/limit.lua, lines 45-65)

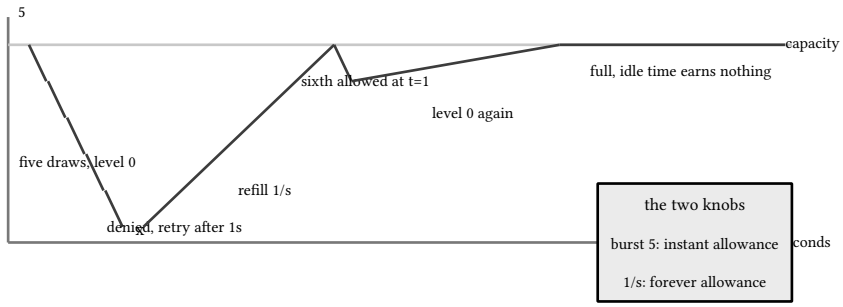


figure 148: bucket level over one draw timeline, the refill climbing between bursts

27.3. the keyed table and its sweep

The go lane builds the teaching bucket and then reads the production limiter out of a library. This lane has no second implementation to read: the first-principles bucket is the production gate, so the lesson moves to the table that keys it. One limiter owns a rules table keyed by route path and a buckets table keyed by limit key, each entry created on first use, and the entry is exactly the bucket of the previous section plus a last-touch stamp. The

constructor refuses to run without an injected clock, an `assert` in the constructor, so a wiring that reaches for `os.time` fails at startup instead of silently testing against the wall.

The sweep is amortized into the draw. Every `idle_ttl` of clock time, the pass a limited route already pays walks the table and retires keys nobody touched, which deletes the janitor entirely: no timer, no ticker, no janitor coroutine parked somewhere the load chapter has to find. A retired entry becomes garbage the collector reclaims on its own incremental schedule, the collector chapter's truth, where the go lane's sweeper frees memory the moment it deletes. The third admission mode the go lane taught, the reservation and its `Wait`, does not exist here on purpose: the vm is single threaded, and a wait would hold the only thread the service has, so an immediate refusal is the only honest answer a limit on this platform can give:

```
-- one decision for the key at the route. the sweep rides the same
-- pass: every idle_ttl of clock time the idle keys retire inside the
-- draw the route already pays. an unrouted path answers nil, meaning
-- not limited, and the layer passes it through untouched
function M.draw(l, path, key)
  local rule = l.rules[path]
  if not rule then
    return nil
  end
  local now = l.now()
  if now - l.last_sweep ≥ l.idle_ttl then
    M.sweep(l, now)
  end
  local b = l.buckets[key]
  if not b then
    b = M.bucket(rule.burst, rule.per_second, now)
    l.buckets[key] = b
  end
  b.last_touch = now
  local d = M.allow(b, now)
  d.message = rule.message
  return d
end
```

listing 216: draw: the sweep amortized into the pass, the bucket created on first use (source: lua/service/limit.lua, lines 157-179)

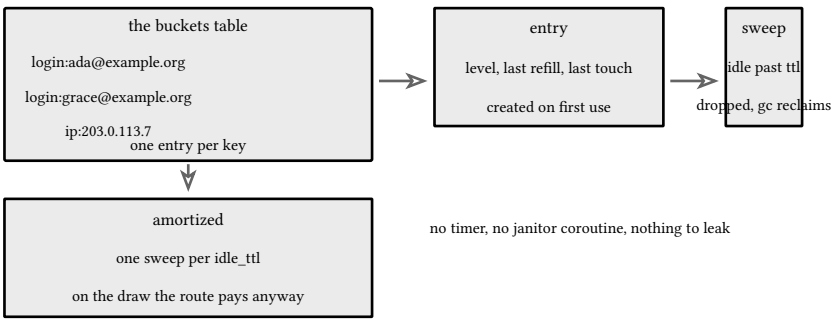


figure 149: key to bucket table, first use creates, idle age retires

27.4. the 429 contract

The wire shape is pinned by the contract and the frozen vector, and one function writes it. Every response from a limited route carries three headers: `X-RateLimit-Limit` is the burst capacity, `X-RateLimit-Remaining` is whole tokens left after this draw, and `X-RateLimit-Reset` is the unix second at which the next token arrives, floored. A denial adds `Retry-After` in seconds, the ceiling of the wait, and answers the standard envelope with code `rate_limited`, never a bare status or a bespoke body.

The layer owns every draw. No handler draws a bucket, no handler stamps a rate header, and the login key normalizes the email exactly once inside the key function. The layer draws before it calls the handler, stamps the three headers on the response the handler returns, and on denial never calls the handler at all, answering through the injected failure writer so the request id in the envelope is the same id the access log records. One boundary is worth stating where the go lane has none: a handler that raises produces no response value at this layer at all, the error travels up and the kernel's recover answers above, so that answer carries no rate headers, where the go writer sets its headers before the handler runs and they ride every later status. The refusal itself still carries the headers, because a denial is answered here, inside the layer. One replay divergence is ruled and stated: the frozen register vectors answer without rate headers, so the replay attaches the login rule only, while the production composition attaches both rules and stamps every limited route:

```
-- the three headers every limited route carries, allowed or denied,
-- plus the 429-only Retry-After: whole tokens, unix seconds floored,
-- wait ceil seconds. appended as ordered pairs, the envelope's header
-- convention, so they survive the render a fail value takes above
-- this layer and ride the response a handler returns below it
function M.stamp(res, d)
  local h = res.headers or {}
  res.headers = h
  h[#h + 1] = M.header_limit
  h[#h + 1] = tostring(d.limit)
  h[#h + 1] = M.header_remaining
  h[#h + 1] = tostring(d.remaining)
  h[#h + 1] = M.header_reset
  h[#h + 1] = tostring(math.floor(d.reset_at))
  if not d.allowed then
    h[#h + 1] = M.header_retry_after
    h[#h + 1] = tostring(math.ceil(d.retry_after))
  end
  return res
end

-- header lookup over the ordered array, the convention the envelope
```

listing 217: stamp: three headers appended in order, retry-after ceil on denial (source: lua/service/limit.lua, lines 181-202)

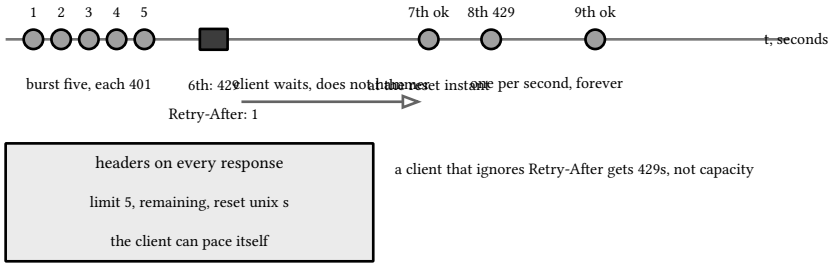


figure 150: a client that honors Retry-After: five through, one refused, the next at the reset instant

27.5. what to key on

A limit is only as good as its key, and the two routes key on different things for opposite reasons. Login keys on the lower-cased email: the attack the rule exists for is guessing one account's password, so the bucket belongs to the account, and the fold happens in exactly one place, the key function, so both casings of one address are one bucket and no other layer re-normalizes. A body without a usable email lands in one shared `login:unparsed` bucket, because garbage must not orbit the limit by being invalid.

Register keys on the client ip, and here the platform tells the truth about provenance: the c host's seam answers the connection's peer through one call, host only, no port, and the kernel stamps it on the request once, so the key

function splits nothing it does not have to. Proxy headers stay unread on purpose: `X-Forwarded-For` is a header any client can write, so a limiter that keys on it verbatim sells a fresh bucket of ten registrations per forged address, one per request, forever. When no peer exists, the internal listener and the in-process tests, the key falls to one shared `ip:unknown` bucket, fail closed: unknown callers share capacity rather than each minting their own:

```
-- stripped the way a host split does, a bracketed ipv6 literal keeping
-- its brackets off. the seam stamps the peer (the c side's
-- getpeername); an internal or in-process caller carries none and
-- lands in the one unknown bucket, the stated fail-closed boundary.
-- proxy headers are client written until a trusted proxy contract
-- strips them, so they are unread here by design
function M.key_peer(remote)
  if type(remote) ~= "string" or remote == "" then
    return "ip:unknown"
  end
  local host = remote
  if host:sub(1, 1) == "[" then
    local close = host:find("]", 1, true)
    if close then
      host = host:sub(2, close - 1)
    end
  else
    local cut = host:reverse():find(":", 1, true)
    if cut then
      host = host:sub(1, #host - cut)
    end
  end
  return "ip:" .. host
end
```

listing 218: key_peer: port split, bracketed ipv6 kept bare, fail closed on no peer (source: lua/service/limit.lua, lines 79-102)

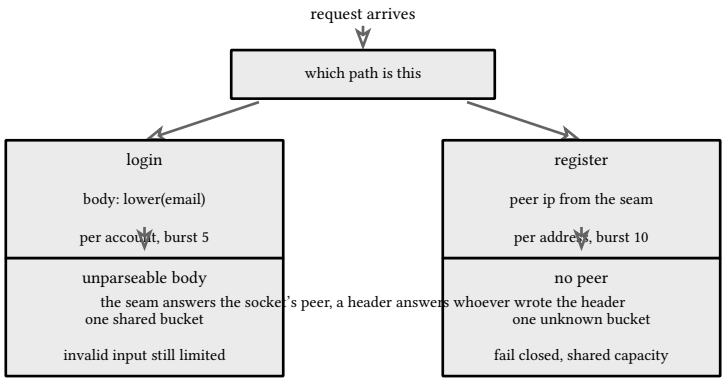


figure 151: key choice: the route decides what the bucket belongs to

27.6. limits under test

Every time-dependent behavior in this chapter is arithmetic on a fixed clock, because a rate limiter tested against the wall clock is tested against a scheduler. The test list walks the bucket through its whole grammar with literal numbers: the burst drain reporting remaining 4 through 0, the sixth same-instant draw denied with the exact one-second wait, the half-second refill raising the level to an unspendable 0.5 and shrinking the wait to 0.5, the fractional level paying its complement, the idle century capping at capacity, and the denial that spends nothing. The frozen sixth-login vector pins the arithmetic to the contract's bytes: the clock at `t0` plus twelve, the reset stamp 1790830813 which is exactly `t0` plus thirteen, `Retry-After` 1, and the message the vector froze. The byte-exact replay runs in the resource family's vector suite, five setup draws and the frozen sixth through the composed stack

with status, headers, and envelope asserted. No test sleeps, and the module reads no clock at all, the constructor refuses one:

```
M[#M + 1] = {
  name = "the frozen sixth login's arithmetic, vector 06's numbers",
  fn = function()
    -- t0 = 1790830800, the vector's clock sits at t0 plus 12 seconds
    local t = 1790830800 + 12
    local b = M.bucket(M.login_burst, M.refill_per_second, t)
    for _ = 1, 5 do
      local d = M.allow(b, t)
      T.ok(d.allowed)
    end
    local d = M.allow(b, t)
    T.ok(not d.allowed)
    T.eq(math.floor(d.reset_at), 1790830813, "the vector's Reset byte, t0 plus 13")
    T.eq(math.ceil(d.retry_after), 1, "the vector's Retry-After byte")
    T.eq(d.remaining, 0)
    T.eq(d.limit, 5)
  end,
```

listing 219: the vector's numbers as a fixed-clock test, t0 plus 12, reset t0 plus 13 (source: lua/service/limit.lua, lines 325-341)

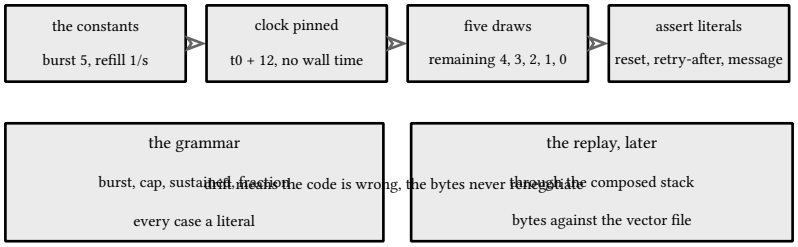


figure 152: the fixed-clock pipeline: constants in, draws through the bucket, literals out

sources: lua.org manual 5.5 sections 6.2 (basic functions, the pairs walk and tonumber used by the table and its tests), 6.5 (string manipulation, the find and sub of the key split), and 6.8 (mathematical functions, min, floor, ceil), RFC 6585 section 4 for status 429 and RFC 9110 section 10.2.3 for the Retry-After field, all accessed 2026-09-27. Verified by the service plain lane under the pinned lua 5.5 with this module's 14 tests among the suite's green run, the frozen vector's numbers asserted from `contract/testdata/06-login-429.json`, and the byte-exact composed replay green in the resource family's vector suite.

28. observability

A service no one can see is a service no one can fix, and a service that logs prose no machine can query is seen only by whoever is reading along live. This chapter builds the service's observability in `service/obs.lua`: one kv line assembler for every line the ops family writes beside the kernel's json request log, a closed redaction set, the W3C traceparent header parsed and formatted from scratch with spans held as a tree, the fixed-table metrics registry rendering the prometheus exposition the frozen vector pins.

28.1. the access line

One function renders every line the ops family writes, so the field order exists exactly once: timestamp, level, message, request id, method, path, status, duration in milliseconds, remote. The request-level access log is the middleware chain's lesson, a json line its own layer writes. The two coexist by role: the chain's line feeds the deployment, the kv line renders the family's own lines and any access line wired to its sink. Each field goes through one kv renderer with a mechanical quoting rule: a value containing a space, a quote, or an equals sign is double quoted with backslash and quote escaped, and every other value rides bare. A lone backslash rides bare: quoting protects the separators.

Two defaults carry real meaning: an absent id renders as a dash, an absent peer renders `remote=-` and the internal listener stamps no peer, and a peer that exists comes from the seam, host only, one stamp:

```
-- THE access-line assembler: every access line in the service renders
-- through this one function, so the field order exists once. an
-- absent peer renders remote=- : internal listeners stamp no peer,
-- and the in-process composition carries none
function M.access_line(a)
  local parts = {
    ("ts=%d"):format(a.ts),
    "level=" .. a.level,
    M.kv("msg", a.msg),
    M.kv("request_id", a.request_id or "-"),
    M.kv("method", a.method),
    M.kv("path", a.path),
    ("status=%d"):format(a.status),
    ("duration_ms=%d"):format(a.duration_ms),
    M.kv("remote", a.remote or "-"),
  }
  return table.concat(parts, " ") .. "\n"
end
```

listing 220: the one access-line assembler, field order fixed by construction (source: lua/service/obs.lua, lines 49-66)

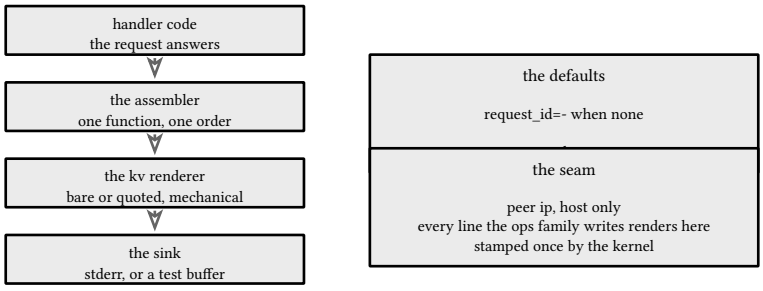


figure 153: the line's own stack: call site through the kv renderer to the sink

28.2. structured beats interpolated

An interpolated message packs data into prose, and prose is lossy: `login failed for ada, attempt 3` cannot be grouped by email, counted by attempt, or joined to its request id without a pattern match that breaks the day the message rewords itself. The structured line carries the same information as named fields that keep their names and their places in an index. The rule is the one the kv line already follows: the message names the event, the fields carry the data, and no handler interpolates a value it could pass as a field.

Event lines and crash lines render through the same attribute renderer, and the attribute list is an array of pairs rather than a hash, because order is data. The request id rides along when there is one, which is the whole correlation story, one id joining the log line, any envelope the request produced, and the span tree the next section builds:

```
-- the attribute renderer: event lines and crash lines through the
-- same kv renderer, masked keys answered with [redacted], the request
-- id carried when there is one, attrs in call order as {k, v} pairs
-- because order is data
function M.attr_line(a)
  local parts = {
    ("ts=%d"):format(a.ts),
    "level=" .. a.level,
    M.kv("msg", a.msg),
  }
  if a.request_id then
    parts[#parts + 1] = M.kv("request_id", a.request_id)
  end
  if a.attrs then
    for _, kv in ipairs(a.attrs) do
      local v = M.masked(kv[1]) and "[redacted]" or kv[2]
      parts[#parts + 1] = M.kv(kv[1], v)
    end
  end
  return table.concat(parts, " ") .. "\n"
end
```

listing 221: the attribute renderer: pairs in call order, the id carried when present (source: lua/service/obs.lua, lines 68-88)

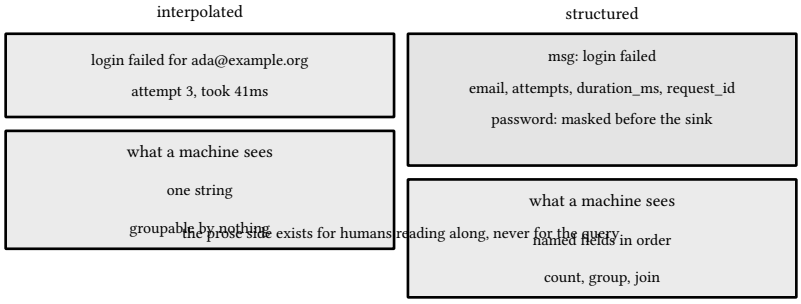


figure 154: the same failure, line-oriented and structured

28.3. redaction as a closed set

Secrets ride in values, so the mask is by key. The redaction set is closed, six keys, folded to lower case on comparison so a handler that logs `Password` or `Set-Cookie` is caught just the same: password, authorization, cookie, set-cookie, secret, token. A member of the set has its value replaced with `[redacted]` before the renderer ever sees it, and a key that is merely similar passes through, because the policy is membership, not similarity. A handler that logs its login body verbatim ships the password:

```lua
-- the closed redaction set, folded to lower case on comparison.
-- secrets ride in values, so the mask is by key: a login handler that
-- logs its password field verbatim ships the secret, and grep never
-- forgets
local masked_keys = {
  password = true,
  authorization = true,
  cookie = true,
  ["set-cookie"] = true,
  secret = true,
  token = true,
}

function M.masked(key)
  if type(key) ~= "string" then
    return false
  end
  return masked_keys[key:lower()] == true
end
```

listing 222: the closed set folded to lower case, the mask applied by key (source: lua/service/obs.lua, lines 29-47)

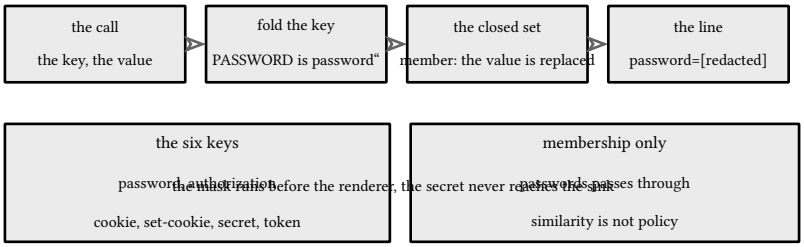


figure 155: one field's trip through the mask

28.4. traceparent from scratch

A trace is the shape of one request's time, joined across services by one shared header. The W3C trace context format is deliberately small, four lowercase hex fields, `version-traceid-spanid-flags`, 2, 32, 16, and 2 digits, and small enough to parse and format by hand, which is what this module does: no sdk, no collector, no span leaves the process. The parser is strict where the spec allows a choice: exact field lengths, version `ff` reserved and refused, all-zero trace or span ids refused, uppercase hex refused because the lowercase rendering is the canonical one, and only the sampled bit of the flags byte kept. The spec's processing model says the rest: an invalid header means the vendor creates a new traceparent, and unknown flag bits are set to 0 on outgoing requests, which is exactly what this module does.

The server side is one function. A good header continues the inbound trace: the same trace id, a fresh span id of our own, the parent set to the remote span, the sampling bit inherited. An absent or malformed header mints a fresh trace and reports that it did, because one bad header must never kill a request. Children opened beneath a span inherit its trace id and its sampling bit, which is the propagation the format exists for, and the tree renders one line per span with two-space indent. Every id comes from the injected source and every instant from the injected clock, so a test pins the rendered tree byte for byte:

```lua
-- the server side of propagation: a good header continues the inbound
-- trace, same trace id, a fresh span of our own, the parent the
-- remote span, the sampling bit inherited. an absent or malformed
-- header mints a fresh trace and reports it: one bad header never
-- kills the request
function M.continue(header, idsrc)
  local trace, parent, sampled = M.parse_traceparent(header)
  if trace == nil then
    return M.mint(idsrc)
  end
  local fresh = M.mint(idsrc)
  fresh.trace_id = trace
  fresh.parent_id = parent
  fresh.sampled = sampled
  fresh.remote = true
  return fresh
end
```

listing 223: continue: a good header joins the inbound trace, anything else mints fresh (source: lua/service/obs.lua, lines 163-179)

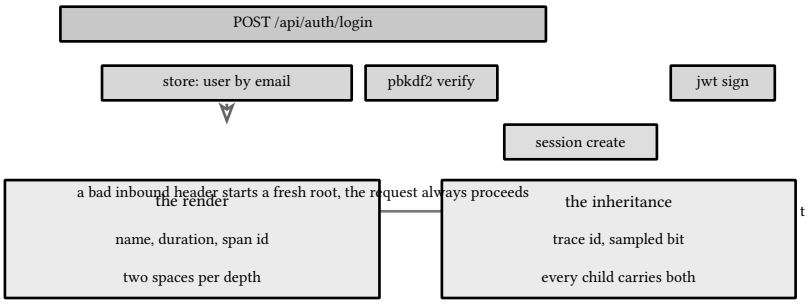


figure 156: one login request's spans over time, the tree the render prints

28.5. the fixed-table metrics registry

Two series carry the load story, and both live in one registry of fixed tables, the c lane's shape carried into lua. The counter is one series per route and status code, `goapi_requests_total{route,code}`, incremented once per request. The histogram is one per route, `goapi_request_duration_seconds`, with the contract's eleven finite bounds tight at the low end where service latencies live and geometric through the tail plus the implicit `+Inf`. An observation increments every bound the seconds fit under, which is what makes the counts cumulative, plus `+Inf`, the sum, and the count.

Two policies differ from the threaded lanes. The tables are bounded: sixty-four counter series and thirty-two routes by default, a full table answers nothing new, the bounded policy stated where the go lane's map grows and the c lane states the same bound, because the lua alternative is an unbounded table the composition root keeps alive forever, which is exactly the leak the load chapter counts. And there is no lock anywhere: the c lane guards the same registry with a process-wide srw lock because its worker threads draw concurrently, while this vm is single threaded, one request at a time by construction, so the registry needs no guard at all, the host's serialization doing the work, a stated truth and not a free lunch:

```lua
-- one counted request at the route and status: find-and-bump, a miss
-- appends when a slot is free
function M.count(m, route, code)
  local k = route .. "\0" .. code
  local i = m.counter_ix[k]
  if i then
    m.counters[i].count = m.counters[i].count + 1
    return
  end
  if m.counter_n ≥ m.series_max then
    return -- the bounded policy, a full table drops the increment
  end
  m.counter_n = m.counter_n + 1
  m.counters[m.counter_n] = { route = route, code = code, count = 1 }
  m.counter_ix[k] = m.counter_n
end
```

listing 224: count: find-and-bump through the index, the bounded policy on the miss (source: lua/service/obs.lua, lines 279-294)

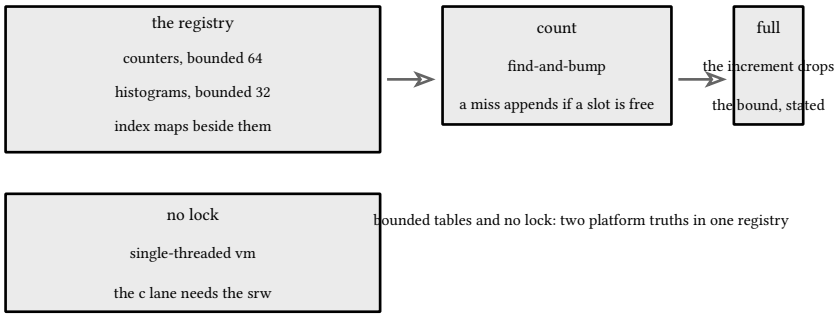


figure 157: the registry: fixed tables, one index, no lock

28.6. the exposition, byte for byte

The exposition writer renders the prometheus text format directly: help and type lines per family, the counter family first, series sorted by label tuple within a family, bucket lines ascending with `+Inf` last, and every number in the shortest fixed decimal that rounds back to the same value, so 3 renders 3, 0.5 renders 0.5, and 0.0025 renders 0.0025. Zero buckets are elided, the one deliberate divergence from client_golang, which prints every finite bucket at zero as well. The format's own requirements are fewer than a reader might fear: the +Inf bucket must exist and equal the count, buckets must ascend by their `le` label, and each family's lines must ride together under their help and type lines. This writer does all three and freezes the rest by test. One slow `/healthz` observation, past every finite bound, renders exactly the eight lines the contract froze: the counter family, then the histogram with only the `+Inf` bucket, the sum, and the count. The vector carries `<observed>` where the sum sits, and the test substitutes its own observation before the byte comparison, pinning both the fixed lines and the sum's rendering:

```
# HELP goapi_requests_total requests by route and code.
# TYPE goapi_requests_total counter
goapi_requests_total{route="/healthz",code="200"} 1
# HELP goapi_request_duration_seconds request latency.
# TYPE goapi_request_duration_seconds histogram
goapi_request_duration_seconds_bucket{route="/healthz",le="+Inf"} 1
goapi_request_duration_seconds_sum{route="/healthz"} <observed>
goapi_request_duration_seconds_count{route="/healthz"} 1
```

listing 225: the frozen vector the exposition renders against, byte for byte (source: lua/service/contract/testdata/15-metrics.txt, lines 1-8)

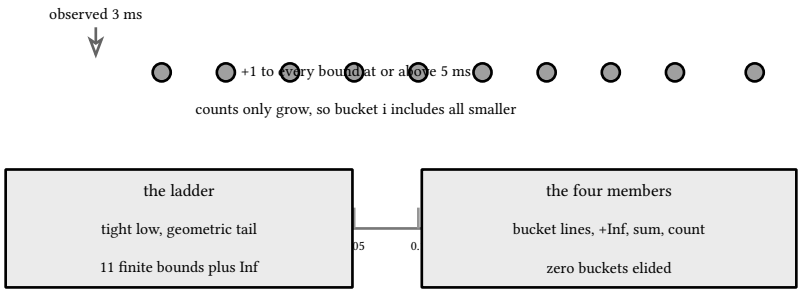


figure 158: cumulative buckets: each observation fills every bound above it

28.7. one request fully observed

Wire the signals around one request and a login tells the whole story. The metrics layer wraps the stack one layer out, so the counter ticks at the status the inner response really carries and the histogram at the duration two clock reads measured around it, and the registry sits behind this layer, so a 429 from the limit layer inside counts like any other status. The route label comes from the router's matched pattern, never the raw path, because a raw path would mint one series per concrete user id and fill the bounded table with uuids nobody graphs, and an unmatched path carries the empty label, the router's own answer, never a standing literal for it. The chain's json log line and the span open at the same request, the id joins them, and the label agrees with the line's path because both read the same match.

Reading it back is what the tests do, separately and without ceremony: the assembler test pins the whole line's bytes, the redaction test pins the mask, the parser test walks every refusal, the exposition test renders the frozen vector, and the layer test drives the layer over an injected label function and asserts the series that come out. The composed walkthrough drives one request through the same layer order the root splices, the id on the line and the response, both series by the matched pattern, the scrape adding its own series for the next scrape to see, the denial counted:

```
-- the metrics layer: one count and one observation per request, the
-- label from the router's matched pattern (deps.route_label(req)),
-- never the raw path, so a concrete id mints no series. the registry
-- lives behind this layer, so denials count too. deps = { now,
-- route_label }
function M.metrics_layer(m, deps)
  return function(req, next)
    local t0 = deps.now()
    local res = next(req)
    local t1 = deps.now()
    local route = deps.route_label(req)
    if route == nil then
      route = "" -- unmatched: the oracle's answer, never its own literal
    end
    M.count(m, route, res.status)
    M.observe(m, route, t1 - t0)
    return res
  end
end
```

listing 226: the metrics layer: matched pattern label, status captured, both series ticked (source: lua/service/obs.lua, lines 398-416)

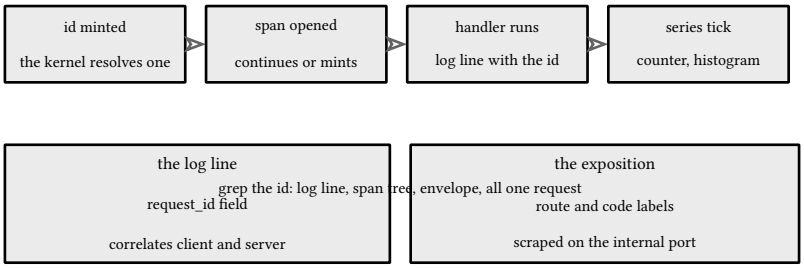


figure 159: one request id through every signal

sources: the W3C trace context level 1 recommendation at w3c.github.io/trace-context, sections 3.2.2.2 through 3.2.2.5 for the traceparent field grammar and sections 4.2 and 4.3 for the processing model whose invalid-header and zeroed-flags rules this chapter quotes, the prometheus text exposition format documentation at prometheus.io, and lua.org manual 5.5 sections 6.5 (string manipulation), 6.7 (table manipulation, concat), 6.2 (tonumber with a base), and 3.4.2 (bitwise operators, the sampled-bit mask), all accessed 2026-09-27, with client_golang's zero-bucket behavior as the go chapter verified it 2026-09-25. Verified by the service plain lane under the pinned lua 5.5, `run.lua` green with this module's 14 among them (the whole-lane census chapter 31 states once), the exposition replayed byte for byte from `contract/testdata/15-metrics.txt`.

29. load and p99

A service that has never been loaded is a service whose p99 is a guess, and this platform says so twice: the vm is single threaded, so concurrency is a script, and the clocks are a millisecond cpu tick and whole-second wall time, so a sub-second latency is not a number this book can honestly quote. This chapter closes the ops family with what is checkable anyway: nearest-rank percentiles and the histogram walk that answers bounds, the closed-loop harness over the real host wire with every seam injected, the committed capture dated and toolchain-labeled, and the leak discipline for a garbage collected language, where a leak is a table the composition root keeps alive and counting is finding.

29.1. what load means here

The c seam owns the only server thread, and that thread accepts one connection and serves it to completion before the next. Offered load beyond that one request at a time does not pile up inside the service, it piles up in the winsock accept backlog, and the queue the load chapter usually draws inside the server is drawn here at the socket layer the kernel does not own. Little's law still holds at the wire, `L = lambda W`, with in-flight capped at one in service plus whatever the backlog holds, which is why the honest load question on this platform is throughput and cost per request rather than a concurrency sweep.

The harness measures exactly that. Every worker fires once per round before any answer is awaited, so each round deposits the worker count into the backlog and drains it in accept order, warmup runs first as round zero and lands nowhere, and the sink's arrival order is the scripted worker order, deterministic by construction. The seams are three injected functions, open, fire, and await, which is what lets the unit tests script the whole interleave over recording fakes while the real driver rides actual sockets:

```
function M.closed_loop(opts)
  local workers, per_worker, warmup =
    opts.workers or 1, opts.per_worker or 1, opts.warmup or 0
  local hs = {}
  for w = 1, workers do hs[w] = opts.on_open(w) end
  local sink = { latencies = {}, statuses = {}, n = 0 }
  -- one round: every worker fires once before any answer is awaited,
  -- warmup is round zero, unrecorded, landing nowhere
  local function round(i, record)
    for w = 1, workers do opts.on_fire(hs[w], i) end
    for w = 1, workers do
      local status, secs = opts.on_await(hs[w])
      if record then
        sink.n = sink.n + 1
        sink.latencies[sink.n] = secs
        sink.statuses[sink.n] = status
      end
    end
  end
  for _ = 1, warmup do round(0, false) end
  for i = 1, per_worker do round(i, true) end
  return sink
end
```

listing 227: the closed loop: round-major fire then await, warmup round zero unrecorded (source: lua/service/load.lua, lines 67-89)

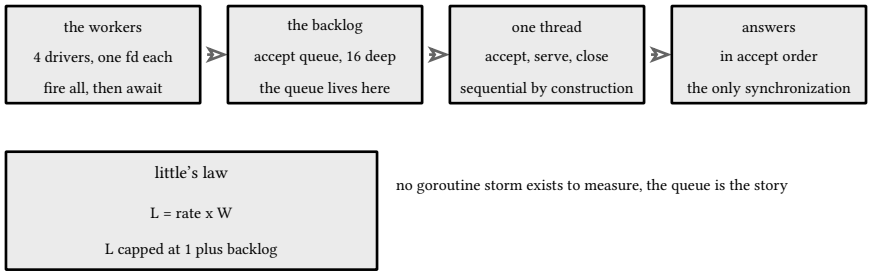


figure 160: the wire under load: workers, backlog, one server thread

29.2. nearest rank and the bucket walk

The percentile definition is nearest rank, the one the profiling chapter pinned: sort the latencies, take the value at `ceil(p/100 * n)`. No interpolation anywhere, p99 of one hundred values is the ninety-ninth smallest, not a blend of the ninety-ninth and hundredth, and a rank that runs past the end clamps to the largest, because a percentile past the data is a bound, not a measurement. The histogram walk answers from what a scraper can see, the cumulative buckets: the smallest finite bound whose cumulative count reaches the rank, which is the bound itself, never a value inside the bucket, and therefore an upper bound on the true percentile that says so. When the rank falls into the `+Inf` bucket the walk answers nil, meaning the true value exceeds the largest finite bound and only worse than that is knowable.

The fixed pair pins the contrast. Six latencies of 1, 1, 2, 3, 5, and 8 milliseconds over millisecond bounds of 1, 2.5, 5, and 10 give an exact p50 of 2 and p99 of 8, while the walk answers 2.5 and 10, the bounds the ranks land in, the resolution a histogram throws away:

```lua
-- nearest rank: sort the samples, take the value at
-- ceil(p/100 * n). no interpolation anywhere: p99 of one hundred
-- values is the ninety-ninth smallest, not a blend of two neighbors,
-- and a rank past the end clamps to the largest, because a percentile
-- past the data is a bound, not a measurement
function M.nearest_rank(sorted, p)
  local n = #sorted
  if n == 0 then
    return nil
  end
  local rank = math.ceil(p / 100 * n)
  if rank < 1 then rank = 1 end
  if rank > n then rank = n end
  return sorted[rank]
end
```

listing 228: nearest rank: the rank ceiled, the ends clamped, nothing interpolated (source: lua/service/load.lua, lines 14-28)

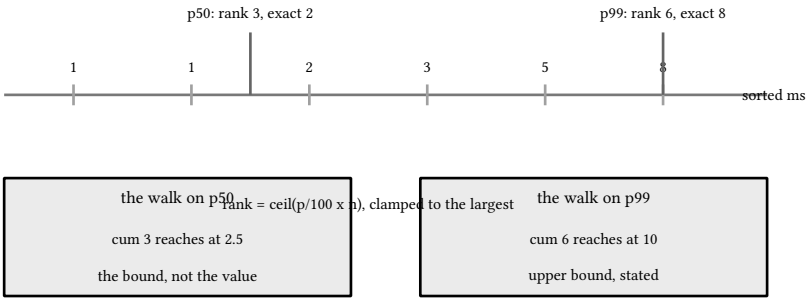


figure 161: the sorted ladder, the exact answer and the bucket's bound beside it

29.3. the harness over the real wire

The driver lives at `tests/load/run.lua`, deliberately outside `tests/drivers/`, because the verify lane runs every driver it finds and a benchmark in verify is a flaky test. It is invoked the way verify builds the host, the documented gcc line from the service root with the mingw `lua55.dll` on the path, and it runs only when a capture is being made.

The model is connection per request, and the serialized accept loop is why: a second kept-alive connection would sit unaccepted while the first is open, so each fire connects and sends, each await reads to the connection end the server's close terminates, and the backlog is the queue where the round's concurrency waits. The fd discipline is the seam-level leak check, every open counted against every close, and the run ends with a scrape of the exposition the load itself produced. The composition this drives is the root's real chain, with one adapter worth its sentence: the ops layers speak `(req, next)` while the kernel chain folds next-handler wrappers, and one three-line adapter in the root states that bridge once, so the same layers serve the plain lane's in-process tests and the wire:

```
local sink = load.closed_loop({
  workers = 4,
  per_worker = 500,
  warmup = 1,
  on_open = function(w)
    return w
  end,
  on_fire = function(w)
    local fd = net.connect(host, port)
    assert(fd, "worker " .. w .. " could not connect")
    opened = opened + 1
    assert(net.send(fd, REQ) == #REQ, "worker " .. w .. " send")
    fds[w] = fd
  end,
  on_await = function(w)
    local resp = read_all(fds[w])
    net.close(fds[w])
    fds[w] = nil
    closed = closed + 1
    local status = resp:match("^HTTP/1%.1 (%d+)")
    assert(status, "worker " .. w .. " got no status line")
    return tonumber(status), 0
  end,
})
```

listing 229: the loop wired to the sockets, one connection per request, fds counted (source: lua/service/tests/load/run.lua, lines 43-66)

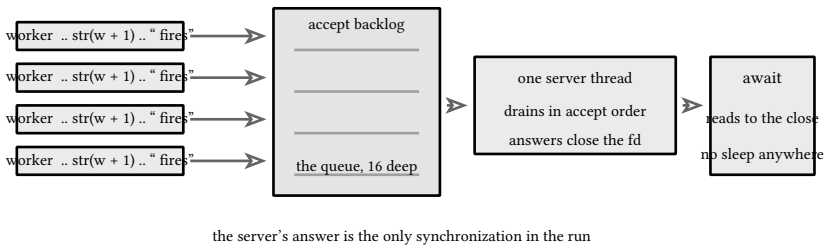


figure 162: one round of the driver: four connects, the backlog, answers in order

29.4. the report and its two sources

One run folds into one row with both percentile sources side by side: the p columns from the raw samples through nearest rank, the b columns from the histogram the registry kept through the walk, unknown where the walk refuses, and the units in the header, because a latency column that does not say its unit is an error waiting for a reader. The golden test pins the bytes with the fixed six-sample set, exact 2 and 8 beside bounded 2.5 and 10, and a

second row whose every sample exceeds the largest finite bound, all three b columns reading unknown, the not-knowable the +Inf bucket honestly is:

```
-- the report: one row per run, both percentile sources side by side,
-- the p columns from the raw samples, the b columns from the
-- histogram the registry kept, unknown where the rank falls into
-- +Inf, units in the header because a latency column that does not
-- say its unit is an error waiting for a reader
function M.report(rows)
  local out = { ("%-12s %3s %7s %7s %7s %8s %8s %8s  %s"):format(
    "run", "n", "p50_ms", "p99_ms", "p999_ms",
    "b50_ms", "b99_ms", "b999_ms", "source") }
  for _, r in ipairs(rows) do
    local s = M.summary(r.samples)
    out[#out + 1] = ("%-12s %3d %7s %7s %7s %8s %8s %8s  %s"):format(
      r.label, s.n,
      F.fmt_number(s.p50), F.fmt_number(s.p99), F.fmt_number(s.p999),
      cell(M.from_cumulative(r.cum, r.bounds, 50)),
      cell(M.from_cumulative(r.cum, r.bounds, 99)),
      cell(M.from_cumulative(r.cum, r.bounds, 99.9)),
      "exact and histogram bound")
  end
  return table.concat(out, "\n") .. "\n"
end
```

listing 230: the report row: exact and bounded percentiles, units in the header, unknown where the walk refuses (source: lua/service/load.lua, lines 112-132)

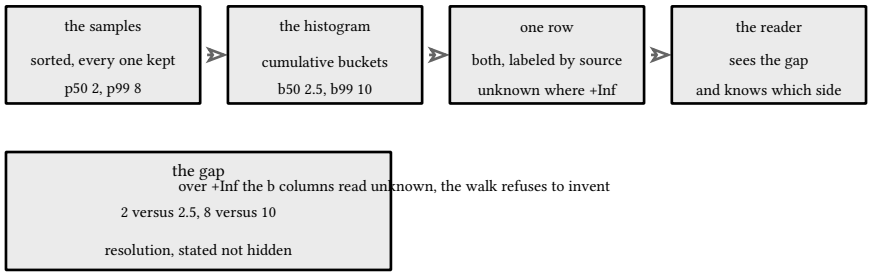


figure 163: one row, two sources, the gap between them is the histogram's resolution

29.5. committed captures

The capture discipline is the profiling chapter's: measure once, commit the output, quote the table, never re-measure in verify. The committed capture carries its own provenance, date and machine, toolchain, the composition level it measured, the model, the clock truths, and the invocation, so a reader knows exactly what ran. The 2026-09-28 capture quotes this run: 2000 measured requests at 4 workers over 20 wall seconds, 100.0 requests per second, process cpu of 10.220 milliseconds per request counting both threads, every request answered 200, every fd closed, and the scrape at the end holding 2004 observations of `/healthz`, warmup included, because the registry counts every request the server served while the driver's sink holds only the measured ones.

The scrape also demonstrates the clock truth better than any argument: every observation sits in the 0.001 bucket and the sum reads 0, because the service clock is `os.time`, whole seconds, and a sub-millisecond request reads as zero elapsed. The histogram is honest about what it saw. A latency distribution this platform cannot resolve is not invented, it is measured coarsely and labeled with the resolution it was measured at:

```
load run: workers 4, per_worker 500, warmup 4
ok 1 measured requests, warmup landed nowhere
ok 2 every request answered 200
ok 3 4 warmup plus 2000 measured connections plus the scrape
ok 4 every fd closed, the scrape's after its read
wall 20 s, 100.0 req/s
process cpu 20.439 s over 2000 requests, 10.220 ms/request, both threads
```

listing 231: the capture's run summary, 2026-09-28, quoted verbatim (source: lua/service/captures/2026-09-28-load.txt, lines 20-26)

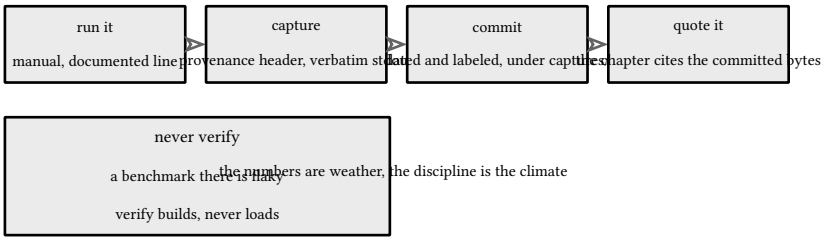


figure 164: the capture pipeline: run, capture, commit, quote, never verify

29.6. the lua leak, found by counting

A leak in this language is not a lost pointer, it is a table the composition root still reaches after the work it served is done, and that means counting is finding. The root's registries answer counts: the limiter's bucket table after a sweep, the metrics registry's series rows, both bounded by policy, so the assertion is steady state, run again, count again, same number. The collector's own figure, live kilobytes under two full collects, is the floor under the counting, two collects because the collector is incremental and one pass can leave the finalizers' garbage for the next. At the seam the same discipline counts file descriptors, every open against every close, and the load driver asserts it at the end of every run.

The unit test demonstrates the failure mode on purpose: a session table whose entries retire returns to zero, a table nothing ever clears holds one hundred, and the difference is the leak, counted. The sweep from the rate limiting chapter is the production instance of the fix, idle buckets retire, the collector reclaims them, and the root's count holds steady no matter how many distinct keys the day brings:

```
-- how many entries one root-held table keeps: the counter the leak
-- discipline asserts with. a lua leak is not a lost pointer, it is a
-- table the composition root still reaches, so counting is finding
function M.root_count(root)
  local n = 0
  for _ in pairs(root) do n = n + 1 end
  return n
end

-- live kilobytes under a full collect: the floor under the counting.
-- two collects because lua's collector is incremental and one pass
-- can leave the finalizers' garbage for the next
function M.gc_kb()
  collectgarbage("collect")
  collectgarbage("collect")
  return collectgarbage("count")
end
```

listing 232: root_count and the full-collect floor, the leak found by counting (source: lua/service/load.lua, lines 134-150)

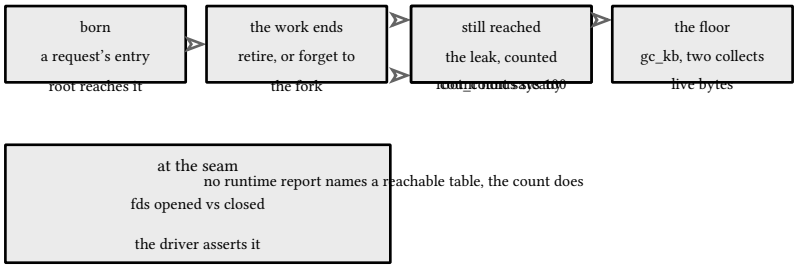


figure 165: a table's life under the root: born, reached, retired or counted

sources: lua.org manual 5.5 sections 6.2 (basic functions, collectgarbage), 6.7 (table manipulation, sort over the samples), and 6.8 (mathematical functions, ceil over the rank), the little's law statement as used across the corpus, and the clock facts from this book's profiling chapter, measured there. Verified by the service plain lane under the pinned lua 5.5, `run.lua` green with this module's 7 tests and the family's 3 composed walkthrough tests among the total, the load driver green over the real wire with 4 checks, and the capture committed at `captures/2026-09-28-load.txt`, measured on this machine, a 12th gen i7-12700F under windows/amd64 with gcc 15.2.0, accessed 2026-09-28, never re-measured by verify.

30. shipping

The service exists as a directory of green tests until it also ships. This chapter turns it into an artifact and a lane: the store story, the wiring the ship marker owns, the secret source, the drain truth, an image that builds its own interpreter because no registry carries lua 5.5, one compose command, and a make gate that replays the contract over the real listener.

30.1. what shipping means here

Two facts decide this chapter's shape. First, no registry image carries lua 5.5, so an honest image compiles the interpreter from the lua.org tarball inside the build. Second, the wire is not lua's to own on any platform: the winsock host owns the windows edge, and a plain vm has no socket at all. The container answer is a posix twin of that host, the same six net functions and the same `handle(fd)` contract ported to berkeley sockets, and the discipline that keeps the port small is that the seam, not either host, is the contract:

```
static void api_register(lua_State *L, int with_connect) {
  static const luaL_Reg every[] = {
      {"recv", net_recv},
      {"send", net_send},
      {"close", net_close},
      {"addr", net_addr},
      {"peer", net_peer},
      {NULL, NULL},
  };
  static const luaL_Reg driver_only[] = {
      {"connect", net_connect},
      {NULL, NULL},
  };
  luaL_newlib(L, every);
  if (with_connect) {
    luaL_setfuncs(L, driver_only, 0);
  }
  lua_setglobal(L, "net");
}
```

listing 233: the seam restated on posix, the same table the winsock host registers (source: lua/service/docker/posix_host.c, lines 164-182)

One divergence earns its comment at the send function: windows has no sigpipe, a dead peer is just a send error, while linux raises a signal that kills by default, and `MSG_NOSIGNAL` keeps the platforms alike. The twin folds the windows host's two files into one because the container needs no second lua state: serve mode is pid 1 with one accept loop, and client mode runs one driver script with `connect` and no listener, how the probe and the replay leg talk to a running stack with no curl in the image.

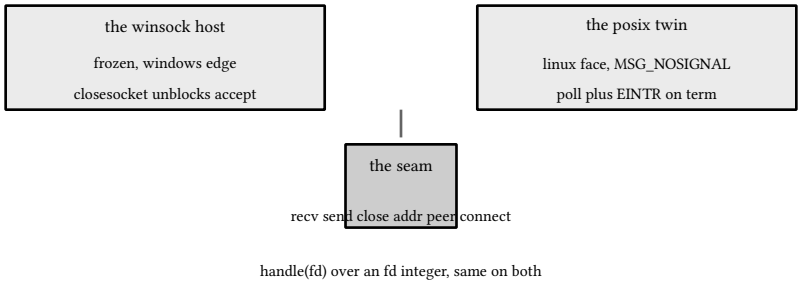


figure 166: two hosts around one seam, the service module never learns which answered

30.2. the oracle learns the report seams

The reports routes waited behind one honest gap: the handlers take `report_create`, `report_load`, and `report_compute` injected, the jit engine carried them, and the oracle that backs the service in this vm did not. The store chapter's ruling settles it: the oracle is the mirror the jit store must match row for row, so the seams belong on it and every lane serves the same surface, events and delete cascade included.

The series is the chapter's one derivation, the engine computes it in one committed query with window functions and the oracle walks tables to the same rows: presence collapses to one member per user per day, the first day inside the window marks the newcomer, and delta carries yesterday's count with today's as the lag default so the first day reads zero. The window bound is the subtle part and the test pins it: narrow the window to one day and both users count as newcomers there, because first day is a property of the window, not of history, exactly the bound the sql presence filter draws. Leaders rank with ties sharing a rank and the user id breaking the order:

```
local walk = {}
for day in pairs(per_day) do walk[#walk + 1] = day end
table.sort(walk)
local rows, prev = {}, nil
for _, day in ipairs(walk) do
  local active, newcomers = 0, 0
  for user_id in pairs(per_day[day]) do
    active = active + 1
    if first_day[user_id] == day then newcomers = newcomers + 1 end
  end
  rows[#rows + 1] = { day = day, active = active,
    newcomers = newcomers, delta = prev == nil and 0 or active - prev }
  prev = active
end
```

listing 234: the series walk: days sorted once, then the counting loop the engine folds into one query (source: lua/service/users.lua, lines 438-451)

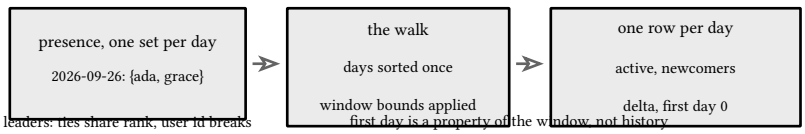


figure 167: events to series, the window bounds every step

30.3. the reports routes and vector 16

The wiring at the ship marker is four moves: the routes mount behind a guard that demands an actor and stamps the idem scope, the policy rows for both routes have sat in the authz table since that chapter froze its audit, the readiness gate takes the kernel's `ready` seam, and the secret line now reads through the shipping module. The guard answers an anonymous caller with `unauthorized` before any store read, and the actor's user id becomes the identity the handler keys its idempotency claim on. The generic idem layer skips this one path by rule, because the sanctioned 202 to 200 upgrade is the reports handler's own decision and a generic replay would answer the stored 202 forever:

```
local ship = require("ship")
app.cfg.ready = ship.readiness().is_ready
local reports = require("reports")
svc.reports = reports.new({ idem = svc.idem, store = svc.store,
  next_id = app.cfg.new_id, clock = app.cfg.clock })
local function guarded(req)
  if not req.actor then
    return nil, envelope.fail("unauthorized", "authentication required")
  end
  req.identity = req.actor.user_id
  return true
end
app.router.route("POST", "/api/reports", function(req)
  local ok, fail = guarded(req)
  if not ok then return fail end
  local res, f = svc.reports.create(req)
  return res or f
end)
app.router.route("GET", "/api/reports/{id}", function(req)
  local ok, fail = guarded(req)
  if not ok then return fail end
  local res, f = svc.reports.get(req.params.id)
  return res or f
end)
```

listing 235: the ship splice: the gate, the reports pair over the oracle, the readiness seam (source: lua/service/main.lua, lines 392-415)

Two facts are stated rather than hidden. The idem layer's clock is milliseconds while the kernel's is whole seconds, so the root crosses units at the construction, a bare seconds feed made every stored row effectively immortal. And the wire lane's first run caught a real defect here: the guard's anonymous branch returned its fail value as a single value, the wrapper destructured two, took the truthy fail table as success, and an anonymous submission reached a 202, invisible to every in-process suite because the vectors compose the harness.

Vector 16 then lands as the scenario the suite was waiting for: the bearer is the session the roles scenario minted at offset 49, asserted byte for byte before the run, the setup op seeds through the oracle's own `event_add`, and the contract's GET line reconciles with the vector's bytes, this service computes on the first read so a GET answers 200 or 404 and pending exists only on the replay before any read has run.

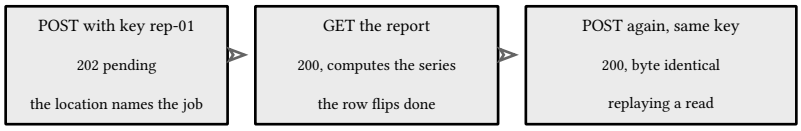


figure 168: vector 16′s three steps, the one sanctioned divergence in the middle

30.4. the secret source

Everything the tokens chapter signed rides one key, and the root's one read of the environment moves into the shipping module where the decision can fail loudly. The rules are three branches: a set secret is the answer once it clears a floor, production without a secret is a startup error, and the unset non-production case gets a named dev constant that is a statement, not a secret. The floor exists because a two-character key is configuration noise that happens to parse, and the production branch exists because a service that boots on a default key is a service nobody configured but everyone can talk to:

```
M.DEV_SECRET = "dev-secret-not-for-production"
M.MIN_SECRET_BYTES = 16
M.PRODUCTION = "production"

-- secret_from is the service tree's one reader of the secret
-- environment: main.lua hands it os.getenv and closes over the answer
-- in a local, so no other module can re-derive the key or disagree
-- with the branch that chose it
function M.secret_from(getenv)
  local value = getenv("LUAAPI_SECRET")
  if value ~= nil then
    if #value < M.MIN_SECRET_BYTES then
      error("LUAAPI_SECRET is set but shorter than " ..
        M.MIN_SECRET_BYTES .. " bytes", 0)
    end
    return value
  end
  if getenv("LUAAPI_ENV") == M.PRODUCTION then
    error("LUAAPI_SECRET is required when LUAAPI_ENV is production", 0)
  end
  return M.DEV_SECRET
end
```

listing 236: the three branches: set and long enough, production refuses absence, dev otherwise (source: lua/service/ship.lua, lines 18-39)

A load-time error is the right shape: the host dies, the container exits, the orchestrator restarts, an operator reads the message. The reader seam is injected, the dev constant is public by construction, and the secret appears nowhere else, the frozen vectors carry their own pinned public key.

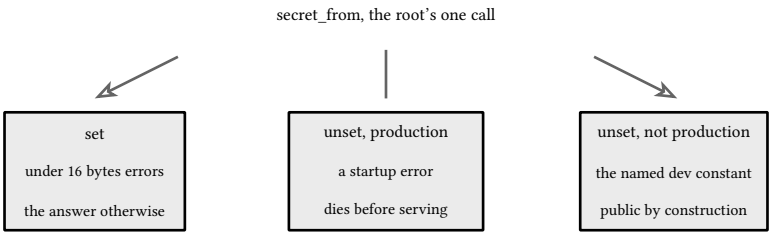


figure 169: one reader of the environment, three answers, one of them fatal

30.5. drain and readiness

The contract says readyz flips to 503 the moment drain starts, and the single-threaded vm has an honest version of that sentence and no more. The lua half is a gate, flipped once and latched because a drain that un-flips lies to the balancer, and the boundary is plain: nothing on the lua side can observe a signal to call it.

The container's drain lives in the twin, and the posix half has one lesson the windows host never needed: windows documents that closing a socket fails the accept blocked on it, linux makes no such promise, so the twin polls and the term interrupts the poll, visible only because the handler installs without `SA_RESTART`, which is why the code uses `sigaction` and not `signal`, whose bsd semantics restart the poll and hide the term. The drain falls out of the loop: the in-flight `handle` runs to its last byte, the next poll reads the flag, and the exit is zero, verified by stopping a healthy container:

```
  memset(&sa, 0, sizeof(sa));
  sa.sa_handler = on_term;
  sigemptyset(&sa.sa_mask);
  sa.sa_flags = 0;
  sigaction(SIGTERM, &sa, NULL);
  sigaction(SIGINT, &sa, NULL);

  while (!g_stop) {
    struct pollfd pfd = { .fd = listen_fd, .events = POLLIN, .revents = 0 };
    int ready = poll(&pfd, 1, -1);
    if (ready < 0) {
      if (errno == EINTR) continue;
      fatal("poll");
    }
    if (ready == 0) continue;
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) {
      if (errno == EINTR) continue;
      fatal("accept");
    }
    // the same bound the windows twin teaches: a client that connects
    // and sends half a head may not park the single thread, recv
    // fails after the deadline budget and the seam closes the rest
    struct timeval tv = { .tv_sec = 5, .tv_usec = 0 };
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
    lua_rawgeti(L, LUA_REGISTRYINDEX, ref);
    lua_pushinteger(L, fd);
    docall(L, 1, 0);
```

listing 237: sigaction without restart, poll interrupted by term, the in-flight connection finishing (source: lua/service/docker/posix_host.c, lines 227-254)

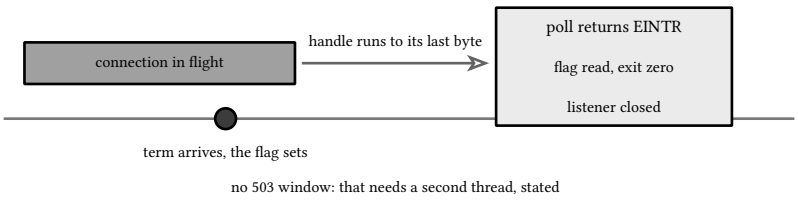


figure 170: the drain a single thread can honestly give

30.6. the image

Two stages, and the boundary carries one binary and one tree. The builder is debian slim plus a compiler toolchain, it fetches the lua tarball by pinned version and checksum so a moved upstream is a loud build failure, and it builds the interpreter with the release's own `make linux` target, proving the artifact in place: the whole plain lane runs green in the builder on the interpreter the image ships. The edge links that build's static `liblua.a` in one gcc line, so the runtime needs no lua at all, only the binary and libc:

```
# the pin: the same 5.5.1 the book's tools/lua55 builds locally, with
# the tarball's sha256 stated so a moved or repacked upstream is a
# loud build failure, never a silent toolchain drift
ARG LUA_VERSION=5.5.1
ARG LUA_SHA256=1c4b4068d67061f2a2231ad2b5422e77acea1487ea9890f6320af614f4373dce
RUN curl -fsSL -o lua.tar.gz https://lua.org/ftp/lua-${LUA_VERSION}.tar.gz \
    && echo "${LUA_SHA256}  lua.tar.gz" | sha256sum -c - \
    && tar xzf lua.tar.gz \
    && cd lua-${LUA_VERSION} \
    && make linux \
    && make install

WORKDIR /src
COPY . .

# the artifact proves itself in the builder: the whole plain lane runs
# green on the interpreter this image just built, then the edge links
# the static lua into one binary that needs nothing but libc at run
RUN lua run.lua \
    && gcc -O2 -Wall -Wextra -o /out-host docker/posix_host.c \
       -I/usr/local/include /usr/local/lib/liblua.a -lm -ldl
```

listing 238: the pin, the build, the self-proof, the static link (source: lua/service/Dockerfile, lines 15-35)

The checksum is the pin that matters, the same 5.5.1 the book's `tools/lua55` builds locally, and `make linux` needs no readline, that is the separate flavor. The runtime is slim debian under uid 1000 and the tree copies whole, the context trade the go lane states rather than paying a dockerignore that drifts.

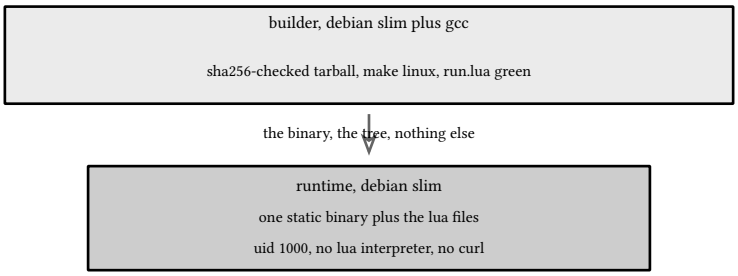


figure 171: two stages, one binary and one tree cross

30.7. compose one command

The compose file is one service and its replay sibling, and the contract it enforces is the return value of a single command. `docker compose up -d --build --wait` builds, starts, and blocks until the healthcheck passes, so a green up means the interpreter compiled, the lane proved itself in the builder, the edge bound its listener, and the probe answered. The probe is the image probing itself: no curl exists in the runtime, so the healthcheck runs the same binary in client mode against its own loopback, one flag apart from the server. The published port is the reserved 19490 fronting the container's 19480, keeping a running stack off the local winsock lane's port, and the ops listener is declined: this kernel owns one listener, metrics already ride the main mux:

```
x-hc: &hc
  interval: 2s
  timeout: 2s
  retries: 30
  start_period: 3s

services:
  api:
    build: .
    environment:
      LUAAPI_SECRET: dev-only-not-a-secret
    ports:
      - "19490:19480"
    healthcheck:
      # no curl, no wget in the image: the same binary in client mode
      # probes its own loopback healthz, 200 and only 200 is healthy
      test: ["CMD", "host", "--client", "/app/docker/probe.lua", "--addr", "127.0.0.1:19480"]
      <<: *hc
```

listing 239: the probe timing anchor, one service, the self-probe, the reserved port fronting it (source: lua/service/compose.yml, lines 13-30)

No volume exists, and the absence is the statement: the oracle is this vm's memory, a restart starts from zero, durability is the jit engine's file store. The file also lacks a stop grace period, because this drain concludes with the in-flight connection.

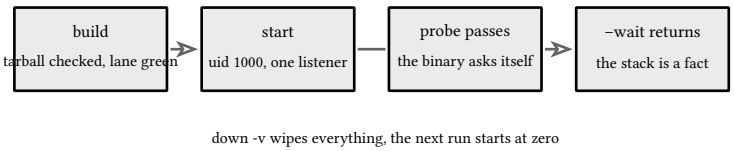


figure 172: what a green up proves, in order

30.8. the docker lane as a gate

The lane is one make target with the sibling lanes' shape: a daemon preflight that fails loudly instead of skipping, teardown under a trap on every path, and the replay leg as the gated step. One lesson cost a stale image and sits in the build line: `up --build` never builds a service whose profile is inactive, so the lane builds with the profile named, and the replay container runs one-shot over the compose network with its exit code as the verdict:

```
verify-lua-docker:
  @docker info >/dev/null 2>&1 || { echo "docker daemon not reachable." >&2; exit 1; }
  @cd books/lua/service && docker compose down -v >/dev/null 2>&1 || true && \
    trap 'docker compose down -v' EXIT && \
    docker compose --profile lane build && \
    docker compose up -d --wait && \
    docker compose --profile lane run --rm replay; rc=$$?; exit $$rc
```

fragment, illustrative, not executed

The nine scenarios replay the contract's guarantees at the layer the frozen bytes cannot reach over a wire: the register replay byte identical, the two 401 shapes differing only in their request id, the etag pair, the stale patch guard, the last admin chain, the exposition, and the reports pair's upgrade. The rate limit scenario is the most honest platform statement: every credential-shaped login miss pays the issue-tier pbkdf2 before its 401, measured around twenty seconds per answer over this wire, and six of those cannot fit the bucket's one-token refill second, the flattened cost being the feature that stops account enumeration. The lane trips the same bucket with the

decode-refusal shape the bind ladder rejects before any verify runs, the sixth read the 429 with its retry-after, where the frozen vector pins the same arithmetic at zero cost. The suite chapter owns the test shapes and the swap, the conc chapter the idem semantics, the load chapter the offered-load harness and the p99.

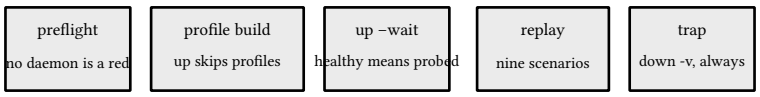


figure 173: the lane: preflight, build with the profile, up, replay, teardown on every path

sources: the lua 5.5 manual section 6.9, operating system facilities, for `os.getenv`, and the lua.org ftp page plus the 5.5 readme for the `linux` target and `liblua.a`, at lua.org, accessed 2026-09-28. Docker's documentation for multi-stage builds, `up --wait`, healthchecks, profiles, and build, at docs.docker.com, accessed 2026-09-28. The linux man-pages via man7.org for `signal(2)` and `sigaction(2)` on `SA_RESTART`, `poll(2)` on `EINTR`, and `send(2)` on `MSG_NOSIGNAL`, accessed 2026-09-28. Verified by the service plain lane at 188 checks on the landed tree, the host lane at 162 and the jit lane at 18 both green, and the docker lane end to end under `make verify-lua-docker`, nine wire scenarios, the trap teardown observed, the stopped container's zero exit read from its state.

31. the test suite

The service part ends the way the language chapters began, with the project runner: `samples/run.lua` pcalled a `{name, fn}` list per chapter in chapter 1, and the service suite is that same runner grown to 21 entries. This chapter turns the suite around as its own artifact: the 2 shapes a test takes here, the 51-line runner and 59-line assert library that carry everything, the swap that moved the frozen vector replay onto the composition root's real chain and caught 3 production defects doing it, the deterministic replay suites that stand in for the property libraries the other lanes import, and the dated facts about `busted` and `luaunit` this vehicle declines.

31.1. what the suite holds

The census, stated once here and nowhere else in the chapter. The plain lane runs 188 checks green: 136 of them live in the array part of 15 family modules, each module carrying its own test list beside the code it tests, and 52 of them live in 6 scenario files under `service/tests/` that compose several families into one story. The jit lane adds 18 checks over the ffi store family, a lane boundary and never a skip, and the host lane drives 162 wire checks through the compiled winsock seam. All 16 frozen vectors are pinned: 01 through 14 and 16 replay byte for byte through the scenario suite, and 15, the metrics exposition, is pinned by the observability module's own list. The runner's table is the census in executable form:

```
local chapters = {
  "scaffold",
  "json",
  "envelope",
  "idem",
  "cache",
  "limit",
  "obs",
  "load",
  "router",
  "crypto",
  "tests/users_test",
  "tests/authn_test",
  "authz",
  "reports",
  "middleware",
  "main",
  "ship",
  "tests/vectors_test",
  "tests/ops_test",
  "tests/swap_test",
  "tests/replay_test",
}
```

listing 240: the chapters table: 15 family modules, 6 scenario files, the store suite's absence stated in the comment above (source: lua/service/run.lua, lines 7-29)

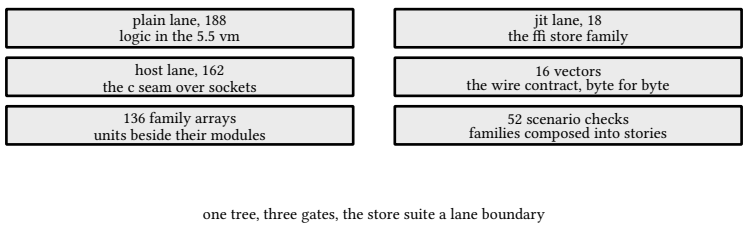


figure 174: three lanes, each proving a different thing about the same tree

31.2. the runner

The runner is 20 lines and no framework. Each entry in the chapters table is required, its array part is walked with `ipairs`, and every test function runs under `pcall`: a pass contributes one to the module's count, a raise is caught, named to standard error as module, test name, and message, and counted as a failure instead of killing the run. The per-module line is the same shape chapter 1 printed. The exit code is the whole contract with the make lane: zero failures exits 0, any failure exits nonzero and `verify-lua` stops:

```
local failed = 0
local passed = 0
for _, name in ipairs(chapters) do
  local mod = require(name)
  local cpass = 0
  for _, t in ipairs(mod) do
    local ok, err = pcall(t.fn)
    if ok then
      cpass = cpass + 1
    else
      failed = failed + 1
      io.stderr:write(name .. " :: " .. t.name .. " FAILED: " .. tostring(err) .. "\n")
    end
  end
  passed = passed + cpass
  print(string.format("%-14s %d/%d", name, cpass, #mod))
end
print(string.format("total %d passed, %d failed", passed, failed))
if failed > 0 then os.exit(1) end
```

listing 241: the loop: pcall per test, failures named, the exit code the only output verify reads (source: lua/service/run.lua, lines 33-51)

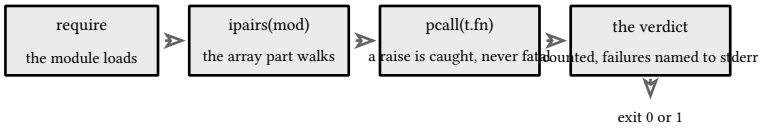


figure 175: one entry's path through the runner

31.3. the t lib

The assert library is 5 functions shaped like `samples/lib.lua`, and 2 of them do all the work. `eq` tries lua equality first, so numbers, strings, and booleans compare for free, and falls to a structural walk for tables that counts members in both directions, so a missing member and an extra member are both differences. The failure message raises at level 2, which points the error at the caller's line rather than inside the library. `throws` wraps `pcall` and requires the message substring, `near` carries the float slack the clock arithmetic sometimes needs, and `show` renders both sides sorted:

```lua
local function deep(a, b)
  if a == b then return true end
  if type(a) ~= "table" or type(b) ~= "table" then return false end
  local seen = 0
  for k, v in pairs(a) do
    if not deep(v, b[k]) then return false end
    seen = seen + 1
  end
  for _ in pairs(b) do seen = seen - 1 end
  return seen == 0
end

-- compares with lua equality first, then structurally for tables
function M.eq(actual, expected, msg)
  if not deep(actual, expected) then
    error((msg or "values differ") .. ": got " .. show(actual) .. ", want " .. show(expected), 2)
  end
end
```

listing 242: deep equality counting members both ways, eq raising at level 2 (source: lua/service/lib.lua, lines 19-36)

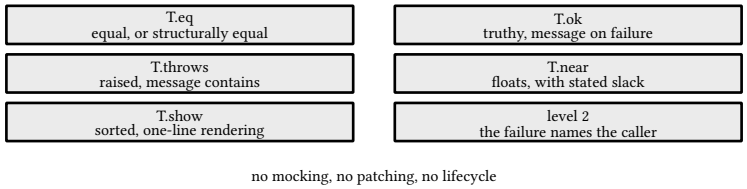


figure 176: each assert and the fact it establishes

31.4. the two test shapes

A test here takes 1 of 2 shapes, and the split is deliberate. The family module carries its own tests in the array part of the table it returns, the mixed-table idiom the json chapter ratified: the array part is the test list the runner walks, the hash part is the module's api, one table serving both, and a module test lives 1 require away from the code it pins. The 6 files under `service/tests/` exist because their subjects are not units: the vector scenarios compose the store, the identity layer, the limiter, and the idem guard into one chain, the ops walkthrough mirrors the root's splice order, and the swap and replay suites drive the root itself. Both shapes splice into the same chapters table, so the runner never learns which shape an entry is:

```
M[#M + 1] = {
  name = "a request parses across chunk boundaries",
  fn = function()
    local req = M.read_request(reader({
      "POST /api/users HTTP/1.1\r\nHost: x\r\nContent-Ty",
      "pe: application/json\r\nContent-Length: 7\r\n\r\n{\"a\":1",
      "}",
    }), function() return "r" end)
    T.eq(req.method, "POST")
    T.eq(req.target, "/api/users")
    T.eq(req.version, "HTTP/1.1")
    T.eq(req.headers["content-type"], "application/json")
    T.eq(req.headers["host"], "x")
    T.eq(req.body, '{"a":1}')
    T.eq(req.trailing, "")
  end,
}
```

listing 243: the family shape: one entry of the kernel module's own list, beside the code it pins (source: lua/service/main.lua, lines 492-508)

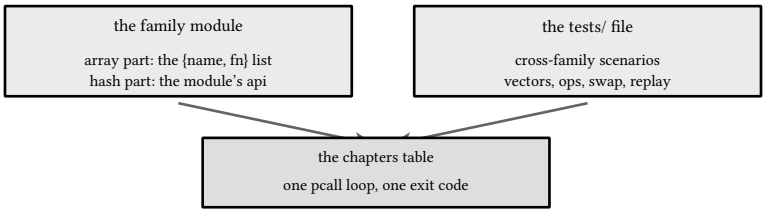


figure 177: the two shapes, one runner

31.5. the harness swap

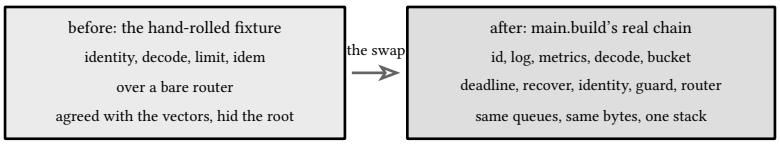
The scenario suite began on a fixture that hand-rolled the whole chain over a bare router, and the fixture's own comment invited the swap: when the composition root grew a builder over injected seams, the fixture could retire and the vectors would drive the production wiring. `main.build(cfg)` is that builder, and the swap is 1 file. The harness still owns the frozen vectors' queues, the constant clock the byte pins assume, the ids and bytes in pop order, the test iterations tier, and the collected sink, but the stack those seams feed is now the root's own. The one ruled divergence rides the rules seam, login only, because the frozen register vectors answer without rate headers and the bytes outrank the prose for replay.

The swap caught 3 production defects the fixture had been hiding, all 3 fixed in the builder's landing commit. The idem store was built on the bare seconds clock against a milliseconds ttl, making the 24 hour expiry effectively immortal, and the fixture had crossed units all along, which is why every byte stayed green while the root was wrong. Nothing in the real chain fed `req.decoded`, so the login rule's email keying collapsed every caller into 1 shared bucket in production, invisible because the fixture decoded by hand. And the idem guard's key-reuse refusal returned a raw fail value no layer above it renders, so the contract's 422 reached the wire as a 500 with the connection closed, confirmed by probe before the fix. A 4th finding closed as a deletion: the root minted a cache object nothing consumed, its ttl on the bare seconds clock besides, and the honest fix was removing it, a reads cache over an in-memory table being theater the go lane's sqlite boundary earns. Every scenario byte held through the swap, 13 of 13: the fixture agreed with the root because the vectors said so, not because the 2 stacks were the same code:

```
  local function now() return h.t end
  local function new_id()
    local id = table.remove(h.ids, 1)
    assert(id, "fixture id queue exhausted")
    return id
  end
  local function rand(n)
    local bytes = table.remove(h.rand, 1)
    assert(bytes and #bytes == n, "fixture rand queue exhausted")
    return bytes
  end

  h.app, h.svc = main.build({
    clock = now,
    new_id = new_id,
    rand = rand,
    secret = M.SECRET,
    iterations = authn.TEST_ITERATIONS,
    rules = opts.rules or { "login" },
    sink = function(line) h.lines[#h.lines + 1] = line end,
  })
  return h
end
```

listing 244: the swap: the queues and seams feed main.build, the stack is the root's own (source: lua/service/tests/harness.lua, lines 29-51)



3 defects found and fixed, the bytes the judge
the immortal ttl, the unfed decode, the 422 that answered 500
13 of 13 vectors byte-identical through the root

figure 178: before and after the swap, the bytes the judge

31.6. the four fresh states

The vectors are scenarios, not 1 story, and the suite says so with fresh state per group. Vector 08 revokes a session family, and vectors 09 through 12 act with the very session that revocation kills, so the read, patch, and list scenarios cannot follow the reuse attack in one continuous state. Four fixtures cover the 4 stories, each with its own queues, the ids a scenario will mint in the order the requests will mint them and the bytes the wire will consume. One ordering rule worth stating: the auth routes draw the dummy verify's salt at wiring time, before any request runs, so the first queue entry is always the dummy's, and scenario a's four 16 byte draws are the dummy, then 3 registration salts:

```
local a = harness.build({
  rand = {
    ("\1"):rep(16), ("\2"):rep(16), ("\3"):rep(16), ("\4"):rep(16),
    REFRESH_A, REFRESH_B, ("\7"):rep(32),
  },
  ids = {
    ADA,
    "01a0f5d5-7450-72d9-b66f-aaa6bf532a80", -- 03's 409
    GRACE, LINUS,
    "01a0f5d5-9390-716c-8712-6a54e12b56bb", -- 04's session
    "01a0f5d5-ed68-78b4-bb40-592c3b78c623", -- 12's 412
    "01a0f5d6-04d8-7b71-bfe0-5d3e9eabadbe", -- grace's session at +39
  },
})
```

listing 245: scenario a's build: the queues carry the vector's own ids and bytes in pop order (source: lua/service/tests/vectors_test.lua, lines 72-85)

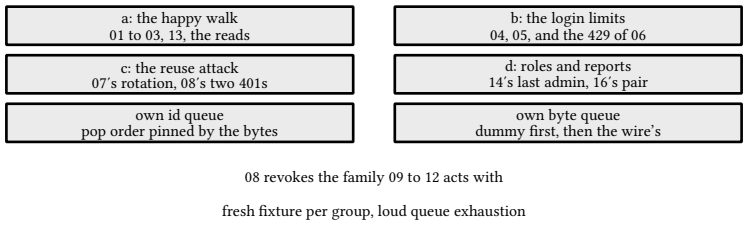


figure 179: four scenarios, four fresh states, what each holds

31.7. replay invariants over seeded streams

The go lane holds its invariants with fuzz targets whose seeds ride the plain gate, and this lane's analog is deterministic replay over a committed seed, because the lua vm has no native fuzzing and the contract rules the platform's own mechanism. The generator is a hand-rolled xorshift64 rather than `math.random`, because the manual declares `math.random`'s algorithm implementation defined, and a seed whose every bit the book can print is the honest version of reproducible. The first stream drives the cache as a state machine, 600 ops over 6 names at capacity 4, each op's move picked by the low 2 bits of the op word and its name by the bits above them. The invariants are named and checked after every op: the bound holds, a hit always answers the last write, a deleted name is gone, and expiry is a cliff, everything written before a tick misses after it. The hit invariant is one directional on purpose, because a bound cache may legally miss what eviction removed, and the claim worth testing is that it never answers a wrong value. The second stream holds the bucket arithmetic the same way, 400 draws at random gaps, the level inside its bounds, a denial free, an allow exactly 1, the reported remaining floored. No test sleeps, every clock is arithmetic, and no assertion depends on map order:

```
    if move == 0 then
      model[key] = op % 97
      c:put(key, model[key])
      T.eq(c:get(key), model[key],
        "the write just made is immediately observable")
    elseif move == 1 then
      local got = c:get(key)
      if got == nil then
        -- a miss is always legal: eviction and expiry both miss
        misses = misses + 1
      else
        hits = hits + 1
        T.eq(got, model[key], "a hit always answers the last write")
      end
    elseif move == 2 then
      c:delete(key)
      model[key] = nil
      T.eq(c:get(key), nil, "a deleted key is gone")
    else
      now = now + 1000
      model = {}
      for _, k in ipairs(keys) do
        T.eq(c:get(k), nil, "expiry is a cliff: pre-tick writes miss")
      end
    end
```

listing 246: the op dispatch: four moves, an invariant under each, the bound after every op (source: lua/service/tests/replay_test.lua, lines 45-69)

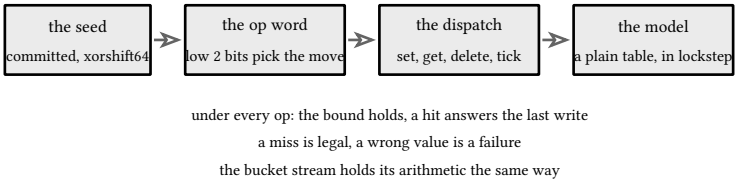


figure 180: seed to stream to model, the invariant under every step

31.8. busted and luaunit as dated meta

The ecosystem's 2 test frameworks both shipped lua 5.5 support in 2026, and both are declined on the standing rule. `busted` 2.3.0, published 2026-01-07 under lunarmodules, brings describe blocks, before_each hooks, spies and mocks, and a command line runner. `luaunit` 3.5, published 2026-03-26, brings an assert family over the xunit shape with xml output. The adoption criteria are stated so the refusal is a decision: a framework earns its keep when the suite is large enough that uniform describe structure beats reading a flat list, when the mocking machinery replaces doubles this tree would otherwise build, and when ci integration consumes the framework's report format. This suite meets none of them. The runner is 20 lines the book has already taught, the doubles are hand-rolled and strict, and the make lane reads an exit code, so a dependency would add a rockspec, a pin, and a second thing to learn, and remove nothing the suite lacks. The strict double is the concrete core of that answer: the store double in the reports module mirrors the real row shape field for field, every call lands in a log a test reads to prove what did not run, and a second create for a live id is an error rather than a silent overwrite, the strictness generated mocks sell:

```lua
-- would hide a contract change
local function fake_store()
  local s = { log = {}, rows = {}, computes = 0 }
  function s.report_create(r)
    if s.rows[r.id] then error("unexpected second create " .. r.id, 2) end
    s.log[#s.log + 1] = "create"
    s.rows[r.id] = { id = r.id, kind = r.kind, from_day = r.from_day,
      to_day = r.to_day, body_sha = r.body_sha, status = "pending" }
    return true
  end
  function s.report_load(id)
    s.log[#s.log + 1] = "load"
    return s.rows[id]
  end
  function s.report_compute(id, from_day, to_day)
    s.computes = s.computes + 1
    s.rows[id].status = "done"
    return {
      { day = "2026-09-25", active = 1, newcomers = 1, delta = 0 },
      { day = "2026-09-26", active = 2, newcomers = 1, delta = 1 },
    }
  end
  return s
end
```

listing 247: the strict store double: the log proves what did not run, the second create errors (source: lua/service/reports.lua, lines 122-146)

the axis	this suite	busted 2.3.0	luaunit 3.5
structure	a flat list	describe blocks	xunit classes
doubles	hand-rolled, strict	spies and mocks	none shipped
ci reads	the exit code	junit and taps	xml output
dependency cost	zero	a rockspec and a pin	a rockspec and a pin

both added lua 5.5 support in 2026, neither adopted

figure 181: the frameworks as dated facts, the axes that would change the answer

sources: lua.org manual 5.5 sections 6.2 (basic functions, the pcall, error, and assert the runner and lib are built on), 6.3 (coroutine manipulation, the resume and status the deadline layer runs under), 6.7 (table manipulation, the remove and concat of the queues), 6.8 (mathematical functions, floor and min in the bucket arithmetic), and 6.10 (operating system facilities, os.exit and the root's os.time edge), all accessed 2026-09-28. `busted` 2.3.0 verified against github.com/lunarmodules/busted/releases and luarocks.org, `luaunit` 3.5 verified against luaunit.readthedocs.io, both accessed 2026-09-28 and deliberately not adopted, criteria stated above. The structural mirror is the go lane's suite chapter, `books/go/chapters/30-suite.typ`. Verified by the service lanes under the pinned toolchain: 188 plain checks, 18 jit checks over the ffi store family, 162 wire checks through the host seam, all 16 frozen vectors byte-exact through the composition root's real chain.

32. capstone: auto chess, human versus bot

The capstone is a complete auto chess game, two players, shops, gold, xp, merges, deterministic combat, one of them a bot, and the whole of it embedded in the allegro host chapter 18 built. The game is pure lua, 10 modules plus a 34 test suite under the pinned 5.5 binary, and the c host that renders it also runs it headless over scripted drivers that pin entire games to one digest string. The human interface is a mouse and four keys, and it is also the automation api: both speak the same command queue.

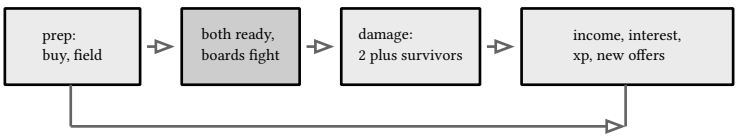
32.1. the game

Two players start at 20 hp, 10 gold, level 2. A round is a prep phase, buy from a five slot shop, field units up to your level, then both sides ready and the boards fight. The loser takes 2 damage plus the winner's survivors, the winner banks a bonus, everyone draws income and interest, and three copies of a unit merge into the next tier. Five unit types carry the tactics: walls soak, blades hit, archers shoot the weakest, pyros splash neighbors, medics heal:

```
M.types = {"wall", "blade", "archer", "pyro", "medic"}

-- stats per tier: hp and damage double-ish per tier, the shape of every
-- auto battler's sheet, fixed numbers so combat stays deterministic
M.sheet = {
  wall   = {{hp = 30, dmg = 1}, {hp = 48, dmg = 2}, {hp = 78, dmg = 3}},
  blade  = {{hp = 12, dmg = 4}, {hp = 20, dmg = 7}, {hp = 32, dmg = 12}},
  archer = {{hp = 8,  dmg = 3}, {hp = 14, dmg = 5}, {hp = 22, dmg = 8}},
  pyro   = {{hp = 10, dmg = 2}, {hp = 16, dmg = 4}, {hp = 26, dmg = 6}},
  medic  = {{hp = 9,  dmg = 1}, {hp = 15, dmg = 2}, {hp = 24, dmg = 3}},
}
```

listing 248: the stat sheet, five types of three tiers each (source: lua/capstone/stats.lua, lines 7-17)



level is the deploy cap, 2 to 6, xp buys level

three copies merge, bench first, recursively to tier 3

selling returns half, income 5, interest 1 per 10 capped at 3

a side with nothing fielded fights an empty board and loses

figure 182: one round of the game, prep to income

32.2. the command queue, the automation seam

book 9, ch 18 (lua 5.5, a c host: allegro 5 with lua embedded) designed the contract, the capstone ships the production queue. Commands are validated tables, `keydown`, `keyup`, `click`, `quit`, and the game consumes exactly one `drain()` per tick, nothing else reads input at all:

```
  local q = {items = {}, first = 1, last = 0}
  function q.push(cmd)
    if type(cmd) ~= "table" or type(cmd.type) ~= "string" then
      error("commands are tables carrying a string type", 2)
    end
    local shape = shapes[cmd.type]
    if not shape then error("unknown command type: " .. cmd.type, 2) end
    for field, want in pairs(shape) do
      if type(cmd[field]) ~= want then
        error(cmd.type .. " wants " .. field .. ": " .. want, 2)
      end
    end
    q.last = q.last + 1
    q.items[q.last] = cmd
    return cmd
  end
```

listing 249: push validates at the door, the only way in (source: lua/capstone/queue.lua, lines 17-32)

The seam is bidirectional automation. In a window, the c host translates allegro events into these tables through the fixed pixel layout, a click at a shop slot buys it, a bench slot selects it, a board cell places or moves, `r` rerolls, `d` buys xp, `e` sells the selection, `space` readies. In the verify suite, scripted drivers push the same tables at chosen ticks, and the game cannot tell the difference, which is the point:

```
function M.input(state, cmd)
  if cmd.type == "quit" then
    state.phase.fire("game_over")
    state.outcome = "quit"
    return
  end
  if cmd.type == "keyup" then return end
  local p = state.human
  if cmd.type == "keydown" then
    if cmd.key == "r" then M.reroll(state, "human")
    elseif cmd.key == "d" then M.buy_xp(state, "human")
    elseif cmd.key == "space" then p.ready = not p.ready
    elseif cmd.key == "e" then
      if state.sel then
        if state.sel.where == "bench" then
          M.sell(state, "human", "bench", state.sel.slot)
        else
          M.sell(state, "human", "board", state.sel.col, state.sel.row)
        end
```

listing 250: the input mapping, clicks against the fixed layout (source: lua/capstone/actions.lua, lines 207-225)

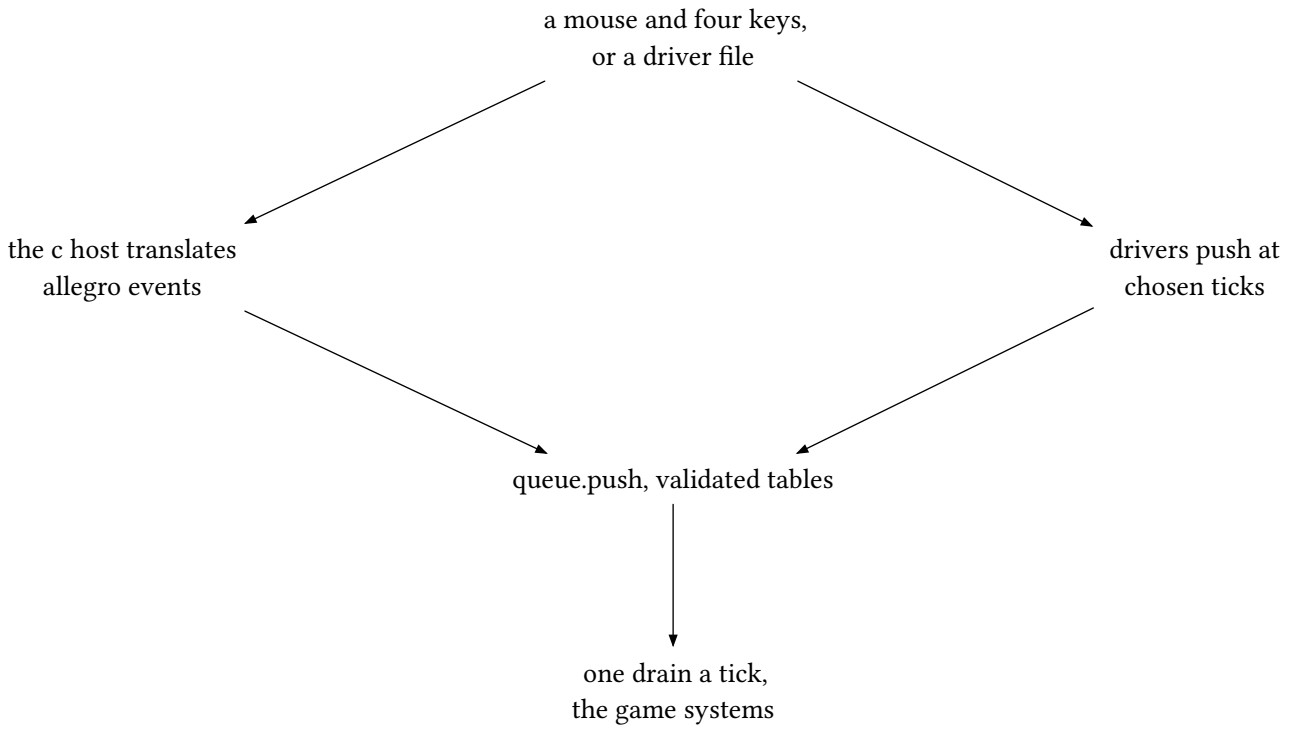


figure 183: two hands, one queue, one game

32.3. ecs in lua

Units and players are entities, integer ids in one world, with a sparse store per component, `unit`, `team`, `loc`, and iteration that picks the smallest store as its anchor, the trick book 15, ch 2 (game systems and architecture, ecs theory) teaches for free queries:

```
function w.each(...)
  local names = {...}
  local anchor, best = nil, math.huge
  for _, name in ipairs(names) do
    local store = w.stores[name]
    local n = 0
    if store then
      for _ in pairs(store) do n = n + 1 end
    end
    if n < best then best = n; anchor = name end
  end
  local out = {}
  local seed = w.stores[anchor] or {}
  for id in pairs(seed) do
    if w.alive[id] then
      local ok = true
      for _, name in ipairs(names) do
```

listing 251: each anchors on the smallest store among the wanted components (source: lua/capstone/ecs.lua, lines 43-59)

The board and bench are indexes beside the stores, cell keys to entity ids, so placement queries stay O(1) while the world owns the truth. Merges and sells kill entities, the collector reclaims the component rows, and the suite pins the lifecycle:

```
  w.alive[id] = true
  return id
end

function w.kill(id)
  w.alive[id] = nil
  for _, store in pairs(w.stores) do
    store[id] = nil
  end
end

function w.set(id, name, value)
```

listing 252: spawn and kill are the whole lifecycle (source: lua/capstone/ecs.lua, lines 11-22)

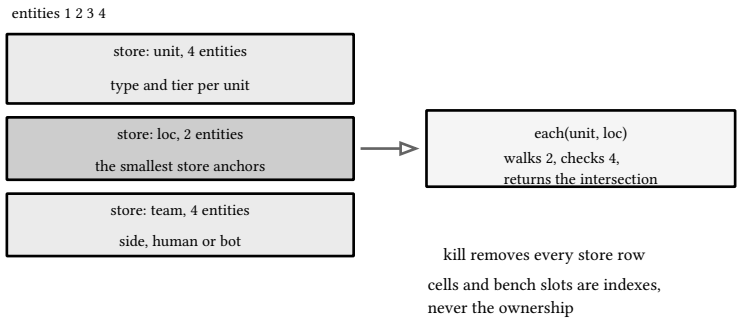


figure 184: entities, per-component stores, the anchor law

32.4. the bot's brain, a state machine

The bot plays through the same action functions the input mapping calls, `buy`, `deploy_from_bench`, `reroll`, `buy_xp`, `ready`, and a four state machine decides which, the shape book 15, ch 5 (game systems and architecture, finite state machines) teaches. `OPENING` buys the cheapest offers until the board is full, `ACCUMULATE` hoards gold and buys only duplicates, `SPEND` rerolls for upgrades and levels, and `STABILIZE` is panic mode at low hp, spending everything:

```
M.EVENTS = {
  bench_full = {OPENING = "ACCUMULATE", ACCUMULATE = "SPEND"},
  gold_changed = {ACCUMULATE = "SPEND"},
  round_result = {SPEND = "ACCUMULATE"},
  low_hp = {OPENING = "STABILIZE", ACCUMULATE = "STABILIZE", SPEND = "STABILIZE"},
  board_full = {OPENING = "ACCUMULATE"},
}

-- the panic threshold on hp, in units of round damage
M.panic_hp = 6
M.rich_gold = 12

function M.new_brain()
  return fsm.new({OPENING = true, ACCUMULATE = true, SPEND = true,
                  STABILIZE = true}, "OPENING", M.EVENTS)
end
```

listing 253: the transition table, five events over four states (source: lua/capstone/bot.lua, lines 14-30)

```lua
if brain.in_state("OPENING") then
  -- buy the cheapest offers in slot order until broke or board full
  local order = {}
  for slot in pairs(me.shop.offers) do order[#order + 1] = slot end
  table.sort(order)
  for _, slot in ipairs(order) do
    local offer = me.shop.offers[slot]
    if offer and stats.buy_cost(offer.tier) ≤ me.gold - 1 then
      actions.buy(state, "bot", slot)
    end
  end
  actions.deploy_from_bench(state, "bot")
elseif brain.in_state("ACCUMULATE") then
  -- hoard: buy only duplicates of what is already owned
  for slot = 1, 5 do
```

listing 254: opening: cheapest offers in slot order, then field them (source: lua/capstone/bot.lua, lines 48-62)

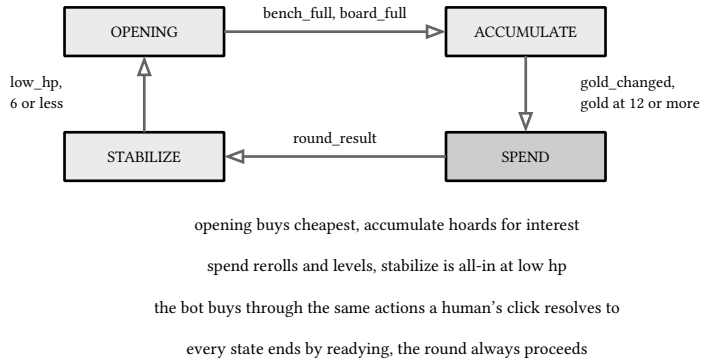


figure 185: the bot's four states and the events that move it

32.5. deterministic combat

Combat has no rng anywhere. Both boards face each other, units act in reading order, front row first, and one tick is one full pass: melee swings at the nearest front enemy, archers and pyros pick the weakest, medics heal the weakest ally, pyro splash halves onto orthogonal neighbors:

```
local function target_for(fight, u)
  local foes = enemies_of(fight, u)
  if #foes == 0 then return nil end
  if u.type == "medic" then
    local allies = living(fight, u.team)
    local best = nil
    for _, a in ipairs(allies) do
      if a.hp < a.hp_max and (not best or a.hp < best.hp) then best = a end
    end
    return best, "heal"
  end
  if stats.ranged[u.type] then
    local best = nil
    for _, f in ipairs(foes) do
      if not best or f.hp < best.hp then best = f end
    end
    return best, "hit"
  end
  local best = nil
  for _, f in ipairs(foes) do
    local d = frontness(u, f.row)
    if not best or d < best.d then best = {u = f, d = d} end
  end
  return best.u, "hit"
end

-- one combat tick: every living unit acts once, in reading order. returns
-- the event log for this tick so the game layer can animate or assert
```

listing 255: targeting: front, weakest, or the weakest ally (source: lua/capstone/rules.lua, lines 62-89)

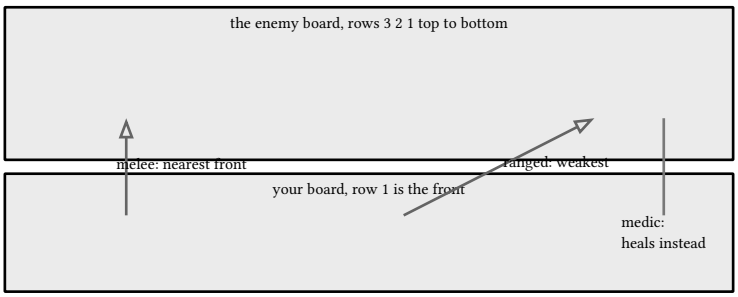
```
function M.tick(fight)
  fight.tick = fight.tick + 1
  local log = {tick = fight.tick, acts = {}}
  for _, u in ipairs(fight.order) do
    if u.hp > 0 then
      local target, mode = target_for(fight, u)
      if target then
        local act = {actor = u.id, target = target.id, kind = mode}
        if mode == "heal" then
          target.hp = math.min(target.hp_max, target.hp + u.dmg + 2)
          act.amount = u.dmg + 2
        elseif u.type == "pyro" then
          act.amount = u.dmg
          target.hp = target.hp - u.dmg
          for _, n in ipairs(board.neighbors(target.col, target.row)) do
            for _, f in ipairs(enemies_of(fight, u)) do
              if f.hp > 0 and f.col == n.col and f.row == n.row then
                f.hp = f.hp - (u.dmg // 2)
                act.splash = true
```

listing 256: one tick, one pass, heal, splash, and plain hits (source: lua/capstone/rules.lua, lines 90-108)

The game layer runs one rules tick every quarter second of sim time until a side is empty or the 40 tick cap draws the fight, and the whole fight is a replay: same boards, same ticks, same result, which the suite pins twice, once for the resolution and once for the per-tick event log:

```
tests[23] = {"resolve is deterministic and counts survivors", function()
  local strong = {{"blade", 3, 1, 1}, {"blade", 3, 2, 1}, {"blade", 3, 3, 1}}
  local weak = {{"wall", 1, 1, 1}}
  math.randomseed(1)
  local a = arena(strong, weak)
  local r1 = rules.resolve(a)
  math.randomseed(1)
  local b = arena(strong, weak)
  local r2 = rules.resolve(b)
  T.eq({r1, rules.survivors(a, 1)}, {1, 3})
  T.eq(r1, r2)
  local cap = arena({{"medic", 1, 1, 1}}, {{"medic", 1, 1, 1}})
  T.eq(rules.resolve(cap), 0)
end}
```

listing 257: the deterministic resolve test, run twice (source: lua/capstone/run.lua, lines 263-277)



reading order, no rng, one pass per tick, cap at 40

survivors feed the damage, 2 plus the count

figure 186: one combat tick, the targeting rules

32.6. the c host, both interop directions

The host is chapter 18's skeleton grown up. Direction one, c calls lua: the timer ticks, the host builds a command table with `lua_createtable` and field pushes, hands it to the queue, and calls `update(dt)` protected. Direction two, lua calls c: the api table registers draw primitives that early-return headless, so the same game draws into a window or into nothing:

```
  if (h.headless) {
    // no timer, no sleep: headless runs as fast as the cpu allows
    long tick = 0;
    while (tick < max_ticks) {
      tick++;
      push_due_driver_cmds(L, tick);
      call_game(L, "update", dt);
      if (outcome_of(L)) break;
    }
    lua_pushinteger(L, tick);
    lua_setglobal(L, "last_tick");
```

listing 258: the headless loop, driver commands then one update (source: lua/capstone/src/host.c, lines 201-211)

```c
static int api_clear(lua_State *L) {
  if (host->headless || !host->target) return 0;
  al_clear_to_color(al_map_rgb((int)luaL_optinteger(L, 1, 0),
                               (int)luaL_optinteger(L, 2, 0),
                               (int)luaL_optinteger(L, 3, 0)));
  return 0;
```

listing 259: headless primitives early-return, never error (source: lua/capstone/src/api.c, lines 12-17)

The verdict at the end of a headless run is a digest, one string summarizing rounds, hp, gold, levels, and every unit with its position, compared against the outcome and digest the driver file freezes. All three drivers end `bot_win`, and the honest result is in the numbers: a casual human script dies by round 6 against a bot that never misses an economy beat, the steady `fixed-open` driver holds to round 6, `rush` and `concede` fall at round 5:

```
fixed-open  3656 ticks  bot_win
rush        2756 ticks  bot_win
concede     2716 ticks  bot_win
```

fragment, illustrative, not executed

32.7. the headless suite

`make verify-lua` runs the 34 lua tests, `make verify-lua-c` adds the c side, 8 host samples plus the capstone host over its three drivers, 118 checks in all, and none of it opens a window. The drivers live beside the game and are ordinary lua:

```lua
  script = {
    {at = 2,   cmd = {type = "click", x = 54, y = 30}},
    {at = 4,   cmd = {type = "click", x = 54, y = 456}},
    {at = 6,   cmd = {type = "click", x = 52, y = 280}},
    {at = 40,  cmd = {type = "keydown", key = "space"}},
    {at = 900,  cmd = {type = "click", x = 54, y = 30}},
```

listing 260: a driver is a tick script of the same command tables (source: lua/capstone/tests/drivers/fixed-open.lua, lines 5-10)

The determinism test is the suite's keystone: two games from one seed, one command script, and the digests must match exactly, or a stray `math.random` has leaked into the game:

```lua
tests[33] = {"same seed, same run, digest equal", function()
  local script = {{at = 10, cmd = {type = "click", x = board.shop_x + 30,
                                   y = board.shop_y + 10}}}
  for round = 1, stats.max_rounds + 4 do
    script[#script + 1] = {at = round * 500,
                           cmd = {type = "keydown", key = "space"}}
  end
  local a = drive(31, script)
  local b = drive(31, script)
  T.eq(a.outcome, b.outcome)
  T.eq(game.digest(a), game.digest(b))
end}
```

listing 261: same seed, same run, digest equal (source: lua/capstone/run.lua, lines 387-398)

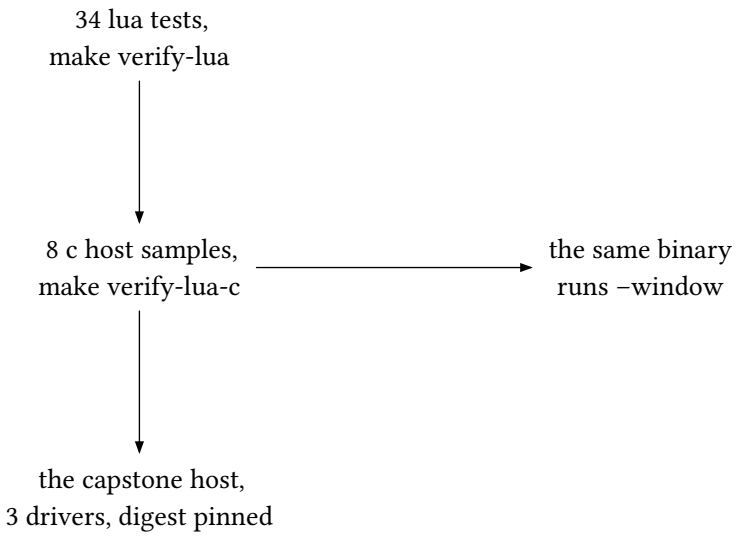


figure 187: the headless gates, three layers, no windows

32.8. profiled, and why

The profiling chapter's census turns on the game once, measured standalone on this machine, 2026-09-21, lua 5.5.1: a full three driver sweep, 9128 game ticks, took a count-hook census at every 200 instructions, 2807 samples. The distribution is the tick loop speaking: `game.lua` carries 45.6 percent, every tick drains the queue and walks the phase machine even when nothing happens, `queue.lua` follows at 9.3 for the same reason, validation runs per command, then `actions.lua` at 5.1, `board.lua` at 4.2, and the combat walker `rules.lua` at 2.7, while the bot's brain never registered a sample at all, it thinks once per prep phase. The shape says where a real engine would spend its optimization budget, and why this one does not need to: the whole sweep is milliseconds of interpreted lua.

The why is the capstone's testing story. Determinism makes the digest pins possible, the digest pins make the drivers regression proof, and the instruction census, not wall time, keeps the claim honest on a machine that is doing anything else. The suite never measures seconds, it counts.

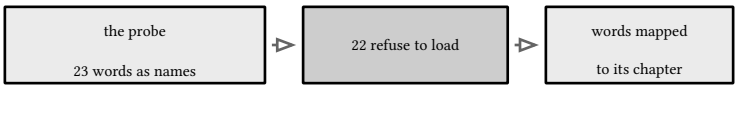
32.9. the service part after the capstone

The game is this book's teaching artifact, and the next evidence layer was built after it, on the same language: the service part under `books/lua/service`, thirteen chapters 19 through 31, closed at version 5.0. The wave's closeout in the findings register, dated 2026-09-28, records the gates green: the plain lane 188 passed 0 failed under `make verify-lua`, 136 family-array checks over 15 modules plus 52 scenario checks over 6 files, the jit lane 18 over the ffi store under `make verify-lua-jit`, and the host lane 162 over the winsock seam under `make verify-lua-c`, with all 16 frozen vectors byte-exact through the composition root's real chain.

sources: the repo's own modules and hosts, all behavior verified in tree: 34 tests through `make verify-lua`, 10 ffi tests through `make verify-lua-jit` from `books/lua/jit/`, 118 c checks through `make verify-lua-c`, driver outcomes and digests measured 2026-09-21 and frozen in `tests/drivers/*.lua`. The ecs and fsm vocabulary cross referenced to book 15, ch 2 (game systems and architecture, ecs theory) and book 15, ch 5 (game systems and architecture, finite state machines).

33. appendices: coverage and sources

The coverage matrices are this book's audit trail. All 23 reserved words of 5.5, `global` included, mapped to the chapter that teaches them, and every standard library family with its status, including the one honest row that stays documented rather than tested, the compact internal arrays, which have no observable semantics. The c api row turned covered in the 3.0 wave: six hosts run under `make verify-lua-c` and the capstone embeds the game in an allegro host. The build enforces the rest: every chapter listing reads a real file that the pinned interpreter executes under `make verify-lua`.

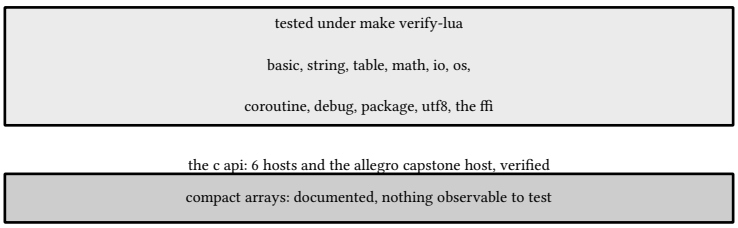


one word survives as a name under compat: global

the matrix below is the result, zero words unmapped

every chapter listing is a real file the pinned interpreter runs

figure 188: how the keyword matrix is proven, word by word



the status column carries this boundary

everything else runs as listings under the pinned binary

figure 189: the coverage boundary, tested against documented only

33.1. keyword coverage

Every reserved word maps to a chapter: 0 unmapped. One word carries a note the whole book repeats: `global`, reserved by the manual, contextual in the stock 5.5.1 build through the `LUA_COMPAT_GLOBAL` default, pinned in book 9, ch 2 (lua 5.5, lexical structure), book 9, ch 8 (lua 5.5, environments, load, gc), and book 9, ch 15 (lua 5.5, incompatibilities with 5.4).

word	kind	taught in	note
and	reserved	ch 4	short circuit, returns an operand
break	reserved	ch 5	innermost loop exit
do	reserved	ch 5	explicit blocks
else	reserved	ch 5	if branches
elseif	reserved	ch 5	if branches
end	reserved	ch 5	block terminators
false	reserved	ch 3	falsy value distinct from nil
for	reserved	ch 5	numeric and generic forms
function	reserved	ch 6	definitions and sugar
global	reserved	ch 2	new in 5.5, compat softened in stock builds
goto	reserved	ch 5	continue idiom, no scope entry
if	reserved	ch 5	conditionals
in	reserved	ch 5	generic for
local	reserved	ch 5	attributes const and close

word	kind	taught in	note
nil	reserved	ch 3	absence
not	reserved	ch 4	below comparisons
or	reserved	ch 4	short circuit, returns an operand
repeat	reserved	ch 5	condition sees body locals
return	reserved	ch 5	multiple results, block end
then	reserved	ch 5	if branches
true	reserved	ch 3	truthy
until	reserved	ch 5	repeat conditions
while	reserved	ch 5	pre tested loop

33.2. stdlib coverage

Component families against chapters, with the status column carrying the honest boundary: what runs under test, what is documented only.

component family	chapter	status
basic: assert, tonumber, select, tostring, ipairs, pairs	ch 11	covered
basic: pcall, xpcall, rawequal, rawget, rawset, rawlen	ch 11	covered
basic: error levels, load and modes	ch 1	covered
string: indexing, format, %q quoting	ch 11	covered
string: patterns, captures, gsub, gmatch, find	ch 11	covered
string: pack and unpack binary formats	ch 11	covered
utf8: len, char, codepoint, offset, codes, charpattern	ch 11	covered
table: insert, remove, concat, move, sort	ch 12	covered
table: pack and unpack	ch 12	covered
math: subtype rules, floor ceil modf integer returns	ch 12	covered
math: fmod, frexp, tointeger, ult, random	ch 12	covered
io: open, read formats, write, seek, io.type	ch 12	covered
io: lines, popen, closable file handles	ch 12	covered, popen also in toolchain
os: time, date, difftime, clock	ch 12	covered
os: getenv, rename, remove, exit	ch 12	covered, exit documented only
coroutine library: create resume yield status wrap close	ch 7	covered
debug: getinfo, traceback, locals, upvalues	ch 13	covered
debug: hooks, registry, getmetatable bypass	ch 13	covered
package: require, loaded cache, preload, searchers, searchpath	ch 13	covered
collectgarbage: modes, param option, count, step	ch 8	covered
collector anatomy: incremental steps, tricolor walk, generational minors and majors	ch 9	covered
lua_Alloc hook: ud, ptr, osize, nsize contract	ch 9	documented only, no host swaps the allocator; counted through count deltas
particle kernel, boxed tables t[1..8], 1m and 10m measured	ch 9	covered, bench.lua opt-in for scale runs
profilers: os.clock, sethook exact counts, instruction census, nearest-rank percentiles	ch 10	covered
jit.p profiling, ffi particle and ffi sqlite row kernel	ch 10	covered, jit/bench.lua opt-in, luajit side only
weak tables and finalizers	ch 8	covered

component family	chapter	status
metatables and metamethods, raw access	ch 3	covered
luac: compile, disassemble, strip	ch 14	covered
c api: stack, registry, refs, userdata, hosts	ch 14	covered, 6 hosts under verify-lua-c, allegro host in capstone
compact internal arrays (5.5 change)	ch 14	documented only, no observable semantics

33.3. pinned sources

Every chapter ends with its own sources line naming the official sections consulted while writing it. The book level pins are:

topic	url	accessed
lua 5.5 reference manual	lua.org/manual/5.5	2026-09-08
lua versions, 5.5 feature list	lua.org/versions.html#5.5	2026-09-08
lua news, release dates 5.5.0 and 5.5.1	lua.org/news.html	2026-09-08
luac manual page	lua.org/manual/5.5/luac.html	2026-09-08
lua 5.5.1 source tarball	lua.org/ftp/lua-5.5.1.tar.gz	2026-09-08
allegro 5.2.11.3 release and prebuilt asset	github.com/liballeg/allegro5/releases/tag/5.2.11.3	2026-09-21
allegro 5 api reference	liballeg.org/a5docs/reference	2026-09-21

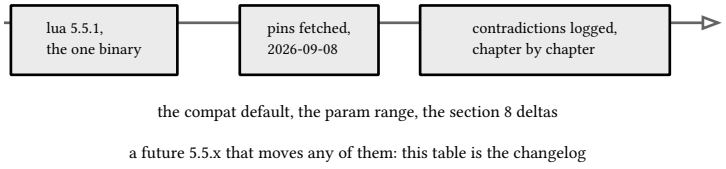


figure 190: the drift rule, one binary, one fetch date, one changelog

NOTE: version drift rule

These pins were fetched 2026-09-08 against lua 5.5.1 built from the official tarball into `tools/lua55`, run on windows with the interpreter and luac from that one build. Findings that contradicted the manual, the compat default for `global`, the out-of-range acceptance of `param` values, the beyond-section-8 changes of book 9, ch 15 (lua 5.5, incompatibilities with 5.4), are recorded in their chapters as verified behavior of this binary, and this table is the changelog if a future 5.5.x moves any of them.